腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

高级Android组件化强化实战

前言

在编程之余，有时候我就在想，什么样的程序员属于高级程序员呢？或者说，高级程序员 有哪些特性呢？工作年限一定不是一个关键的指标，许多工作多年的程序员依然写不出优雅的 程序。无论是在Android 开发还是其他领域，高级程序员一定是勤奋的，可以快速地掌握大量 的新技术、新框架，不仅懂得原理，还能把新的技术落地到公司的产品中去。这是衡量程序员 工作能力的一个重要标准，那么怎样才能将技术运用自如呢？唯有实践。基于此，我想把组件化 在日常实践中的一些大厂经典案例，编著成一本成体系的书，以便为想要进步的Android 程序员增 加更多的实战经验，这也是编写本书的核心目的所在。

编写本书的另外一个目的，是帮助程序员建立产品的思想，对于技术而言，孤立的存在是 没有任何意义的，技术只有与需求相结合，才能具有自身的价值。技术人员在开发的过程中， 要时刻了解所完成的功能可以为公司带来哪些价值，是提升用户的访问兴趣，还是提升用户的 使用流畅度，抑或是其他。当以产品思维去思考技术的时候，就会有动力、有目的地学习更多 有价值的技术，而不是哗众取宠地学一些“看似有用”的新技术。

第一章 Android组件化初识

1.1 组件化和模块化的区别

- 组件化的重心主要是放在业务逻辑层，主要是为了拆分业务逻辑，只针对业务逻辑
- 模块化的重心主要是为了功能的重用，那功能拆分一个个的插件，针对整个项目

1.2 组件化和插件化的区别

- 组件化我们不管把我们的业务拆分为多少个模块，最终在打包上线的时候我们都会生成一个apk
- 而插件化也是拆分为很多插件模块，但是在最终打包之后，成为了很多apk，最终我们把它上传到我们的服务器上面，用户使用的时候，只需要下载响应的apk即可，然后使用动态加载技术，加载里面相应的Activity

1.3 组件化开发的优势

- 相当于我们每次运行不需要整个项目运行，而是运行单一的组件即可
- 如果我们要将某一个模块用到新项目中去，就很简单了，因为我们每一个模块都是一个独立的Application
- 因为Application是不能依赖其他的Application的
- 我们就不需要解耦合了、资源等等
- 大团队开发中，组件化开发是开发的基石

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

1.4 业务逻辑层

公共层

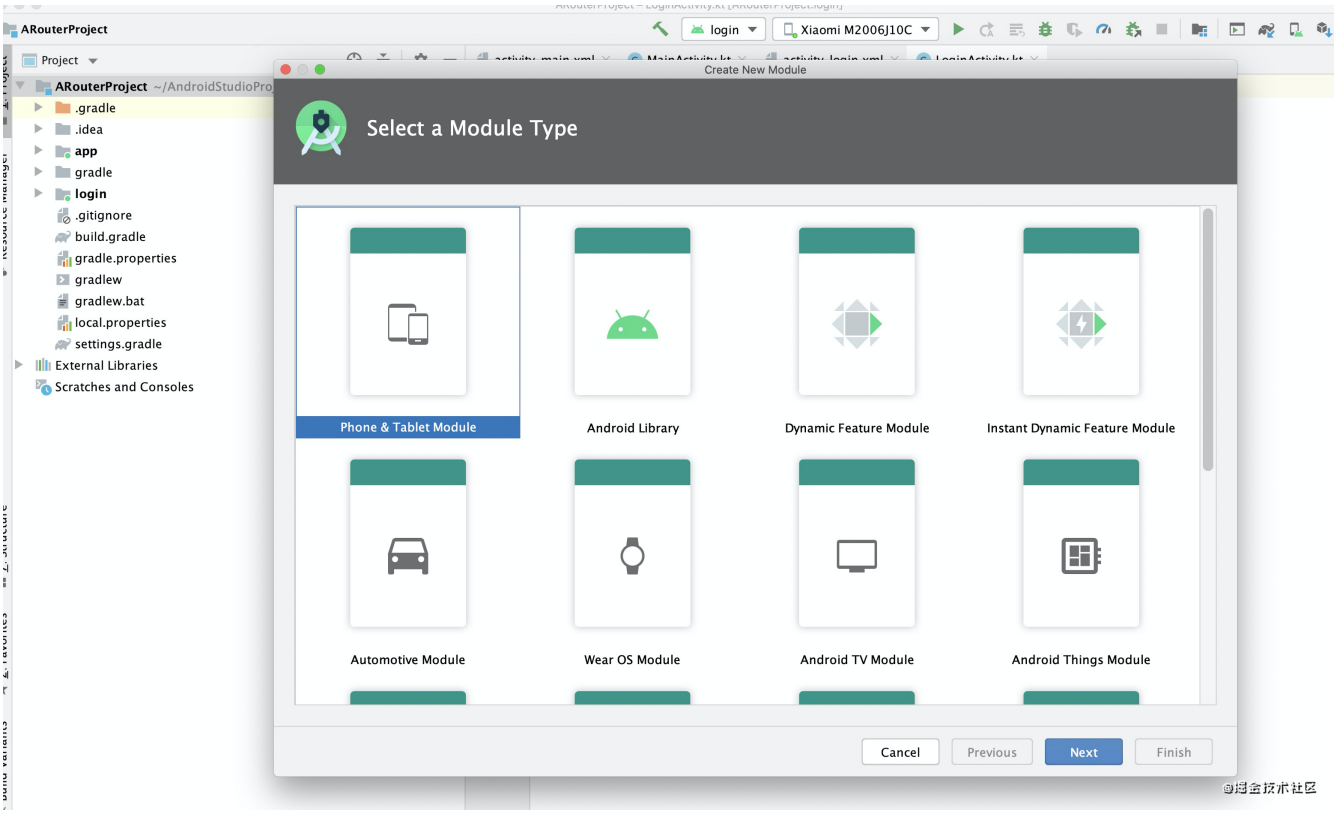
系统层

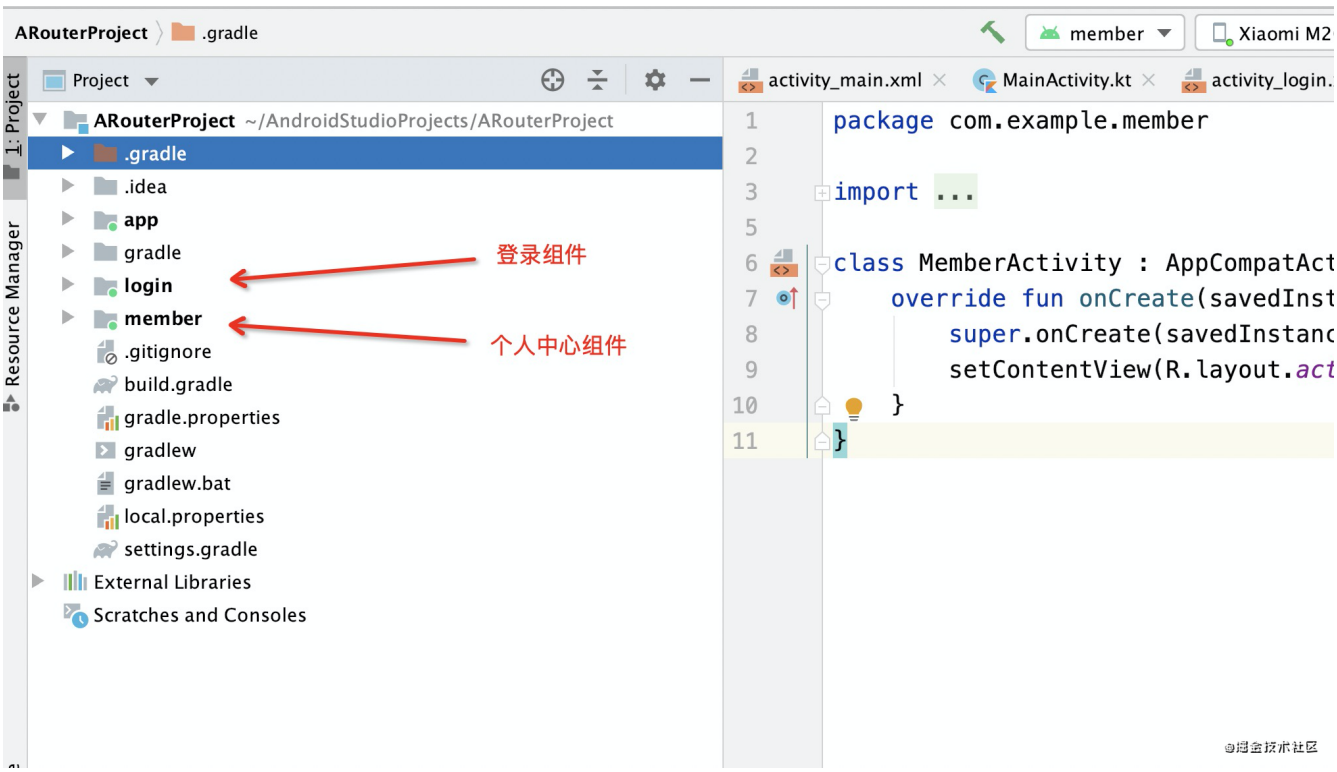
1.5 组件化开发要遇到的问题

1.6 从组件化实战来解决问题

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

- 创建组件化项目
- 创建两个Application组件，注意修改MainActivity的类名，以防止资源命名冲突问题

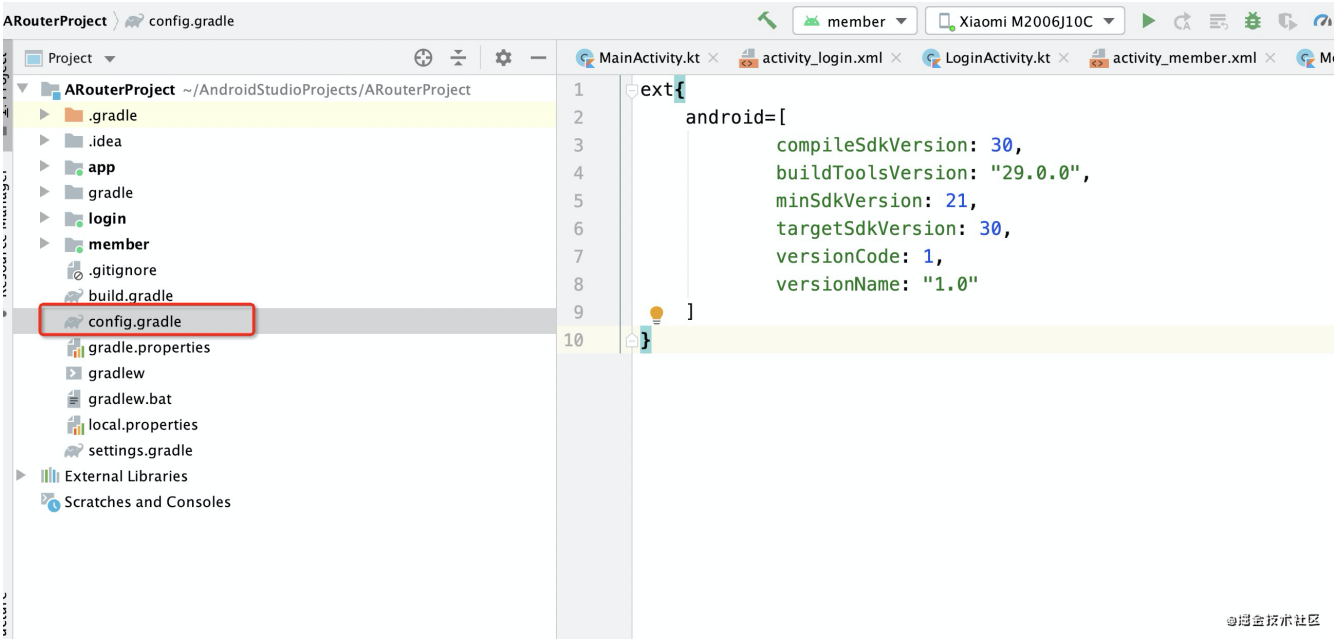




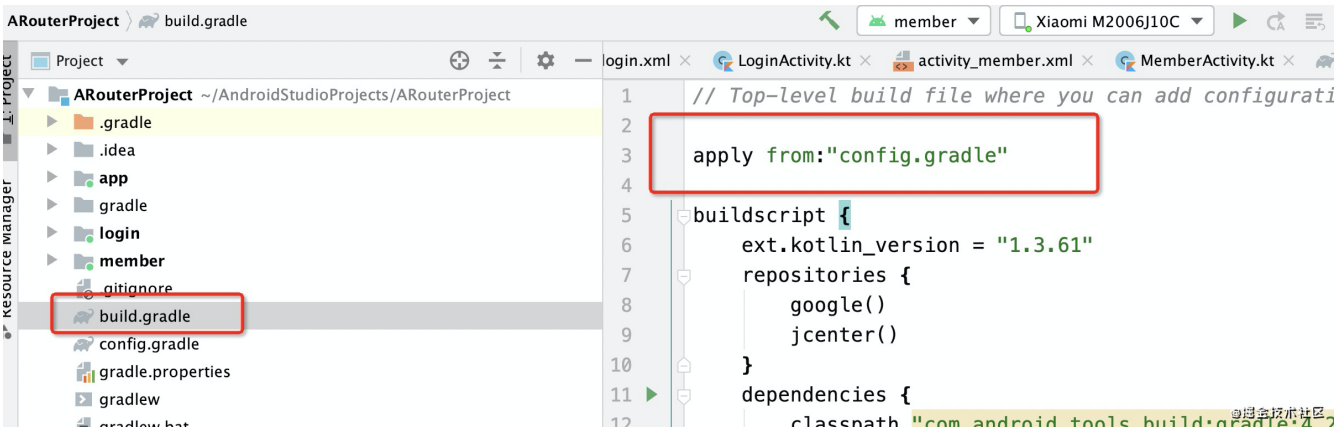
- 我们增加了这么多模块，那么每个模块里面都有build.gradle，假设后面我们需要升级某一个模块中依赖的组件的版本，我们需要修改每一个模块中的版本号么？这样肯定是不行的，所以我们需要创建一个文件，统一管理我们的版本号，而在各个模块中使用相同的版本控制工具才行
- 这里有三种方式
- 第一种方式：直接定义到gradle.properties，简单粗暴

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

- 第二种方式：定义到我们整个项目的build中
- 第三种方式：我们自己创建一个Gradle文件，专门去装我们这些参数
- 我们采用第三种方式

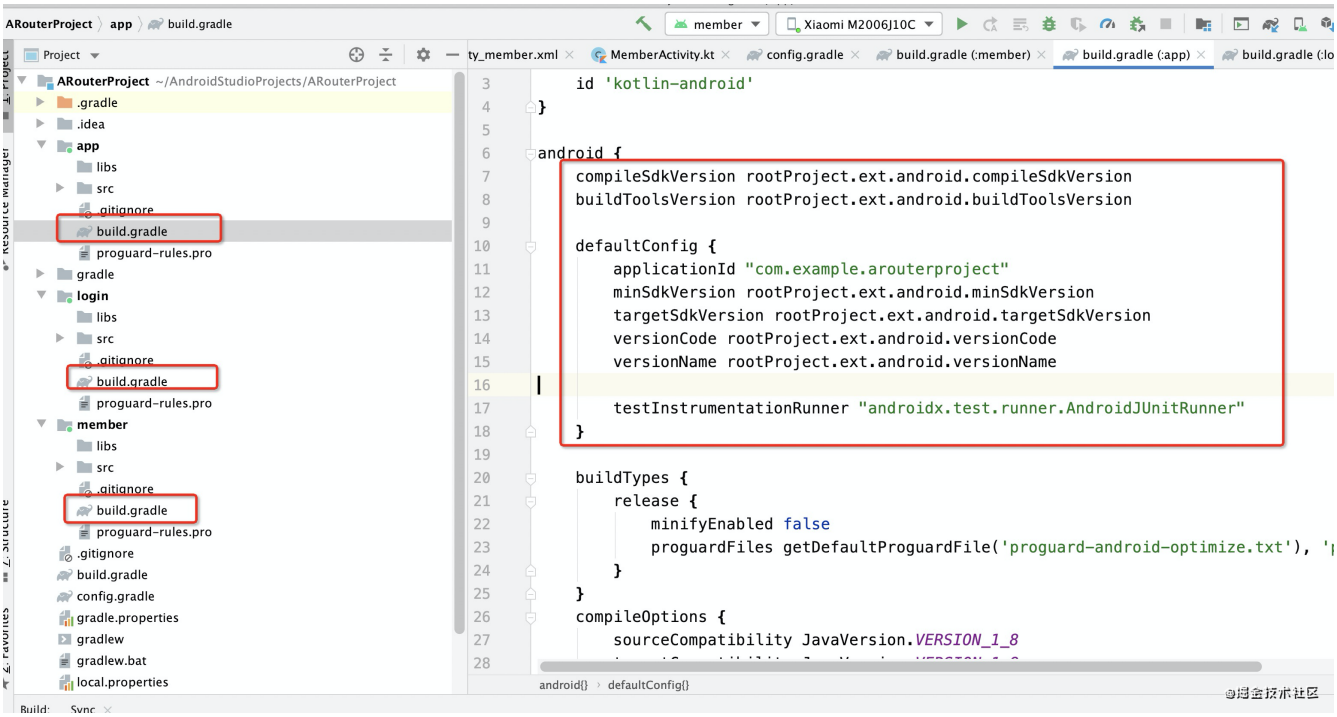


- 文件创建好了，但是我们的项目是不知道的，怎么让整个项目都能使用呢？
- 在项目的build.gradle中声明即可



- 这样声明了就是告诉项目在我们程序加载的时候去加载我们的config
- 然后我们就可以在主App和子Module下面使用我们的config文件了

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



- 改完之后，Sync now一下即可，没有报错就是成功了
- 新的问题来了，我们通常修改完SDK版本号之后，我们下面的各种依赖库的版本也要与之对应，怎么办呢？
- 当然，我们也可以通过同样的方式，定义一个数组来解决问题

```
ext {
    android = [
            compileSdkVersion: 29,
            buildToolsVersion: "30.0.2",
            minSdkVersion    : 19,
            targetSdkVersion : 29,
            versionCode      : 1,
            versionName      : "1.0"
    ]

    dependencies = [
            publicImplementation: [
                    "org.jetbrains.kotlin:kotlin-stdlib:$kotlin_version",
                    'androidx.core:core-ktx:1.3.1',
                    'androidx.appcompat:appcompat:1.2.0',
                    'com.google.android.material:material:1.2.1',
                    'androidx.constraintlayout:constraintlayout:2.0.2'
            ]
    ]
}
```

- 同样的方式，在各个组件的依赖下面引入即可

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

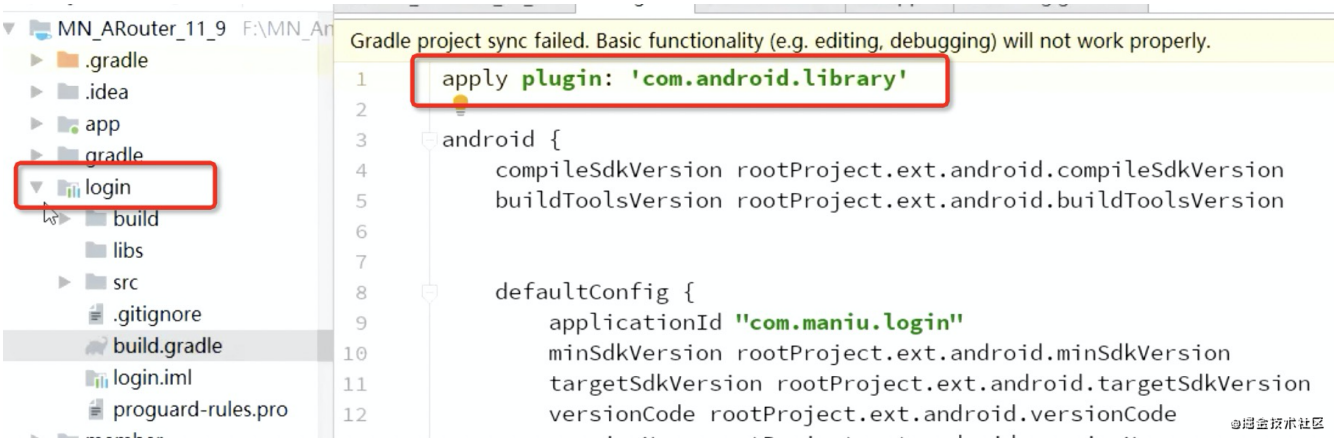
```
34
35  dependencies {
36
37      implementation rootProject.ext.dependencies.publicImplementation
38
39      testImplementation 'junit:junit:4.+'
40      androidTestImplementation 'androidx.test.ext:junit:1.1.2'
41      androidTestImplementation 'androidx.test.espresso:espresso-core:3.3.0'
42  }
```

@掘金技术社区

- 写到这里，会有疑问，gradle文件里面，为什么有的直接用大括号？有的用等于号，有的用冒号，这里解释下，直接用大括号表示数据集的意思，后面等于号表示一个对象的意思，冒号则是某个属性啦
- 写到这里，我们就将各种依赖库的版本号给统一管理啦
- 下一个问题

1.6.1 组件如何在Application和各种Module之间来回切换？

- 其实就是修改我们的组件的打包方式即可



@掘金技术社区

- 把此处的Application改为library
- 但是如果我们每次都需要手动修改，就比较麻烦了
- 我们可以添加一个开关

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
ext {
    android = [
            compileSdkVersion: 29,
            buildToolsVersion: "30.0.2",
            minSdkVersion    : 19,
            targetSdkVersion : 29,
            versionCode      : 1,
            versionName      : "1.0",
            is_application   : true
    ]

    dependencies = [
            publicImplementation: [
                    "org.jetbrains.kotlin:kotlin-stdlib:$kotlin_version",
                    'androidx.core:core-ktx:1.3.1',
                    'androidx.appcompat:appcompat:1.2.0',
                    'com.google.android.material:material:1.2.1',
                    'androidx.constraintlayout:constraintlayout:2.0.2'
            ]
    ]
}
```

@掘金技术社区

- 然后在module中根据开关配置一下即可

```
if (rootProject.ext.android.is_application) {
    apply plugin: 'com.android.application'
} else {
    apply plugin: 'com.android.library'
}

apply plugin: 'kotlin-android'
```

复制代码

- 这个时候，我们把开关设置为false，就会发现，组件都变为Library了

1.6.2 AndroidManifest.xml文件的区分

- 因为application和library的AndroidManifest.xml文件是不同的，这个时候我们就需要区分了
- 我们再module模块中，添加一个manifest文件夹，并拷贝一份manifest文件进去
- 然后library的manifest文件是不需要intent的，这个时候就需要我们去掉intent以及Application里面的部分代码

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.login">

    <application>
        <activity android:name=".LoginActivity">
        </activity>
    </application>

</manifest>
复制代码
```

- 我们把每个Module里面的manifest修改了之后，就需要设置一下了，根据is_applitaion属性来动态设置manifest文件
- 在每个module模块下面添加这样一段代码即可

```
sourceSets{
        main{
            if (rootProject.ext.android.is_application) {
               manifest.srcFile 'src/main/AndroidManifest.xml'
            } else {
                manifest.srcFile 'src/main/manifest/AndroidManifest.xml'
            }
        }
    }
复制代码
```

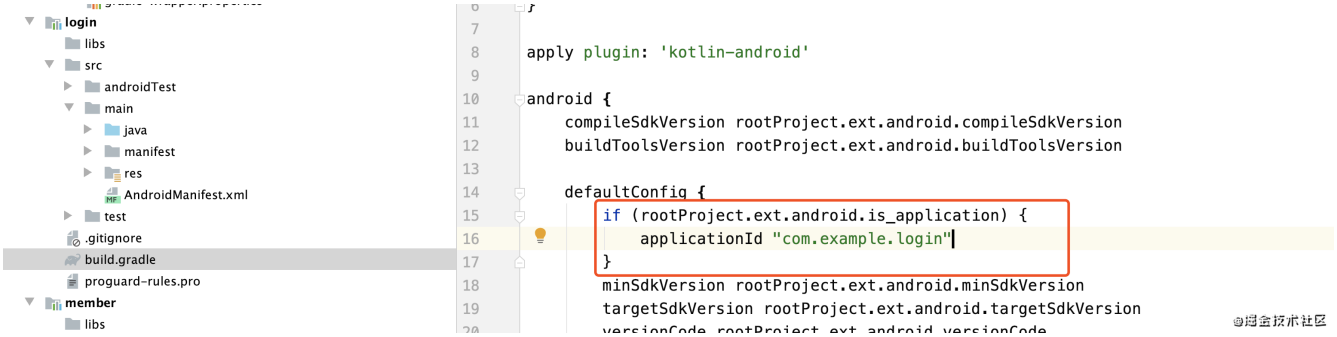
1.6.3 library模块中不能有applicationId

- 这个异常是因为什么呢？

是因为我们的defaultConfig中设置了applicationId,但是library中是不需要的

- 怎么解决这个异常呢？
- 我们在设置applicationId的时候添加判断即可

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



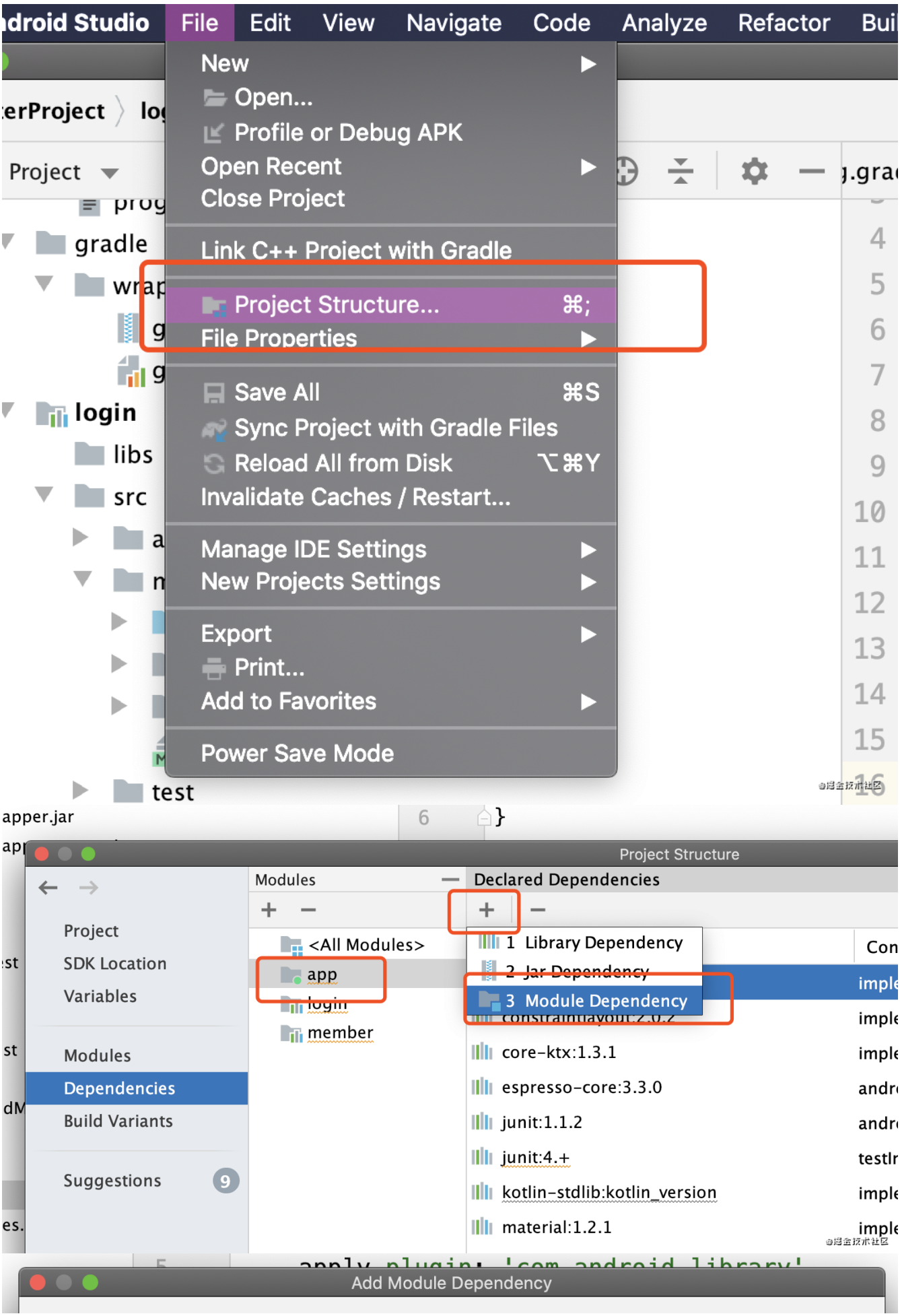
- 如上即可

1.6.4 模块设置

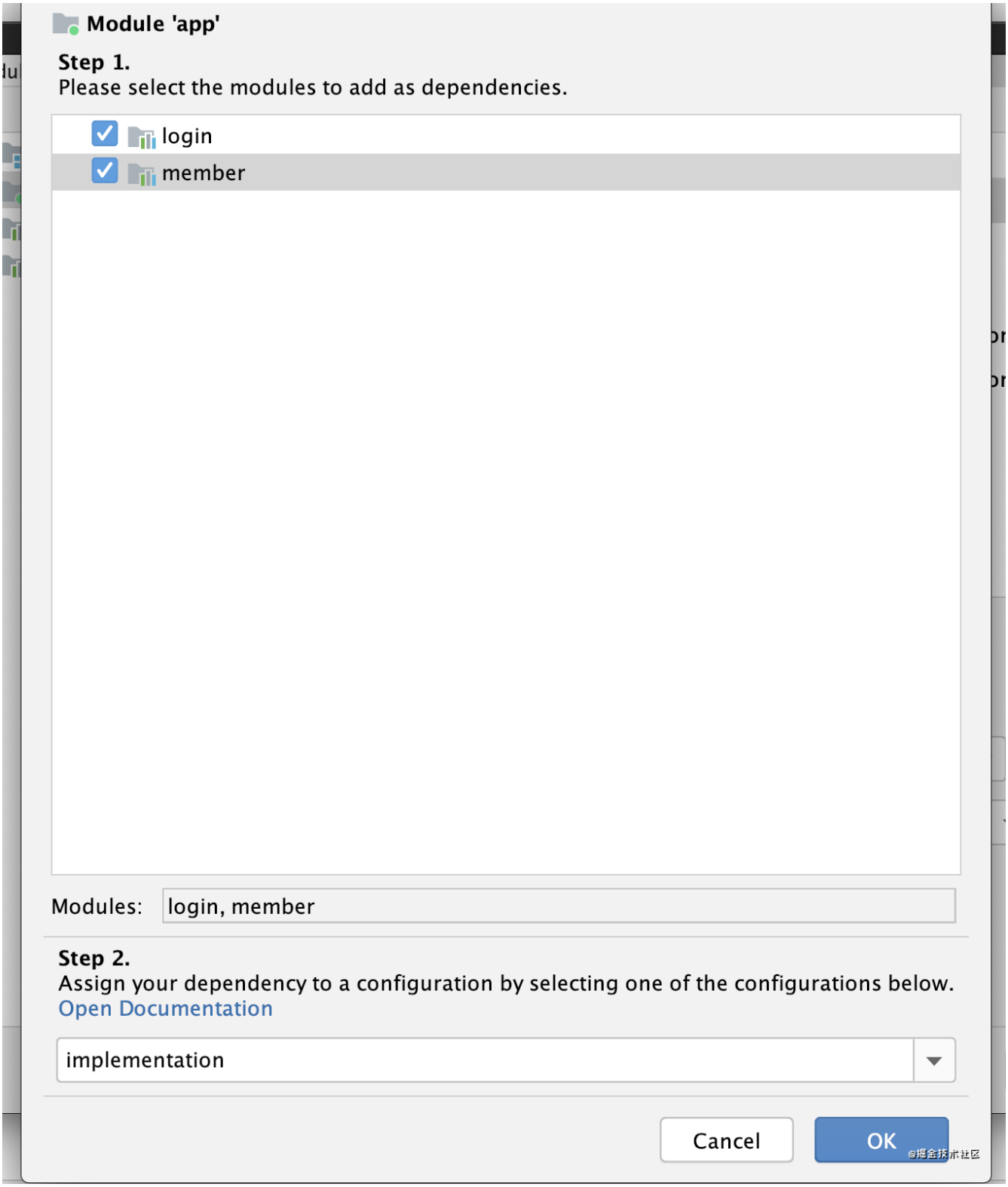
模块设置完毕了，那么最终我们需要发布的时候，就需要让我们的主App去依赖我们的Module了

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



- 如上图所示，我们让App依赖下面的两个模块

1.6.5 依赖之后，我们已经发布过了，但是现在我们又需要将我们的模块修改为application怎么办呢？

- 我们改为true，发现抛出了异常
- 异常的意思就是我们的App模块不能依赖我么的login模块和member模块，因为都是application，application不能依赖application模块
- 那么我们就需要修改了，修改成子模块为library的时候，我们才去依赖那两个模块，否则不依赖
- 修改app下面的build.gradle

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
dependencies {

    implementation rootProject.ext.dependencies.publicImplementation

    if (!rootProject.ext.android.is_application) {
        implementation project(path: ':login')
        implementation project(path: ':member')
    }

    testImplementation 'junit:junit:4.+'
    androidTestImplementation 'androidx.test.ext:junit:1.1.2'
    androidTestImplementation 'androidx.test.espresso:espresso-core:3.3.0'
}
```
复制代码

以上就是一些基本问题，下面还有一些问题

1.6.6 比如我们在项目中需要知道当前项目是application还是library

- 这个时候就需要我们创建给基础库了
- 为什么需要基础库呢？因为我们每个模块都需要用到这些东西

- 接下来问题来了，基础库创建好了，我们要怎么写才能让每个Module都去依赖它呢？
- 我们先在config中定义一个other库，然后再build.gradle中通过each语法，遍历，依赖即可

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
    dependencies = [
            publicImplementation: [
                    "org.jetbrains.kotlin:kotlin-stdlib:$kotlin_version",
                    'androidx.core:core-ktx:1.3.1',
                    'androidx.appcompat:appcompat:1.2.0',
                    'com.google.android.material:material:1.2.1',
                    'androidx.constraintlayout:constraintlayout:2.0.2'
            ],
            other:[
                    ':basic'
            ]
    ]
}
```

@掘金技术社区

```
dependencies {

    implementation rootProject.ext.dependencies.publicImplementation
    rootProject.ext.dependencies.other.each{
        implementation project(it)
    }
    if (!rootProject.ext.android.is_application) {
        implementation project(path: ':login')
        implementation project(path: ':member')
    }

    testImplementation 'junit:junit:4.+'
    androidTestImplementation 'androidx.test.ext:junit:1.1.2'
    androidTestImplementation 'androidx.test.espresso:espresso-core:3.3.0'
}
```

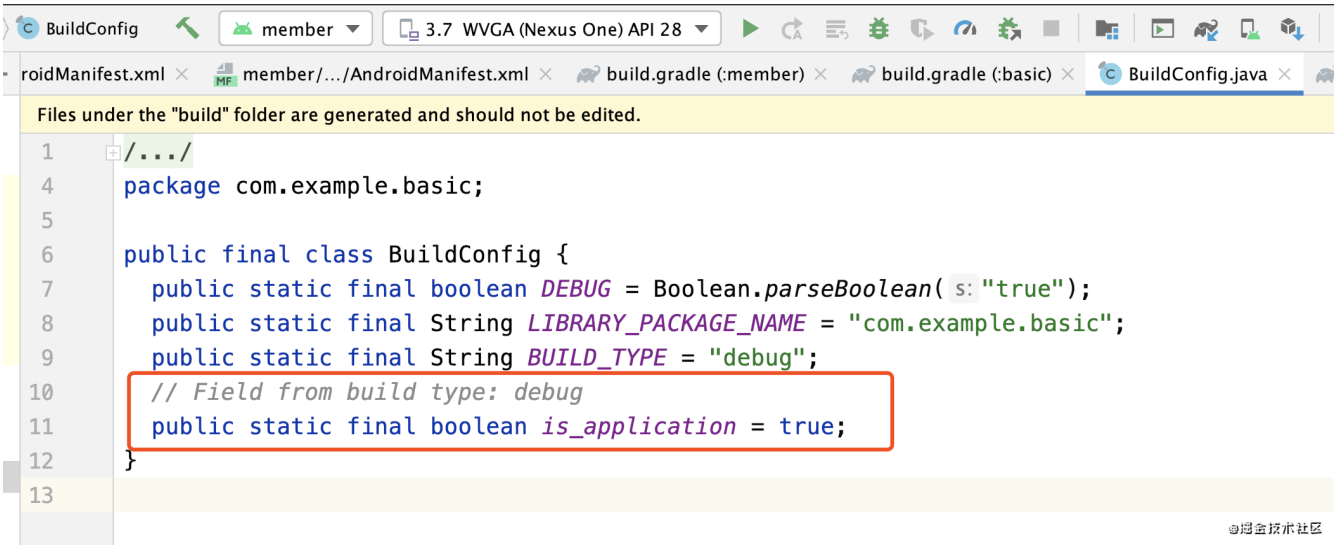
@掘金技术社区

- 这样，我们以后就可以只改动一个地方，就好了
- 接下来我们要处理参数传递的问题，我们需要将is_application参数传递给每个模块的buildConfig文件中去
- 我们需要在我们basic模块下的build.config文件中，添加下面的代码，注意各种环境都要添加该属性

```
buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'),
'proguard-rules.pro'
            buildConfigField 'boolean', 'is_application',
rootProject.ext.android.is_application.toString()
        }
        advanced {
            buildConfigField 'boolean', 'is_application',
rootProject.ext.android.is_application.toString()
        }
        debug {
            buildConfigField 'boolean', 'is_application',
rootProject.ext.android.is_application.toString()
        }
    }
复制代码
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

- 接着我们Make一下工程
- 查找basic的buildConfig文件，发现属性已经添加好了



- 接下来我们就可以在basic模块下面使用了
- 注意这里我们需要在basic模块下的application下面声明一个is_application的静态变量
- 这样才能让我们的每一个模块都能使用上这个属性
- 如下

```
package com.example.basic

import android.app.Application

/**
 * <pre>
 *     author: Jafar
 *     date  : 2020/11/11
 *     desc  :
 * </pre>
 */
class BaseApplication : Application() {

    companion object {
        const val is_application = BuildConfig.is_application
    }

    override fun onCreate() {
        super.onCreate()
    }
}
```
复制代码

- 当然，我们每一个模块不一定都能用上application，那么我们的baseActivity无疑是最合适的

```
package com.example.arouterproject

import android.os.Bundle
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
import androidx.appcompat.app.AppCompatActivity

/**
 * <pre>
 *     author: Jafar
 *     date  : 2020/11/11
 *     desc  :
 * </pre>
 */
class BaseActivity : AppCompatActivity() {

    companion object {
        const val is_application = BuildConfig.is_application
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
    }
}
```

复制代码

- 我们有时候，需要在模块里面编译的时候加载一些用到的类，但是打包的时候不需要，怎么设置呢？

- 如上图所示，我们只需要创建一个文件夹，把java类拷贝进去，然后在sourceSets中添加这样一行代码即可
- 其实这样一行代码的意思就是声明我这个目录就是一个java的文件夹
- 这样做的好处就是，开发的时候，我们使用Application编译，使用临时的代码进行编译，但是呢？打包的时候，我们就不需要用到了，也就是当其变为library的时候，我们就不需要了。减少了无所谓的编译
- 这种做法，就会防止我们因为不断地切换而产生的BUG
- 解决了资源浪费的问题
- 到此为止。我们就把细节都处理的差不多了

模块之间的跳转怎么做呢？手写路由即可！

1.7 Android组件化基础

当 App 项目复杂一定的程度，将项目组件化是必不可少的，组件化可以更好的进行功能的划分，提到组件化有人可能会想到模块化，其实组件化和模块化的本质是一样的，都是为了代码重用的业务解耦，模块化主要按照业务划分，而组件化主要按照功能划分，从组件化最基础的几个方面打开组件化的大门。

1. 组件之间的跳转
2. 动态创建
3. 资源冲突

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

4. 静态常量

1.7.1 组件之间的跳转

组件化中两个功能模块时不直接依赖的，其依赖规则是通过 Base module 间接依赖，当组件之间的 Activity 进行界面跳转时，由于没有相互依赖的关系，往往会无法引用另一个 module 中的 Activity。

隐式跳转

隐式跳转是通过 Android 原生 Intent 匹配机制来实现相应跳转，就是使用 Action 来跳转到对应的 Activity，这样使用隐式跳转的方式就可以跨 module 实现 Activity 之间的跳转了，注意一点，如果移出 Activity 所在的 module 而不移出相应的跳转，如果继续跳转会出现异常，使用隐式 Intent 跳转需要验证是否会接收该 Intent，需要对该 Intent 对象调用 resolveActivity() 方法来判断至少有一个应用能够处理该 Intent，通过隐式跳转的方式还可以设置 exported 为 false 来确保只有自己的 App 才能够启动对应的组件。

ARouter跳转

在 Android 开发中可将 module 看成不同的网络，而对应的 Router 就是连接各个 module 的中转站，这个中转站可以对页面跳转的参数等进行统一处理，ARouter 是阿里开源出来的一个页面跳转路由，使用 ARouter 可以替代隐式跳转来完成不同 module、不同组件之间的跳转以及跳转过程的监听、参数的传递等，ARouter 支持路径跳转和 URL 跳转两种方式，使用也非常灵活，ARouter 的具体使用这里不做介绍，其具体使用会在单独一篇文章中详解，ARouter 与 Android 传统跳转方式的对比如下：

1. 显示跳转需要依赖于类，而路由跳转通过指定的路径跳转；
2. 隐式跳转通过 AndroidManifest 集中管理，导致协作开发困难；
3. 原生使用 AndroidManifest 来注册，而路由使用注解注册
4. 原生 startActivity 之后跳转过程交由 Android 系统控制，而路由跳转采用的是 AOP 切面编程可对跳转过程进行拦截和过滤。

动态创建

组件化开发中最重要的一点就是各个模块、各个组件之间要尽可能解耦，这样很容易就会想到使用 Java 中的反射机制，使用反射可在运行状态下获取某个类的所有信息，然后就可以动态操作这个类的属性和方法了。如果 Fragment 单独作为一个组件来使用时，当这个 Fragment 组件不需要被移出后，如果是常规的 Fragment 则会因为索引不到该 Fragment 而使得 App 崩溃，想一下如果使用反射创建 Fragment 的方式则至少不会引起 App 崩溃，这里可以捕捉异常完成相关逻辑，这样是不是降低了耦合呢。可见，虽然反射有一定的性能问题，但使用反射确实能在一定程度上降低耦合，学习组件化 Java 反射机制应该是必须的一部分。

组件化开发中要求每个组件都能独立运行，一般情况下每个组件都有一定的初始化步骤，最好的一种情况是项目需要的几个组件的初始化基本相同，那就可将初始化放在 BaseModule 中进行统一初始化，但是这种情况毕竟比较理想，一般情况是每个组件的初始化都不一样，可能你会想到在各自的 Application 初始化，如果在各自的 Application 中初始化，当在最终编译由于 Application 的合并难免会出一些问题，这种方式也不可取，到这里又想到了反射，在各组件中创建初始化文件，然后在最终的 Application 中通过反射完成各个组件的初始化操作，这里通过 Java 的反射机制完成了组件化开发中 Application 的动态配置。

资源冲突

组件化开发过程中，如果 ModuleA 的 AmdroidManifest 文件中使用 android:name 属性指定了相应的 Application，而主 App Module 的 AndroidManifest 文件中也使用 android:name 属性指定了相对应的 Application，此时就必须在 **主App Module** 的 AndroidManifest 文件中使用 tools:replace="android:name" 来解决冲突，使用 replace 属性表示该属性也就是在 标签下的 android:name 属性可在编译过程中被替换，这样根据 AndroidManifest 文件替换规则最终指定的 Application 应该是 App Module 中的指定的 Application。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?tid=8674704361863085

举一个例子，我在项目中的某个功能 Module 中使用 SMSSDK 来完成短信验证的功能，因为其他地方不用，所以只引入到了要使用的功能 Module 中，如果其他 Module 会使用应该将 SMSSDK 引入到 BaseModule 中，使用 SMSSDK 如果不指定该 Module 的 Application，MobSDK 会将 com.mob.MobApplication 指定为该 Module 的 Application，此时在整体编译打包时就会出现 AndroidManifest 文件的 android:name 属性冲突，当然了解决方法就是使用 replace 属性了。 AndroidManifest 文件合并后的主要冲突也就是这个问题了，当然 下的其他属性有冲突，也是使用 replace 属性。在实际的开发中多验证会更有收获喔。

组件化开发中另外需要注意的一点是防止资源名称一样导致最终合并的时候，因为冲突造成资源引用错误或者某些资源丢失等，如字符串、颜色值等资源等合并的时候会被后面加载的相同名称的资源所替换，解决的思路是在资源命名上要有一定的规则，可以在 build.gradle 文件中配置 "resourcePrefix "组件名称"" 的方式强制约束开发者确保资源名称唯一，建议 Module 中资源的命名格式为 "Module名称*功能*其他"。

静态常量

组件化开发中，最终合并时每个组件都是以 Lib Module 的形式存在，而 Lib Module 中 R.java 文件中定义的静态变量没有声明为 final，这就意味着不能在组件 Module 中使用相对应的常量了，如在时候 switch 语句就不能使用了，这就要求在组件中要使用 if 语句来替代 switch 语句，当然在组件独立运行的时候是没有这个问题的。

开发中经常会使用到 Butterknife，Butterknife 可非常方便的对 View 及 View 的事件等进行注解操作，它采用的是编译时注解机制，注解中只能使用常量，所以在 Butterknife 在组件化开发中应该使用 R2 代替 R，R2 实际上是 R 的拷贝， R2 对应声明的变量是 final，所以在组件化开发中如果使用 Butterknife 在相应的注解中要使用 R2 替代 R。

第二章 Android组件化初探

2.1 组件化演示案例

demo地址：https://github.com/syg13579/assembleDemo

2.1.1 概述

软件开发进程也是架构的演进过程，就拿Android来说，从最开始的MVC ,MVP ,MVVP ,再到后来的组件化，插件化，但归根到底一切的一切，都是为了项目更好的维护、迭代，降低开发成本。

在一个项目的开发过程中，前期我们可能把所有的功能模块都放到了一个moudle中，这样能够快速的开发，但随着项目壮大，开发人员和功能的增加，就回导致代码越来越臃肿，各个模块之间的耦合越来越重，牵一发而动全身，这个时候为了保证项目质量，我们就需要对项目进行重构。

我们可以根据业务模块进行查分，把不同的业务模块放到不同的moudle中，实现各个业务之间的结构，他们又共同依赖底层公共库，这就是**模块化的概念**，但是当多个模块中涉及到相同功能时代码的耦合又会增加，例如有两个模块都需要视频播放的功能，把视频播放放到两个组件中就会出现代码重复的问题，放到公共库感觉也不是很好，这时候就用组件化来解决这个问题

2.1.2 模块化和组件化

2.1.2.1 模块化

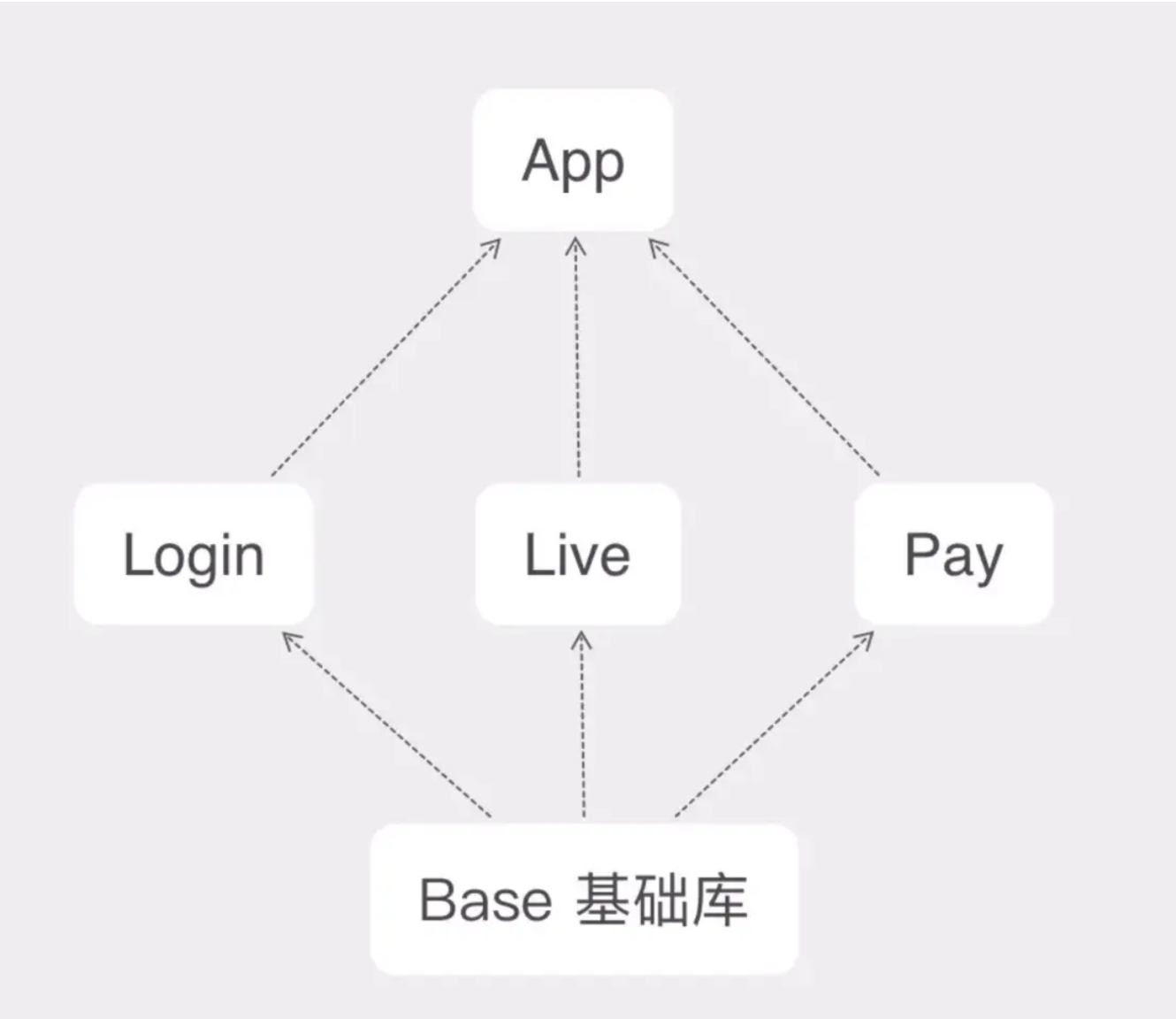
具体的业务模块，例如商品详情模块，商品发布模块 ，搜索模块

2.1.2.2 组件化

单一的功能组件，如视频播放组件、分享组件等，每个组件都可以以一个单独的 module 开发，并且可以单独抽出来作为 SDK 对外发布使用

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

模块化和组件化的思想是一样的，都是对代码进行拆分，但模块化是按功能模块进行查分（业务导向），组件化是按功能模块进行查分（功能导向），模块化的颗粒度更大一些，组件的颗粒度更小一些，**一个项目中模块和组件同时存在也是很常见的，各自负责各自的事情**



如上图所示 是个组件化项目的基本架构

- **基础库、公共库**：项目所需要的基础操作类，工具类 ，第三方库的引入封装 ，app宿主功能，各个模块，各个组件都依赖这个库
- **组件层**：项目用的功能模块或者业务模块，如：登录模块，视频播放组件，分享组件等
- **应用层**：宿主工程，APP的主项目，APP入口和主架子

2.1.3 组件化Demo

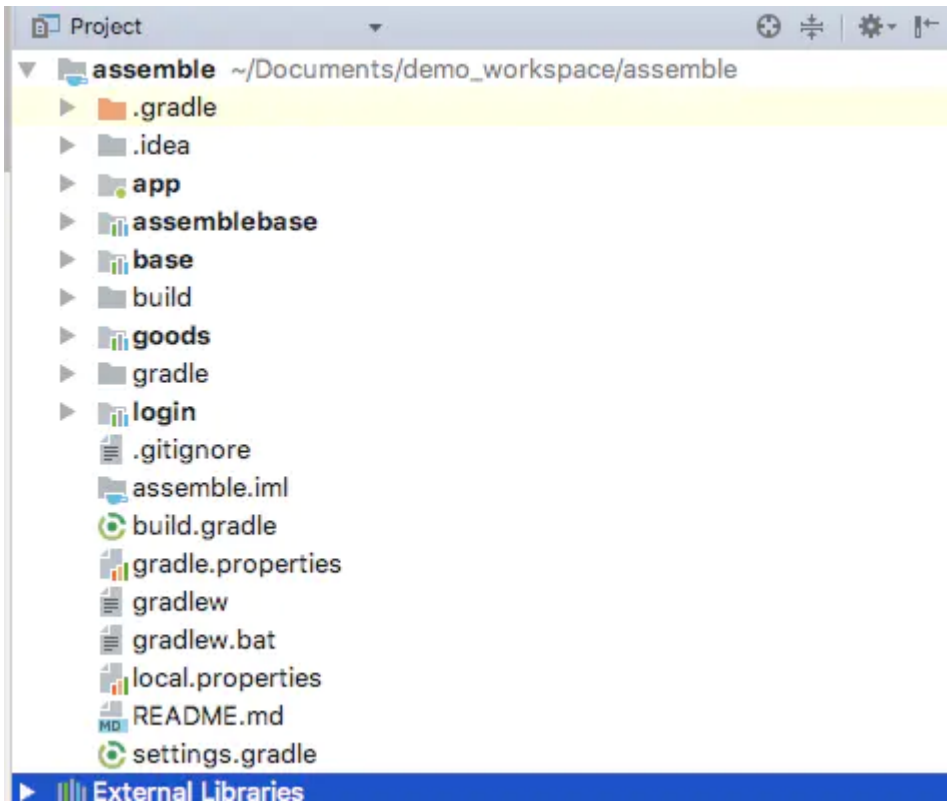
地址如下: https://github.com/syg13579/assembleDemo 我根据demo项目从以下几个方面来讲解

- 1：项目分析
- 2：组件application和library动态切换
- 3：组件间的数据传递和方法调用

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

- 4：组件类（例如：Fragment）的获取,以及夸组件页面跳转和通讯

2.1.3.1 项目分析



如上图所示，项目的主要结构

- 应用层：app 项目的主入口
- 组件层：goods login 商品详情页和登录组件
- 基础库层：assemblebase用来各个组件数据和方法交互的 ，base是常用的工具类，各种类库的封装

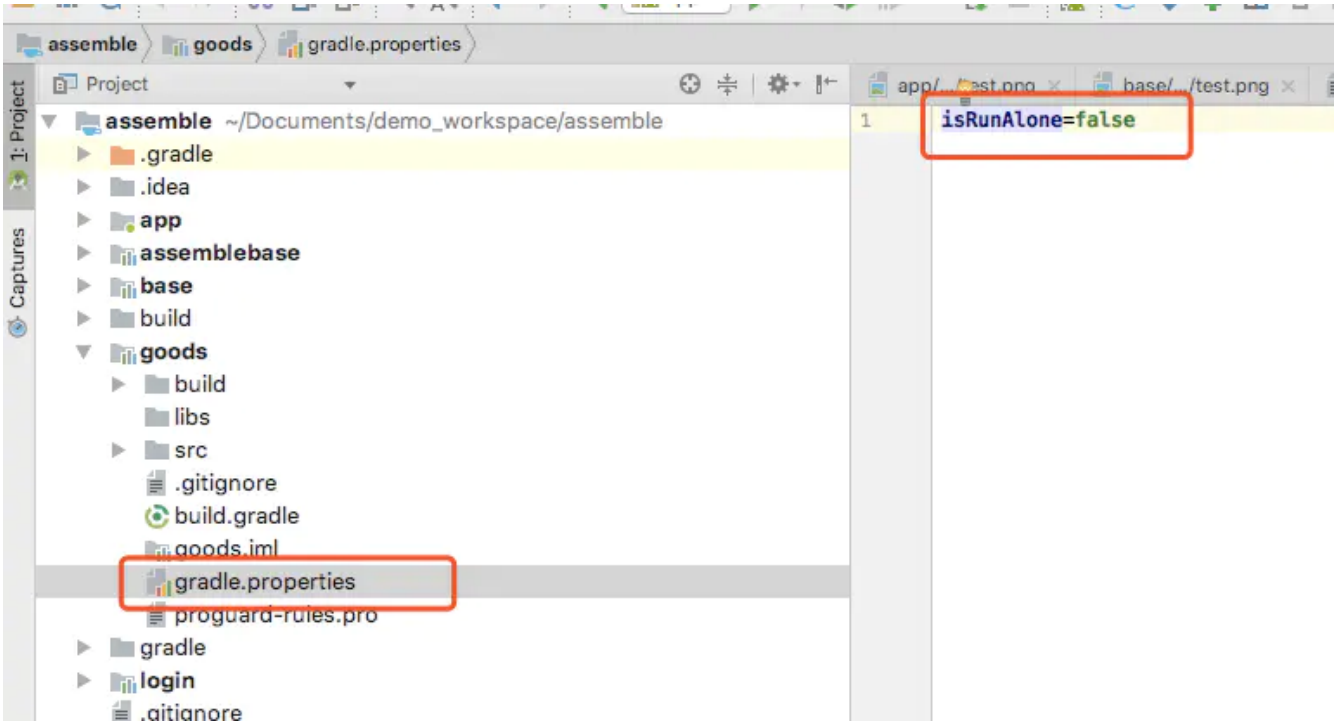
2.3.2 组件application和library动态切换

在开发过程中，为了能够实现快速开发，组件能够独立运行就显的特别重要,moudle一般分为两种

- App 插件，id: com.android.application
- Library 插件，id: com.android.library
-

我们可以通过配置可动态进行application和library的切换，我们在各个组件的gradle.properties文件中配置一个控制切换的变量

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



然后在build.gradle中就可以通过isRunAlone变量来进行application和library的切换了，主要设计的点有三部分

- plugin属性的配置
- applicationId的配置
- AndroidManifest的配置
-

```
if (isRunAlone.toBoolean()) {
    apply plugin: 'com.android.application'
} else {
    apply plugin: 'com.android.library'
}

android {
    compileSdkVersion 26



    defaultConfig {
        if (isRunAlone.toBoolean()) {
            applicationId "ppzh.jd.com.goods"
        }
        minSdkVersion 15
        targetSdkVersion 26
        versionCode 1
        versionName "1.0"

        testInstrumentationRunner "android.support.test.runner.AndroidJUnitRunner"

    }
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android.txt'), 'proguard-
 rules.pro'
        }
    }

    sourceSets {
        main {
            if (isRunAlone.toBoolean()) {
                manifest.srcFile 'src/main/manifest/AndroidManifest.xml'
            } else {
                manifest.srcFile 'src/main/AndroidManifest.xml'
            }
        }
    }

}
复制代码
```

如果以上配置就可以实现application和library的切换了

2.1.3.3 组件间的数据传递和方法调用

由于主项目、组件之间，组件和组件之间不能直接通过引用进行数据传递和方法调用，那么在开发的过程中怎么进行数据传递和方法调用呢，可以通过「接口」+「实现」的方式进行，

assemblebase基础库就是用来进行数据传递和方法调用的，它被所有组件所依赖，assemblebase提供各个组件对外提供数据和方法调用的抽象service ,同时还有serviceFactory对service进行操作，各个组件在初始化的时候对各自的service进行实现。同时中也会提供所有的 Service 的空实现，以避免引起的空指针异常

就以登录模块为例，对外提供两个数据

```
public interface ILoginService {

    /**
     * 是否已经登录
     *
     * @return
     */
    boolean isLogin();

    /**
     * 获取登录用户的 AccountId
     *
     * @return
     */
    String getAccountId();

}
复制代码
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

相关的serviceFactory类如下，可以通过serviceFactory拉取相关service的实例

```
public class ServiceFactory {

    private ILoginService loginService;
    private IGoodsService goodsService;

    /**
     * 禁止外部创建 ServiceFactory 对象
     */
    private ServiceFactory() {
    }

    /**
     * 通过静态内部类方式实现 ServiceFactory 的单例
     */
    public static ServiceFactory getInstance() {
        return Inner.serviceFactory;
    }

    private static class Inner {
        private static ServiceFactory serviceFactory = new ServiceFactory();
    }


//    ------------------------LoginService-----------------------
    /**
     * 接收 Login 组件实现的 Service 实例
     */
    public void setLoginService(ILoginService loginService) {
        this.loginService = loginService;
    }

    /**
     * 返回 Login 组件的 Service 实例
     */
    public ILoginService getLoginService() {
        if (loginService == null) {
            loginService = new EmptyLoginService();
        }
        return loginService;
    }
```

复制代码

在login组件中只需要实现ILoginService，并通过serviceFactory进行设置

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
public class LoginService implements ILoginService {
    @Override
    public boolean isLogin() {
        return false;
    }

    @Override
    public String getAccountId() {
        return null;
    }
}
```
复制代码

在login的appliction中进行service的设置

```
public class LoginApp extends BaseApp {

    @Override
    public void onCreate() {
        super.onCreate();
        initModuleApp(this);
        initModuleData(this);
    }

    @Override
    public void initModuleApp(Application application) {
        ServiceFactory.getInstance().setLoginService(new LoginService());
    }

    @Override
    public void initModuleData(Application application) {

    }
}
```
复制代码

但是有这样一个问题：在集成到app中，LoginApp是没有被执行的，这个怎么解决呢，我们可以通过反射进行解决

```
public class AssembleApplication extends BaseApp {
    @Override
    public void onCreate() {
        super.onCreate();
        initModuleApp(this);
        initModuleData(this);
        initComponentList();
    }

    @Override
    public void initModuleApp(Application application) {

    }
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
    @Override
    public void initModuleData(Application application) {

    }

    //初始化组件
    //通过反射初始化
    private void initComponentList(){
        for (String moduleApp : AppConfig.moduleApps) {
            try {
                Class clazz = Class.forName(moduleApp);
                BaseApp baseApp = (BaseApp) clazz.newInstance();
                baseApp.initModuleApp(this);
                baseApp.initModuleData(this);
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }


}
复制代码
```

如上所示就完成了

2.1.3.4 组件类（例如：Fragment）的获取,以及夸组件页面跳转和通讯

fragment的获取也是通过service来完成的

```
public interface IGoodsService {

    /**
     * 创建 GoodsFragment
     * @param bundle
     * @return
     */
    Fragment newGoodsFragment(Bundle bundle);
}
复制代码
```

相关组件实现该接口就行

各个组件间页面的跳转可以通过阿里的ARouter实现，我是通过设置ComponentName来实现的，**但这种方式好像并没有实现真正的代码隔离**

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
/**
     *
     * 去登陆
     *
     * 跨组件页面跳转
     */
    private void toLogin(){
        Intent intent = new Intent();
        intent.setComponent(new ComponentName(mContext, "ppzh.jd.com.login.LoginActivity"));
        startActivityForResult(intent,LOGIN_REQUEST_CODE);
    }
```

复制代码

2.1.4 总结

通过上面就整体实现了项目组件化，在以后也可以更多的运用组件化来进行项目开发

2.2 WanAndroid APP 组件化项目实战附demo

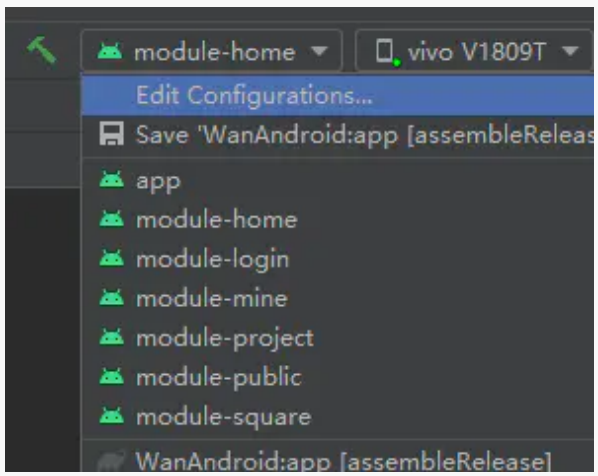
2.2.1 简介

本项目基于 组件化 + Arouter + Jetpack + Rxjava + Retrofit + AOP 等框架实现的一款开源项目。如有任何疑问或bug欢迎给我提issues,项目会一直维护下去，一起努力打造一个完美的app。
https://github.com/1170762202/WanAndroid

2.2.2 版本更新

- module单独编译运行
- module-main迁移至app，并移除module-main

1.png

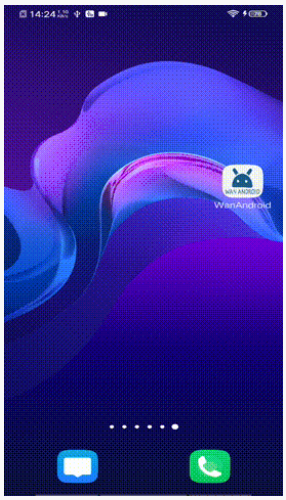


2.png

历史版本

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

2.2.3 效果图



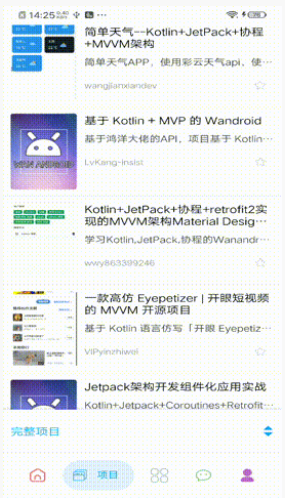
1.gif

2.gif

3.gif

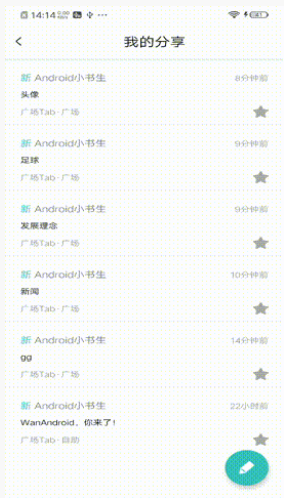
4.gif

1.gif



2.gif

3.gif



4.gif

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

1.gif

1.gif

1.gif

1.gif

2.2.3 主要功能

- 首页、项目、广场、公众号、我的
- 登录、注册 动画交互
- 搜索页面共享元素动画过渡
- 项目页面仿高德地图滑动面板交互
- 广场页面tab跟随滑动系数渐变、列表采用谷歌爸爸的flexboxlayout流式布局
- 广场页面根据滑动系数给indicator添加动画效果
- 公众号页面点击左上角为一个90°的arc交互动画，列表数据从下往上过渡的动画效果
- 我的页面仿百度外卖个人中心水波纹效果
- 整体采用Material Design设计风格
- 首页有彩蛋哦！

2.2.4 项目目录结构

```
|- WanAndroid
||-- app // app 入口
    ||librarys //library库
      ||--library-aop// aop 封装（登录校验、点击）
      ||--library-db// room数据库封装
      ||--library-network// 网络请求封装（livedata+rxjava+retrofit）
      ||--library-base// 基础封装（BaseAc、BaseFg、BaseUtil等）
      ||--library-common//共用的组件、适配器、api返回实体类等
      ||--library-widget// 控件封装
  ||--modules// 功能模块
    ||--module-home// 首页模块
    ||--module-login// 登录模块
    ||--module-project// 项目模块
    ||--module-square // 广场模块
    ||--module-public //公众号模块
    ||--module-mine//我的模块
    ||--module-web//网页模块
||-- README.md
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

module单独编译运行说明

gradle.properties 文件下有个"集成开发模式" 和 "组件开发模式"的切换开关 true表示组件独立运行，false表示一个library

```
isRunModule=true
```

2.2.5 主要开源框架

- flexbox-layout
- RxJava
- RxAndroid
- Retrofit
- okhttp
- Glide
- BaseRecyclerViewAdapterHelper
- EventBus
- Arouter
- ImmersionBar
- Particle
- banner
- LoadSir
- MagicIndicator
- MMKV
- SmartRefreshLayout
- AgentWeb
- aop
- PersistentCookieJar

点我下载

致谢

- 阿里矢量图
- WanAndroid

第三章 架构演化（大厂篇）

3.1 从智行 Android 项目看组件化架构实践

3.1.1 前言

智行火车票早期以火车票业务起步，随着整体的业务发展和扩张，先后增加了汽车票、机票和酒店模块，逐渐打造成了一个提供出行、旅行和住宿一站式预订服务的 OTA 平台。

在业务扩张过程中，之前 Android 项目单一工程的架构模式慢慢暴露出一些问题，例如业务间耦合较多，整体项目编译耗时等，渐渐无法满足业务开发需求。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

为了解决面临的问题，综合主流的 Android 项目架构方案，团队选择了组件化架构方案对项目进行了调整和实践，抽离出基础组件库、独立的业务模块，实现了各独立业务的拆分和独立运行，可以单独进行需求开发，发版时再合并到一起编译打包和发布。

在组件化架构实践过程中，团队解决了组件化调整中遇到的一些难题，对组件化技术在 Android 项目中的应用有一定的参考价值和实践经验。同时根据业务需求，还实现同一个项目进行多个应用差异化适配打包的功能，便于开发和维护团队旗下的其他应用。

3.1.2 概述

本文主要根据智行 Android 团队在组件化架构调整中的实践过程以及最终的实践成果，从以下几个方面来进行阐述：

- 为什么要进行组件化架构调整
- 组件化结构调整的实施步骤
- 组件化调整过程中遇到的难题以及解决方案
- 组件化架构调整的成果

3.1.2.1 组件化调整的原因和目标

如前面提到的，在调整之前，项目是单一工程的架构模式，这也是常见的 Android 项目架构模式，但是一旦项目整体业务增多，扩张出相对较为独立的业务模块，这种架构就会带来一些问题，例如：

- 业务间代码层面耦合太重，业务之间隔离不明确：由于各业务间代码存在较多的耦合，经常出现某个业务线功能开发迭代影响其他业务线，出现代码冲突，影响其他业务功能。
- 项目整体源码较多，编译耗时久：各业务开发人员主要开发各自业务线需求，但是需要编译整个项目，耗时较多，影响开发效率。
- 多应用差异化适配方案不完善：在业务扩张过程中，还衍生出一些独立应用，例如智行旗下的订票助手、智行机票等应用，实际是使用同一个项目打包，更改一些主题配色和首页入口，进行差异化的编译打包。之前使用的多应用打包方案存在一些的问题，逐渐无法满足实际需求。

参考技术社区的 Android 架构方案，以及结合项目实际情况和业务场景，我们选择了组件化方案来进行架构的调整。

Android 项目组件化，最早是冯森林老师在 2016 年 MDCC 大会上的《回归初心，从容器化到组件化》演讲中提出来的，当时该方案刚提出，实际应用到项目中的还是比较少得，毕竟一般的公司项目业务不是很复杂，项目结构也是较为单一，没有使用组件化的必要。

但对于此时的智行 Android 项目而言，正是组件化架构最适合实践的项目，多个业务线，项目整体比较庞大，业务间不必要的耦合过多，因此组件化架构的调整方案也就应运而生。

在进行调整之前，团队也定下了调整预期的目标：

1) 业务解耦，使得各业务模块可以独立运行，同时可以组合编译打包

2) 拆分基础组件，抽离出基础组件 Library

3) 各业务间通信和业务交叉调用的实现

4) 实现多应用差异化适配打包

以上大的目标点主要是来解决之前遇到的问题，也是项目架构调整的首要目的。

3.1.2.2 组件化架构调整的整体规划

3.1.2.2.1 基础组件的拆分

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

智行 Android 项目的基础组件主要分为业务基础组件和功能基础组件，其中业务基础组件包含登录组件、自定义 View 组件、项目网络层组件等，这些和业务有关联，提供给各业务模块的基础组件，根据具体情况拆分成 aar 或者 library，像登录，基础网络层这样较为稳定的组件，一般直接打包成 aar，减少编译耗时。而像自定义 View 组件，由于随着版本迭代会有较多变化，就直接以源码形式抽离成 Library。

基础组件的调整相对较为简单，主要就是按照功能或者业务拆分成 Library ，处理好之前的引用的地方即可，但是对于拆分出来的 Library 的质量和后续维护工作是要求相对较高的，作为基础的组件，是需要为各业务模块提供基础的功能的，重要性是相对较高的。

基础组件库的编译版本设置一般是和主工程同步的，为了方便后续升级和维护配置，可以使用如下的方式来实现 library 使用同一份配置：

```
ext.libDefaultConfig = {
```

复制代码

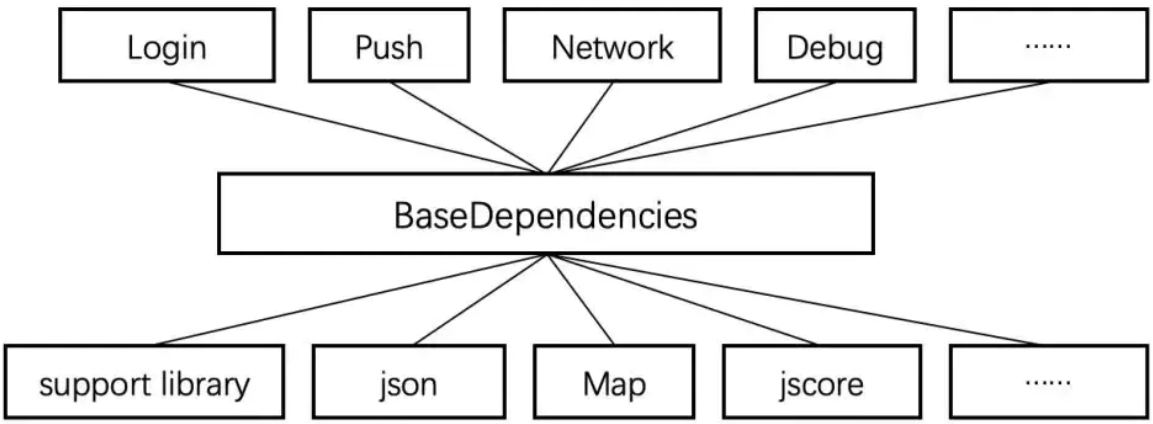
定义一个通用的 DefaultConfig 配置，设置统一的 SDK 版本信息和编译选项，在 Library 的 build.gralde 文件中使用如下方式即可应用到配置：

```
android {
```

这样既可以保证基础组件库的编译配置统一，也方便后期统一修改和升级。

对于再基础点的组件，例如 Support Library、json 库等，绝大多数基础组件都会使用到。为了避免每个独立基础组件都去引入对应的依赖，还要尽可能得保证版本的统一，我们使用了一个空壳 Library 来一次性引入这些基础的依赖组件。

这个 Library 叫做 BaseDependencies，然后其他的基础组件去依赖 BaseDependencies，这样就可以保证基础组件对这些基础的依赖版本做到一致，后续的升级改动位置也相对较为集中。如此调整后的依赖如下图所示：



当然这样的也有一定的弊端，就是每个基础组件都可能存在引入冗余的依赖，对于后续可能需要提供给第三方的基础组件，还需要进行改动才可以独立出来。

3.1.2.2.2 业务模块的拆分和配置

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

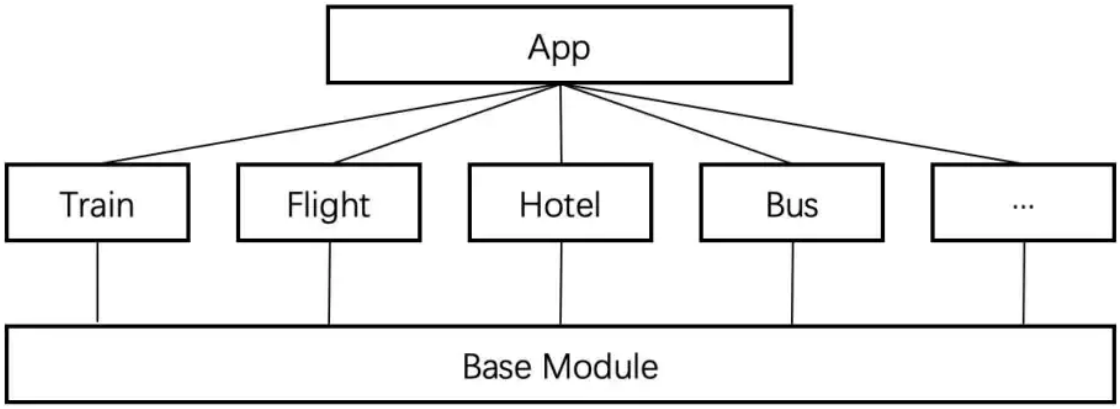
业务模块的调整是组件化中最重要的，这里的模块也是组件，对于这类组件的调整目标就是做到能够独立运行。业务模块的调整主要分两步：

1) 业务模块的拆分独立

2) 业务模块的配置

由于之前各业务的代码以不同包名来进行区分，各业务代码也是比较集中的，拆分出来还是相对较为容易的。遇到两个业务模块都会使用到的类的话，就将对应的类下沉到 Base Module，当然这种情况也是尽可能去避免，否则 Base Module 会越来越臃肿，如果不加以控制，那么业务模块就变成了一个空壳，失去了组件化原本的意义。

拆分成独立模块业务，彼此之间是平级的关系，无依赖关系，从而从结构层面达成了分离的目的，避免了之前一不小心就出现的类相互引用，耦合严重的问题。



业务模块拆分成独立的 Library 以后，就是对其进行配置，这也是组件化的关键步骤，既要使得各个业务模块可以独立运行，又要保证各个模块作为整体 App 的一部分，关键就在于不同场景下给每个业务模块应用不同的插件类型。

独立运行时，需要使用：

```
apply plugin: 'com.android.application'
```

而分独立运行时，则需要使用：

```
apply plugin: 'com.android.library'
```

为了方便业务模块的在这两种模式下的快速切换和统一调整，我们使用了以下的设置方式：

```
// 项目的 build.gradle 中配置模块是否独立运行
```

这样在切换时，只需要修改 isSingleCompile 的值，就可以在独立运行和作为模块运行之间切换。

当业务模块独立运行时，还需要配置独立的 Application 和启动页，以及一些特殊的资源文件，这里同样是根据 isSingleCompile 的值来配置 sourceSets 中的属性：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
sourceSets {
```

这里配置内容不再赘述，可以参考 Android 官网的说明进行设置，主要是针对独立运行时配置 Manifest 文件和添加入口页面的调整。

3.1.2.2.3 业务间的通信

对于拆分成独立部分的业务模块而言，彼此业务出现关联的场景还是比较多的，比如火车票推荐酒店，就需要从火车票模块跳转到酒店模块。

对于这样的业务场景，除了之前提到的将部分业务下沉到 BaseModule 以外，针对页面间跳转，我们采用了路由的方式。由于跳转的场景可能是原生页面、网页和 React Native 页面，我们制定了一套规则来进行通用的跳转。

跳转的链接按照以下的格式来实现：

```
sy://suanya.cn/xxx?url=xxx&type=1
```

其中的 type 则是跳转的类型， url 参数的值就是实际的跳转的地址。对于 网页和 React Native 页面，url 的值是比较容易直观的，对于原生页面，则引入了 ARouter 实现，对于需要传递的参数的场景，则采用 query parameters 的方式进行传递，在统一的地方进行处理，转换成 ARouter 传参的形式。

3.1.2.3 组件化架构调整中遇到的一些问题

3.1.2.3.1 业务模块的 Manifest 文件维护

在之前提到，业务模块独立运行时需要指定 Application 和启动页面，Manifest 文件内容如下：

```
<application
```

其中的 ModuleApplication、ModuleLaunchActivity 和 ModuleHomeActivity 合并打包时，根据 sourceSets 的设置，是不会编译进来的，需要调整 AndroidManifest 文件，最简单的办法就是写两份 AndroidManifest 文件，通过 sourceSets 中的 manifest.srcFile 来指定。但是这样存在一个问题，例如添加一个 ModuleXXXActivity，这个在独立和非独立运行时都需要的 Activity，则需要在两个 AndroidManifest 中都添加一次，这样显然是不够合理的，对开发而言是不友好的。

我们通过 manifest merge 规则找到解决办法，业务模块只需要维护独立运行时的一份 AndroidManifest 文件即可，在合并的 App 的 AndroidManifest 中，对 application 节点，使用 replace 操作：

```
<application
```

而对于只有模块独立运行才使用到的 activity ，则采用 remove 操作进行移除：

```
<activity
```

如此操作之后，就可以保证最终合并打包时，业务模块设置的 application 被替换成 app 的，而模块独立运行才应用到的 activity 则被移除了，只需要维护一份模块的 AndroidManifest 文件即可。

3.1.2.3.2 多应用差异化配置打包

在前面也提到，我们的业务场景对于同一个项目打包出不同的应用，这种需求的我们使用 ProductFlavors 进行了实现。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

通过设置不同的 ProductFlavors，通过 manifestPlaceholders 来配置每个应用差异化的参数，例如接入微信的 appId，地图的 key 等。对于每个应用使用不同的主题色和资源问题，则采用在对应的 ProductFlavors 文件夹中以同名文件、同名资源名称的方式进行覆盖设置，这种同名的资源最终以 ProductFlavors 文件夹中的设置为准。

除此以外，还需要针对每个应用的签名配置进行设置：

```
productFlavors.all { flavor ->
```

需要定义 getSigningConfigsByFlavorName 方法来根据 flavor name 获取到对应的 signingConfigs。

3.1.3 组件化架构的实践成果

根据之前设定的目标，组件化调整后基本都完成的预期的目标：

1) 业务模块分离，从结构层面做到了代码隔离，减少了之前不必要的耦合

2) 基础组件的拆分，按照业务和功能拆分出基础组件，便于后期开发和维护

3) 简单实现了业务间的通信，实现了跨模块的多种类型的通用跳转

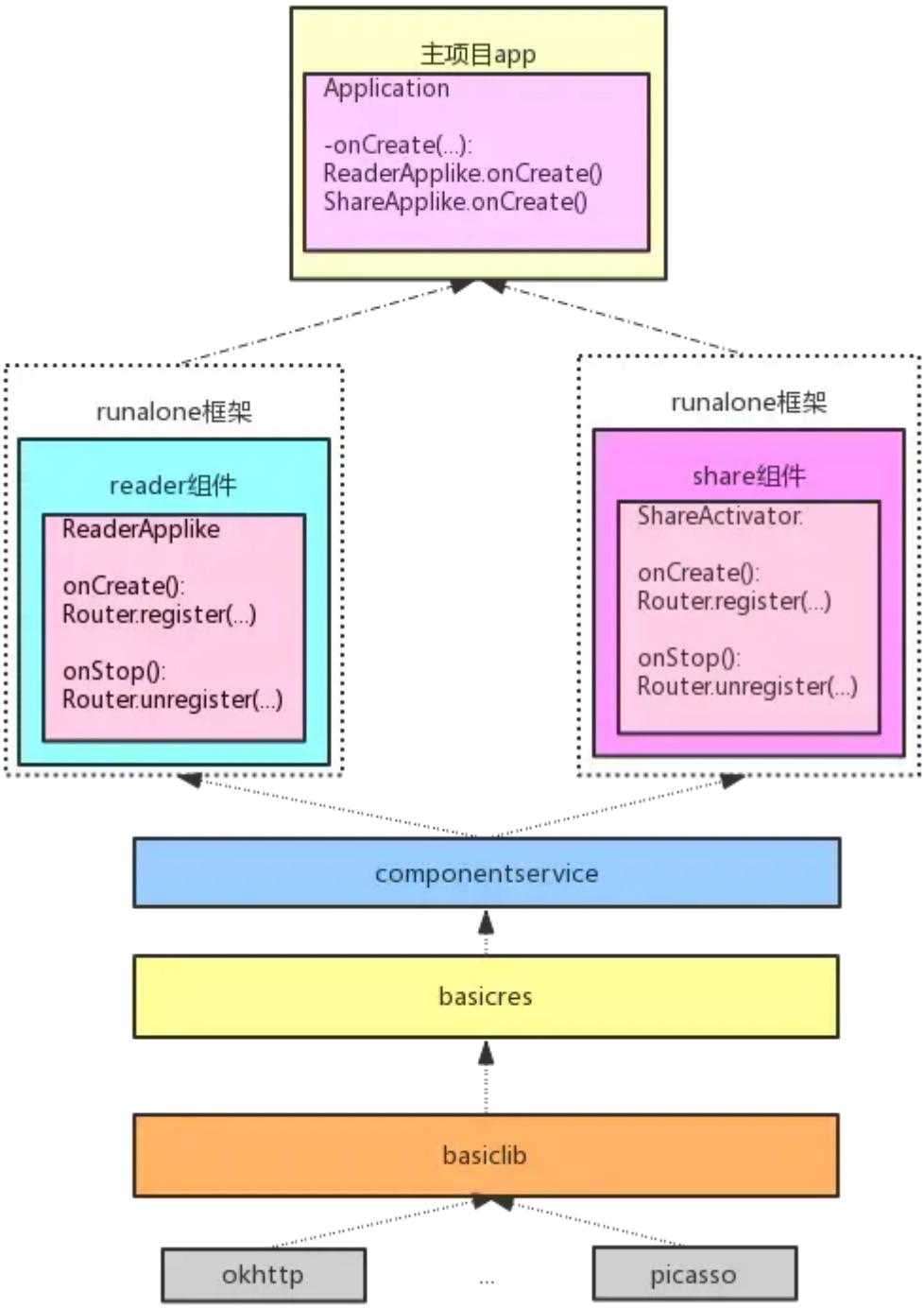
4) 实现同一个项目多应用差异化适配打包，支持主题适配等

整体项目进行组件化调整以后，模块的划分更为清晰，结构上实现了代码隔离，减少了耦合。业务模块支持独立运行和整体打包，单个模块完整编译耗时约 20 秒左右，合并打包完整编译整个项目耗时约 1 分钟，极大地提升了开发效率。

3.2 得到 App

- 提到了组件生命周期、服务注册的实现
- 提到了公共层定义组件服务、base层定义通用资源
- 提到了 implementation 与 runtimeOnly 的代码 / 资源隔离效果；
- 提到了 JIMU 插件的调试切换、智能配置功能；
- 提到了 2 种调用组件声明周期的方法： javassist 和反射；
- 提到了有序初始化组件的解决方案：StartUp、DAU

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



3.2.1 Android彻底组件化demo发布

得到Android组件化方案已经开源，参见Android组件化方案开源。方案的解读文章是一个小的系列，这是系列的第二篇文章： 1、Android彻底组件化方案实践 2、Android彻底组件化demo发布 3、Android彻底组件化-代码和资源隔离 4、Android彻底组件化—UI跳转升级改造 5、Android彻底组件化—如何使用Arouter

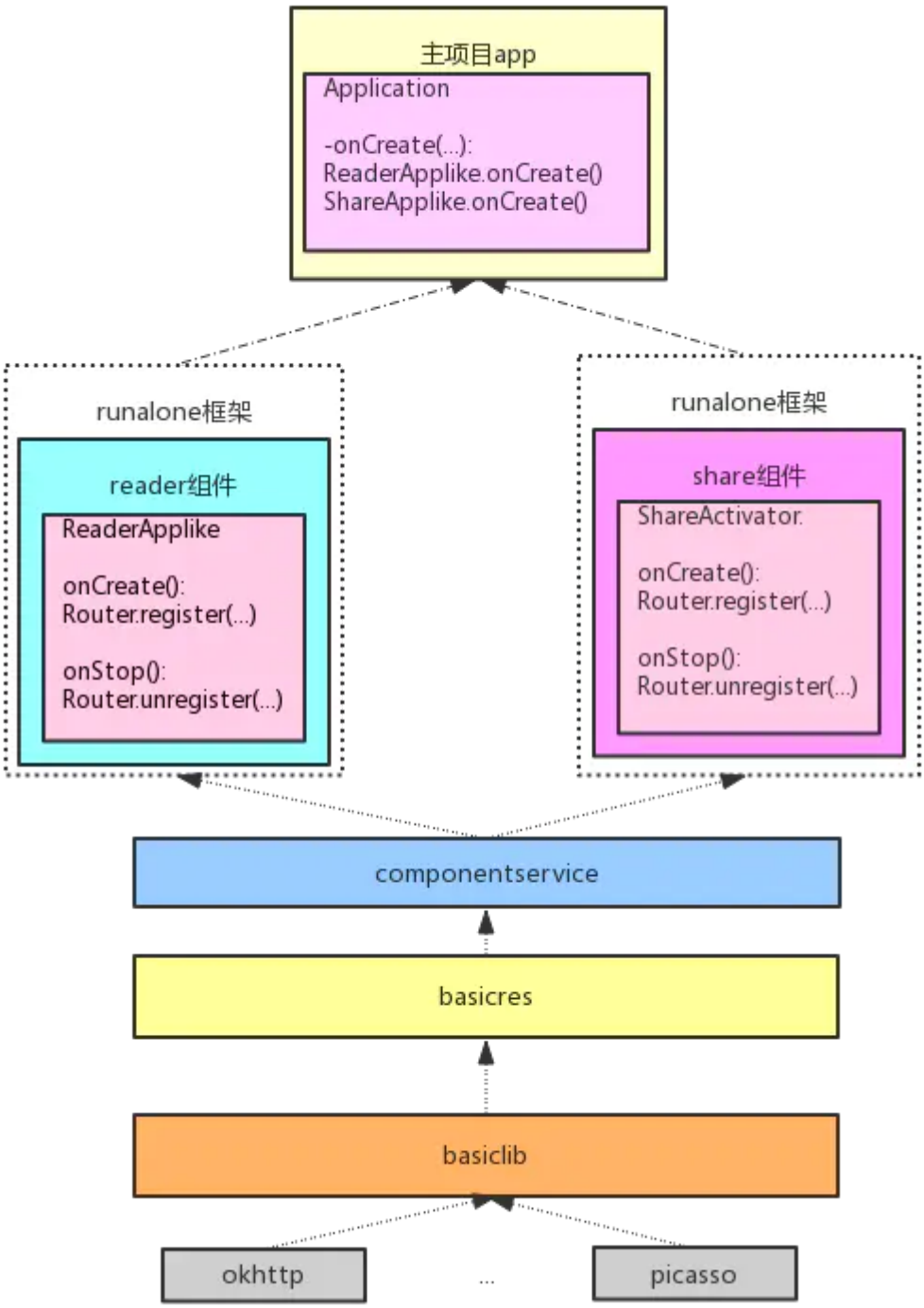
腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

今年6月份开始，我开始负责对“得到app”的android代码进行组件化拆分，在动手之前我查阅了很多组件化或者模块化的文章，虽然有一些收获，但是很少有文章能够给出一个整体且有效的方案，大部分文章都只停留在组件单独调试的层面上，涉及组件之间的交互就很少了，更不用说组件生命周期、集成调试和代码边界这些最棘手的问题了。有感于此，我觉得很有必要设计一套完整的组件化方案，经过几周的思考，反复的推倒重建，终于形成了一个完整的思路，整理在我的第一篇文章中Android彻底组件化方案实践。这两个月以来，得到的Android团队按照这个方案开始了组件化的拆分，经过两个开发周期的努力，目前已经拆分五个大的业务组件以及数个底层lib库，并对之前的方案进行了一些完善。从使用效果上来看，这套方案完全可以达到了我们之前对组件化的预期，并且架构简单，学习成本低，对于一个急需快速组件化拆分的项目是很适合的。现在将这套方案开源出来，欢迎大家共同完善。代码地址：https://github.com/mqzhangw/JIMU

3.2.1.1 JIMU使用指南

首先我们看一下demo的代码结构，然后根据这个结构图再次从单独调试（发布）、组件交互、UI跳转、集成调试、代码边界和生命周期等六个方面深入分析，之所以说“再次”，是因为上一篇文章我们已经讲了这六个方面的原理，这篇文章更侧重其具体实现。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



代码中的各个module基本和图中对应，从上到下依次是：

- app是主项目，负责集成众多组件，控制组件的生命周期
- reader和share是我们拆分的两个组件
- componentservice中定义了所有的组件提供的服务
- basicres定义了全局通用的theme和color等公共资源
- basiclib中是公共的基础库，一些第三方的库（okhttp等）也统一交给basiclib来引入

图中没有体现的module有两个，一个是componentlib，这个是我们组件化的基础库，像Router/UIRouter等都定义在这里；另一个是build-gradle，这个是我们组件化编译的gradle插件，也是整个组件化方案的核心。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

我们在demo中要实现的场景是：主项目app集成reader和share两个组件，其中reader提供一个读书的fragment给app调用（组件交互），share提供一个activity来给reader来调用（UI跳转）。主项目app可以动态的添加和卸载share组件（生命周期）。而集成调试和代码边界是通过build-gradle插件来实现的。

1 单独调试和发布

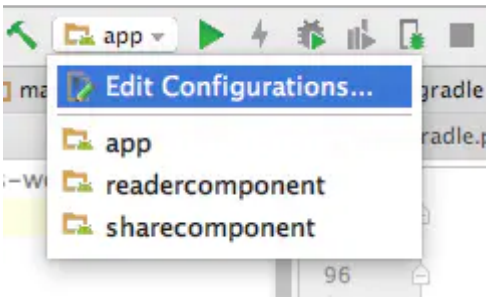
单独调试的配置与上篇文章基本一致，通过在组件工程下的gradle.properties文件中设置一个isRunAlone的变量来区分不同的场景，唯一的不同点是在组件的build.gradle中不需要写下面的样板代码：

```
if(isRunAlone.toBoolean()){
apply plugin: 'com.android.application'
}else{
 apply plugin: 'com.android.library'
}
```

而只需要引入一个插件com.dd.comgradle（源码就在build-gradle）,在这个插件中会自动判断apply com.android.library还是com.android.application。实际上这个插件还能做更"智能"的事情，这个在集成调试章节中会详细阐述。

单独调试所必须的AndroidManifest.xml、application、入口activity等类定义在src/main/runalone下面，这个比较简单就不赘述了。

如果组件开发并测试完成，需要发布一个release版本的aar文件到中央仓库，只需要把isRunAlone修改为false，然后运行module:assembleRelease命令就可以了。这里简单起见没有进行版本管理，大家如果需要自己加上就好了。值得注意的是，发布组件是唯一需要修改isRunAlone=false的情况，即使后面将组件集成到app中，也不需要修改isRunAlone的值，既保持isRunAlone=true即可。所以实际上在Androidstudio中，是可以看到三个application工程的，随便点击一个都是可以独立运行的，并且可以根据配置引入其他需要依赖的组件。这背后的工作都由com.dd.comgradle插件来默默完成。



项目中有三个application工程.png

2 组件交互

在这里组件的交互专指组件之间的数据传输，在我们的方案中使用的是接口+实现的方式，组件之间完全面向接口编程。

在demo中我们让reader提供一个fragment给app使用来说明。首先reader组件在componentservice中定义自己的服务

```
public interface ReadBookService {
    Fragment getReadBookFragment();
}
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

然后在自己的组件工程中，提供具体的实现类ReadBookServiceImpl：

```
public class ReadBookServiceImpl implements ReadBookService {
    @Override
    public Fragment getReadBookFragment() {
        return new ReaderFragment();
    }
}
```

提供了具体的实现类之后，需要在组件加载的时候把实现类注册到Router中，具体的代码在ReaderAppLike中，ReaderAppLike相当于组件的application类，这里定义了onCreate和onStop两个生命周期方法，对应组件的加载和卸载。

```
public class ReaderAppLike implements IApplicationLike {
    Router router = Router.getInstance();
    @Override
    public void onCreate() {
        router.addService(ReadBookService.class.getSimpleName(), new ReadBookServiceImpl());
    }
    @Override
    public void onStop() {
        router.removeService(ReadBookService.class.getSimpleName());
    }
}
```

在app中如何使用如reader组件提供的ReaderFragment呢？注意此处app是看不到组件的任何实现类的，它只能看到componentservice中定义的ReadBookService，所以只能面向ReadBookService来编程。具体的实例代码如下：

```
Router router = Router.getInstance();
if (router.getService(ReadBookService.class.getSimpleName()) != null) {
    ReadBookService service = (ReadBookService)
router.getService(ReadBookService.class.getSimpleName());
    fragment = service.getReadBookFragment();
    ft = getSupportFragmentManager().beginTransaction();
    ft.add(R.id.tab_content, fragment).commitAllowingStateLoss();
}
```

这里需要注意的是由于组件是可以动态加载和卸载的，因此在使用ReadBookService的需要进行判空处理。我们看到数据的传输是通过一个中央路由Router来实现的，这个Router的实现其实很简单，其本质就是一个HashMap，具体代码大家参见源码。

通过上面几个步骤就可以轻松实现组件之间的交互，由于是面向接口，所以组件之间是完全解耦的。至于如何让组件之间在编译阶段不不可见，是通过上文所说的com.dd.comgradle实现的，这个在第一篇文章中已经讲到，后面会贴出具体的代码。

3 UI跳转

页面（activity）的跳转也是通过一个中央路由UIRouter来实现，不同的是这里增加了一个优先级的概念。具体的实现就不在这里赘述了，代码还是很清晰的。

具体的功能介绍和使用规范，请大家参见文章： android彻底组件化—UI跳转升级改造

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

4 集成调试

集成调试可以认为由app或者其他组件充当host的角色，引入其他相关的组件一起参与编译，从而测试整个交互流程。在demo中app和reader都可以充当host的角色。在这里我们以app为例。

首先我们需要在根项目的gradle.properties中增加一个变量mainmodulename，其值就是工程中的主项目，这里是app。设置为mainmodulename的module，其isRunAlone永远是true。

然后在app项目的gradle.properties文件中增加两个变量：

```
debugComponent=readercomponent,com.mrzhang.share:sharecomponent
compileComponent=readercomponent,sharecomponent
```

其中debugComponent是运行debug的时候引入的组件，compileComponent是release模式下引入的组件。我们可以看到debugComponent引入的两个组件写法是不同的，这是因为组件引入支持两种语法，module或者modulePackage:module，前者直接引用module工程，后者使用componentrelease中已经发布的aar。

注意在集成调试中，要引入的reader和share组件是不需要把自己的isRunAlone修改为false的。我们知道一个application工程是不能直接引用（compile）另一个application工程的，所以如果app和组件都是isRunAlone=true的话在正常情况下是编译不过的。秘密就在于com.dd.comgradle会自动识别当前要调试的具体是哪个组件，然后把其他组件默默的修改为library工程，这个修改只在当次编译生效。

如何判断当前要运行的是app还是哪个组件呢？这个是通过task来判断的，判断的规则如下：

- assembleRelease → app
- app:assembleRelease或者 :app:assembleRelease → app
- sharecomponent:assembleRelease 或者:sharecomponent:assembleRelease→ sharecomponent

上面的内容要实现的目的就是每个组件可以直接在Androidstudio中run，也可以使用命令进行打包，这期间不需要修改任何配置，却可以自动引入依赖的组件。这在开发中可以极大加快工作效率。

5 代码边界

至于依赖的组件是如何集成到host中的，其本质还是直接使用compile project（...）或者compile modulePackage:module@aar。那么为啥不直接在build.gradle中直接引入呢，而要经过com.dd.comgradle这个插件来进行诸多复杂的操作？原因在第一篇文章中也讲到了，那就是组件之间的完全隔离，也可以称之为代码边界。如果我们直接compile组件，那么组件的所有实现类就完全暴露出来了，使用方就可以直接引入实现类来编程，从而绕过了面向接口编程的约束。这样就完全失去了解耦的效果了，可谓前功尽弃。

那么如何解决这个问题呢？我们的解决方式还是从分析task入手，只有在assemble任务的时候才进行compile引入。这样在代码的开发期间，组件是完全不可见的，因此就杜绝了犯错误的机会。具体的代码如下：

```
  /**
 * 自动添加依赖，只在运行assemble任务的才会添加依赖，因此在开发期间组件之间是完全感知不到的，这是做到完全隔离的关键
 * 支持两种语法：module或者modulePackage:module,前者之间引用module工程，后者使用componentrelease中已经发布的aar
 * @param assembleTask
 * @param project
 */
private void compileComponents(AssembleTask assembleTask, Project project) {
    String components;
    if (assembleTask.isDebug) {
        components = (String) project.properties.get("debugComponent")
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
    } else {
        components = (String) project.properties.get("compileComponent")
    }
    if (components == null || components.length() == 0) {
        return;
    }
    String[] compileComponents = components.split(",")
    if (compileComponents == null || compileComponents.length == 0) {
        return;
    }
    for (String str : compileComponents) {
        if (str.contains(":")) {
            File file = project.file("../componentrelease/" + str.split(":")[1] + "-
release.aar")
            if (file.exists()) {
                project.dependencies.add("compile", str + "-release@aar")
            } else {
                throw new RuntimeException(str + " not found ! maybe you should generate a
new one ")
            }
        } else {
            project.dependencies.add("compile", project.project(':' + str))
        }
    }
}
```

6 生命周期

在上一篇文章中我们就讲过，组件化和插件化的唯一区别是组件化不能动态的添加和修改组件，但是对于已经参与编译的组件是可以动态的加载和卸载的，甚至是降维的。

首先我们看组件的加载，使用章节5中的集成调试，可以在打包的时候把依赖的组件参与编译，此时你反编译apk的代码会看到各个组件的代码和资源都已经包含在包里面。但是由于每个组件的唯一入口ApplicationLike还没有执行oncreate()方法，所以组件并没有把自己的服务注册到中央路由，因此组件实际上是不可达的。

在什么时机加载组件以及如何加载组件？目前com.dd.comgradle提供了两种方式，字节码插入和反射调用。

- 字节码插入模式是在dex生成之前，扫描所有的ApplicationLike类（其有一个共同的父类），然后通过javassist在主项目的Application.onCreate()中插入调用ApplicationLike.onCreate()的代码。这样就相当于每个组件在application启动的时候就加载起来了。
- 反射调用的方式是手动在Application.onCreate()中或者在其他合适的时机手动通过反射的方式来调用ApplicationLike.onCreate()。之所以提供这种方式原因有两个：对代码进行扫描和插入会增加编译的时间，特别在debug的时候会影响效率，并且这种模式对Instant Run支持不好；另一个原因是可以更灵活的控制加载或者卸载时机。

这两种模式的配置是通过配置com.dd.comgradle的Extension来实现的，下面是字节码插入的模式下的配置格式，添加applicationName的目的是加快定位Application的速度。

```
combuild {
    applicationName = 'com.mrzhang.component.application.AppApplication'
    isRegisterCompoAuto = true
}
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?tid=8674704361863085

demo中也给出了通过反射来加载和卸载组件的实例，在APP的首页有两个按钮，一个是加载分享组件，另一个是卸载分享组件，在运行时可以任意的点击按钮从而加载或卸载组件，具体效果大家可以运行demo查看。

3.2.1.2 组件化拆分的感悟

在最近两个月的组件化拆分中，终于体会到了做到剥丝抽茧是多么艰难的事情。确定一个方案固然重要，更重要的是克服重重困难坚定的实施下去。在拆分中，组件化方案也不断的微调，到现在终于可以欣慰的说，这个方案是经历过考验的，第一它学习成本比较低，组内同事可以快速的入手，第二它效果明显，得到本来run一次需要8到10分钟时间（不过后面换了顶配mac，速度提升了很多），现在单个组件可以做到1分钟左右。最主要的是代码结构清晰了很多，这位后期的并行开发和插件化奠定了坚实的基础。

总之，如果你面前也是一个庞大的工程，建议你使用该方案，以最小的代价尽快开始实施组件化。如果你现在负责的是一个开发初期的项目，代码量还不大，那么也建议尽快进行组件化的规划，不要给未来的自己增加徒劳的工作量。

本文提出的Android组件化方案已经开源，参见Android组件化方案开源项目

3.2.2 Android彻底组件化—代码和资源隔离

得到Android组件化方案已经开源，参见Android组件化方案开源。方案的解读文章是一个小的系列，这是系列的第三篇文章： 1、Android彻底组件化方案实践 2、Android彻底组件化demo发布 3、Android彻底组件化-代码和资源隔离 4、Android彻底组件化—UI跳转升级改造 5、Android彻底组件化—如何使用Arouter

最近Google正式推出AS3.0版本，同时gradle插件也升级为3.0.0，目前各大开源库都在做gradle3.0.0的兼容，我也把得到开源的组件化方案JIMU进行了升级，结论是：JIMU在**gradle3.0上是没有兼容问题的，可以直接使用**。关于如何迁移到gradle3.0.0，请参见官方迁移指南。

虽然没有兼容问题，但在升级的过程中也收获了意外之喜，那就是发现gradle3.0.0对代码隔离的支持越来越好。为什么对"代码隔离"这么关注呢？大家可以回顾前两篇文章Android彻底组件化方案实践和Android彻底组件化demo发布，在这两篇文章中提到的JIMU组件化方案，被我冠以"彻底"二字，虽然有些说大话，但主要是为了强调JIMU与之前其他组件化方案的不同之处就在于，JIMU实现了组件之间的绝对隔离，不同组件之间在代码开发阶段是完全不可见的，是一种彻底解耦的思想。为了实现这种隔离，我人为在编译和运行期做了一次判断和区分，既在编译期间（开发期间）组件之间没有任何依赖关系，但在打包和运行时，再偷偷添加依赖。具体可以参见前两篇文章和github源码。

不得不说，当时这种实现是迫不得已，我本来想直接使用gradle提供的功能来做这种隔离，其实gradle也的确提供一个类似的功能，那就是apk依赖语法，其作用就是保证依赖库只在运行期间对外可见，但在编译期间是不可见的。按说这已经满足我的要求了，但是遇到了一个坑：在gradle2.+版本，apk依赖只能是jar，不能是aar，但是我们的组件因为含有各种资源，输出产物就是aar！所以最终选择了放弃apk这种语法。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?tid=8674704361863085

而在最新的gradle3.0.0上，apk被替换为runtimeOnly语法，其作用还是一样的，但是我发现**runtimeOnly可以添加aar依赖**！这的确让我很兴奋，这不就是我梦寐以求的功能吗？有了这个尚方宝剑，组件化的方案就可以做的更薄了啊。于是我对在得到app上进行了实验，结论是：**runtimeOnly的确可以解决一些问题，但是还不够。**下面我从代码隔离、资源隔离和调试切换（单独和集成）三个方便仔细阐述，也顺便再讲一下JIMU所能实现的功能。

3.2.2.1 代码隔离

在讲代码隔离之前，先大致看一下gradle3.0.0对添加依赖的语法变化。

首先compile被废弃了，而是分成了两个：implementation和api，其中api与之前的compile功能基本一致，不再赘述；implementation就比较高级了，其作用就是，使用implementation添加的依赖不会再编译期间被其他组件引用到，但在运行期间是完全可见的。这也是一种代码隔离。举个例子，

```
组件A依赖lib1，既A implementation lib1
组件B依赖组件A，既B api A
```

在gradle3.0.0之前，B是完全可以引用到lib1里面的类的，但是现在B在编译期间就做不到了，只能在运行期可以。这种思想有点类似于“下属的下属不是你的下属”的思想。但是这种隔离在组件之间是不起作用的，在上面的例子中A的所有类对B还是完全可见的，也就是没有做任何隔离的。不过implementation的确是一种有效减少编译时间的方式，还是上面的例子，lib1发生了变化，现在只需要编译A就可以了，而在之前B有可能也使用到了lib1，所以需要同时编译B和A。按照官方建议，大部分情况下都应该使用implementation来进行添加依赖。

此外还有两种变化，原来的apk语法被runtimeOnly取代，provided被compileOnly取代，其作用还是没变。上文也讲了，runtimeOnly有个极大的改动就是可以支持aar了，但是compileOnly还是只能支持jar！

先做一个小结，目前gradle3.0.0的四种语法的功能和代码隔离效果见下图：

语法	旧语法	功能	支持类型	代码隔离效果
implementation	compile	编译期间对其他组件不可见 运行期间可见	jar aar	“隔代”编译期间隔离
api	compile	编译和运行期间都可见	jar aar	没有隔离
compileOnly	provided	只参与编译，不打包到apk	jar	没有隔离
runtimeOnly	apk	编译期间不可见，参与打包到apk 运行期间可见	jar aar	编译期间隔离

四种语法的功能和代码隔离效果

从上图可以看出，在代码隔离效果上，runtimeOnly的效果是最好的！**但是就可以直接使用了吗，答案是否定的。**

3.2.2.2 资源隔离

在前面的文章中，一直在强调代码隔离，其实组件之间的完全隔离还有一层就是资源隔离，否则还是容易造成组件之间的耦合。这个在文章的“单独调试”章节中提到了一句，就是每个组件都需要指定一个资源前缀resourcePrefix，以避免集成后资源名冲突的问题。也就是说，一个彻底的组件化不仅要做到代码不能直接引用，资源也是不能引用的！

但是runtimeOnly目前还做到资源隔离，我在JIMU的开源库上做了试验，app通过runtimeOnly引用sharecomponent组件，虽然sharecomponent的代码是不可见了，但是资源还是可以被app直接使用的并能成功运行。

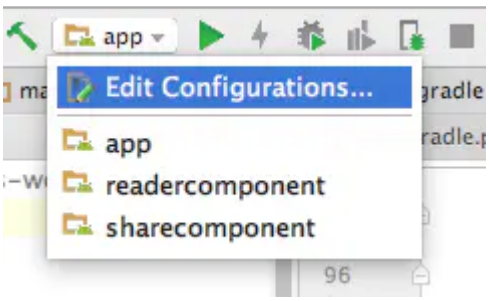
腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

从这一点上看，直接替换成runtimeOnly是不行的，为了达到这种效果，目前还是需要像JIMU一样，人为的加一层控制，所以从组件化方案的角度上看并没有变的更薄，不过幸好JIMU已经很简单了，有一定的gradle基础的人可以比较容易的理解。

3.2.2.3 调试切换

除了上面说的资源隔离导致不能直接用runtimeOnly之外，还有一个使用上的问题需要解决，这也是JIMU中compbuild插件提供的一个功能：自动切换单独调试和集成调试。在单独调试时，组件是一个application工程，其输出产物是apk文件，而在集成调试时，被依赖的组件是一个library工程，其输出产物是aar文件。对于runtimeOnly来说，对aar和jar是支持的，但是不能支持apk，所以如果想在单独调试和集成调试之间切换的话，需要人工修改runalone配置并修改build.gradle配置文件，然后还需要sync之后才能生效，这种修改是相当繁琐的。

在JIMU中，这个问题的解决是通过"智能"识别当前要调试的组件来解决的，对于要调试的组件将其设置为application工程，而将其依赖的其他组件默默修改为library工程，这种修改是即时生效的，对开发者是完全透明的。开发者直接点击AS的run功能区就可以随意的调试任意组件。AS的run功能区的图如下：



随意的调试切换

3.2.2.4 总结

综上所述，我们对JIMU和gradle3.0.0做几点总结：（1）升级到gradle3.0.0之后，可以继续使用JIMU，不需要专门做兼容（2）gradle3.0.0提供了implementation和runtimeOnly两种语法，它们都能实现一定程度的代码隔离效果，建议大家在今后优先使用（3）implementation和runtimeOnly目前在资源隔离和调试切换上还不能满足组件化的要求，所以还是需要使用JIMU提供的完全隔离和随意切换功能。

在JIMU的源码中我增加了gradle3.0.0分支，依赖语法做了相应的替换源码地址：DDComponent

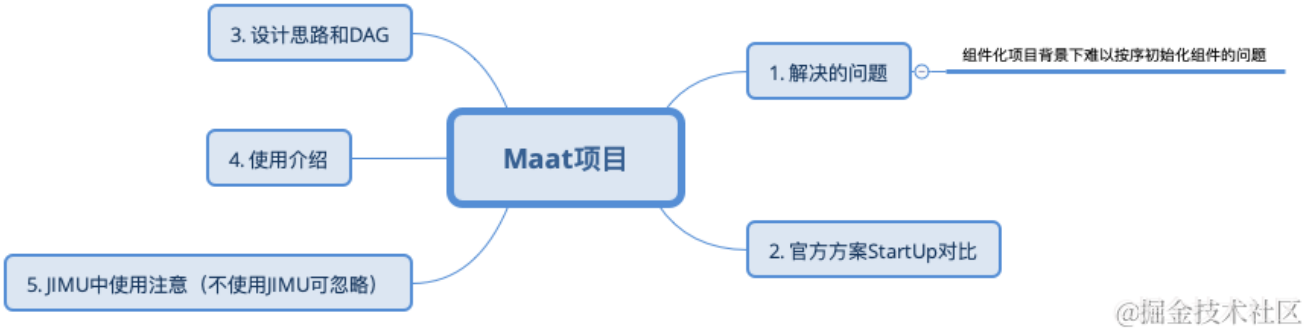
3.2.3 组件化：代码隔离也难不倒组件的按序初始化

3.2.3.1 前言

时至今日，Android项目中的组件化大家都已经非常熟悉了，但在各个细节方面还是有一些门门道道的内容，如果没有趁手的中间件支持，推行组件化的过程中还是会遇到阻碍。继2017年逻辑思维得到项目团队开源其组件化方案思路和核心gradle构建插件后，笔者一直投身其中并致力于**插件的功能升级和中间件生态完善**。

其实在2018年就有部分同学提出了"增加按序初始化组件"的需求，限于个人精力以及需求的优先级，当时被搁置了，这个需求一拖也就拖了两年了。这次终于抽出时间完成了这个中间件，特此写了一篇博客，介绍其中的一些知识点和这个中间件Maat。Github仓库

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



方便阅读，导图附上。

注：即使你没有使用上面提到的得到组件化方案（DDComponentForAndroid）或者我们后续维护的JIMU，也不影响本篇文章的理解。

3.2.3.2 问题的根源

这里我们再花一点时间来了解下问题的根源：组件化的基础是模块化，在做到模块化的同时，模块与模块在编写、编译期间也就达成了完全代码隔离，组件间的交互依靠 **底层接口+服务发现（或者服务注册）** 或者更加抽象为 **“基于协议、隐藏实现”**。这带来了编写、编译期间激增的代码耦合 *（注：此处语境遗漏，在达成编写、编译期间完全代码隔离的条件下，想要用比较原始的、直面问题的方式解决组件按序初始化问题，例如使用反射+无分支遗漏的逻辑涵盖所有组件组合情况，会导致耦合激增。2020年10月20日补）*。

我知道这样说实在是太晦涩了，一点也不接地气，我们以一个简单的例子来配合说明。

```
interface IComponent {
    fun onCreate()
    fun onDestroy()
}
```

我们定义这样的接口来代表一个组件模型。案例设定为：一个宿主H+两个互无关联的组件A、B

那么有：

```
class A : IComponent {
    override fun onCreate() {
        // A初始化逻辑
    }

    override fun onDestroy() {
    }
}

class B : IComponent {
    override fun onCreate() {
        // B初始化逻辑
    }

    override fun onDestroy() {
    }
}
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

另有

```
class H :Application {
    override fun onCreate() {
        A().onCreate()
        B().onCreate()
    }
}
```

我们以最简单的代码演示组件的加载和初始化环节。**这里隐藏了一个问题：如果是手工编码，那么是存在代码边界的，编写、编译期间H无法直接访问A和B，我们只能通过反射去实现（否则编译不通过）。当然，也可以通过字节码技术实现**

如果我们要让B先于A初始化，那么就调整其顺序，这对于手工编码方式而言，可能就是将编码变为：

```
XXX.loadComponent("Bpackage.B") //"Bpackage.B"为B的类路径
XXX.loadComponent("Apackage.A")
复制代码
```

而利用字节码技术的，则需要增加排序功能或者读取全量配置功能。

案例2： 此时A组件依赖于B，必须等B组件初始化成功并得到结果后才能初始化。

思路1：先加载和初始化B，利用代码同步的特性，再初始化A

思路2：先加载和初始化B，修改组件模型，增加callback作为入参，异步初始化A

思路1存在很大的限制，比如其初始化需要参与网络通信或者数据库操作；思路2对于手工编码来说，会产生回调地狱，而对于字节码技术实现而言，就是一个噩梦

而且，JIMU已经投入使用挺长一段时间了，如果不是毫无选择，对于基类或者接口做无法版本兼容的操作都不应该被采纳

思路2的改进版：增加上下文，使得回调嵌套扁平化。

既然我们决定增加一个上下文，那么将初始化的管理工作进行封装就成了顺理成章的事情

3.2.3.3 为什么不使用官方StartUp而选择造轮子

在思考这个问题时，我们必须要清楚Startup的设计意图

Startup 中文介绍

> 可在应用启动时简单、高效地初始化组件。

借助 App Startup 库，可在应用启动时简单、高效地初始化组件。库开发者和应用开发者都可以使用 App Startup 来简化启动序列并显式设置初始化顺序。

我们知道，在Startup发布之前，各大SDK采用的初始化方式一般为两种：

- 显式API调用，需要Application实例
- 内部提供一个ContentProvider，并在其中获取Application实例。因为其特性，会在应用启动时被自动加载，而不再需要使用者显式的API调用

一般为了方便开发者，在manifest文件中写入SDK参数配置并利用Context（为了不造成泄漏，使用Application是最好的选择）读取配置的做法更受推荐。所以第二种方式的使用越来越多。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?tid=8674704361863085

这就带来了一个问题：引入越多的SDK就会引入更多的ContentProvider，他们并不会随着初始化工作完成而消亡，而且加重了应用启动时AMS的负担。

业内存在一个著名的编程范式：约定优于配置，既然使用ContentProvider作为初始化入口已经被广泛接受，那么Google作为生态维护者提供一个官方库，使用统一的初始化入口，使用者只需要按照约定暴露初始化逻辑，并且提供了前置依赖使得任务可排序的功能。

到这里我们就可以明白这样几件事情：

- StartUp中使用异步和其排序加载之间存在"矛盾"
- StartUp不提供依赖有向无环图校验

因为StartUp更主要的是面向SDK，提供统一标准。SDK库之间出现"存在性上的先后关系"的场景本身就非常小，如果有"依赖"，SDK生产者在库内部都处理好了，一般也不会出现代码边界。

所以，Maat并不是一个和StartUp一较长短的功能库，而是为了解决特定问题而编写的功能库。这些问题又恰恰是StartUp所不涉及的

3.2.3.4 设计思路

相信大家对"同步"和"异步"都有比较深的理解，我们先提出三个参与初始化的角色：

- **任务**： 初始化工作的最小单元，清晰的知道自己的所依赖的任务，只有依赖的任务都执行完毕后才能执行，我们以`Task=Name[dependency1,dependency2,...]`来表示任务，例如 `B[]` ==> 无前置依赖的任务B, `A[B]` ==> 任务A、依赖任务B
- **任务集**：所有任务的集合，可分析任务的所有前置依赖并判断是否存在循环依赖，对任务进行排序,记为`TaskGroup={Task1,Task2,...}`
- **任务调度器**：从任务集中取出任务派发执行的调度器

回顾我们最开始给出的例子，组件之前有存在性先后关系，*必须要让依赖的组件完成初始化后才能开始加载。* **那么任务调度器的工作方式是"同步"的，在"被依赖的任务"执行完毕前，依赖他的任务都必须阻塞等待。**

但是思考一个问题：两个互相独立的任务，必须阻塞等待吗？答案显然，不是必须的。

这里举一些例子：

有任务集：`{A[],B[],C[A,B]}`,A和B是无依赖的，C依赖任务A和B，

那么任务调度器可以按照A、B、C的顺序进行调度，

也可以按照B、A、C的顺序进行调度每个任务执行中，任务调度器都阻塞等待，

也可以让AB两个任务并发（需要分配到不同线程）阻塞等待AB均完成后调度C。*在第一个版本设计中，我还没有采用这个方案，目前让库保持足够轻量。当存在多组初始化路径时，其复杂程度远大于本处的例子*

3.2.3.4 有向无环图（DAG）

接下来我们适当花一些篇幅来讨论DAG。在我们上面提到的任务集这一角色中，我们使用了DAG来处理拓扑排序和依赖无环校验。

我们将任务看做是图中的顶点，任务的依赖关系看做是边，方向和依赖方向相反，即`A[B]`意味着有从B到A的边。将所有的任务合并起来后我们将得到一份有向图，显然，成环的依赖是不被允许的。

为了更好的理解，我们人为的添加一个虚拟的顶点Start，作为初始化任务集的第一个任务，将所有无依赖的任务人为添加一个前置依赖：Start。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

一个合法的任务集，必然没有成环的依赖，所以一定不是强连通图，在我们添加了虚拟顶点start后，其基图一定是连通图，故而合法的任务集（包含虚拟Start节点）是一个弱连通图

3.2.3.5 环校验

我们采用DFS方式递归遍历，受益于我们制定的虚拟顶点Start，我们可以直接从这个顶点开始。

定义深度集合 deepPathList，选定起始顶点S， 定义回环顶点列表 loopbackList， 定义路径列表 pathList

直接上代码 *getEdgeContainsPoint(startPoint, Type.X) 代表取出所有以startPoint为起始点的边*

```
fun recursive(startPoint: T, pathList: MutableList<T>) {
    if (pathList.contains(startPoint)) {
          loopbackList.add("${debugPathInfo(pathList)}->${startPoint.let(nameOf)}")
          return
      }
      pathList.add(startPoint)
      val edgesFromStartPoint = getEdgeContainsPoint(startPoint, Type.X)
      if (edgesFromStartPoint.isEmpty()) {
          val descList: ArrayList<T> = ArrayList(pathList.size)
          pathList.forEach { path -> descList.add(path) }
          deepPathList.add(descList)
      }
      edgesFromStartPoint.forEach {
          recursive(it.to, pathList)
      }

      pathList.remove(startPoint)
}
```

如果loopbackList不为空，则代表存在回环，回环的信息就存放在loopbackList中

3.2.3.6 契合需求的排序方式

上面我们已经提到了深度优先遍历（DFS），但是这种方式作出的拓扑排序不适合我们的需求，他适合寻找最优或者最差路径。而广度优先遍历（BFS）才契合需求。

直接给出代码：

```
private fun DAG<JOB>.bfs(): JobChunk {

    val zeroDeque = ArrayDeque<JOB>()
    val inDegrees = HashMap<JOB, Int>().apply {
        putAll(this@bfs.inDegreeCache)
    }
    inDegrees.forEach { (v, d) ->
        if (d == 0)
            zeroDeque.offer(v)
    }

    val head = JobChunk.head()
    var currentChunk = head

    val tmpDeque = ArrayDeque<JOB>()
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
    while (zeroDeque.isNotEmpty() || tmpDeque.isNotEmpty()) {
        if (zeroDeque.isEmpty()) {
            currentChunk = currentChunk.append()
            zeroDeque.addAll(tmpDeque)
            tmpDeque.clear()
        }
        zeroDeque.poll()?.let { vertex ->
            currentChunk.addJob(vertex)

            this.getEdgeContainsPoint(vertex, Type.X).forEach { edge ->
                inDegrees[edge.to] = (inDegrees[edge.to] ?: 0).minus(edge.weight).apply {
                    if (this == 0)
                        tmpDeque.offer(edge.to)
                }
            }
        }
    }
    return head
}
```

其中JubChunk是一组无关联的Job *即前文提到的初始化任务*，前面提到目前没有让任务的执行可并发，JobChunk是为了可支持并发做准备的

关于DAG的部分我们就不再花篇幅介绍了，有兴趣的同学可以自行查阅相关资料

3.2.3.7 任务的描述

先上代码：

```
abstract class JOB {
    abstract val uniqueKey: String

    abstract val dependsOn: List<String>

    abstract val dispatcher: CoroutineDispatcher

    internal fun runInit(maat: Maat) {
        MainScope().launch {
            flow {
                init(maat)
                emit(true)
            }
                .flowOn(dispatcher)
                .catch {
                    maat.onJobFailed(this@JOB,it)
                }.flowOn(Dispatchers.Main)
                .collect {
                    maat.onJobSuccess(this@JOB)
                }
        }
    }
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
    abstract fun init(maat: Maat)
}
```

考虑到kotlin已经被官方推荐很长时间了，并且在去年Retrofit已经开始支持协程，姑且认为大部分项目中都已经开始使用协程了。所以很偷懒的直接使用了协程和Flow

- uniqueKey 是当前任务名，需要人为确保唯一性
- dependsOn 是当前任务所依赖的任务的uniqueKey的集合，虽然使用了List，但是顺序无关。
- dispatcher 指定任务执行被分配到的线程类型
- fun init(maat: Maat) 实际初始化逻辑，**注意：按需求分析初始化代码块是否需要 “同步、阻塞”，如果部分代码是“异步、基于回调”且无法更改，这个实际场景（必须要异步获取结果，且该结果被另一个组件使用）想来很少见，第一个版本中我没有考虑，下个版本我会加上**

示例代码模拟了4个初始化任务，有点长，具体的使用可以看一下Demo

```
val maat = Maat.init(application = this, printChunkMax = 6,
    logger = object : Maat.Logger() {
        override val enable: Boolean = true


        override fun log(msg: String, throws: Throwable?) {
            Log.d("maat", msg, throws)
        }

    }, callback = Maat.Callback(onSuccess = {}, onFailure = { maat, job, throwable ->

    })
)

maat.append(object : JOB() {
    override val uniqueKey: String = "a"
    override val dependsOn: List<String> = emptyList()
    override val dispatcher: CoroutineDispatcher = Dispatchers.IO

    override fun init(maat: Maat) {
        Log.e(
            "maat",
            "run:" + uniqueKey + " isMain:" + (Looper.getMainLooper() == Looper.myLooper())
        )
        //test exception
//                throw NullPointerException("just a test")
    }

    override fun toString(): String {
        return uniqueKey
    }

}).append(object : JOB() {
    override val uniqueKey: String = "b"
    override val dependsOn: List<String> = arrayListOf("a")
    override val dispatcher: CoroutineDispatcher = Dispatchers.Main /* + Job()*/

    override fun init(maat: Maat) {
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
        Log.e(
            "maat",
            "run:" + uniqueKey + " isMain:" + (Looper.getMainLooper() == Looper.myLooper())
        )
    }

    override fun toString(): String {
        return uniqueKey
    }

}).append(object : JOB() {
    override val uniqueKey: String = "c"
    override val dependsOn: List<String> = arrayListOf("a")
    override val dispatcher: CoroutineDispatcher = Dispatchers.IO /* + Job()*/

    override fun init(maat: Maat) {
        Log.e(
            "maat",
            "run:" + uniqueKey + " isMain:" + (Looper.getMainLooper() == Looper.myLooper())
        )
    }

    override fun toString(): String {
        return uniqueKey
    }

}).append(object : JOB() {
    override val uniqueKey: String = "d"
    override val dependsOn: List<String> = arrayListOf("a", "b", "c")
    override val dispatcher: CoroutineDispatcher = Dispatchers.Main

    override fun init(maat: Maat) {
        Log.e(
            "maat",
            "run:" + uniqueKey + " isMain:" + (Looper.getMainLooper() == Looper.myLooper())
        )
    }

    override fun toString(): String {
        return uniqueKey
    }

}).start()
```

3.2.3.8 在JIMU中使用

JIMU是一种很彻底的组件化方案，意味着编写代码时存在代码边界，即使是空壳宿主和业务组件之间也存在。前面也提到了，JIMU是使用字节码技术织入的组件加载代码（设置为自动加载组件时），而织入的代码是在Application的onCreate最后执行。

这这一前提下，如果通过javasist实现Maat的任务设置部分，他的可维护性将很差。所以我建议将任务设置部分放在组件的初始化入口处，这样可读性和可维护性都相对好一点.

以原先的分享业务组件为例：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
public class ShareApplike implements IApplicationLike {

    UIRouter uiRouter = UIRouter.getInstance();

    @Override
    public void onCreate() {
        uiRouter.registerUI("share");
        Log.e("share","share on create");
        Maat.Companion.getDefault().append(new JOB() {
            @NotNull
            @Override
            public String getUniqueKey() {
                return "share";
            }

            @NotNull
            @Override
            public List<String> getDependsOn() {
                return Collections.singletonList("reader");
            }

            @NotNull
            @Override
            public CoroutineDispatcher getDispatcher() {
                return Dispatchers.getMain();
            }

            @Override
            public void init(@NotNull Maat maat) {
                Log.d("share", "模拟初始化share，context：" +
maat.getApplication().getClass().getName());
            }

            @Override
            public String toString() {
                return getUniqueKey();
            }
        });
    }

    @Override
    public void onStop() {
        uiRouter.unregisterUI("share");
    }
}
```

当然，务必不要忘记在Application的onCreate()中先初始化Maat:

```
Maat.Companion.init(this, 8, new Maat.Logger() {
            @Override
            public boolean getEnable() {
                return true;
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
            }

            @Override
            public void log(@NotNull String s, @Nullable Throwable throwable) {
                if (throwable != null) {
                    Log.e("maat",s,throwable);
                } else {
                    Log.d("maat",s);
                }
            }
        }, new Maat.Callback(new Function1<Maat, Unit>() {
            @Override
            public Unit invoke(Maat maat) {
                Maat.Companion.release();
                return null;
            }
        }, new Function3<Maat, JOB, Throwable, Unit>() {
            @Override
            public Unit invoke(Maat maat, JOB job, Throwable throwable) {
                return null;
            }
        }));
```

而Maat的启动API调用，自然由javasist织入了。配合最新的gradle插件 build-gradle：1.3.4方可使用，启用开关为：

```
combuild {
    useMaat = true/false
}
```

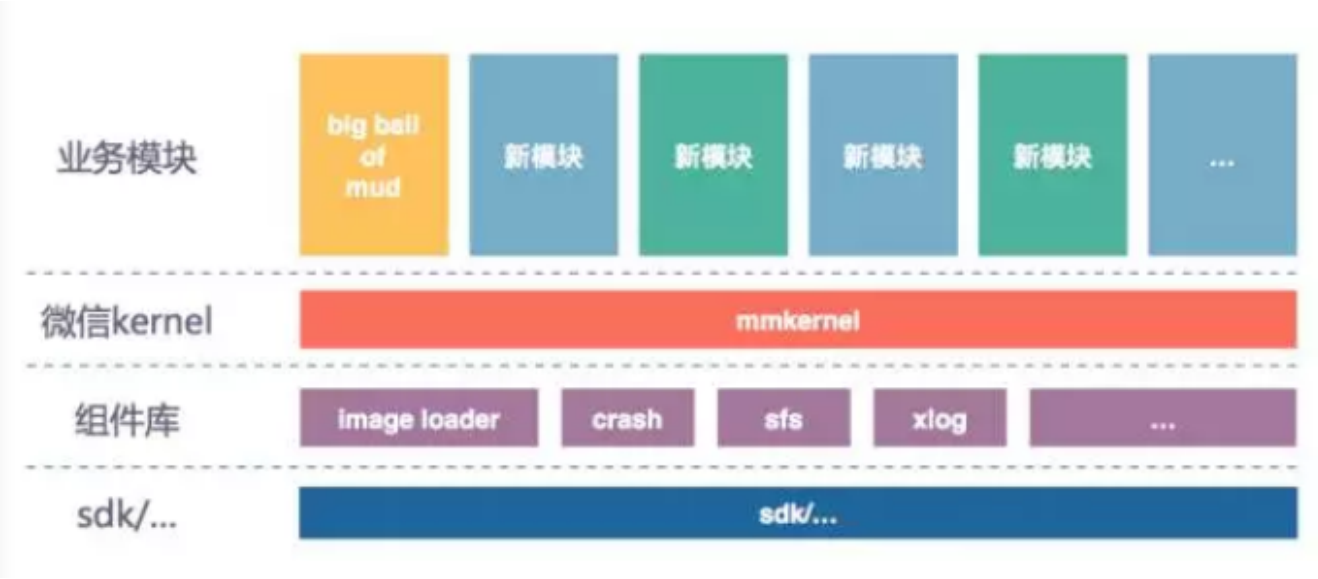
3.2.3.9 重要事项

请务必分析项目的组件初始化场景，在Maat适用你的应用场景时再使用。

3.3 微信 App

- 提到了微信架构演进：简单分层架构 -> 多进程架构 -> 多子工程并行开发架构；
- 提到了基础工程存在中心化的问题：越来越多的代码/事件很“自然的”被下沉到基础工程；
- 提出了重塑模块化的 3 个目标：改变通信方式、重新设计模块、约束代码边界
- 提出了服务注册的通信方式
- 提出了新颖的接口暴露方法：将文件后缀修改为`.api`（需配合include_with_api 命令）
- 设计了模块生命周期：dependency()、configure()、execute()
- 设计了 pins 工程结构，用于约束代码边界
- 提出了组件化的考量问题：动态性、隔离性

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



3.3.1 微信Android模块化架构重构实践

3.3.2 微信Android架构历史

微信Android诞生之初，用的是常见的分层结构设计。这种架构简单、清晰并一直沿袭至今。这是微信架构的v1.x时代。

图1-架构演进

到了微信架构的v2.x时代，随着业务的快速发展，消息通知不及时和Android 2.3版本之前webview内存泄露问题开始突显。由于代码、内存、apk大小都在增长，对系统资源的占用越来越多，导致微信进程容易被系统回收。因此微信开始转向多进程架构，独立的通信进程保持长连接的稳定性，独立的webview进程也阻隔了内存泄露导致的问题。

时间继续推进，我们也遇到了65535问题和LinearAlloc问题。这时的微信已经具备了许多功能像朋友圈、摇一摇、附近的人等等，分离核心功能和其他业务模块变得越发重要。为此，微信开启了第三次架构改造(v3.x)。我们对各种产品功能进行解耦并拆分到相互独立的p_xxx工程中，这是微信第一次进行模块化架构的重构。经过几个月的努力，微信拆出了几十个p工程，它们都通过基础组件访问网络、存储等服务，互相独立并行。新的p工程架构支撑了微信更

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

快速的业务发展，配合多分支开发模式的改进，能够支持团队多分支多team的并行开发。

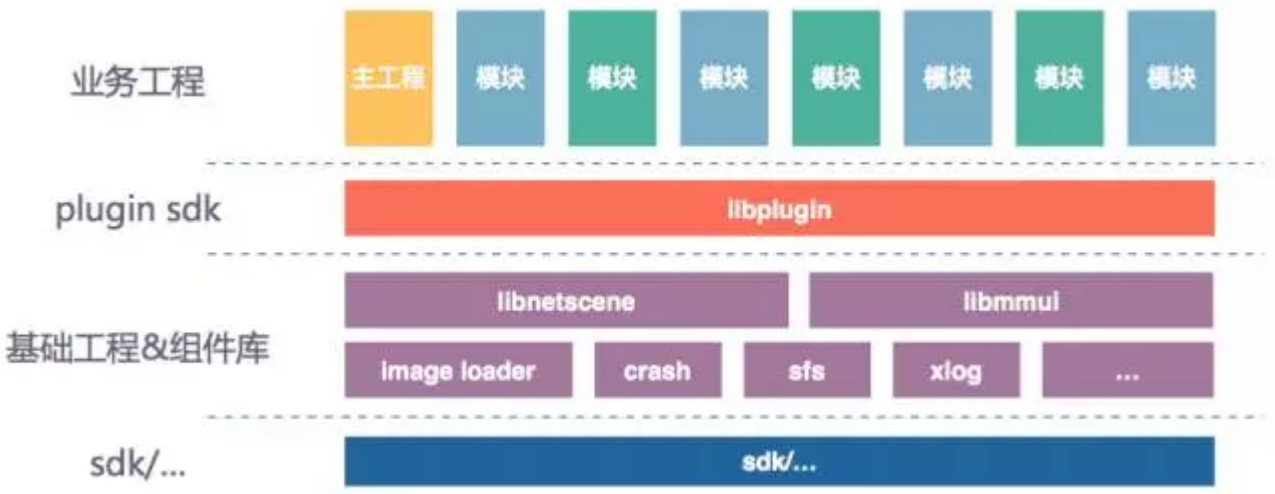


图2 - 架构图

3.3.3 为何再次重构微信

原本好好的架构出了什么问题？

从上个架构之后的两年多时间里，微信Android基本没有大的架构改动。配合gradle的编译，以及git的多分支并行开发，微信的模块工程数量不断增多，支撑了游戏、支付等大功能，可以说这段时间里原有架构起到了很好的作用。

然而随着代码继续膨胀，一些问题开始突显出来。首先出问题的是基础工程libnetscene和libplugin。基础工程一直处于不断膨胀的状态，同时主工程也在不断变大。同时基础工程存在中心化问题，许多业务Storage类被附着在一个核心类上面，久而久之这个类已经没法看了。此外当初为了平滑切换到gradle避免结构变化太大以及太多module，我们将所有工程都对接到一个module上。缺少了编译上的隔离，模块间的代码边界出现一些劣化。虽然紧接着开发了工具来限制模块间的错误依赖，但这段时间里的影响已经产生。在上面各种问题之下，许多模块已经称不上“独立”了。所以当我们重新审视代码架构时，以前良好模块化的架构设计已经逐渐变了样。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

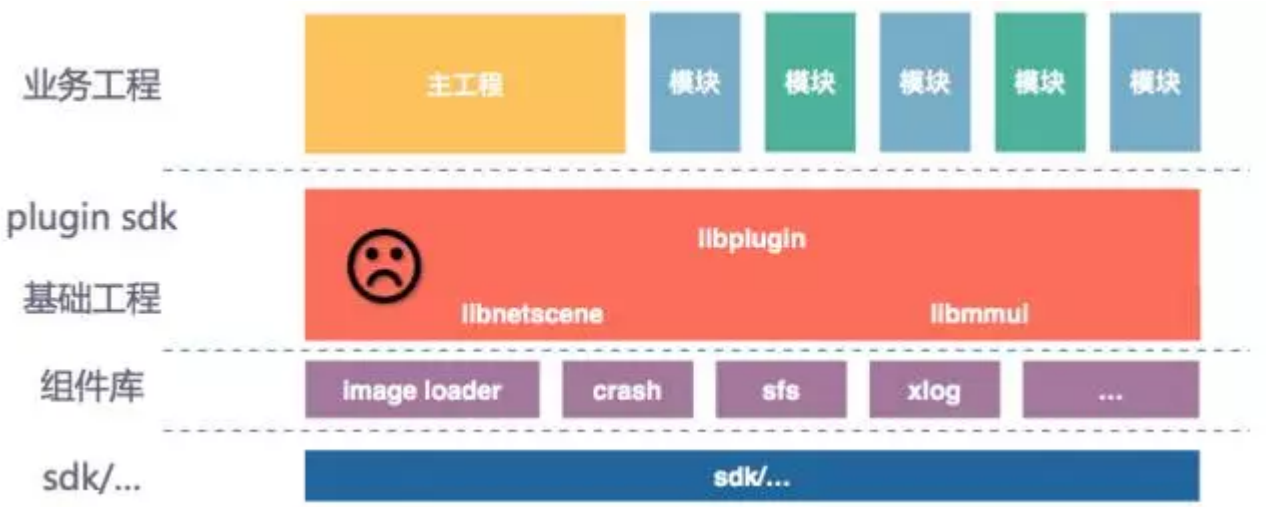


图3 - 架构逐渐的变化

“君有疾在腠理，不治将恐深”，在我们还在犹豫到底要不要重构的时候，硬件同学向我们提出了需求。希望将微信Android代码移植到类似微信相册这样产品中。这样就可以快速跟进微信业务最新的支撑组件、协议、安全性、后台服务等能力，而且代码要尽可能精简，可以选择和定制模块，可以移植模块来实现原型尝试。但就之前的情况来说，微信一时难以满足。这下定了，还得重构。

于是我们回过头仔细看之前的设计，找找问题究竟是怎么来的。

问题出在哪

先寻找代码膨胀的原因。

翻开基础工程的代码，我们看到除了符合设计初衷的存储、网络等支持组件外，还有相当多的业务相关代码。这些代码是膨胀的来源。但代码怎么来的，非要放这？一切不合理皆有背后的逻辑。在之前的架构中，我们大量适用Event事件总线作为模块间通信的方式，也基本是唯一的方式。使用Event作为通信的媒介，自然要有定义它的地方，好让模块之间都能知道Event结构是怎样的。这时候基础工程好像就成了存放Event的唯一选择——Event定义被放在基础工程中；接着，遇到某个模块A想使用模块B的数据结构类，怎么办？把类下沉到基础工程；遇到模块A想用模块B的某个接口返回个数据，Event好像不太适合？那就把代码下沉到基础工程吧……

就这样越来越多的代码很“自然的”被下沉到基础工程中。

我们再看看主工程，它膨胀的原因不一样。分析一下基本能确定的是，首先作为主干业务一直还有需求在开发，膨胀在所难免，缺少适当的内部重构但暂时不是问题的核心。另一部分原因，则是因为模块的生命周期设计好像已经不满足使用需要。之前的模块生命周期是从“Account初始化”到“Account已注销”，所以可以看出在这时机之外肯定还有逻辑。放在以前这不是个大问题，刚启动还不等“Account初始化”就要执行的逻辑哪有那么多。而现在不一样，再简单的逻辑堆积起来也会变复杂。此时，在模块生命周期外的逻辑基本上只能放主工程。

此外的问题，模块边界破坏、基础工程中心化，都是代码持续劣化的帮凶。

总之在模块化上我们忽视了一些重要的问题，必须重塑。

3.3.4 重塑模块化

重塑模块化，我们分解为三个目标：

- 改变通信方式

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

- 重新设计模块
- 约束代码边界

改变通信方式

前面讲过，我们使用Event总线作为模块间通信的媒介，这种设计很常见。然而当回顾整体代码时能发现，Event并非所有通信需要的最佳形式。它的特点适合一对多的广播场景，依赖关系弱。一旦遇到需要一组业务接口时，用Event写起来那是十分痛苦的。也正因如此，这种情况下大家都跳过了Event的使用，直接将代码下沉到了基础工程，共享代码，进而导致基础工程的不断膨胀。

所以选个合适的通信方式很有必要，我们希望兼顾考虑开发的便利性和协议的约束性。

Event不合适。协议通信如何？

我们理解的协议通信，是指跨平台/序列化的通信方式，类似终端和服务器间的通信或restful这种。现在这种形式在终端内很常见了。协议通信具备一种很强力解耦能力，但也有不可忽视的代价。无论什么形式的通信，所有的协议定义需要让通讯两方都能获知。通常为了方便会在某个公共区域存放所有协议的定义，这情况和Event引发的问题有点像。另外，协议如果变化了，两端怎么同步就变得有点复杂，至少要配合一些框架来实现。在一个应用内，这样会不会有点复杂？用起来好像也不那么方便？更何况它究竟解决多少问题呢。

所以我们想要简单点。经过权衡，我们决定用模块提供"SDK"的方式作为它与其他模块进行通信的手段。

通常"SDK"提供的是什么，是接口 + 数据结构。这种方式好处明显：实现简单也能解决问题，IDE容易补全、调用接口方便，不用配合工具，协议变化直接反映在编译上，维护接口也简单了。

其实想想，用协议的方式在终端内作为通信手段，开发效率低，也容易出错。因此可能会诞生各种框架和工具来提升这里损失的效率。到头来，是不是大家都实现了一套类似RPC这样的封装。其实本地的通信，能用接口就挺好，不能用的时候，再用协议封装也来得及。

确定了方案，实现起来就很简单。我们的注册方式和接口访问都很简单。用接口注册，再用接口访问，不暴露实现细节。如下图。

```
// 注册插件接口别名
alias(IPluginZero.class);

// 注册服务
registerService(IMyService.class, myService);
```

图4 - 注册接口

```
// 访问插件
plugin(IPluginZero.class);

// 访问服务
service(IMyService.class).doServiceWork();
```

图5 - 访问接口

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

接下来，怎么暴露接口更方便？

模块暴露“SDK”的方式无非就是新建个“SDK”工程，剥离接口和数据结构到该工程里面，然后让其他模块引用编译。但这样有点麻烦，能不能再方便点？

当然有办法。我们实现了另一种接口暴露的形式——“.api化”。

使用方式和思路都很简单。对于java文件，将工程里想要暴露出去的接口类后缀名从“.java”改成“.api”，就可以了。

而且并不只是java文件，其他文件如果也想暴露，在文件名后增加".api”，也一样可以。

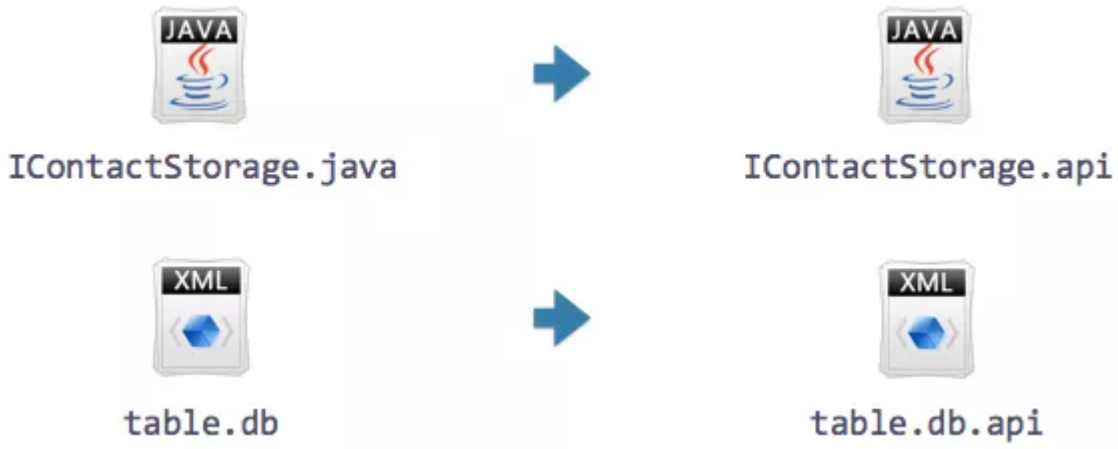


图6 - “.api化”

当然，要让工程支持这样的方式，gradle文件肯定会有一点改变。

settings.gradle

```
// settings.gradle
include_with_api(project: ":plugin-messenger-foundation")
```

build.gradle

```
// build.gradle
dependencies {
    compileApi(':plugin-messenger-foundation')
}
```

图 7 - settings.gradle & build.gradle

就这样，可以说暴露接口变得非常容易，不用担心实现类也被人引用到。而它的实现原理也相当简单：自动生成一个“SDK”工程，拷贝.api后缀文件到工程中就行了，后面其他工程依赖编译的只是这个生成的工程。简单好用。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

还有个细节，如果想编辑.api后缀的java文件，为了能让Android Studio继续高亮该怎么办？可以在File Type中把.api作为java文件类型。

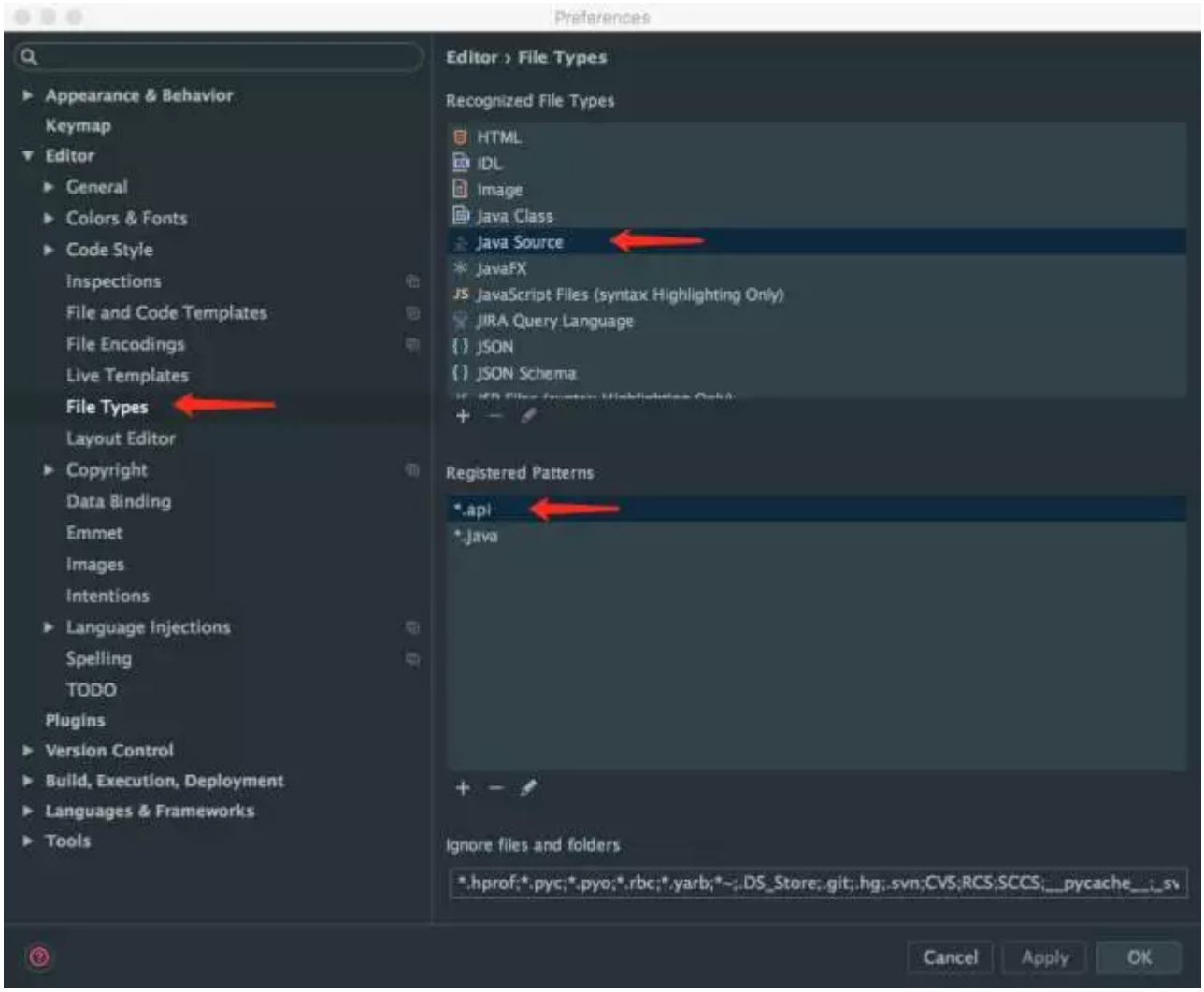


图8 - 设置File Types

重新设计模块

要把模块重新设计，还要做好几件事。首先，消灭代码经常下沉的“三不管区域”——基础工程。这意味着原来的模块要把之前下沉的代码重新认领回去。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

图9 - 分层结构改造

为了巩固替代基础工程的mmkernel层，不被滥用为新的代码堆放处，顺便还要解决中心化问题。就必须强化它的职责和设计。

mmkernel结构可以很通用的定义为CoreAccount/CoreNetwork/CoreStorage三个部分，分别提供了核心账号状态（初始化、注销）、网络状态回调（链接建立）、存储状态生命周期（db创建、销毁、用户存储路径切换、sdcard挂起）。

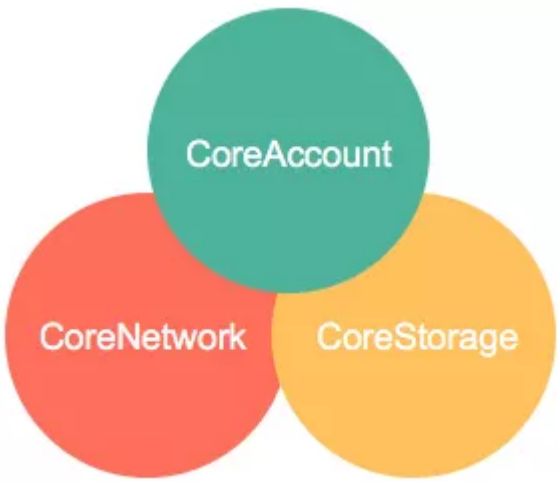


图10

再然后是生命周期问题，我们需要重新设计正确的生命周期。

之前讲过，我们的模块生命周期大体上只有“Account初始化”和“Account注销”两个阶段。这已经不够用了。

所以扩大模块的生命周期，就给了模块实现各种代码需要的时机，才能避免大家往主工程塞代码。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

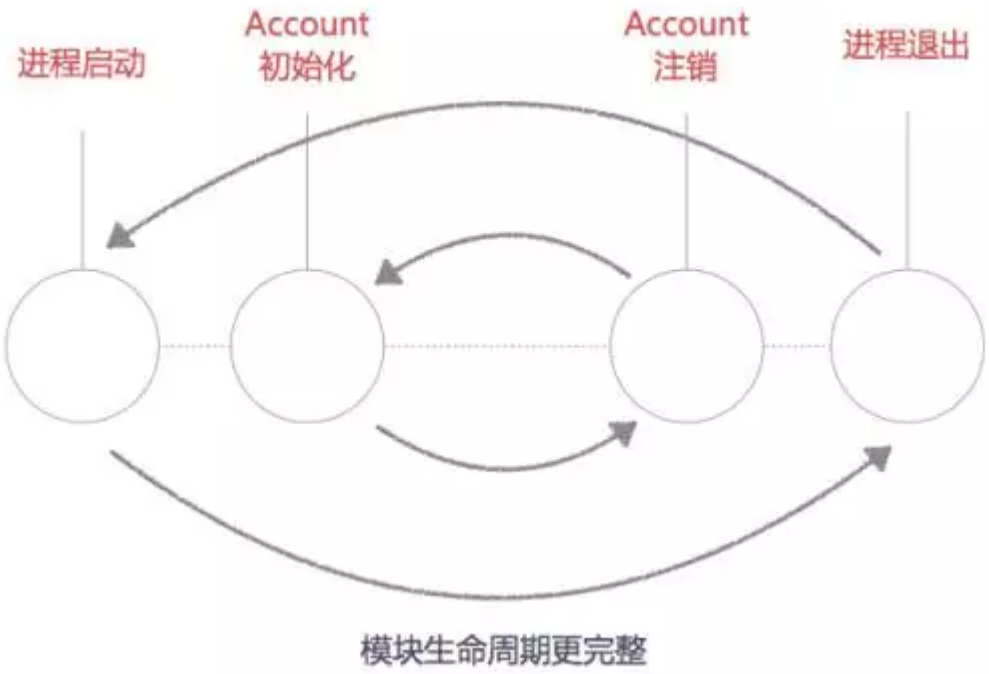


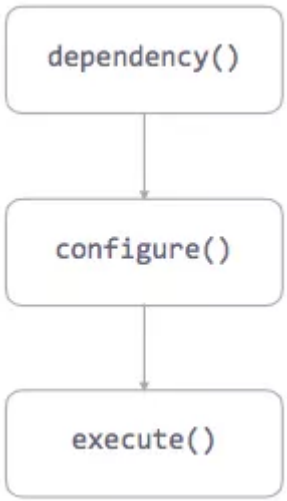
图11

实现新的生命周期是一个正确的选择，同时产生了解决另一个问题的机会——复杂的启动流程。

要知道主工程的代码一部分原因是启动流程堆积造成的，逻辑多了代码自然多。随之而来的问题就是代码多了，逻辑也就跟着复杂起来。微信的初始化逻辑是顺序排列在一起并从上到下执行，某种情况下还会异步启动。当程序启动流程比较复杂时，这样的代码会产生“隐性依赖”的问题。“隐性依赖”顾名思义就是：原本并应该存在依赖的代码，随着版本的迭代逐渐产生了依赖，而且还不明显。这样的情况会让情况恶化，大家只敢往里面堆代码，但却不敢“乱动”。

所以重新设计的模块应该要彻底避免这些问题。

我们重新定义了模块的生命周期，将模块的生命周期延长到应用启动和退出。而后，每个模块都可以定义一个Plugin类，作为模块的“支柱”或“起点”。作为解决初始化问题的手段，它具备几个主要阶段：**dependency()**、**configure()**、**execute()**



腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

图12 - plugin初始化的几个阶段

dependency()阶段，用于设置需要依赖的其他Plugin，当然提供那个Plugin的别名接口类就可以了。

```
@Override
public void dependency() {

    // 设置依赖
    dependsOn(IPluginZero.class);

}
```

图13 - 设置dependency

依赖阶段我们会生成整个模块的依赖树。这与编译时的依赖不同。通常的依赖关系是分为两种的，一种是类型依赖也就是编译期依赖，需要被依赖模块提供具体类型才能编译通过；另一种依赖则是运行期的逻辑依赖或数据一致性依赖，当一个模块用这种方式依赖另一个模块，就意味着，前者的执行要依赖后者执行已完成，通常是为了数据准备妥当或保证所需服务已被注册。

显性的运行期依赖把之前启动逻辑的"隐性依赖"完全暴露在阳光之下，改启动逻辑不用提心吊胆。

图14 - 依赖关系树状图

configure()阶段，该阶段是根据之前的依赖树遍历执行。通常用于初始化一些数据配置、注册IService服务、向前面依赖的模块注册一些回调等等。

此外这个阶段还有额外的作用是插入BootTask，用于后面execute()阶段的执行。

```
@Override
public void configure(ProcessProfile profile) {

    // 注册服务
    registerService(IMyService.class, myService);

    // 注册小文件存储初始化BootTask
    new SFSInitTask()
            .after(plugin(IPluginZero.class))
            .before(this);

}
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

图15 - configure阶段

execute()阶段，为了改变启动流程不清楚的情况，强调启动逻辑之间的依赖关系，我们现在将每个要执行的启动步骤封装为BootTask。前面的configure阶段时，我们可以将BootTask插入到通过dependency()得到的依赖树。每个Plugin同时也是一个BootTask，也因此拥有execute()接口。最终得到了包含所有BootTask的启动树，将遍历执行所有节点执行execute()。

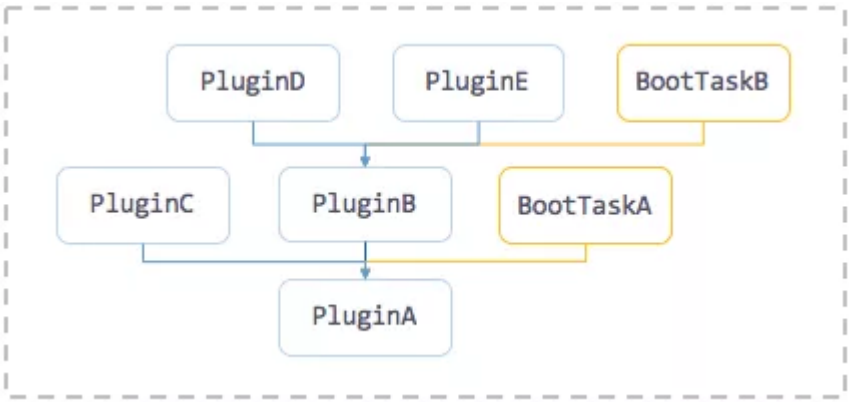


图16 - BootTask

独立使用BootTask的方式并不十分常见，通常Plugin本身的execute已经够用。不过在一些通用型组件初始化尝试会需要用到，如某些给某个全局使用的预加载资源提前初始化的逻辑。

为何设计configure()和execute()，这可以理解为“收集任务”和“执行任务”的两个阶段。另外这样的抽象还可以实现从外部调度execute的执行线程，将启动逻辑和启动异步代码分开。顺便解决了，原来异步启动代码混乱不堪的情况。

约束代码边界

从之前的经验看，要想约束好代码的边界不被破坏，编译上的隔离是唯一法宝。

除了工程和工程之间的分割，在工程的内部如果也能实现约束代码就更好了，算是将问题扼杀在摇篮里。之前通常的做法是以module工程为单位的相互分离，但在工程内部并不限制代码相互引用。所以为了规范代码，常能看到用包名作为约定，区分内部功能职责，靠约定维持解耦。随着时间推移，很快就能发现包名约定作为约束太弱了，在快速迭代的代码上很难一直维持下去。不管怎么样解决，总要通过一些手段审查代码引用的对不对。感觉有点防不胜防。为此，我们实现了一种简单易用、粒度更细的工程组织结构——pins工程结构

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

图17 - pins工程示例图

使用pins工程结构，我们考虑的是要能对接兼容以前结构下那么多伪工程，又能在开始设计某个业务模块时，能用极小粒度的代码解耦，限制错误引用，以防后患。

pins工程能在module之内再次构建完整的多子工程结构，通过project.properties来指定编译依赖关系。通过依赖关系在编译时找到所有的资源和源码路径。而对边界的约束需要配合code-check工具在编译期进行检查，杜绝依赖关系之外的代码引用。

```
apply plugin: 'com.tencent.mm.toolkit.codecheck'
```

图18 - code-check

这样的工程组织形式的两个明显好处：

- 约束代码粒度和小代码边界的利器

粒度极小，一个pin工程也许只有一个源文件，只要它能表达一个独立职责。对于任何一个模块，从内部约束自己的功能结构，是对整体代码边界约束的极大补充。以前面插的结构为例，一个gallery业务可能提供了几种不同的产品功能，以及支撑能力。那么将其相互独立的代码进行区分，避免混杂，就会显得十分必要。清晰的结构，意味着后期维护成本的降低和开发效率的提高，留下了灵活性。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

- 避免的超量module的创建，轻量

pins工程某种程度上能减少一些粒度太小的module工程，也一定程度的缓解太多module工程时的gradle编译性能问题。

至此，我们基本完成了重塑模块化的设计目标，解决掉很多之前没有考虑的问题。算是模块化的加强版。另外设计是一方面，拆分解耦原来代码以及迁移还是另一回事，这个过程也是十分艰难和枯燥这里就不细讲了。接下来想办法看看重构的效果。

看看效果

重新设计的模块化加上代码的重构。我们终于能满足之前硬件同学的需要。同时一并解决许多拖欠的问题。

在编译上，整体编译速度会因为module增多而下降一些。但拆分module之后，却能显著加快单工程增量编译的速度。和之前相比，一行代码的增量编译耗时能减少60%。

除了满足需求外，架构设计的效果并不好量化，不过我们尝试用一个demo来说明。

WeChat nano

基于前面介绍过的轻量的微信内核mmkernel层，再配合一个不包含界面的基础聊天模块和Auth模块，可以在短时间里开发出一个及精简版本的微信——WeChat nano。

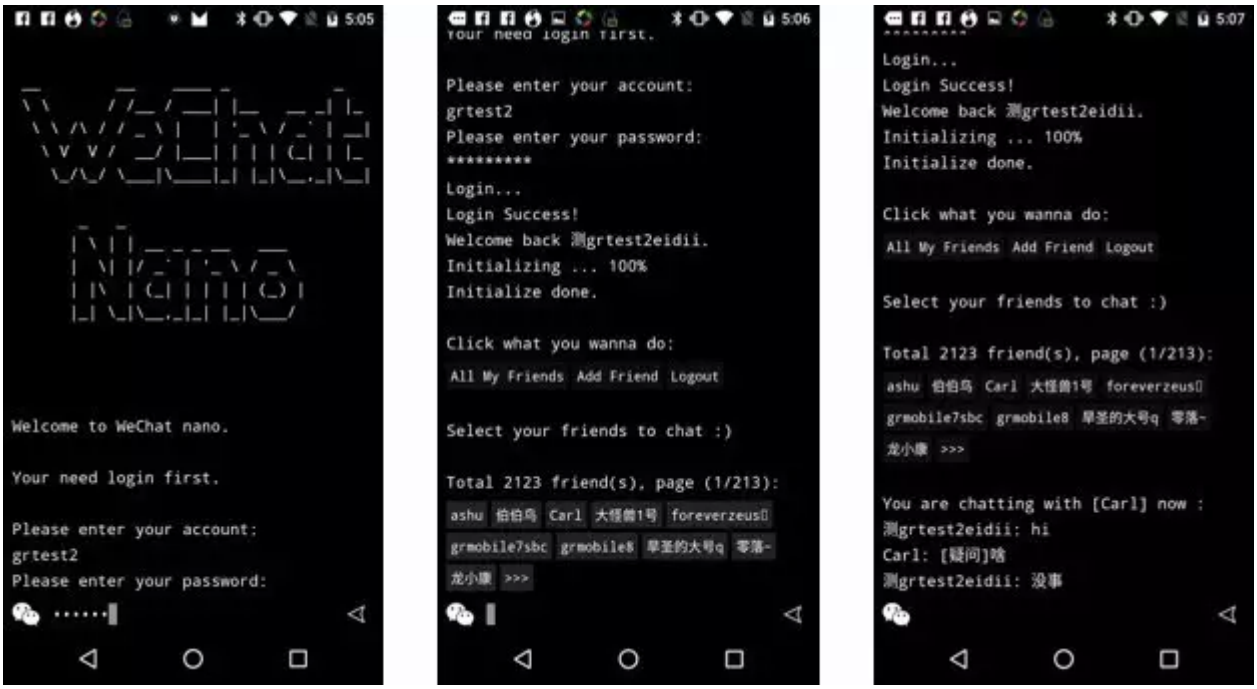


图19 - WeChat nano

模拟这个console的界面是单独开发的，时间的大头都花在这上面。

它的效果不错：

- 可以让安装包大小缩减到3.5M，大概是完整版本的10%
- 能大幅减小内存占用，约占用完整版本的25%（注：只计算应用相关有不同的部分PSS）

大概就是这样。

3.3.5 取舍和选择

对于架构重构，我们也曾放眼行业内已经发布过的各种方案，希望从中找到一些解决思路。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

我们参考了很多业界开放和发表的架构设计。总的来说，目前Android端App整体架设计上，除了聚焦在“大前端”之外，基本上都在“插件化/应用沙盒”上面下功夫。可以参考如atlas、small、DroidPlugin、DynamicApk等等方案，不难发现让模块最终具备动态性是它们最核心的能力。

“大前端”是个热门的方向。我们结合自身情况，考虑到迁移已有代码不是一件容易事，让团队彻底切换编程工具短期不现实。这种“远水解不了近渴”的感觉，也只能暂且作罢。

我们把目光投向了“插件化/应用沙盒”。沙盒的好处是它具备很强的隔离性，有些方案可以彻底隔离代码和资源，边界限制性极强，效果很好。此外，多数方案也都具备动态更新能力、补丁能力，动态性基本成为标配。所以这时我们在考虑需不需要走上这样的路。

一切从自身情况出发。在微信的角度看，我们最关心的问题是如何能约束住代码边界，如何防止架构劣化，如何提高开发效率。这样的情况下，重新审视了具备动态性的插件化和沙盒方案。先从动态性上考虑，回看业内的使用经验，目前的动态性通常有两个主要的用途：1）作为热补丁使用；2）业务并行发布需求强烈，例如运营需求，作为快速发布的手段。对于第二点，就目前的微信团队来说是较少遇到的弱需求。相较之下，工具类和电商类应用常有符合的使用场景。而对于第一点，我们则期望在实现模块化上，目标相对纯粹一些，专心解决代码自身问题，之后再通过tinker在开发阶段之外实现热补丁上的动态性需要。

再来看隔离性。需要说明的是，这里的隔离型主要来自于沙盒架构效果。对于非独立插件化工程（需要编译依赖其他插件）除了动态性，它的作用和普通module在代码隔离上没有区别，不做讨论。

我们心里很期盼代码与代码之间那种干净清爽的分割效果，但事与愿违。

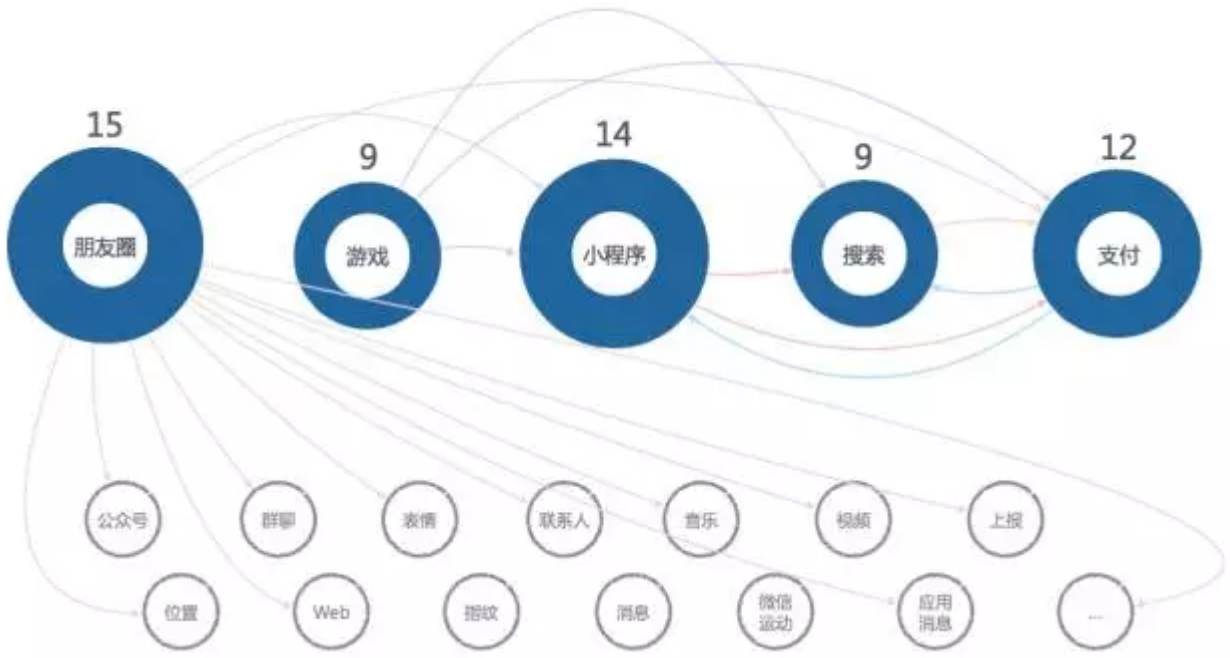


图20 - 模块依赖示意图

从上面这张模块依赖示意图看到，微信业务模块之间数据关系相当复杂，模块间相互访问数据、共享某些功能的行为如此普遍。而实际情况比示意图更麻烦。

面对微信业务数据相互间频繁的调用，沙盒隔离容易导致代码复用困难和相互调用麻烦，想在微信上实现，目前困难极大。重构难度不谈，单是隔离的收益就很有可能无法弥补开发效率上的损失。

从开发模式上看也是一样。微信Android团队目前每个迭代大概三四十人参与，内部沟通成本不算高到不可接受。通常开发同学可能要同时开发和修改几个模块并保证他们相互模块独立，同时又可能有频繁的模块间通信。这种时候，模块调用方不方便显得很重要。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

此外还需要考虑的问题，从几个成熟方案中都能看到hook Android框架、修改aapt、替换或包装android gradle plugin、代理组件等等设计。这些方案的复杂度和兼容性代价，不能忽视。使用和维护它们需要仔细权衡。

所以思前想后我们仍选择了重走最开始的模块化之路。一切问题的根本还是在那个说的很多年的代码边界、解耦和内聚上。只要有了清晰独立的代码模块，才有将来其他变化的可能性。

3.3.6 代码之外，架构之内

模块负责人制度

有这样一句话，“不被监管的权利一定会发生腐败”。如果放到软件开发的行当来说，就是“不被监管的代码也一定会发生劣化”。所以代码应该要接受“监管”。

为了能长期有效的保持代码质量，我们开始执行新的代码审查机制——模块负责人制度。

代码审查的好处毋庸置疑。在这之前，微信由于业务发展快速，同学们经常会变换需求的开发方向。面对着业务模块数量比人多的情况，开发同学经常一个人需要开发多个模块。也因此许多模块被无数人维护过，基础的支撑工程更是如此。这种类似游击战的方式，开发效率很高，支撑了微信快速的研发节奏，但也导致了“无主代码”特别多。大家缺少对代码的“归属感”，也降低了改进优化模块的欲望。

另外在这之前，代码审查是由leader对申请回流主干的Merge Request进行review，这导致效率较低且容易遗漏问题。合理的代码审查更应该是全员性质的。

模块负责人制度尝试改变这些现状。通过大家认领模块，对模块的代码和设计负责，对模块对外提供的接口服务负责，对其他人修改自己模块的行为进行监督。这些情况明显提高开发同学的代码所有感，改变大家修改优化和修改代码的动机。

推动模块负责人制度，渐进式的推动了大范围的代码审查，这样的方式很适合像微信这样没有从一开始执行全员性质Code Review的项目。目前模块负责人机制运转顺利，代码审查率和模块认领率都在提高。

重构与开发者心态的关系

在一个长期没有改进的框架下，开发者的习惯可能会逐步变成跟随式、保守式的开发。这大概可以被描述成“只要别人这样做，我也这样做，哪怕这么样的设计不好，但也不会错”。随着心态逐渐普遍，另一种情形出现：经常能听到有同学吐槽一些代码，却更少看到代码在被改进。这说明一些沉积的问题不是没有被大家发现，只是没有人愿意去修改。这种情况下代码和框架会随着时间变得越来越差，有些问题逐渐变成“陈年旧病”。 面对这个问题首先要说，这不是开发者合格与否的问题，实际上有想法的开发人员有很多，但想将每个想法转换成代码并让大家接受，并不是一件很容易的事。尤其在一个大框架下，尝试改变的代价很大。如果他的主要任务不在改进某些模块上，那么很多想法最后都无法变成现实。这也是为什么保守和跟随的习惯会逐渐变的普遍。

保守的气氛需要被打破。当开启一次重构之后，你会发现团队中会有很多积极的声音响应，他们会把积压的想法和意见抛出来。一次问题的解决，可能会为另一个问题的解决带来机会，其他开发同学的一些想法也许就能更容易落地。所以不定期的推动一些模块的重构，将一些对代码的不满释放出来，是一种不错的激活。

此外在重构之后，还要考虑引导开发的代码组织方式切换，多用模板、正确的代码实例等，让他们可以放心参考。

模块划分经验谈

维持代码边界

代码的边界就像一堵墙，架构的劣化都是从这堵墙的瓦解开始的。从以往的经验来看，编译上的隔离是最好的约束手段，单纯的约定或准则并不能永远的保持下去。 所以在任何情况下都尽可能不要放开编译上的约束。接着，将接口和实现分离，其他工程只依赖接口而不依赖实现，这样的边界效果更好。当然破坏无处不在，例如遇到某个紧急需求要某模块新增若干接口，就可能出现跳过接口直接依赖实现工程进行开发的情况。这时可以考虑通过代码的审查进行监督，也可以通过开发简单的编译脚本，检查是否有不当依赖产生。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

划定模块边界的细节问题

当对代码进行解耦时，即便大体上的模块职责划分已经清晰，但因为模块间的各种业务关系，细节上仍会遇到纠缠不清的情况。事实上，因为需求及功能的不同，并没有哪一种模块划分的规则是完全适用于每个应用的。随着业务的发展和变化，模块边界出现不合适的情况完全符合预期。

那么如何让模块划分更让大家觉得合理，或者说当遇到一个两难选择时，按照什么样的方式大家会更好理解？我们建议的方法其实也很简单：试着对代码"讲一个符合逻辑的故事"，哪个故事讲得通，你就可以将之作为拆分的选择。因为代码解耦从来不是问题，纠结的只是解耦行为能不能让人理解。例如一些模块间通信用的数据结构究竟属于那个模块的问题就可以用这种方式仲裁。在纠结的时候，能自圆其说的方案往往就足够了。我们要尽力避免的，应该是随意拼凑和单纯为了类型解耦而解耦的情况。

模块的一般组织方式

设计一个模块，我们有一个一般性的组织方式，可以将模块分成三个工程：implementation工程、api工程、library工程。

implementation工程提供逻辑的实现。api工程提供对外的接口和数据结构。library工程，则提供该模块的一些工具类。

从另一个角度看，implementation工程实际上是和应用的状态、生命周期相关的，它的执行依赖于各种应用状态。而library工程则不关心这些状态。因此也可以看做library提供某种功能，implementation则是如何运用这种功能。例如，我们实现一个表情模块，library工程提供表情的资源、表情的渲染和播放能力，api工程提供了使用表情的服务接口，implementation工程则提供了api的实现，及何时开始加载表情资源、缓存管理、以及其他表情功能例如商店等等。

当然，这是一个指导性的建议，很多时候library工程和api工程之间没有明显边界也很正常。但强烈建议至少要有implementation工程和api工程。

分析依赖关系的工具

解耦代码时，快速分析代码的依赖关系能很好的提升工作效率。Android Studio提供了一个不错的工具。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

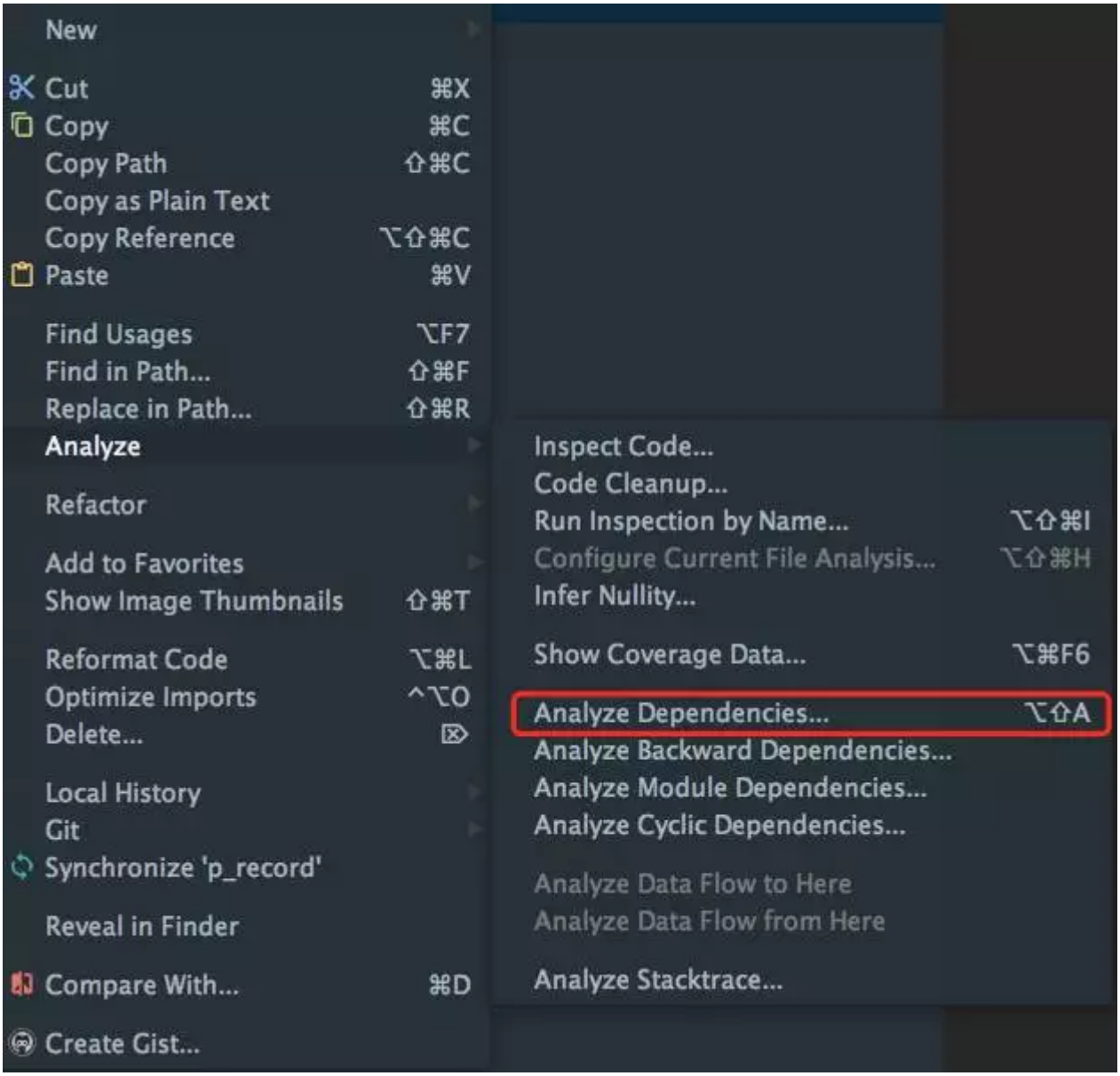


图21 - Analyze dependency工具

文件、资源以及工程，都可以进行依赖分析。有了分析结果，接下来一步一步把代码分离就简单多了。

3.3.7 最后

重构整体架构不是一件容易事，通常也不太可能让整个团队停下来只做重构。所以一直以来微信的重构都是随着版本迭代进行“拆分”-> “灰度” -> “回流”的循环节奏。

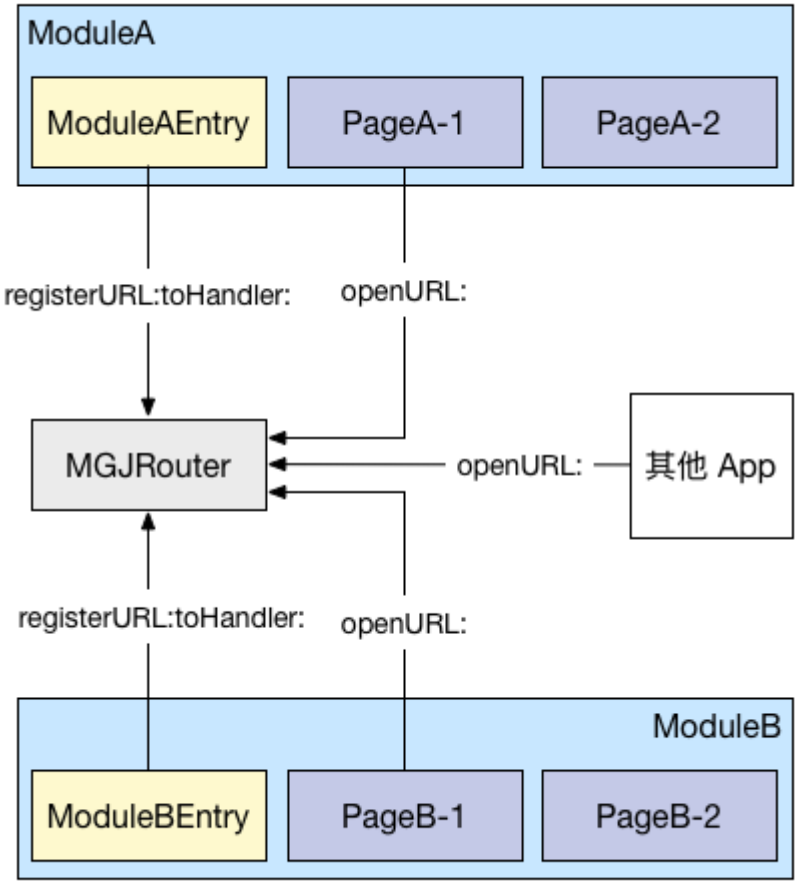
“设计系统的组织，其产生的设计和架构等价于组织间的沟通结构”。对于微信几年间走过的路程，时至今日团队内的沟通形式还可以做到较多的直接沟通。这些情况决定了微信如今的技术选择。所以在方案选择上我们就更愿意寻求相对简单合适的方式解决问题——用纯粹的模块化保持后续架构的灵活性和健壮性，重新强调依赖、强调应用状态和生命周期、强化代码的边界。

除了代码上的设计，代码之外我们也做了些努力。我们认同代码审查的意义，也开始推行模块负责人的审查机制。此外我们还打算强化文档的使用，当然这个还在规划中。

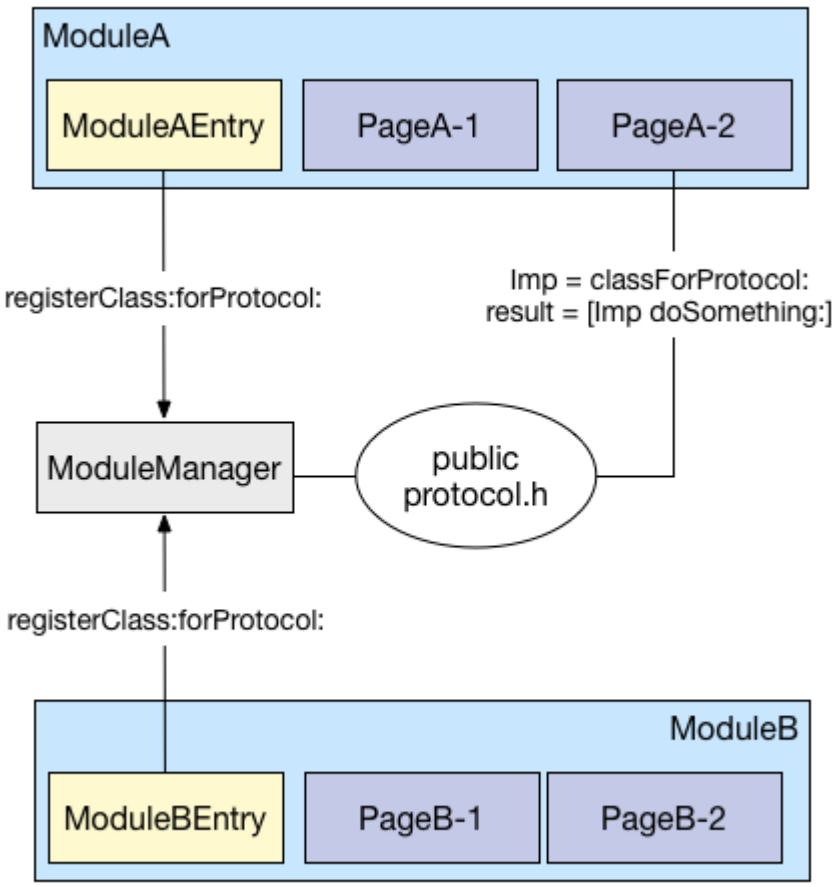
3.4 蘑菇街 App

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

- 提到了组件间通信的实现：URL 统跳、协议下沉（需要有一个公共的地方来容纳这些 public protocl）、隐式 Intent
- 提到了组件生命周期管理：监听系统通知、ModuleManager 中手动遍历调用
- 提到了组件版本管理与持续集成经验



腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



3.4.1 蘑菇街 App 的组件化之路

在组件化之前，蘑菇街 App 的代码都是在一个工程里开发的，在人比较少，业务发展不是很快的时候，这样是比较合适的，能一定程度地保证开发效率。

慢慢地代码量多了起来，开发人员也多了起来，业务发展也快了起来，这时单一工程开发模式就会显露出一些弊端：

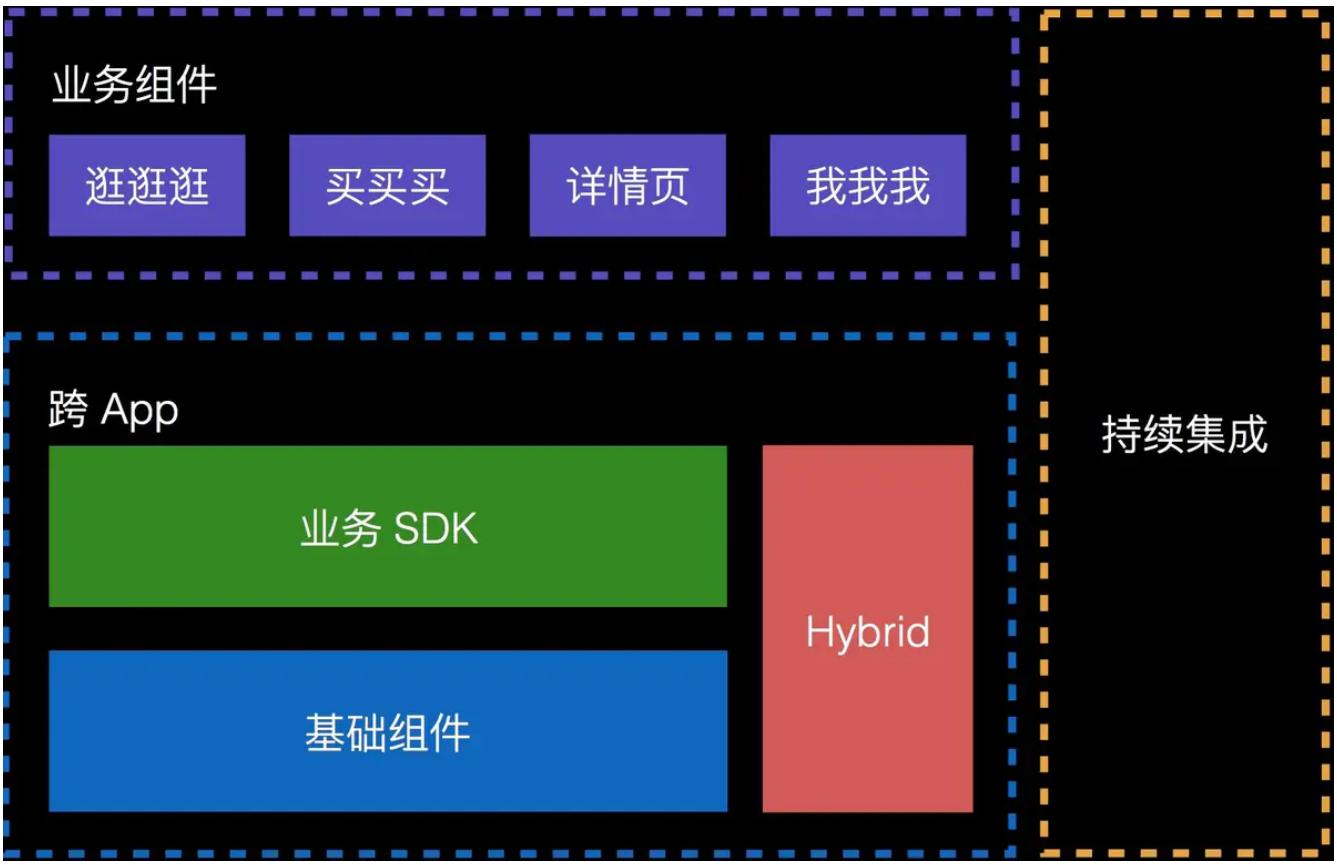
- 耦合比较严重（因为没有明确的约束，「组件」间引用的现象会比较多）；
- 容易出现冲突（尤其是使用 Xib，还有就是 Xcode Project，虽说有脚本可以改善）；
- 业务方的开发效率不够高（只关心自己的组件，却要编译整个项目，与其他不相干的代码糅合在一起）。

为了解决这些问题，就采取了「组件化」策略。它能带来这些好处：

- 加快编译速度（不用编译主客那一大坨代码了）；
- 自由选择开发姿势（MVC / MVVM / FRP）；
- 方便 QA 有针对性地测试；
- 提高业务开发效率。

先来看下，组件化之后的一个大概架构：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



「组件化」顾名思义就是把一个大的 App 拆成一个个小的组件，相互之间不直接引用。那如何做呢？

3.4.1.1 实现方式

组件间通信

以 iOS 为例，由于之前就是采用的 URL 跳转模式，理论上页面之间的跳转只需 open 一个 URL 即可。所以对于一个组件来说，只要定义「支持哪些 URL」即可，比如详情页，大概可以这么做的：

```
[MGJRouter registerURLPattern:@"mgj://detail?id=:id" toHandler:^(NSDictionary
*routerParameters) {
    NSNumber *id = routerParameters[@"id"];
    // create view controller with id
    // push view controller
}];
```

首页只需调用 `[MGJRouter openURL:@"mgj://detail?id=404"]` 就可以打开相应的详情页。

那问题又来了，我怎么知道有哪些可用的 URL？为此，我们做了一个后台专门来管理。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

ID	变量名	短链	描述	开发人员	需求方	创建时间	操作
4	MGJPAGE_ABOUT	mgj://about	关于蘑菇街	慧能	慧能	2015-12-17 19:28	约束 废弃
5	MGJPAGE_ADDRESS	mgj://address	管理收货地址	慧能	慧能	2015-12-17 19:28	约束 废弃
6	MGJPAGE_APPENDRATE	mgj://appendrate	追加评价	慧能	慧能	2015-12-17 19:28	约束 废弃
7	MGJPAGE_BEAUTY	mgj://beauty	美妆	慧能	慧能	2015-12-17 19:28	约束 废弃

然后可以把这些短链生成不同平台所需的文件，iOS 平台生成 .{h,m} 文件，Android 平台生成 .java 文件，并注入到项目中。这样开发人员只需在项目中打开该文件就知道所有的可用 URL 了。

目前还有一块没有做，就是参数这块，虽然描述了短链，但真想要生成完整的 URL，还需要知道如何传参数，这个正在开发中。

还有一种情况会稍微麻烦点，就是「组件A」要调用「组件B」的某个方法，比如在商品详情页要展示购物车的商品数量，就涉及到向购物车组件拿数据。

类似这种同步调用，iOS 之前采用了比较简单的方案，还是依托于 `MGJRouter`，不过添加了新的方法 `-(id)objectForURL:`，注册时也使用新的方法进行注册：

```
[MGJRouter registerURLPattern:@"mgj://cart/ordercount" toObjectHandler:^id(NSDictionary
*routerParamters){
    // do some calculation
    return @42;
}]
```

使用时 `NSNumber *orderCount = [MGJRouter objectForURL:@"mgj://cart/ordercount"]` 这样就拿到了购物车里的商品数。

稍微复杂但更具通用性的方法是使用「协议」 <-> 「类」绑定的方式，还是以购物车为例，购物车组件可以提供这么个 Protocol:

```
@protocol MGJCart <NSObject>
+ (NSInteger)orderCount;
@end
```

可以看到通过协议可以直接指定返回的数据类型。然后在购物车组件内再新建个类实现这个协议，假设这个类名为 `MGJCartImpl`，接着就可以把它与协议关联起来 `[ModuleManager registerClass:MGJCartImpl forProtocol:@protocol(MGJCart)]`，对于使用方来说，要拿到这个 `MGJCartImpl`，需要调用 `[ModuleManager classForProtocol:@protocol(MGJCart)]`。拿到之后再调用 `+ (NSInteger)orderCount` 就可以了。

那么，这个协议放在哪里比较合适呢？如果跟组件放在一起，使用时还是要先引入组件，如果有多个这样的组件就会比较麻烦了。所以我们把这些公共的协议统一放到了 `PublicProtocolDomain.h` 下，到时只依赖这一个文件就可以了。

Android 也是采用类似的方式。

3.4.1.2 组件生命周期管理

理想中的组件可以很方便地集成到主客中，并且有跟 `AppDelegate` 一致的回调方法。这也是 `ModuleManager` 做的事情。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

先来看看现在的入口方法：

```
- (BOOL)application:(UIApplication *)application didFinishLaunchingWithOptions:(NSDictionary
*)launchOptions
{
    [MGJApp startApp];

    [[ModuleManager sharedInstance] loadModuleFromPlist:[[NSBundle mainBundle]
pathForResource:@"modules" ofType:@"plist"]];
    NSArray *modules = [[ModuleManager sharedInstance] allModules];
    for (id<ModuleProtocol> module in modules) {
        if ([module respondsToSelector:_cmd]) {
            [module application:application didFinishLaunchingWithOptions:launchOptions];
        }
    }

    [self trackLaunchTime];
    return YES;
}
```

其中 `[MGJApp startApp]` 主要负责一些 SDK 的初始化。`[self trackLaunchTime]` 是我们打的一个点，用来监测从 `main` 方法开始到入口方法调用结束花了多长时间。其他的都由 `ModuleManager` 搞定，`loadModuleFromPlist:pathForResource:` 方法会读取 bundle 里的一个 plist 文件，这个文件的内容大概是这样的：

Root	Array	(40 items)
Item 0	String	ComponentManager
Item 1	String	MiscModule
Item 2	String	UISkeletonModule
Item 3	String	WatchModule
Item 4	String	URLHandlerModule
Item 5	String	MGJIndexModule

每个 `Module` 都实现了 `ModuleProtocol`，其中有一个 `-(BOOL)applicaiton:didFinishLaunchingWithOptions:` 方法，如果实现了的话，就会被调用。

还有一个问题就是，系统的一些事件会有通知，比如 `applicationDidBecomeActive` 会有对应的 `UIApplicationDidBecomeActiveNotification`，组件如果要做响应的话，只需监听这个系统通知即可。但也有一些事件是没有通知的，比如 `- application:didRegisterUserNotificationSettings:`，这时组件如果也要做点事情，怎么办？

一个简单的解决方法是在 `AppDelegate` 的各个方法里，手动调一遍组件的对应的方法，如果有就执行。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
- (void)application:(UIApplication *)application
didRegisterForRemoteNotificationsWithDeviceToken:(NSData *)deviceToken
{
    NSArray *modules = [[ModuleManager sharedInstance] allModules];
    for (id<ModuleProtocol> module in modules) {
        if ([module respondsToSelector:_cmd]) {
            [module application:application
didRegisterForRemoteNotificationsWithDeviceToken:deviceToken];
        }
    }
}
```

3.4.1.2 壳工程

既然已经拆出去了，那拆出去的组件总得有个载体，这个载体就是壳工程，壳工程主要包含一些基础组件和业务 SDK，这也是主工程包含的一些内容，所以如果在壳工程可以正常运行的话，到了主工程也没什么问题。不过这里存在版本同步问题，之后会说到。

3.4.1.3 遇到的问题

组件拆分

由于之前的代码都是在一个工程下的，所以要单独拿出来作为一个组件就会遇到不少问题。首先是组件的划分，当时在定义组件粒度时也花了些时间讨论，究竟是粒度粗点好，还是细点好。粗点的话比较有利于拆分，细点的话灵活度比较高。最终还是选择粗一点的粒度，先拆出来再说。

假如要把详情页迁出来，就会发现它依赖了一些其他部分的代码，那最快的方式就是直接把代码拷过来，改个名使用。比较简单暴力。说起来比较简单，做的时候也是挺有挑战的，因为正常的业务并不会因为「组件化」而停止，所以开发同学们需要同时兼顾正常的业务和组件的拆分。

版本管理

我们的组件包括第三方库都是通过 Cocoapods 来管理的，其中组件使用了私有库。之所以选择 Cocoapods，一个是因为它比较方便，还有就是用户基数比较大，且社区也比较活跃（活跃到了会时不时地触发 Github 的 rate limit，导致长时间 clone 不下来… 见此），当然也有其他的管理方式，比如 submodule / subtree，在开发人员比较多的情况下，方便、灵活的方案容易占上风，虽然它也有自己的问题。主要有版本同步和更新/编译慢的问题。

假如基础组件做了个 API 接口升级，这个升级会对原有的接口做改动，自然就会升一个中位的版本号，比如原先是 1.6.19，那么现在就变成 1.7.0 了。而我们在 Podfile 里都是用 ~

指定的，这样就会出现主工程的 pod 版本升上去了，但是壳工程没有同步到，然后群里就会各种反馈编译不过，而且这个编译不过的长尾有时能拖上两三天。

然后我们就想了个办法，如果不在壳工程里指定基础库的版本，只在主工程里指定呢，理论上应该可行，只要不出现某个基础库要同时维护多个版本的情况。但实践中发现，壳工程有时会莫名其妙地升不上去，在 podfile 里指定最新的版本又可以升上去，所以此路不通。

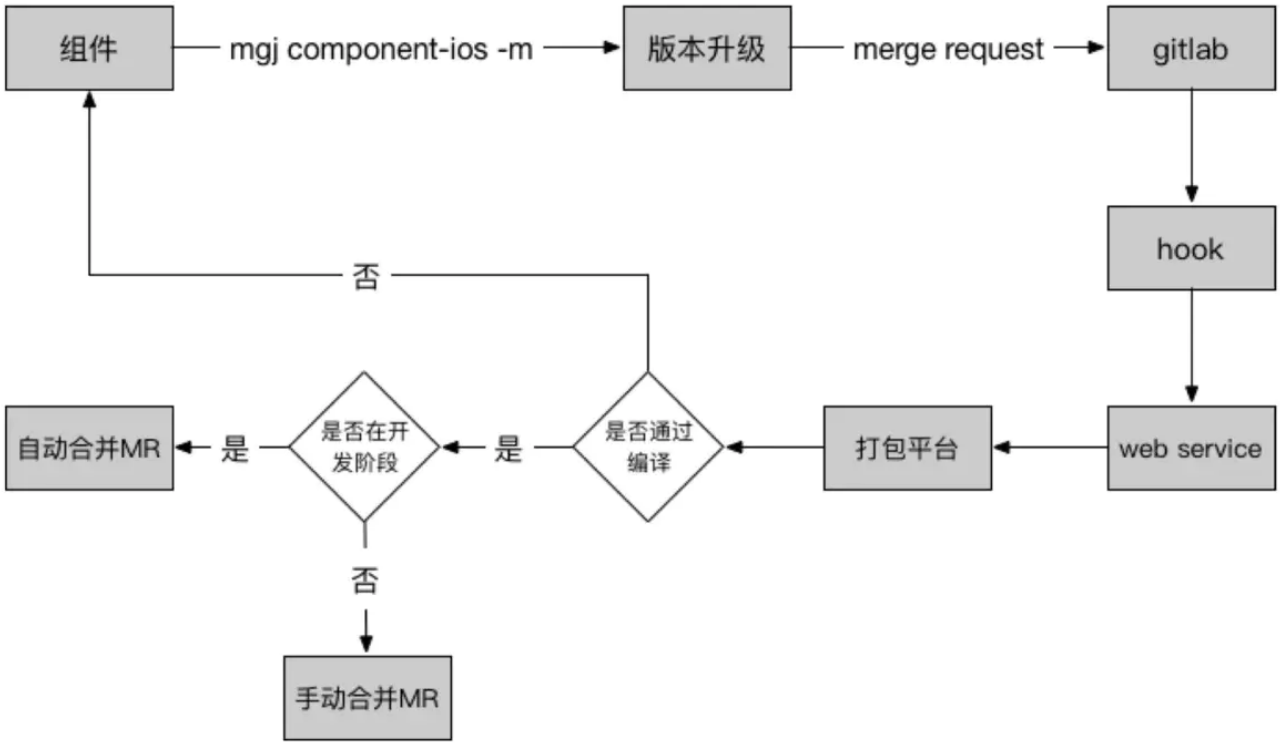
还有一个问题是`pod update`时间过长，经常会在`Analyzing Dependency`上卡 10 多分钟，非常影响效率。后来排查下来是跟组件的 Podspec 有关，配置了 subspec，且依赖比较多。

然后就是 pod update 之后的编译，由于是源码编译，所以这块的时间花费也不少，接下去会考虑 framework 的方式。

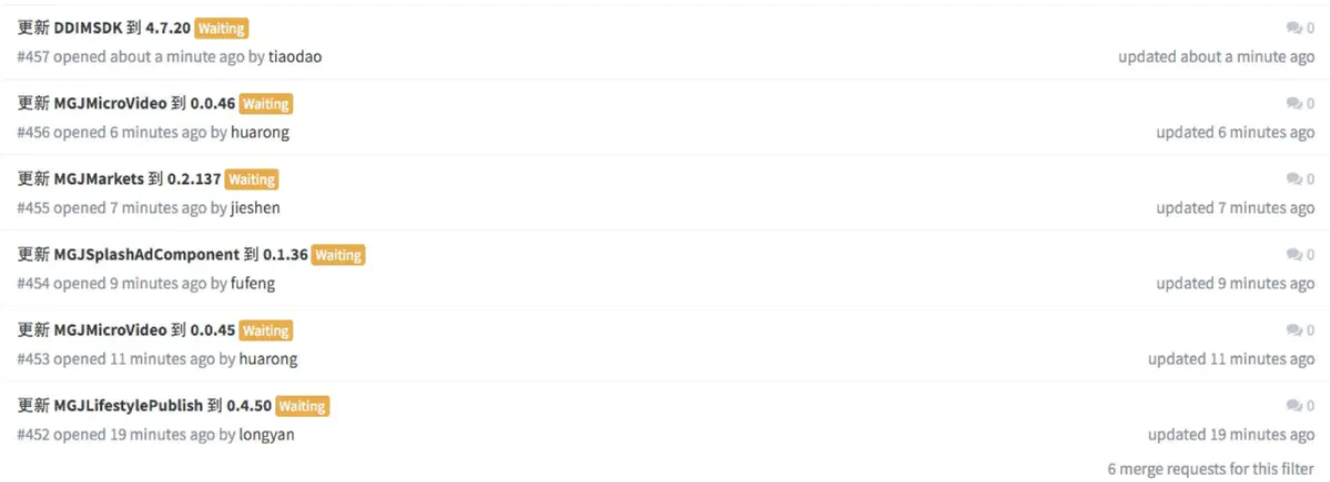
3.4.1.4 持续集成

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

在刚开始，持续集成还不是很完善，业务方升级组件，直接把 podspec 扔到 private repo 里就完事了。这样最简单，但也经常会带来编译通不过的问题。而且这种随意的版本升级也不太能保证质量。于是我们就搭建了一套持续集成系统，大概如此：



每个组件升级之前都需要先通过编译，然后再决定是否升级。这套体系看起来不复杂，但在实施过程中经常会遇到后端的并发问题，导致业务方要么集成失败，要么要等不少时间。而且也没有一个地方可以呈现当前版本的组件版本信息。还有就是业务方对于这种命令行的升级方式接受度也不是很高。

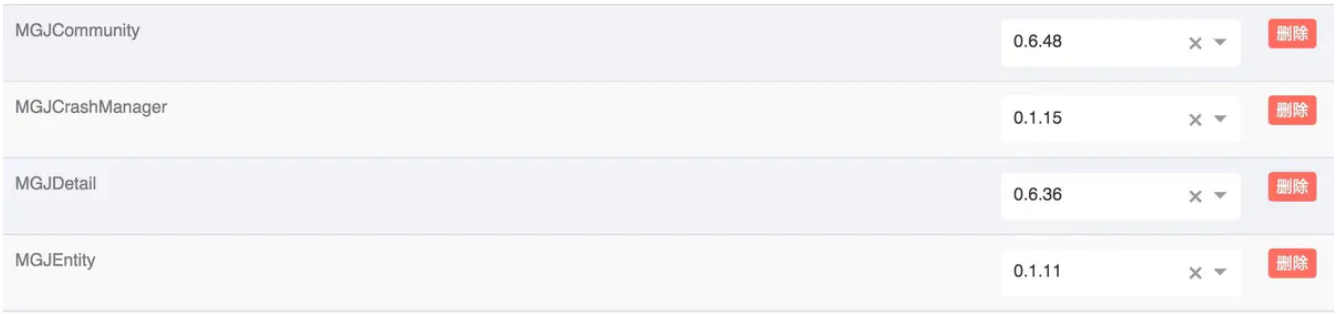


基于此，在经过了几轮讨论之后，有了新版的持续集成平台，升级操作通过网页端来完成。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

大致思路是，业务方如果要升级组件，假设现在的版本是 0.1.7，添加了一些 feature 之后，壳工程测试通过，想集成到主工程里看看效果，或者其他组件也想引用这个最新的，就可以在后台手动把版本升到 0.1.8-rc.1，这样的话，原先依赖 `~> 0.1.7` 的组件，不会升到 0.1.8，同时想要测试这个组件的话，只要手动把版本调到 0.1.8-rc.1 就可以了。这个过程不会触发 CI 的编译检查。

当测试通过后，就可以把尾部的`-rc.n`去掉，然后点击「集成」，就会走 CI 编译检查，通过的话，会在主工程的 podfile 里写上固定的版本号 0.1.8。也就是说，podfile 里所有的组件版本号都是固定的。



3.4.1.5 周边设施

基础组件及组件的文档 / Demo / 单元测试

无线基础的职能是为集团提供解决方案，只是在蘑菇街 App 里能 work 是远远不够的，所以就需要提供入口，知道有哪些可用组件，并且如何使用，就像这样（目前还未实现）

这就要求组件的负责人需要及时地更新 README / CHANGELOG / API，并且当发生 API 变更时，能够快速通知到使用方。

公共 UI 组件

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

组件化之后还有一个问题就是资源的重复性，以前在一个工程里的时候，资源都可以很方便地拿到，现在独立出去了，也不知道哪些是公用的，哪些是独有的，索性都放到自己的组件里，这样就会导致包变大。还有一个问题是每个组件可能是不同的产品经理在跟，而他们很可能只关注于自己关心的页面长什么样，而忽略了整体的样式。公共 UI 组件就是用来解决这些问题的，这些组件甚至可以跨 App 使用。（目前还未实现）

3.4.1.6 小结

「组件化」是 App 膨胀到一定体积后的解决方案，能一定程度上解决问题，在提高开发效率的过程中，踩坑是难免的，希望这篇文章能够带来些帮助。

3.4.2 蘑菇街 App 的组件化之路·续

前几天在「移动学习分享群」分享了关于蘑菇街组件化方面的一点经验，由于时间和文字描述方面的限制，很多东西表述的不是很清楚，让一些同学产生了疑惑，casatwy老师也写了篇文章来纠正其中的一些实现，看完之后确实有不少启发。

3.4.2.1 统一的调用实现

将「URL 调用」和「组件间调用」通过 runtime 达到统一，通过 prefix 的方式来避免安全上的一些漏洞。看起来确实会舒服些，也比较灵活。

3.4.2.2 通过 Category 来统一组件对外暴露的接口

支持 `openURL:` 但最终还是走的 target-action，跟内部调用无差别。 这也是我们目前有待提升的点，想知道某个组件支持哪些 URL 或 哪些 Protocol 不够方便，URL 的参数传递也是个问题，将来 URL 发生变动的话，调整起来也比较麻烦。后续会在这块再加强下。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

当初决定使用 `openURL:` 来做页面间的跳转，而不是方法调用，主要是考虑到我们的大部分场景都可以通过这种方式解决，因此就这么定了。`openURL:` 更像 Android 里的 「隐式 Intent」，不关心谁来处理这个 URL，由系统（MGJRouter）来决定。而方法调用更像「显式 Intent」或者 RPC，明确地知道应该由谁来处理。前者的好处是可以更少地关心业务逻辑，后者的好处是可以很方便地完成参数传递。

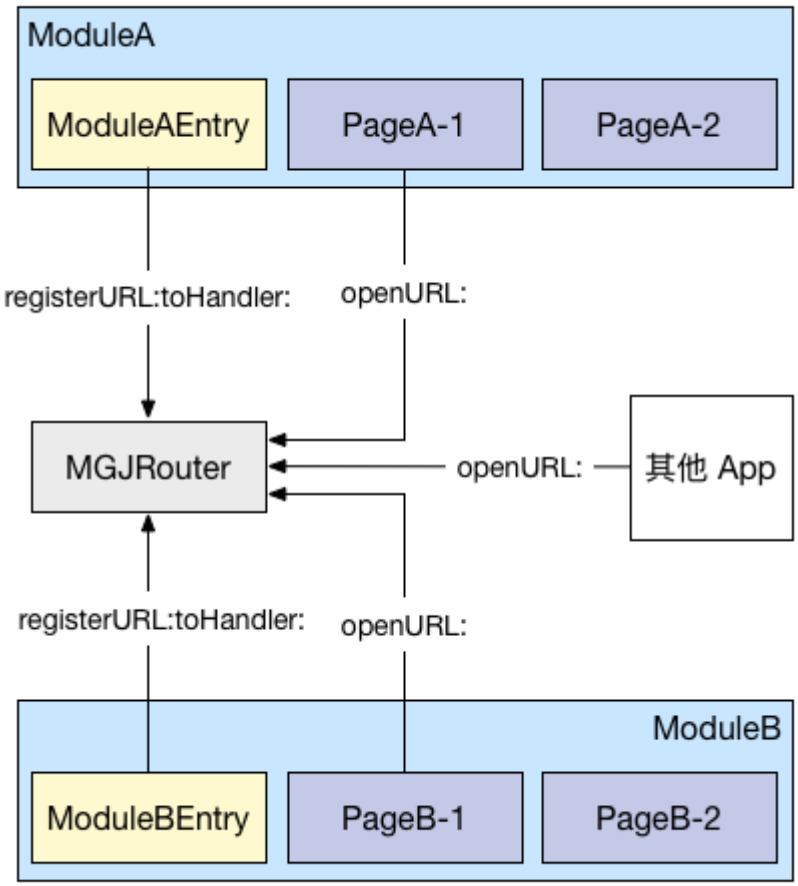
3.4.2.3 更明确的表述

1. `openURL` 只是页面间的调用方式
2. 组件间的调用通过 protocol 来实现

每个组件都有一个 `Entry`，这个 `Entry`，主要做了三件事

- 注册这个组件关心的 URL
- 注册这个组件能够被调用的方法/属性
- 在 App 生命周期的不同阶段做不同的响应

3.4.2.4 注册这个组件关心的 URL




```
[MGJRouter registerURLPattern:@"mgj://detail?id=:id" toHandler:^(NSDictionary
*routerParameters) {



    NSNumber *id = routerParameters[@"id"];
```

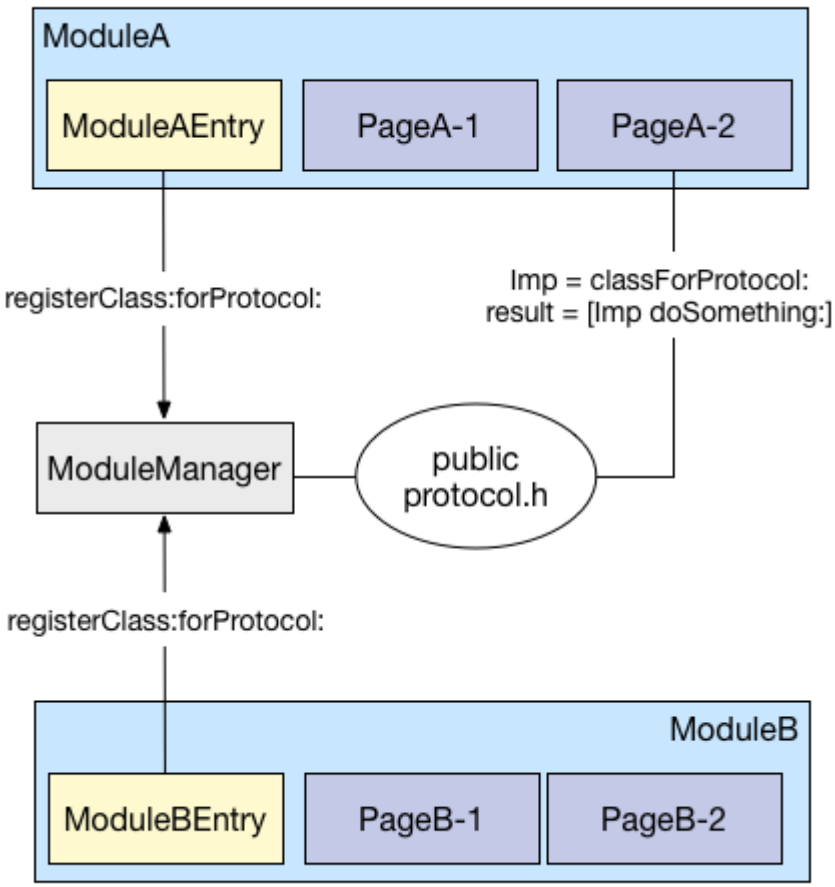
腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
    // create view controller with id


    // push view controller


}];
```

URL 的注册会有对应的 block，拿到这个 URL 后，想怎么折腾就怎么折腾。

3.4.2.5 注册这个组件能够被调用的方法/属性

当有一些场景不适合用 URL 的方式时，就可以通过注册 protocol 来实现



`[ModuleManager registerClass:ClassA forProtocol:ProtocolA]` 的结果就是在 MM 内部维护的 dict 里新加了一个映射关系。

`[ModuleManager classForProtocol:ProtocolA]` 的返回结果就是之前在 MM 内部 dict 里 protocol 对应的 class，使用方不需要关心这个 class 是个什么东东，反正实现了 `ProtocolA` 协议，拿来用就行。

这里需要有一个公共的地方来容纳这些 public protocl，也就是图中的 `PublicProtocl.h`

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

3.4.2.6 在 App 生命周期的不同阶段做不同的响应

上一篇文章中有提到，这里简单说下，`ModuleEntry`，实现某个特定的协议(该协议继承自`UIApplicationDelegate`)，然后实现对应的方法即可。

3.4.2.7 针对casatwy那篇文章的一些回应

> 单纯以openURL的方式是无法胜任让一个App去实施组件化架构的

同意，所以我们并不只有 `openURL` 一种方式

> 根本无法表达非常规对象

单纯地通过 `openURL` 确实不太好表达，但我们并不只有 `openURL` 一种方式

> 注册URL的目的其实是一个服务发现的过程，在iOS领域中，服务发现的方式是不需要通过主动注册的，使用runtime就可以了。另外，注册部分的代码的维护是一个相对麻烦的事情，每一次支持新调用时，都要去维护一次注册列表。如果有调用被弃用了，是经常会忘记删项目的。runtime由于不存在注册过程，那就也不会产生维护的操作，维护成本就降低了。
>
> 由于通过runtime做到了服务的自动发现，拓展调用接口的任务就仅在于各自的模块，任何一次新接口添加，新业务添加，都不必去主工程做操作，十分透明。

尽管通过 runtime 可以做到这些，但最终还是要通过维护 `Category` 来暴露新增的 Target-Action，所以 runtime 虽然不存在注册过程，但实际使用过程中，还是会有注册过程，还是需要去维护。

> 没有拆分远程调用和本地间调用

从上面的图可以看到，我们其实是分为「组件间调用」和「页面间跳转」两个维度，只要 app 响应某个 URL，无论是 app 内还是 app 外都可以，而「组件间」调用走的完全是另一条路，所以也不会有安全上的问题。

> 以远程调用的方式为本地间调用提供服务

同上

> 本地间调用无法传递非常规参数，复杂参数的传递方式非常丑陋

同上，使用 Protocol

> 必须要在 app 启动时注册 URL 响应者

是的，就蘑菇街的方案来说，这步不可避免。

> 这个不必要的操作会导致不必要的维护成本

维护只是在组件内部做调整，并不需要在主工程里做修改。如果采用 Category 的方式，好处是不用在启动时注册，但当组件的接口有变动时，依然要维护 Category，这个成本是免不了的。

> 新增组件化的调用路径时，蘑菇街的操作相对复杂 在本文给出的组件化方案中，响应者唯一要做的事情就是提供Target和Action，并不需要再做其它的事情

提供了 Target-Action 之后，还是要在 Category 里添加一个 wrapper 的吧?

> 没有针对 target 层做封装 这种做法使得所有的跨组件调用请求直接hit到业务模块，业务模块必然因此变得臃肿难以维护，属于侵入式架构。应该将原本属于调用相应的部分拿出来放在target-action中，才能尽可能保证不将无关代码侵入到原有业务组件中，才能保证业务组件未来的迁移和修改不受组件调用的影响，以及降低为项目的组件化实施而带来的时间成本。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

「将原本属于调用相应的部分拿出来放在target-action中」并不是唯一可行的方式，使用 Protocol/URL 注册也可以达到效果。

3.2.4.8 小结

casatwy 的一些思路和思考问题的角度挺不错的，也从他的文章中收获了不少，希望这篇文章能把之前模糊的一些观念说得足够清楚，还有问题的话欢迎继续交流：）

3.5 爱奇艺 App

3.5.1 关于组件化

随着项目结构越来越庞大，模块与模块间的边界逐渐变得不清晰，代码维护越来越困难，甚至编译速度都成为影响开发效率的瓶颈。

组件化拆分是比较常见的解决方案，一方面解决模块间的耦合关系、将通用模块下沉，另一方面做到各模块代码和资源文件的隔离，这样便可以放心进行模块按需编译、单独测试等等。

但随之而来的问题也愈加突出，模块的精细化拆分不可避免的增加了模块间的通信成本。通信的两侧是一个C/S架构，如果服务端与客户端同属一个进程我们称之为本地服务，如果分属不同进程称之为远程服务。注意这里的服务不仅限于Android中的Service组件，而是一种可以对外提供功能或数据的能力。

对于同进程的通信比较简单，通过注册本地接口和实现就可以完成，如果你已经接入ARouter，直接声明服务类继承IProvider+Router注解就完成了服务的注册。

但是对于跨进程的通信就比较复杂了，在Android系统中IPC通信通过Binder实现，对参与通信的数据格式做了限制，也就是基本数据类型或者实现Parcelable接口的类型。

多进程的好处是可以占用更多的系统资源，并且独立核心进程可以免受非核心业务出现异常情况导致整个APP崩溃不可用。

跨进程通信业务场景比较复杂，既要保证服务端的可靠性，还需要支持callback，通常Service是首选。

3.5.2 基于Service的IPC通信

我们回想一下是如何使用Service进行跨进程通信的。

1. 声明提供服务的AIDL接口。
2. 创建Service，并在onBind方法返回实现Stub接口的Binder对象。
3. Client端通过intent bindService，并传入ServiceConnection对象，在onServiceConnected回调获取Service提供的Binder对象。

本质上是将Binder对象（准确的说是代理对象）在进程间进行传递，而Service只是一个载体。

在组件化的大业务背景下，模块间的通信接口数量可能很多，按这套方案会有很多问题。

1. 需要书写AIDL文件和Service类。
2. bindService是异步操作，需要写回调，与本地服务调用方式不统一。
3. 没用统一的Binder管理者，如何处理Binder Die，如何实现Binder缓存等问题。

这样我们可以总结出一个好的组件化通信框架需要具备特点或者说要实现的诉求。

3.5.3 组件化跨进程通信的核心诉求

- 可不可以不写AIDL文件，用声明普通接口类的方式声明一个远程服务接口；可不可以不写Service，因为IPC通信的本质只是传递Binder而已。
- 我们希望像调用本地服务一样调用远程服务，避免回调地狱，即远程服务的获取是阻塞式调用。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

- 如何管理各个进程提供的远程服务，保证高可用。

啰嗦了这么半天回到我们今天的主题Andromeda，文章有点长，希望你耐心阅读，一定有收获！

3.5.4 Andromeda

Andromeda是爱奇艺开源的组件化IPC通信解决方案，它解决了上述的问题2和3，同时不需要书写Service，但是仍需要写AIDL文件。

对于这个问题，饿了吗早前开源的 Hermes框架 可以做到，原理是利用动态代理+反射的方式来替换AIDL生成的静态代理，但是不支持oneway、in、out、inout等修饰符。

再后来，爱奇艺又开源 InterStellar ，实现了不需要书写AIDL文件，当使用跨进程接口时，声明@oneway/@in等注解完成IPC修饰符的添加。这样算是彻底的实现了**远程调用像本地调用一样简单**。但不知为何与Andromeda没有合并到一个项目中，工程代码也很久没有人维护。

此外**Andromeda**还有一些Feature：

- 加入了跨进程通信的事件总线，即跨进程版EventBus。
- 加入了对增强进程稳定性的考量，通过为各个进程预先插桩Service，在获取远程服务时用前台UI组件（Activity/Fragment/View）绑定插桩的Service，最终提升后台服务进程优先级。
- 支持IPCCallback。
- 支持配置Binder分发管理中心（Dispatcher）所属进程。

Andromeda Github地址

我们先来看一下简单的使用

```
//注册本地服务 第一个参数是接口class将来用作key，第二参数为接口实现类。
Andromeda.registerLocalService(ICheckApple.class, new CheckApple());
//使用本地服务
ICheckApple checkApple = Andromeda.getLocalService(ICheckApple.class);
------------------------------
//注册远程服务 第二个参数为IBinder类型，将来会在进程间传递
Andromeda.registerRemoteService(IBuyApple.class, BuyAppleImpl.getInstance());
//使用远程服务，传入UI组件(this)尝试提升远程服务进程的优先级
Andromeda.with(this).getRemoteService(IBuyApple.class);
复制代码
```

整体API的设计清晰且全部都是同步完成，详细使用见工程示例，本篇的重点是分析内部原理。

虽然是源码分析，但我不准备贴过多的源码，这样阅读体验并不好；我会尽量克制，真正有需求的小伙伴请自行查阅源代码，我的目标是把核心思想讲清楚。

3.5.5 架构分析

我们先理清几个概念，无论是事件总线还是服务分发都需要一个中转存储中心，这个中心在Andromeda框架中叫Dispatcher。

3.5.5.1 Dispatcher

它是一个AIDL接口，各个进程在注册服务时需要首先拿到DispatcherProxy，然后将本进程服务Binder传送给DispatcherProxy存储，当其他进程需要使用该服务时，也需要先获取一个DispatcherProxy，然后读取DispatcherProxy中的缓存Binder，并在自己进程存储一份缓存，这样本进程下次获取相同的服务时就不需要进行IPC调用了。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

我们来看一下Dispatcher提供了哪些功能。

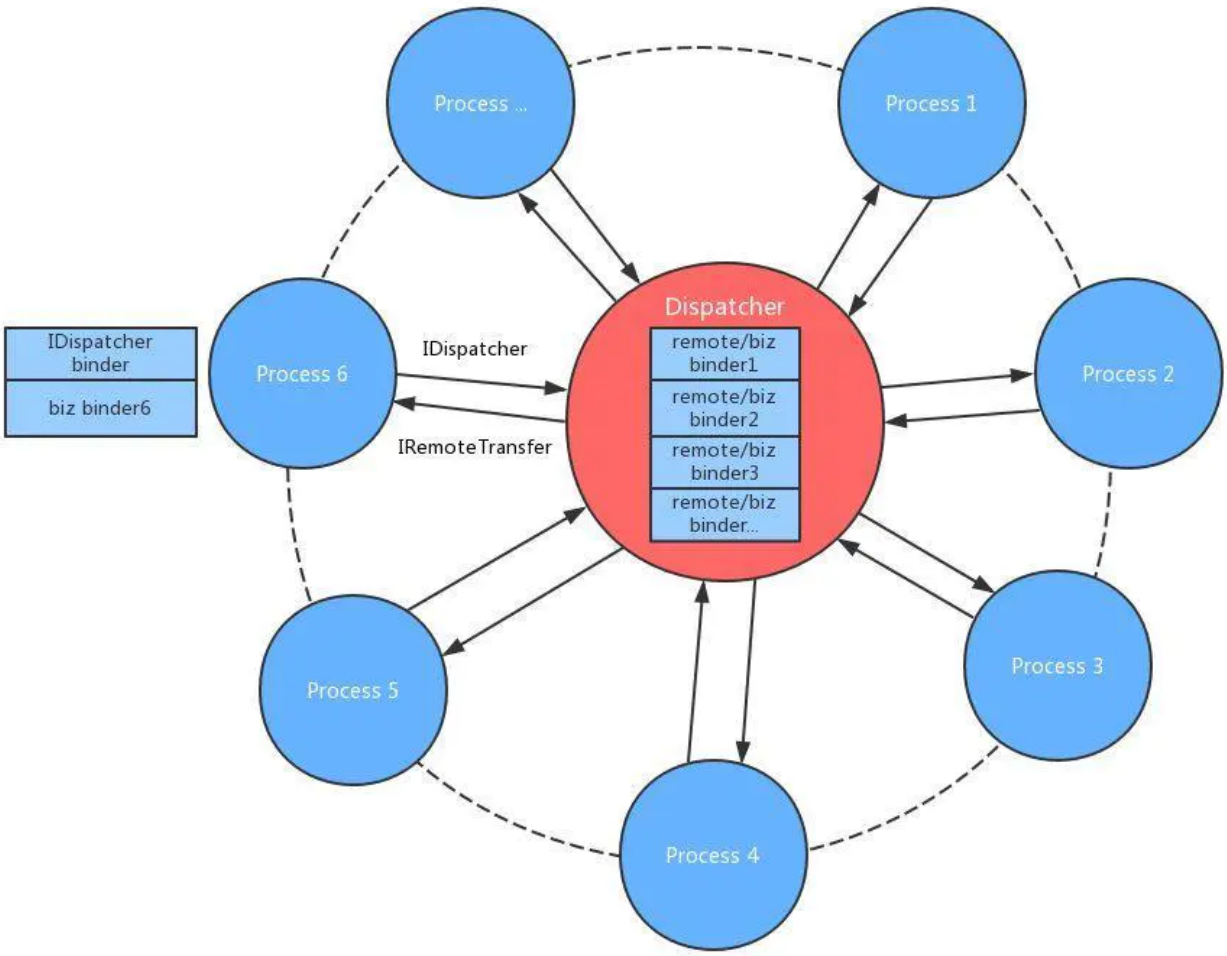
```
# IDispatcher.aidl
interface IDispatcher {
   //通过服务名称获取Binder包装类BinderBean
   BinderBean getTargetBinder(String serviceCanonicalName);
   //保留接口暂时为空实现
   IBinder fetchTargetBinder(String uri);
   //注册本地的RemoteTransfer
   void registerRemoteTransfer(int pid,IBinder remoteTransferBinder);
   //注册/反注册远程服务
   void registerRemoteService(String serviceCanonicalName,String processName,IBinder
Binder);
   void unregisterRemoteService(String serviceCanonicalName);

   //发送事件
   void publish(in Event event);
}
复制代码
```

Dispatcher所在进程可以是主进程也可以用户自定义的进程，为什么要讨论Dispatcher所属进程呢？因为作为组件化通信核心的Center一旦狗带，将导致之前注册服务不可用，所以需要将它放在应用生命周期最长的进程中，通常这个进程是主进程，但对于类似音乐播放器相关的app来说，可能是一个独立的播放器进程，所以框架为我们提供了一个配置项可以显式的声明Dispatcher所在进程。

```
#主工程的build.gradle添加声明
dispatcher{
    process ":downloader"
}
复制代码
```

Dispatcher架构图

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



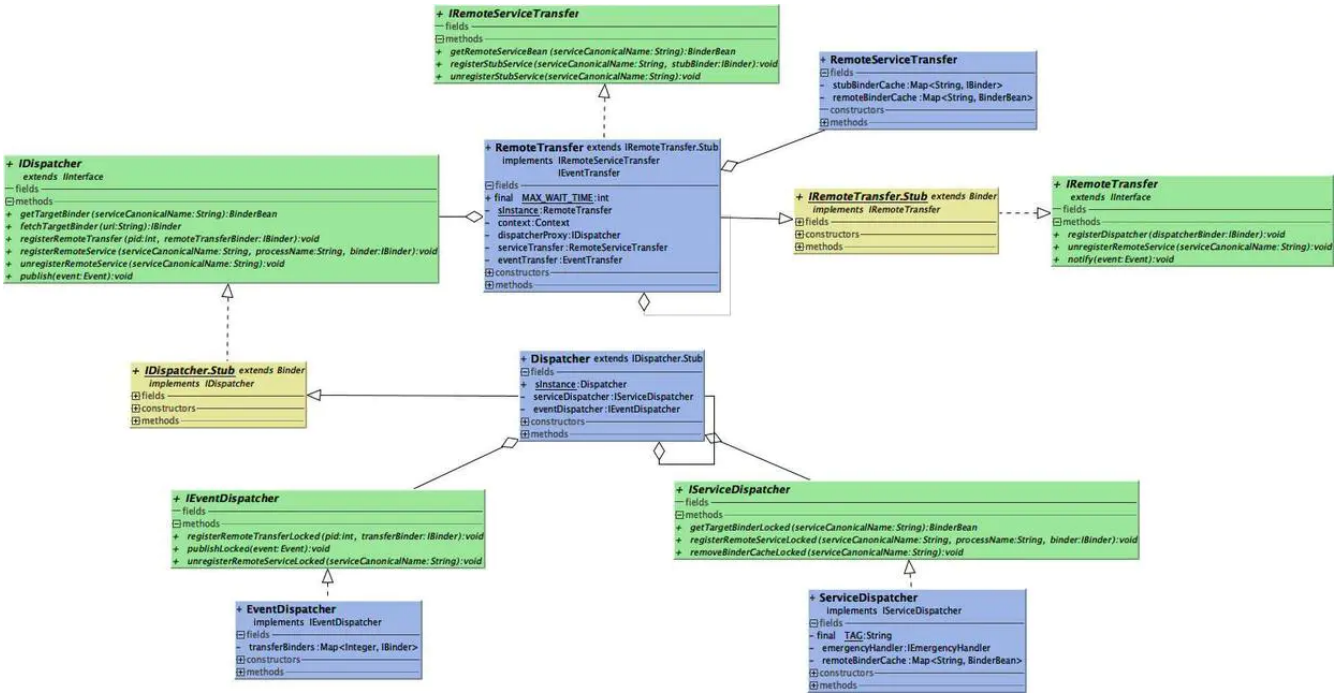
3.5.5.2 RemoteTransfer

上面提到各个进程自己本身也需要管理（缓存）从Dispatcher获取的Binder，防止重复的IPC请求；另外由于事件总线的需求，各个进程需要向Dispatcher进程注册本进程组件管理员，这样当事件pubish后，Dispatcher才能将事件发送给各个进程，这个各个进程管理员就是RemoteTransfer。

IRemoteTransfer是一个AIDL接口，RemoteTransfer是它的实现类，RemoteTransfer还实现了IRemoteServiceTransfer接口。

这里需要一张类图来帮你理清思路：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?tid=8674704361863085




```
#IRemoteTransfer.aidl
interface IRemoteTransfer {
    //① 将Dispatcher代理返回给RemoteTransfer
    oneway void registerDispatcher(IBinder dispatcherBinder);

    oneway void unregisterRemoteService(String serviceCanonicalName);

    oneway void notify(in Event event);
}

#IRemoteServiceTransfer.java
public interface IRemoteServiceTransfer {
    //②获取远程服务包装
    BinderBean getRemoteServiceBean(String serviceCanonicalName);

    //注册/反注册 远程服务
    void registerStubService(String serviceCanonicalName, IBinder stubBinder);
    void unregisterStubService(String serviceCanonicalName);
}
复制代码
```

两个问题需要注意：

① 方法的调用方在Dispatcher中，这样就把Dispatcher的远程代理回传给了当前进程，之后注册远程服务就可以通过这个DispatcherProxy完成。

② 无论是注册还是获取远程服务，都是不能直接传递Binder的，因为Binder并没有实现Parcelable接口，因此需要将Binder包装在一个实现了Parcelable接口的类中传递，BinderBean就是其中一个包装类。

主体逻辑已经讲清楚了，我们正式开始分析功能。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

- 通过ContentProvider方式同步的获取Dispatcher的代理，这个ContentProvider属于Dispatcher进程，且通过插桩的方式织入manifeset文件。
- 获取远程服务时传递当前进程的Activity或Fragment，并bind预先插桩好的StubService，这个StubService属于远程服务所在进程。

这是整个Andromeda工程最最核心的原理，你是不是快看不懂了，没关系，下面会结合时序图、关系图详细分析实现过程。

3.5.5.3 本地服务

本地服务没什么讲的，内部通过维护一个Map关系表，来记录注册服务的名称和实现类。

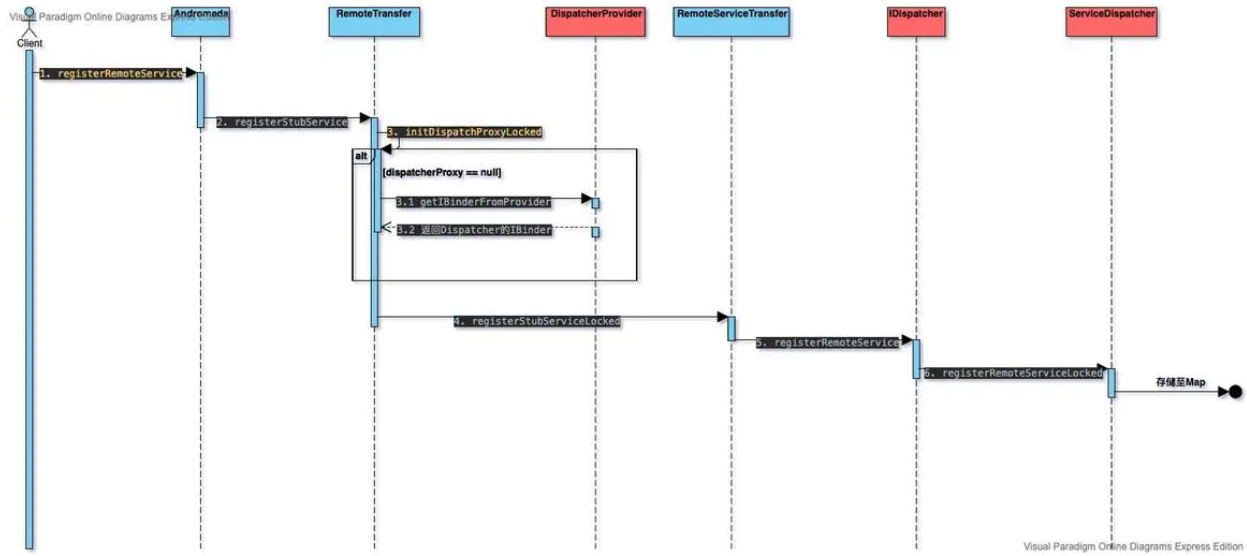
```
# LocalServiceHub
public class LocalServiceHub implements ILocalServiceHub {
    private Map<String, Object> serviceMap = new ConcurrentHashMap<>();

    @Override
    public Object getLocalService(String module) {
        return serviceMap.get(module);
    }

    @Override
    public void registerService(String module, Object serviceImpl) {
        serviceMap.put(module, serviceImpl);
    }

    @Override
    public void unregisterService(String module) {
        serviceMap.remove(module);
    }
}
复制代码
```

3.5.5.4 远程服务

远程服务是框架的核心，对远程服务的操作就是两个，一是注册远程服务，二是获取远程服务。

我们先来看服务的注册，时序图如下↓

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



1. 客户端通过`<T extends IBinder> registerRemoteService(String serviceCanonicalName, T stubBinder)`注册本进程可提供的远程服务，stubBinder即服务实现类。
2. 调用RemoteTransfer的registerStubService方法。
3. registerStubService内部先初始化DispatcherProxy，如果为空跳转3.1。
    - 3.1-3.2 要实现服务的同步注册，本质上是同步获取DispatcherProxy，这是一次IPC通信，Andromeda的方案是在Dispatcher进程插桩一个ContentProvider，然后返回一个包含DispatcherProxy的Cursor给客户端进程，客户端解析Cursor拿到DispatcherProxy。
4. RemoteTransfer请求RemoteServiceTransfer帮忙完成真正的注册。
5. RemoteServiceTransfer通过第3步获取的DispatcherProxy，做一次IPC通信，将Binder传递到Dispatcher进程。
6. Dispatcher进程请求ServiceDispatcher类帮忙完成服务的注册，其实就是将Binder存储在一个Map当中。

图中蓝色的节点表示注册服务的当前进程，也就是Server进程，红色节点表示Dispatcher进程。

整个过程重点在第三步，我们再重点分析一下：

```
# RemoteTransfer
private void initDispatchProxyLocked() {
    if (null == dispatcherProxy) {
        //从contentprovider取Binder
        IBinder dispatcherBinder = getIBinderFromProvider();
        if (null != dispatcherBinder) {
            //取出后asInterface创建远程代理对象
            dispatcherProxy = IDispatcher.Stub.asInterface(dispatcherBinder);
            registerCurrentTransfer();
        }
    }
    ...
}

private void registerCurrentTransfer() {
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
    //向Dispatcher注册自己这个进程的RemoteTransfer Binder
    dispatcherProxy.registerRemoteTransfer(android.os.Process.myPid(), this.asBinder());
    ...
}

private IBinder getIBinderFromProvider() {
    Cursor cursor = null;
    try {
        //通过contentprovider拿到cursor
        cursor = context.getContentResolver().query(getDispatcherProviderUri(),
DispatcherProvider.PROJECTION_MAIN,
                null, null, null);
        if (cursor == null) {
            return null;
        }
        return DispatcherCursor.stripBinder(cursor);
    } finally {
        IOUtils.closeQuietly(cursor);
    }
}
```

复制代码

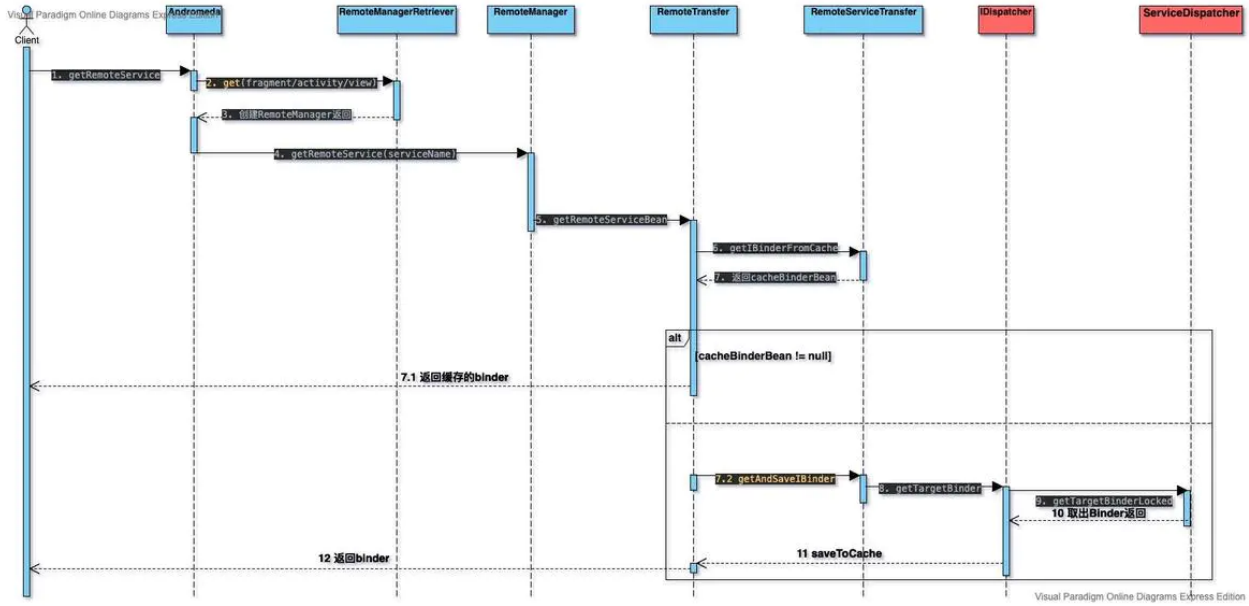
我们来看这个DispatcherProvider

```
public class DispatcherProvider extends ContentProvider {
    ...
    @Override
    public Cursor query(Uri uri, String[] projection, String selection, String[]
selectionArgs, String sortOrder) {
        //将Binder封装到cursor中返回
        return DispatcherCursor.generateCursor(Dispatcher.getInstance().asBinder());
    }
}
```

复制代码

接下来我们看服务的获取，同样的先看时序图↓

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



1. Andromeda入口通过getRemoteService获取远程服务。

2-4. 与提升进程优先级有关，我们暂且不讨论。

5. 向RemoteTransfer请求获取远程服务的包装bean。

6-7. RemoteTransfer请求RemoteServiceTransfer帮忙先从本进程的缓存中查找目标Binder，如果找到直接返回。

7.2. 如果没有命中缓存调用getAndSaveIBinder方法，通过方法名可知，获取后会将Binder缓存起来，这就是6-7步读取的缓存。

8. RemoteServiceTransfer通过DispatcherProxy发起IPC通信，请求远程服务Binder。

9-10. Dispatcher请ServiceDispatcher帮忙查找进程中的服务注册表。

11. 回到客户端进程将Binder缓存。

12. 将Binder返回给调用方。

同样图中蓝色的节点表示获取服务的进程，也就是Client进程，红色节点表示Dispatcher进程。

至此，远程服务的注册与获取流程分析结束。

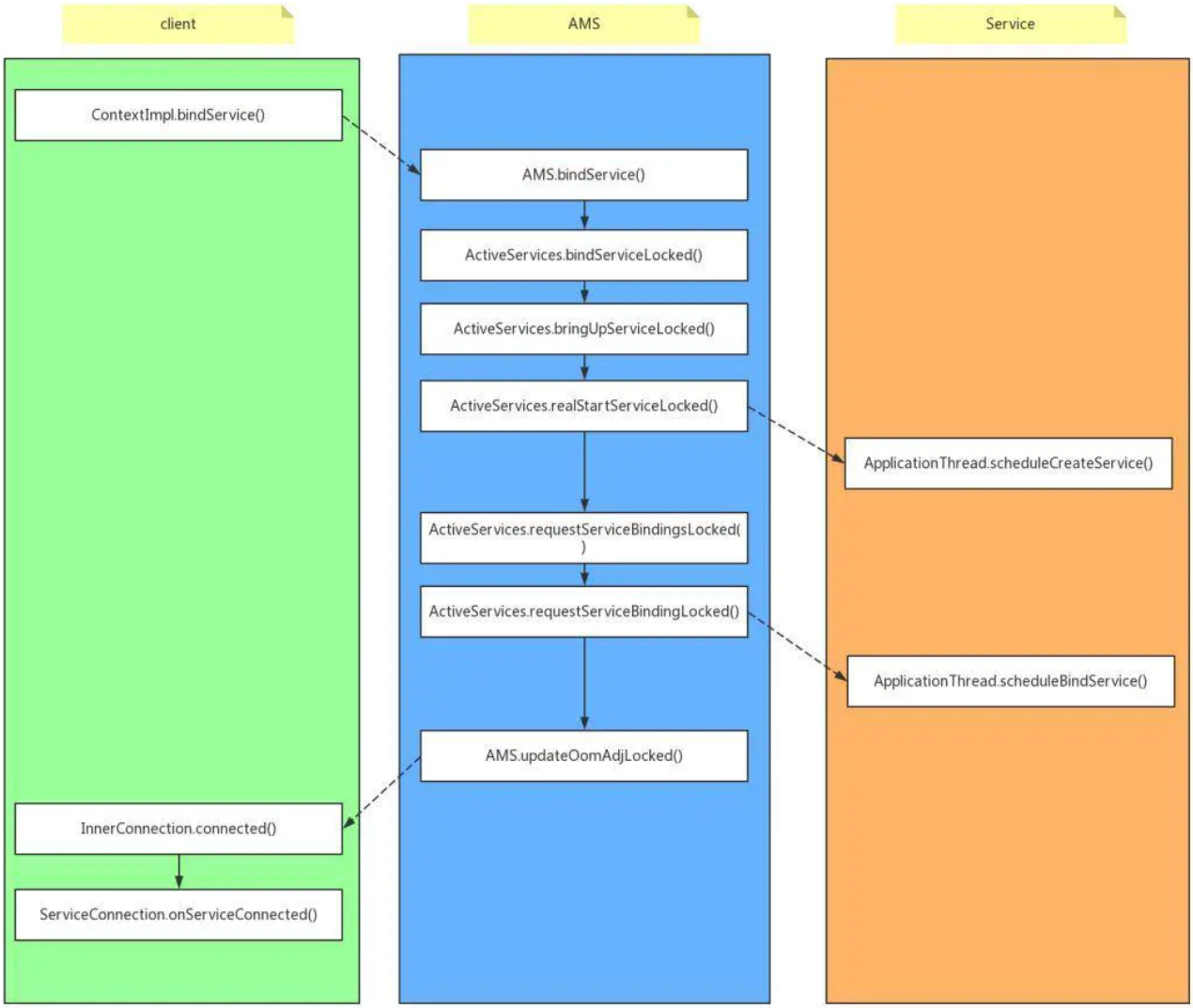
3.5.5.6 进程优先级

上面提到在获取远程服务时，框架做了提升进程优先级的事情。通常情况下使用远程服务的端（简称Client端）处于前台进程，而Server端进程已经注册完毕，往往处于后台。为了提升Server端的稳定性，最好能将Server端的进程优先级与Client保持接近，否则容易出现被LMK（Low Memory Killer）回收的情况。

那如何提升Server端进程的优先级呢？这里的做法是用前台的UI组件（Activity/Fragment/View）bind一个Server端预先插桩好的Service。

整套流程最终通过AMS的updateOomAdjLocked方法实现。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



回到Andromeda实现，这个预先插桩的Service如下：

```
public class CommuStubService extends Service {

    public CommuStubService() {}

    @Override
    public IBinder onBind(Intent intent) {
        return new ICommuStub.Stub() {
            @Override
            public void commu(Bundle args) throws RemoteException {
                //do nothing now
            }
        };
    }

    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        //这样可以使Service所在进程的保活效果好一点
```

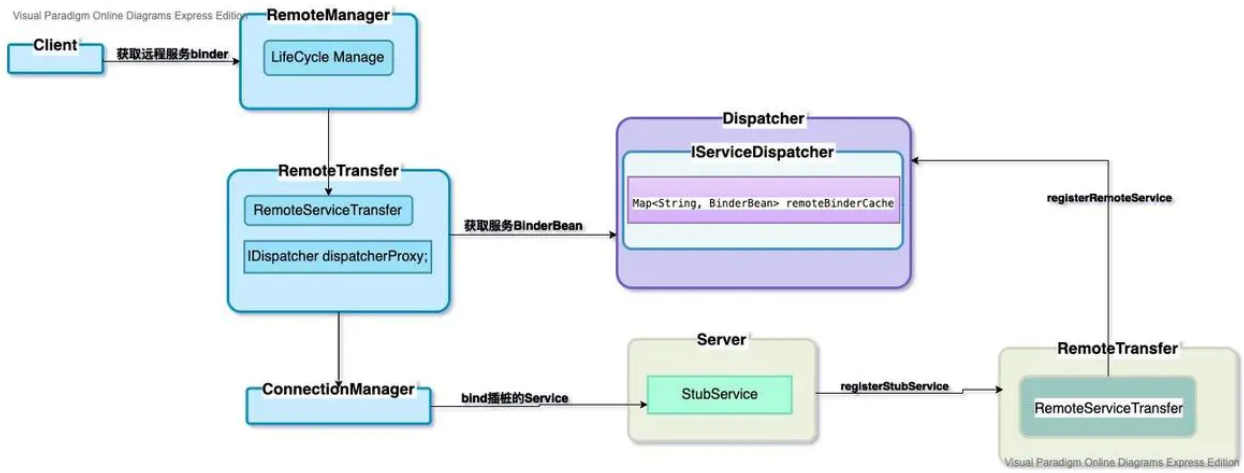
腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
        return Service.START_STICKY;
    }

    public static class CommuStubService0 extends CommuStubService {}
    public static class CommuStubService1 extends CommuStubService {}
    public static class CommuStubService2 extends CommuStubService {}
    ...
    public static class CommuStubService14 extends CommuStubService {}
}
```

复制代码

可见框架预置了15个Service供进程使用，也就是最多支持15个进程，这绝大数场景下足够了；另外维护了一个进程名和Service名称的映射表，否则怎么知道应该bind那个Service，这个映射表也是在编译阶段插桩完成的。

这个service的bind过程发生在上一章节获取远程服务时，流程如下图：



图中模块根据所在进程分为三部分：

1. 蓝色表示Client进程，发起获取远程服务请求。
2. 浅灰色表示Server进程，它事先将服务注册到Dispatcher中。
3. 紫色表示Dispatcher进程，内部缓存了各个进程的服务的Binder对象。

我们重点关注的是蓝色模块ConnectionManager部分，实际上当Client向Dispatcher请求远程服务之后，会立即通过ConnectionManager绑定这个远程服务所在进程的插桩的StubService，如此一来就提升了Server所在进程的优先级。

至此bind操作已经完成了，那何时unbind呢？显然是当UI组件销毁时，因为此时已不在前台，需要降低进程优先级。

如此一来就需要监听UI组件的生命周期，在onDestroy时进行unbind操作。

这就是图中RemoteManager做的事情，它内部维护了前台组件的生命周期。Andromeda提供了几种with方法，用于获取对应RemoteManager：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
public static IRemoteManager with(android.app.Fragment fragment) {return
getRetriever().get(fragment);}
public static IRemoteManager with(Fragment fragment) {return getRetriever().get(fragment);}
public static IRemoteManager with(FragmentActivity fragmentActivity) {return
getRetriever().get(fragmentActivity);}
public static IRemoteManager with(Activity activity) {return getRetriever().get(activity);}
public static IRemoteManager with(Context context) {return getRetriever().get(context);}
public static IRemoteManager with(View view) {return getRetriever().get(view);}
复制代码
```

这是借鉴Glide的做法，这些方法最终被转换为两类：

1. 具备生命周期的UI组件，最终是Activity或Fragment。
2. ApplicationContext。

对于第一种情况，框架会为当前Activity或Fragment添加一个不可见的RemoteManagerFragment以监听生命周期。

对于使用ApplicationContext，获取远程服务的场景不做unbind操作。

事实上用Jetpack lifecycle组件也可以方便的监听Activity/Fragment的生命周期，但是这有个前提，那就是Activity必须继承android.support.v4.app.FragmentActvity，而Fragment必须继承 android.support.v4.app.Fragment，且v4库的版本必须大于等于26.1.0，从这个版本开始支持了Lifecycle。

3.5.5.7 事件总线

在上述通信框架基础之上，实现事件总线简直易如反指。

我们来看一下使用

```
//订阅事件，这里MainActivity实现了EventListener接口
Andromeda.subscribe(EventConstants.APPLE_EVENT,MainActivity.this);

//发布事件
Bundle bundle = new Bundle();
bundle.putString("Result", "gave u five apples!");
Andromeda.publish(new Event(EventConstants.APPLE_EVENT, bundle));
复制代码
```

这里的Event是事件传递的载体。

```
public class Event implements Parcelable {
    private String name;
    private Bundle data;
    ...
}
复制代码
```

至于原理，回想一下我们在注册远程服务的过程中，同时将本进程的RemoteTransfer的Binder也注册到了Dispatcher中。

当我们订阅一个事件时，只是将Event名称和监听器存储在了本进程的RemoteTransfer中，当另一个进程发布事件时，会通过一次IPC调用将Event对象发送到Dispatcher，Dispatcher收到事件后，会向注册过的RemoteTransfer依次发送回调信息，也就是说这一步可能进行多次IPC调用，效率问题需diss一下。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

事件到达订阅进程后会根据事件名称，提取所有关于此名称的监听器，最终发送给监听者。

注意：**这里的监听器常常使用的是Activity，但显然RemoteTransfer是属于进程生命周期的，因此保存监听器时需使用弱引用。**

3.5.5.8 插桩

上面分析原理过程中反复提到了插桩，总结一下共有几处：

1. 将属于Dispatcher进程的DispatcherProvider和DispatcherService插入到manifest中（StubServiceGenerator）。
2. 将各个进程的预置StubService插入到manifest中（StubServiceGenerator）。
3. 将进程名与StubService的关系表插入到StubServiceMatcher类的map中（StubServiceMatchInjector）。

对于manifest的操作，框架内提供了不少工具方法，比如获取所有声明的进程，值得好好学习一下；对于class的操作使用的是javasisst，这在之前的AOP文章中也介绍过，感兴趣的同学自行查阅。

在读源码过程中发现两个值得关注的问题：

一是DispatcherProvider伪造的DispatcherCursor继承MatrixCursor，它通常用于返回几条固定的已知记录，不需要从数据库查询这种场景。

二是跨进程传递bundle对象时，如果bundle中存放了parcelable对象需要手动设置setClassLoader。

```
#DispatcherCursor
public static IBinder stripBinder(Cursor cursor) {
    if (null == cursor) {
        return null;
    }
    Bundle bundle = cursor.getExtras();
    //从cursor中取出bundle需要设置classLoader
    bundle.setClassLoader(BinderWrapper.class.getClassLoader());
    BinderWrapper BinderWrapper = bundle.getParcelable(KEY_Binder_WRAPPER);
    return null != BinderWrapper ? BinderWrapper.getBinder() : null;
}
复制代码
```

因为默认情况下bundle传输使用的ClassLoader是BootClassLoader，而BootClassLoader只能加载系统类，我们本工程的class需要使用PathClassLoader进行加载，因此需要额外的调用bundle的setClassLoader方法设置类加载器，详见Bundle.setClassLoader()方法解析 。

3.5.5.9 缺点

- 服务需要手动注册，这个时机不好把握。最好能提供一个自动注册服务的开关，上层不需要关注服务的注册。
- 发送一次事件需要多次IPC调用效率低，有优化空间。
- 仍需要书写AIDL文件。

至此，Andromeda核心的原理我们就分析完了，虽然有些问题有待完善，但已经给我们提供了很多优秀的解决问题的思路，无论是继续优化还是精简一下本地化都是不错的选择。

3.6 美团 App

- 设计了组件消息总线 modular-event

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

- 设计了组件化路由框架 WMRouter

3.6.1外卖客户端容器化架构的演进

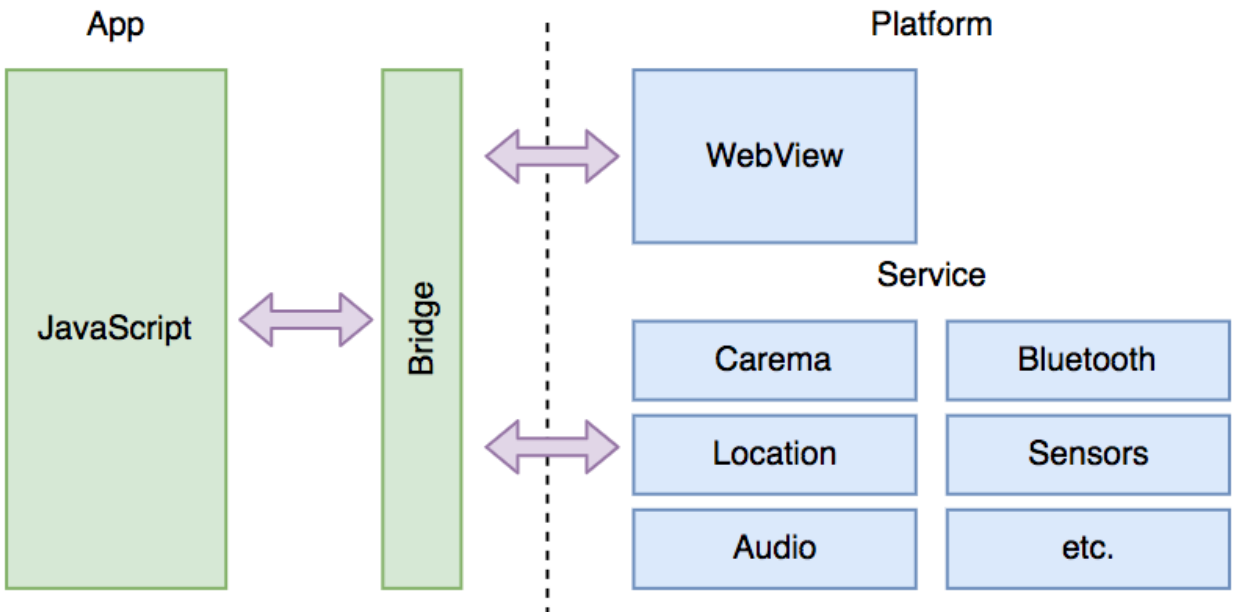
3.6.1.1 背景

1.1 移动端跨平台技术的介绍

移动端的跨平台技术不是一个新话题，早在几年前，WebView容器、React Native、Weex、Flutter、小程序等移动端跨平台框架就风起云涌。为什么跨平台这么有吸引力呢？我们设想一下如果可以做到一次开发，多端复用，那么对于公司来说，就可以降低用人成本。对于开发来说，只需要学习一个框架，就可以在Android和iOS双平台上开发。节约下来的成本，可以投入到产品快速验证、快速上线。这对所有人来说都有着极大的吸引力。本节先针对部分移动端跨平台技术进行一些简要的介绍，以便读者能够更好地理解后面的内容。

1.1.1 WebView容器

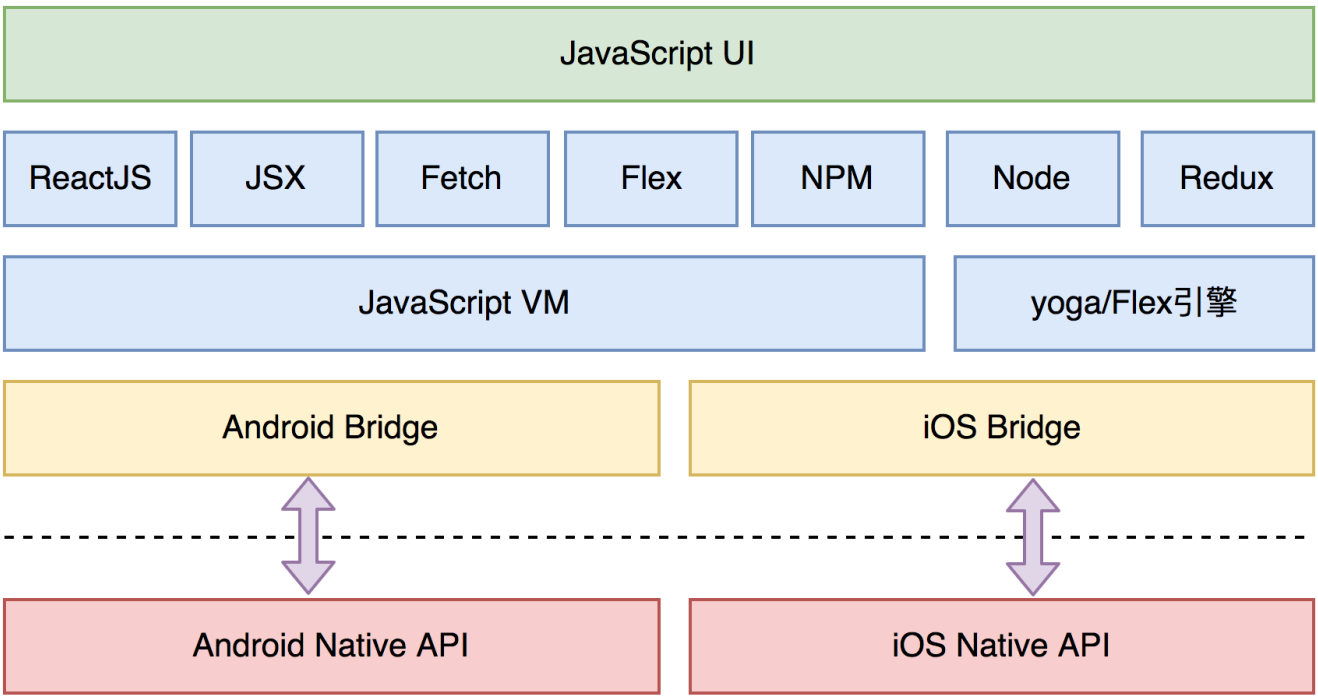
WebView容器的工作原理是基于Web技术来实现界面和功能，通过将原生的接口封装、暴露给JavaScript调用，JavaScript编写的页面可以运行在系统自带的WebView中。这样做的优势是，对于前端开发者比较友好，可以很快地实现页面跨端，同时保留调用原生的能力，通过搭建桥接层和原生能力打通。但这种设计，跨端的能力受限于桥接层，当调用之前没有的原生能力时，就需要增加桥。另外，浏览器内核的渲染独立于系统组件，无法保证原生体验，渲染的效果会差不少。



1.1.2 React Native

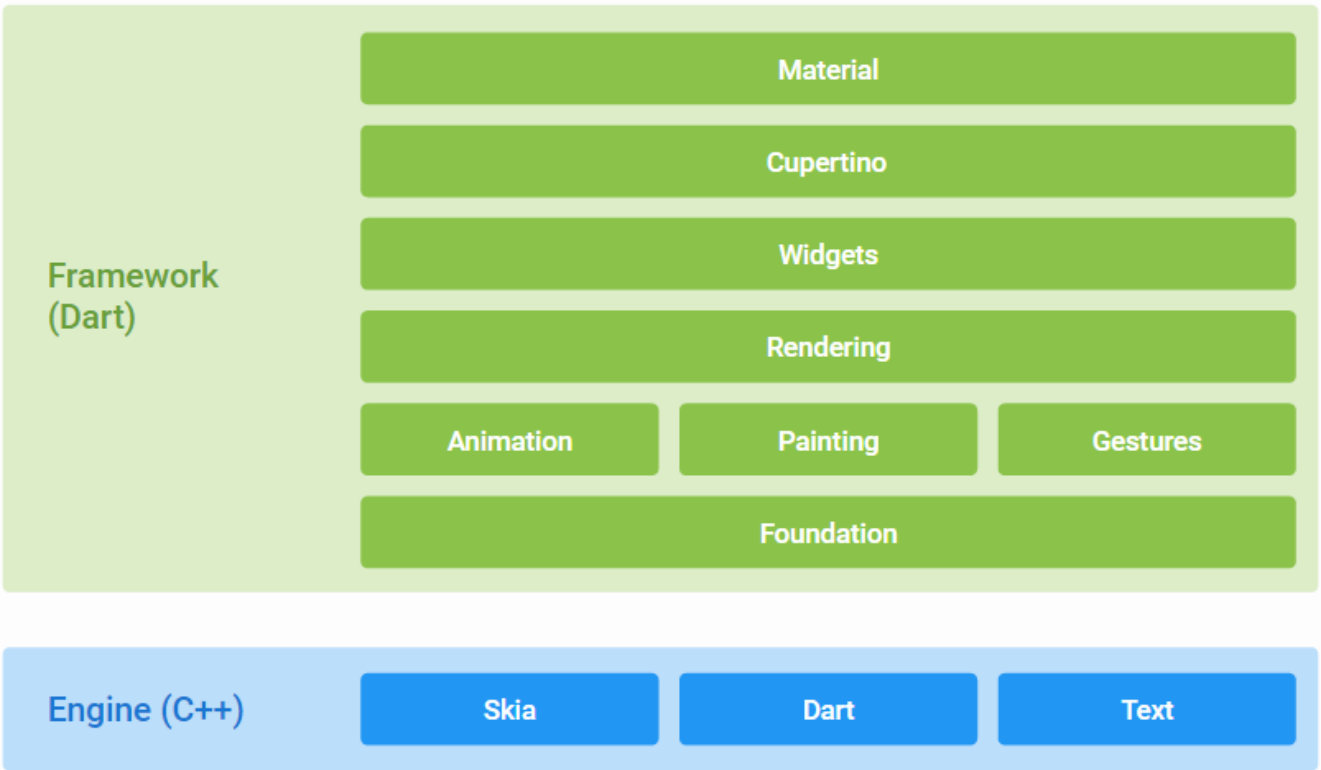
2015年，Facebook推出了React Native，一经推出就备受关注。它的思路是最大化地复用前端的生态和Native的生态，和WebView容器的最大区别在于View的渲染体系。React Native抛弃了低效的浏览器内核渲染，转而使用自己的DSL生成中间格式，然后映射到对应的平台，渲染成平台的组件。相对WebView容器，体验会有一定的提升。不过，渲染时需要JavaScript和原生之间通信，在有些场景可能会导致卡顿。另外就是，渲染还是在Native层，要求开发人员对Native有一定的熟悉度。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



1.1.3 Flutter

2018年Google推出Flutter，通过Dart语言构建一套跨平台的开发组件，所有组件基于Skia引擎自绘，在性能上可以和Native平台的View相媲美。Flutter站在前人的肩膀上，参考了React的状态管理、Web的自绘制UI、React Native的HotReload等特点，同时考虑了与Native通信的Channel机制、自渲染、完备的开发工具链。Flutter与上述Recat Native、WebView容器本质上都是不同的，它没有使用WebView、JavaScript解释器或者系统平台自带的原生控件，而是有一套自己专属的Widget，底层渲染使用自身的高性能C/C++ 引擎自绘。但大部分移动端发展到今天，都已经形成了自己的架构，在现有基础上加上Flutter，会形成原有架构和Flutter双平台共存的问题。目前，对新的App来说，是最被看好的跨端方案。



3.6.1.2 美团外卖业务介绍

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

作为中国领先的生活服务电子商务平台，美团致力于用科技连接消费者和商家，提供服务以满足人们日常“吃”的需求，并进一步扩展至多种生活和旅游服务。而作为公司最为重要的业务之一，美团外卖从2013年创建以来，已经从单一的品类扩展到附近美食、水果、蔬菜、超市、鲜花、蛋糕等多品类，从早午晚餐，发展到下午茶、宵夜，中餐、西餐、家常菜、小吃、快餐、海鲜、火锅、川菜、蛋糕、烤肉、水果、饮料、甜点等多种类餐饮。美团外卖可以说是当前电商领域，最为复杂的业务之一。

业务的复杂，给系统架构也带来了不小的挑战。美团外卖业务之所以说是当前电商领域最为复杂的业务，主要源于以下几点特征：

- 美团外卖业务承载在三个App上，美团外卖App、美团App外卖频道、点评App外卖频道。
- 美团外卖作为美团公司重要的用户入口，还承担着流量平台的作用，提供平台能力支撑频道业务的发展，如闪购、跑腿、金融等。
- 美团外卖作为已经成熟的业务，需提供可复用的平台能力，支撑新业务的发展，例如菜大全App的发展。
- 美团外卖作为一个超级业务方，业务内又运营多个方向业务，如流量业务、交易业务、商家业务、商品业务、营销业务、广告业务等。

综上所述，可以发现美团外卖不仅仅自身业务比较复杂，而且对外的角色也很复杂。在美团内部，外卖不仅仅是美团平台的一个频道业务，而且自己本身也是一个平台业务，同时美团外卖还承担着新业务发展的平台角色。这意味着想要支持好美团外卖业务的发展是一件非常有挑战的事情。

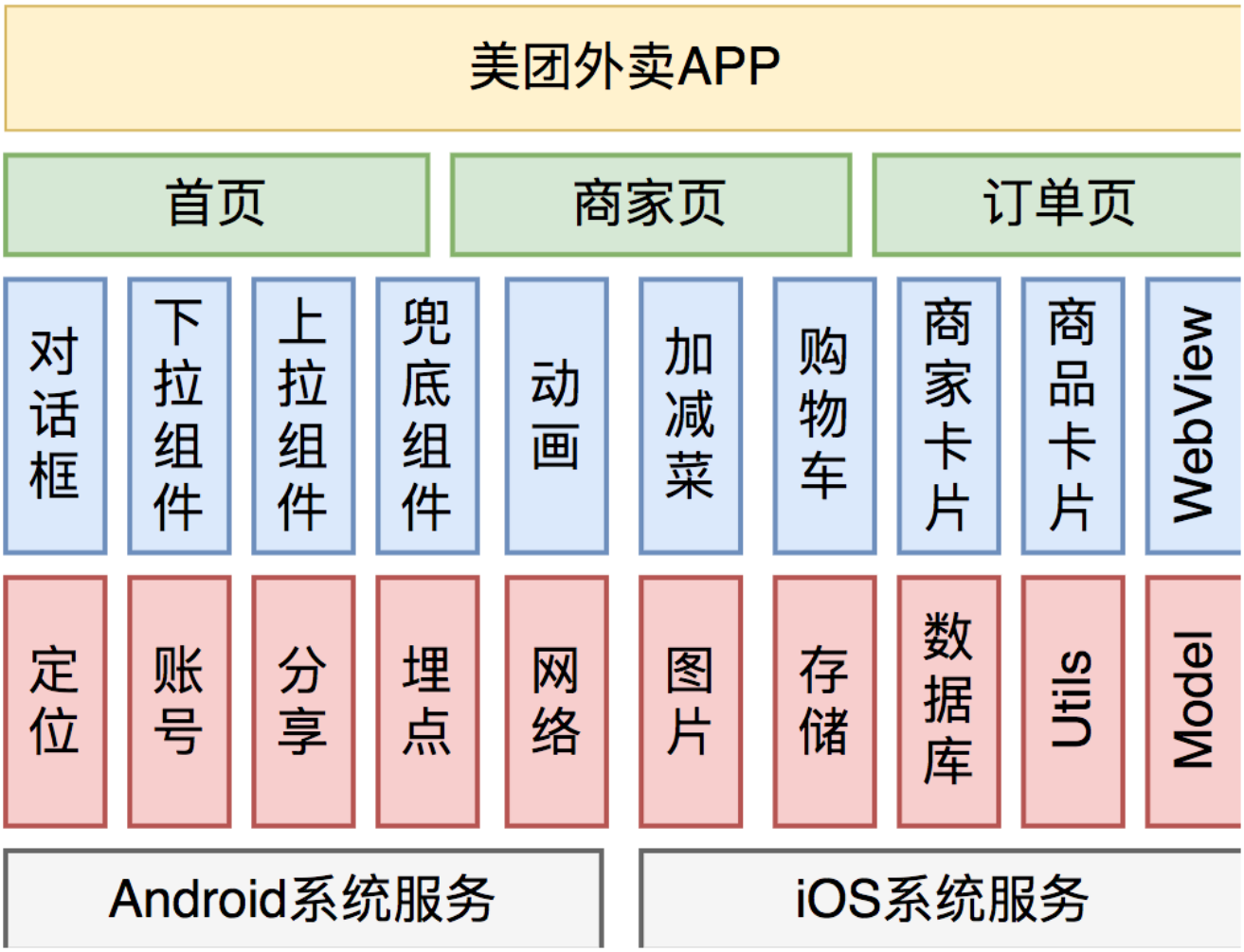
3.6.1.3 美团外卖移动端历史架构概述

好的架构源于不停地衍变而非设计。美团外卖的架构，历史上也是经历了很多次迭代。由于外卖业务形态不断地发生变化，原有的设计也需要不断地跟随业务形态进行演进。在不断探索和实践过程中，我们经历了若干个大的架构变迁。从考虑如何高效地复用代码支持外卖App，逐渐地衍变成如何去解决多端代码复用问题，再从多端的代码复用到支持其他频道业务的平台架构上。在平台化架构建设完成后，我们又开始尝试利用动态化技术去支持业务快速上线的诉求。如今，我们面临着多端复用、平台能力、平台支撑、单页面多业务团队、业务动态诉求强等多个业务场景问题。下文我们针对美团外卖移动端架构的变迁史，做一些简单的概述，以便读者阅读本文时能有更好的延续性。

1.3.1 组件化架构

早期阶段，美团外卖作为公司的一个孵化业务，在2013年底完成了美团外卖App的1.0版本。随着外卖业务的验证成功和跑通，订单量也快速增长，在2014年底突破了日订单量100万。随后在2015年2月，外卖以Native的形式接入美团App，成为美团App的一个业务频道。在接入过程中，我们从美团外卖App拷贝了大量的代码到美团App的外卖频道，两个App上的外卖业务代码也分别由两个独立的团队维护。早期外卖业务变化快，App迭代频繁，写代码的方式也比较粗放，同时美团App也处在一个平台化转变的时期，代码的稳定性和质量都在变化和提升当中。这些因素导致了外卖代码内各子系统之间耦合严重，边界模糊，“你中有我，我中有你”的情况随处可见。这对代码质量、功能扩展以及开发效率都造成很大的影响。此时，我们架构重构的目的，就是希望将各个子系统划分为相对独立的组件，建设组件可以直接复用，架构如下图所示：

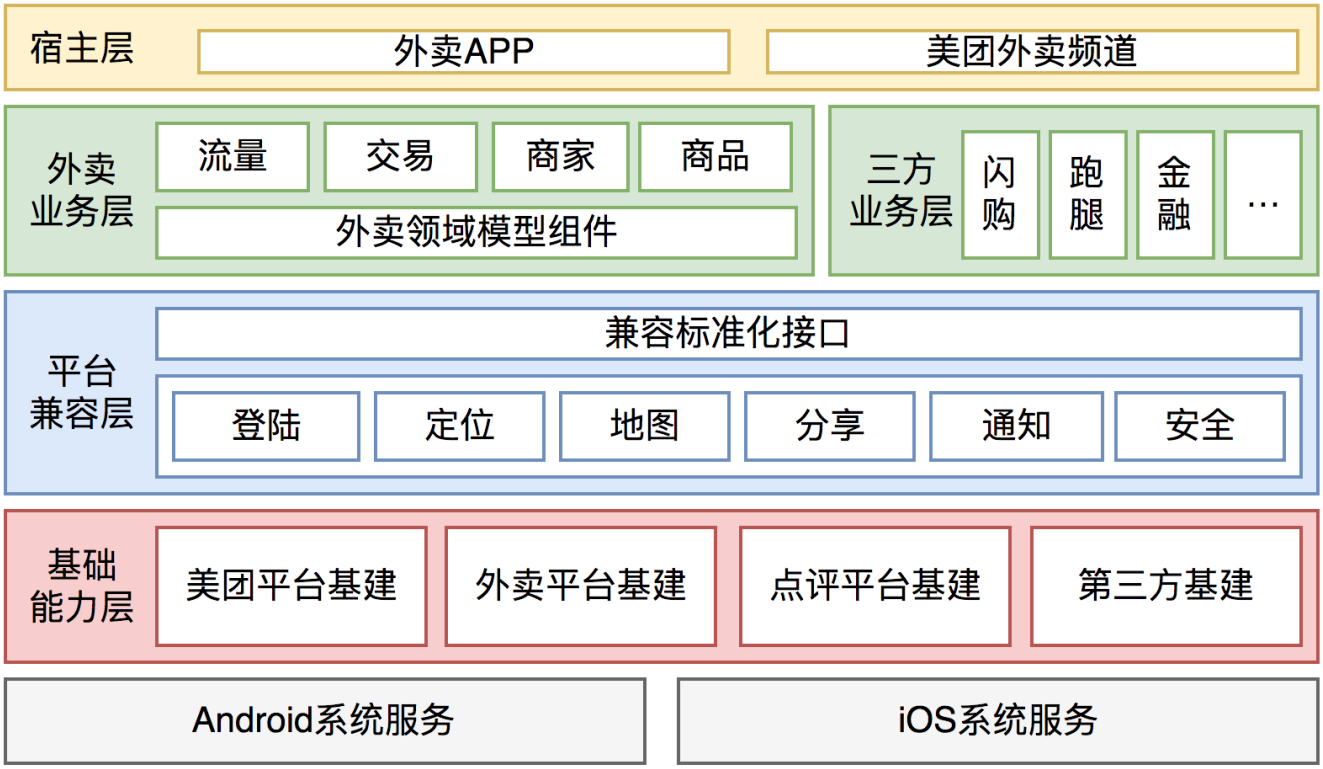
腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



1.3.2 平台化架构

如上文所述，大家可以知道美团外卖和美团外卖频道是由不同的团队在维护发展。2015年，公司考虑到业务发展的一致性，将美团外卖频道团队正式归于美团外卖。从组织架构上来说，美团外卖和美团外卖频道，逐渐融合成一个团队，但是两端的差异性，导致我们不得不仍然阶段性地维持原有的两班人马，各自去维护独立外卖App和美团外卖频道。如何解决这个问题？两端代码复用看起来是唯一的途径。另外，随着业务的快速发展，外卖App所承载的业务模块越来越多，产品功能越来越复杂，团队规模也越来越大，如闪购、跑腿等业务想以独立的Native包的形态接入外卖App，还有外卖的异地研发团队的建立，都带来了挑战。这使得我们在2017年开始了第二次架构重构——平台化架构，目标是希望能够支持多端复用和支持不同团队的业务发展。通过抽象出平台能力层、业务解耦、建立壳容器，最终实现了平台化架构，架构如下图所示：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



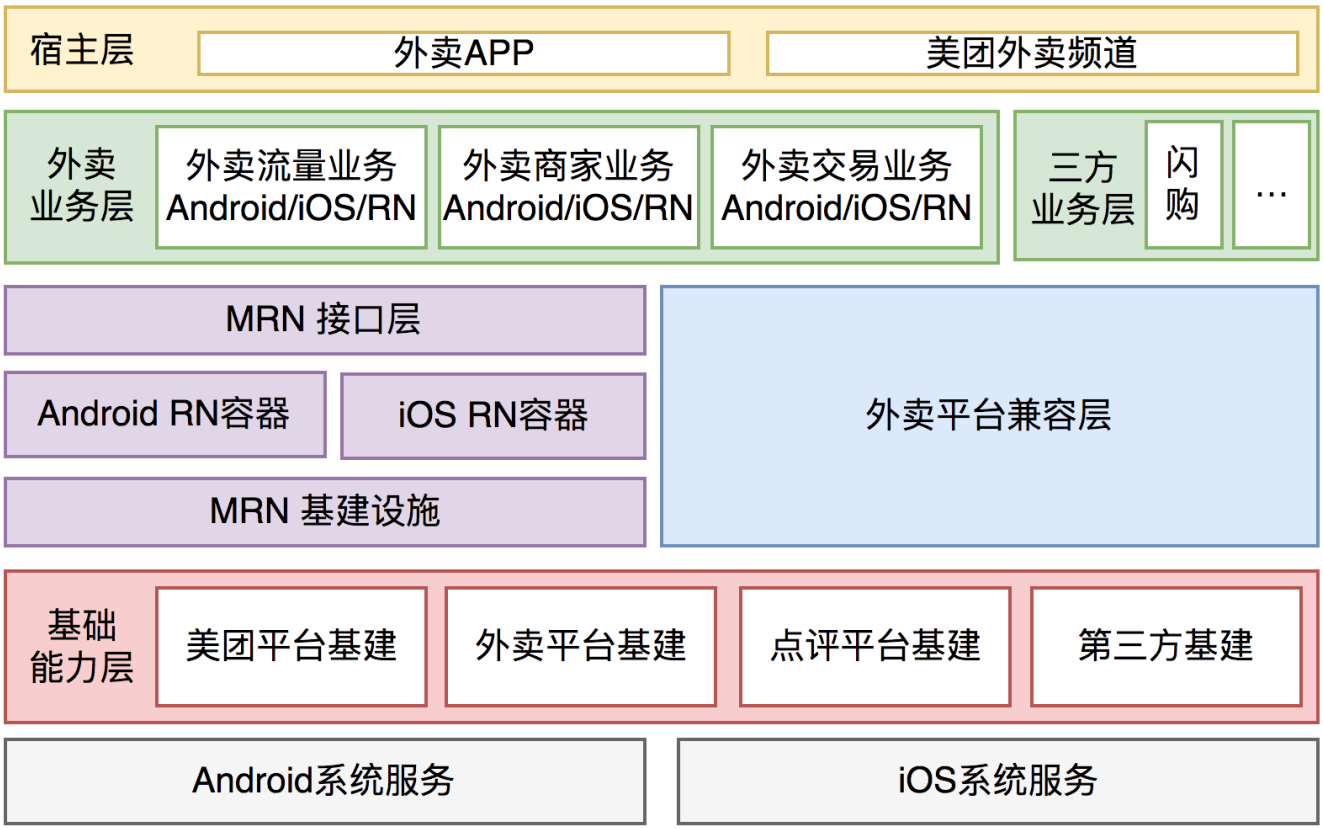
1.3.3 RN混合架构

在平台化架构之后，美团外卖功能持续增加，美团外卖客户端安装包的体积也在持续增加。回顾2017年和2018年，每年几乎都增长100%。如果没有一个有效的手段，安装包将变得越发臃肿。另外，由于原生应用需要依托于应用市场进行更新，每次产品的更新，必须依赖用户的主动更新，使得版本的迭代周期很长。业务上的这些痛点，不断地督促我们去反思到底有没有一种框架可以解决这些问题。

在2015年的时候，Facebook发布了非常具有颠覆性的React Native框架，简称RN。从名字上看，就可以清楚的明白，这是混合式开发模式，RN使用Native来渲染，JS来编码，从而实现了跨平台开发、快速编译、快速发布、高效渲染和布局。RN作为一种跨平台的移动应用开发框架，它的特性非常符合我们的诉求。美团也积极的探索RN技术。在RN的基础上，美团在脚手架、组件库、预加载、分包构建、发布运维等方面进行了全面的定制及优化，大幅提升RN的开发及发布运维效率，形成了MRN（Meituan React Native）技术体系。

从2018年开始，美团外卖客户端团队开始尝试使用MRN框架来解决业务上的问题。使用RN的另一方面的好处是，能逐渐的抹平Android和iOS开发技术栈带来的问题，使用一套代码，两个平台上线，理论上人效可以提升一倍，支持的业务需求也可以提升一倍，架构如下图所示：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



3.6.2 美团外卖容器化架构全景图

3.6.2.1 什么是容器化架构

上文说到，外卖业务已经发展到多App复用、单页面多业务团队开发的业务阶段。要满足这样的业务场景下，寻求一个可持续发展的业务架构是件不容易的事情。经过我们之前架构演进，我们获得了宝贵的经验：在平台化架构的时候，我们将App和业务进行解耦，将App做成壳容器，业务形成独立的业务库，集成到壳容器里面，从而屏蔽了多App的问题，提高了业务的复用度。在RN混合式架构里面，我们引入了RN容器，通过这个容器，使得业务屏蔽了Android和iOS的平台差异。借助这些成功的经验，我们进一步思考，如果我们尝试进一步的细分外卖的业务场景，将不同场景下的基础能力建设成壳容器，业务集成到容器内，是否可以更好的支撑我们多App复用、单页面多业务团队的当前现状呢？

容器化架构的愿景是：

- 希望将前端呈现业务的环境抽象出来，将能力进行标准化，形成统一的容器，通过容器去屏蔽平台和端的差异。容器提供上层标准统一的能力接口，使得业务开发人员专注于容器内的业务逻辑的实现，最大复用已有的能力，而不用关注现在的环境是Android还是iOS，现在的端是美团App还是大众点评App。
- 容器和远端达成呈现协议，使得端上的内容具备随时可变化的能力。容器化架构的实现是存在一定前提的，如果业务的发展本身处在一个探索阶段，还有较多可变的因素，是无法形成稳定的能力层的，这时候建设容器化架构反而使得架构偏向复杂。但对于外卖业务场景来说，经过多年的沉淀固定，外卖业务逐渐形成了一套稳定的业务形态，已经进入到场景细分和快速迭代业务模块的阶段。在这样的阶段下，容器化架构才有可实施的前提。

3.6.2.2 容器化架构的优势

当我们把承载外卖业务的环境进行了抽象和标准化后，就可以获得以下若干点好处。首先动态化属性提升，我们可以把原有必须在客户端上写的业务放到了远端，业务的动态性得到很大的提升，具备随时上线业务的可能。对于开发过程而言，编译部署的速度也得到了极大提升。如果涉及到客户端的代码改动，那客户端的编译打包，即使是增量的编译，也至少是秒级的编译速度。而容器化后，我们只打包必要的业务，把业务动态下发到容器呈现，客户端代码本身

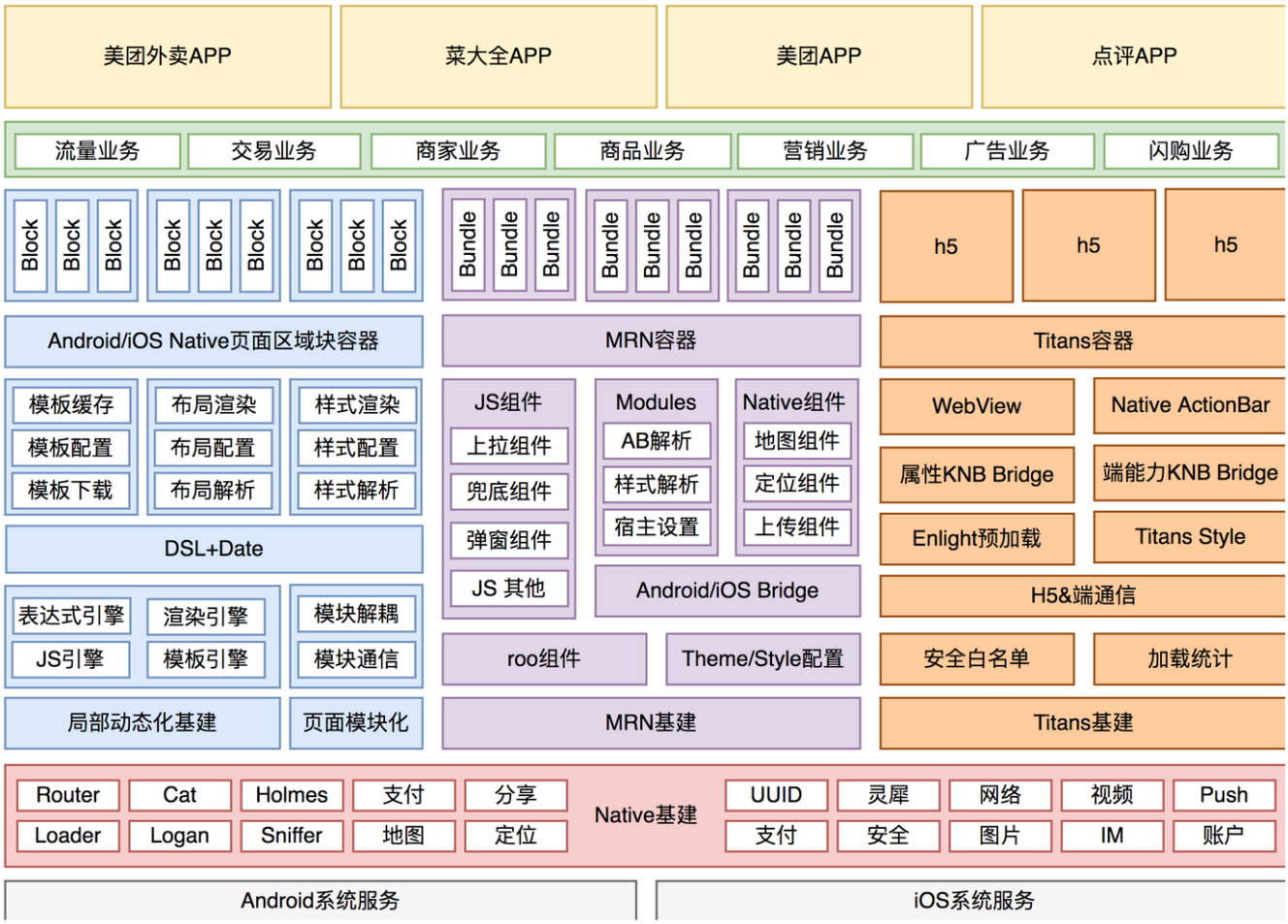
腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

不会有变化，这样就可以从秒级的编译减少到毫秒级的编译。同样，业务动态下发，对减少客户端的包大小也有很大的帮助。

然后，容器位于应用之内，我们向应用中引入相同的容器SDK，容器屏蔽了应用之间的差异，对于Android和iOS平台，在设计上，通过容器这一层去尽可能屏蔽平台之间的差异，使业务开发人员只需要认识容器，不需要花费大量的精力去关注应用和平台之间的差异，从而使得开发效率得到了极大的提升。

其次，容器化后，容器对承载的内容是有接口协议要求的，承载的内容只有满足容器定义的协议才能得到容器带来的好处，这促使业务得到了更细粒度的细分，业务开发时候，对模块化的意识得到了保障。另外，容器这一层提供的接口在Android和iOS上是标准化的，业务的开发也因为依赖的标准化，而趋向标准化，双端的业务一致性得到了提升。这些潜在的架构好处，对未来的业务维护和扩展都打下了比较好的地基。

3.6.2.3 外卖容器化架构全景图



整个外卖容器化架构可以按照从下到上，从左到右的视角进行解读：

最底层是系统服务，因为我们采用了H5和RN这样跨端的技术栈，使得iOS系统和Android系统成为了最底层。

系统服务之上是集团基于Native建设的基建，全公司通用，覆盖了研发工程中方方面面的基础服务。

在基建之上是我们定义的容器层。我们尝试用单一技术栈解决所有问题。但经过我们的探索，觉得不太可能实现。好的架构要匹配业务形态，业务的诉求决定了我们不能选择唯一的技术栈去解决所有问题，细分外卖的业务场景可得到以下3个方向的页面分类：

- 高PV主流程页面，例如首页、金刚页、商家页等，这类页面的PV远高于其他页面。这类页面的特点是，交互强、曝光度高、多团队参与建设和维护。针对这类页面，为了给用户提供极致的体验，是无法采用H5或RN实现的，即使性能上能够满足，但是这些页面是多团队共同参与建设，如果用H5或RN实现，那么单一团队改动，都

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

会造成全页面受到无法预料的影响，其他未改动的业务方肯定是无法接受的。所以这类页面，我们采用的是局部动态化+页面模块化的方案，我们针对页面进行容器化改造，将页面变成容器，容器承载模块。每个模块归不同的业务方进行维护，从模块的维度进行解耦，每个模块都可以动态配置和下发。

- 低PV辅助页面，例如帮助、足迹、收藏等，这类页面的PV低，但胜在数量多，都用原生实现成本比较高，如果都用H5来实现，不少页面和原生的关联还是比较近，不是非常适合。针对这类页面，我们采用集团提供的MRN基建去承载，MRN作为跨端的技术栈，我们已经在之前的RN混合架构的时候，建设了较为丰富的组件，针对自定义的MRN容器做了比较丰富的建设。同时，MRN具备动态下发的能力，满足业务的诉求。
- 向外投放的运营活动页面，例如圣诞节活动，时效性比较强。这类页面由H5技术栈去完成，一方面可以满足时效性，另一方面H5的动态下发能力也是最强的，这样的特性能够充分的满足业务的诉求。我们使用集团提供的Titans基建，通过建设业务自定义的Titans容器，支撑业务的发展。

再往上，就是垂直的业务，外卖目前有流量业务、交易业务、商家业务、商品业务、广告业务、营销业务、闪购业务等。业务都是垂直向下依赖，直接可见容器，可见基建，能够很好地获取到各种已经建设的能力去完成业务的需求。

最上面是承载的App端，目前有四端，包括外卖、点评、美团、闪购等等。

右侧是测试发布和线上监控，相对于常规的移动端App架构而言，容器化架构的测试发布和监控是更为精细化的。不仅仅要关注端本身的可用性，还需要关注容器、容器承载的模块、模块展示的模板，模板里面的样式这些的可用性。

3.6.2.4 容器化的挑战

容器化架构相对常规的移动端架构而言，它从管理移动端的代码转变成管理移动端的容器建设代码和业务远端开发代码，多出了容器和业务远端下发。这不仅仅是对技术上的挑战，对长期做客户端开发同学，也需要一个思维转变的跳转。

一致性的挑战：容器需要在多个宿主应用之中运行，宿主应用的环境一致性直接影响了容器的一致性。我们的策略是两手准备，一方面利用外卖业务的优势推动宿主应用的环境对齐；另一方面将容器建设成SDK，通过SDK将长期保持容器的一致性，也通过SDK内部的设计屏蔽应用之间的差异；对于Android和iOS平台，我们通过分层的设计，尽可能屏蔽平台的差异。综上所述，一致性的挑战在于（1）容器运行的宿主应用的环境一致性；（2）不同应用不同版本容器的一致性；（3）Android和iOS平台容器的对业务的一致性。

动态发布的挑战：长期以来，客户端同学的开发概念里面只有App版本的概念，而当我们逐渐把业务代码做成远端下发时，将会新增一个线上动态发版的概念。当我们在发布业务的时候，相对以往的工作，多出需要去考虑这个业务的版本，可以运行的容器对应的App上下界版本。另外，发版的周期也会新增业务的发版周期，不仅仅是App的发版周期。这两者在一起将会产生新的火花，业务的版本和App的版本如何适配的问题，业务动态发版的周期和App的发版周期如何适配的问题。外卖这边的解决方式是建设主版本迭代+周迭代的模型。

监控运维的挑战：以往的移动端架构，我们更加关注的是端本身的可用性，然而当我们演进到容器化架构的时候，仅仅关注端的可用性已经远远不能确定业务是可用的了。我们需要从端的可用性延伸出下载链路、加载链路，使用链路上的可用性，针对每个重要的环境，都做好监控运维。

3.6.3 外卖跨端容器建设

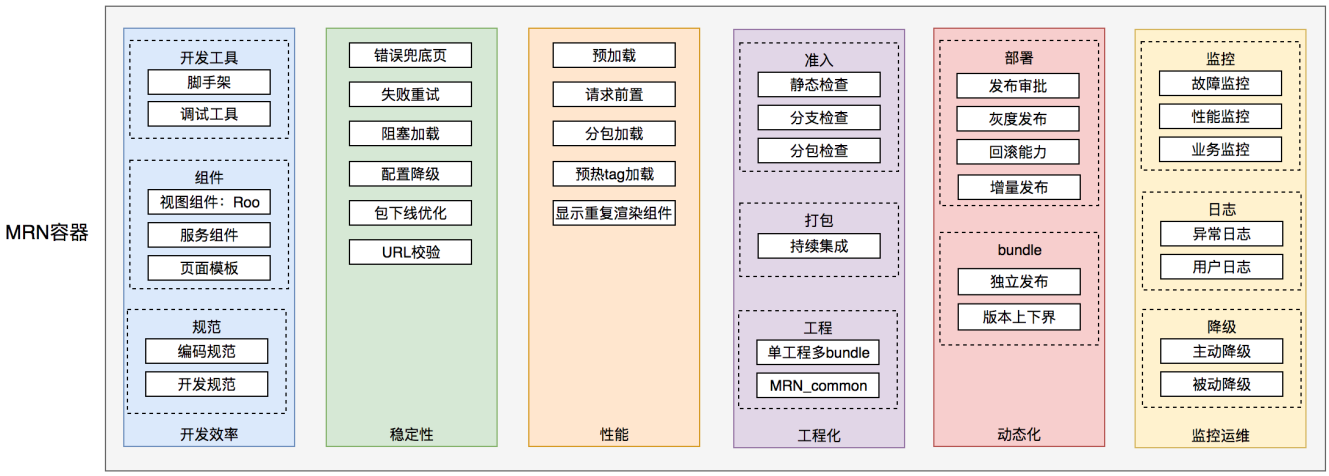
3.6.3.1 MRN容器

3.1.1 MRN容器简介

React Native框架本身只是一个运行时环境中的渲染引擎，可以将同一套JS代码分别在Android和iOS系统上最终以Native的方式渲染页面，从而为App提供了基础的跨端能力。但从工程化的角度来看，如果想在App中大规模地应用RN技术，除了RN框架本身外，还需要在开发、构建、测试、部署、运维等诸多方面的配合。MRN（Meituan React Native）是美团基于React Native框架改造并完善而成的一套动态化方案，在RN的基础上提供了容器化能力、动态化能力、多端复用能力和工程化保障。MRN在开发效率、稳定性、性能体验、动态化和监控运维等多方面进行了能力

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

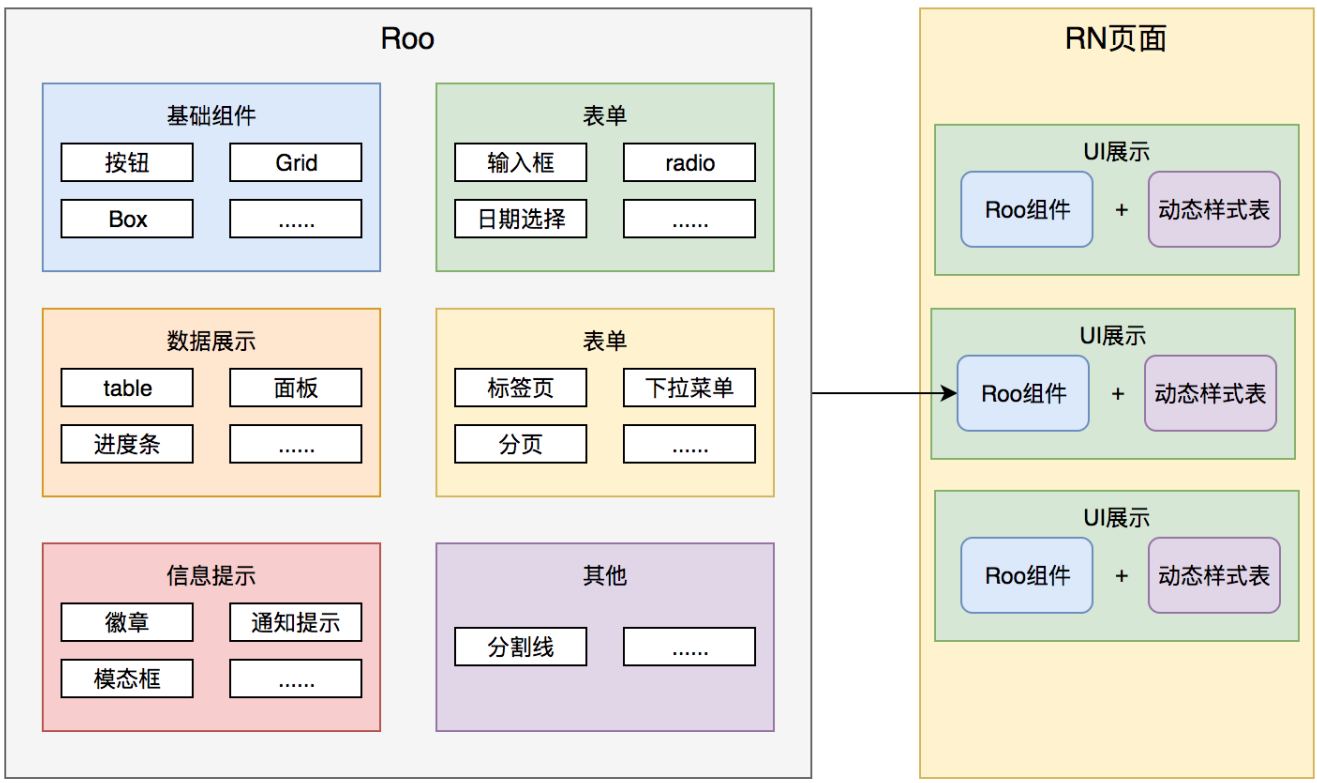
升级和扩展，满足了美团RN开发工程化的需要。目前，MRN已接入美团40多个App，核心框架及生态工具有超过100位内部代码贡献者，总PV超过4亿。



3.1.2 Roo组件库

下面再介绍一下外卖建设的两个UI相关的技术项目，Roo组件库和组件样式动态配置。

- **Roo组件库**：外卖在RN及MRN框架提供的UI组件基础之上，又扩展了适用于外卖业务的标准化UI组件库。UI组件库一方面统一了我们的设计规范和开发规范，提高了UI一致性；另一方面，组件封装也提升了RN页面的开发效率和质量。
- **组件样式动态配置**：有了标准的Roo组件，我们进一步给标准组件的动态添加了样式动态配置能力。在使用组件时，很多样式是支持动态下发的，例如字体、圆角、背景色等，方便我们进行UI的适配和改版。



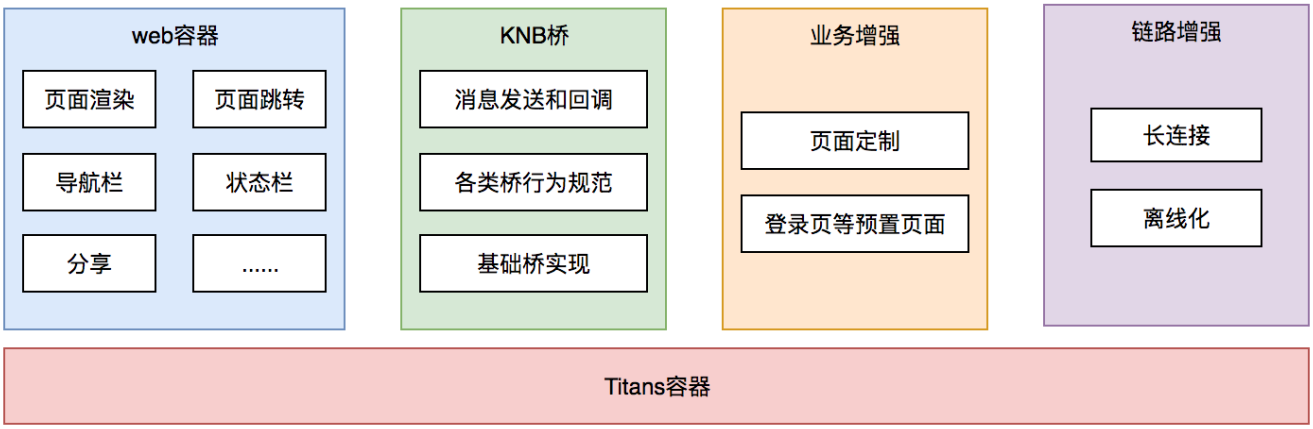
外卖在2018年底开始试验MRN容器在外卖业务上的应用，并在2019年上半年进行了大面积的页面落地。目前，外卖已有近60个RN页面上线，占外卖页面比例超80%，其中包括Tab页面"我的"、提单选择红包页、订单评价页等高PV页面。MRN容器的接入，给外卖App的容器化、动态化、人效提升、包大小瘦身等方面都做出了不小的贡献。

3.6.3.2 Titans容器

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

3.2.1 Titans容器简介

Titans容器是美团系App统一的Web容器组件，基于苹果提供的WebView组件，将WebView容器化，统一了WebView的UI展示和交互方式，规范了桥协议的使用范式，同时预置了诸多基础能力和业务能力。Titans容器大大提高了Web页面的开发效率和用户体验上的一致性。



- **Web容器**：Titans容器提供了统一的UI展示和自定义样式，例如导航栏样式、页面Loading状态、进度条样式等；还有统一的交互方式，例如页面跳转、Scheme协议的解析等。
- **KNB统一桥服务**：Web容器虽然在动态化和Android、iOS双端复用上很好地弥补了Native的不足，但在很多地方体验上又难以达到Native的标准。因此，KNB桥应运而生，KNB定义了Native和JS通信的标准方式，方便开发时进行桥协议扩展，同时KNB也内置众多的Native基础能力，极大地提高了Titans容器的用户体验和开发效率。
- **Web业务增强能力**：Titans容器中预置了丰富的基础业务页面，例如登录页面、分享弹窗等。
- **提供链路增强**：提供了长连接、离线化等方式来提高网络请求的速度和成功率。
- **WebView预加载**：在App启动之后，用户点击网页入口之前，提前加载好HTML主文档和基础库，这样当用户点击页面入口时，App直接使用已准备好的WebView，仅需加载少量的业务代码。从而达到白屏时间短、加载页面迅速的效果。

Titans容器在外卖业务中的使用场景非常丰富，其中最重要的使用场景是各种运营页和活动页，例如点击首页顶部Banner的广告落地页、为你优选、限时秒杀等活动运营页等；还有客服页、帮助反馈页、商家入驻页、美团公益页等功能性页面；作为一级入口页面的美团会员页面，也是一个基于Enlight的Titans容器。

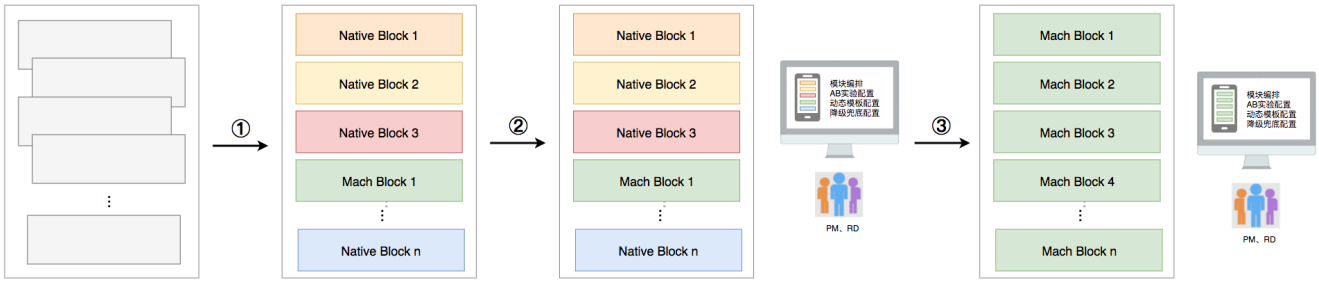
3.6.4 外卖页面容器建设

外卖容器化建设，首先需要要区分的是核心页面和非核心页面。外卖业务中对核心页面的定义是页面DAU>美团DAU的5%或者是下单关键路径。为什么要先按照是否为核心页面进行拆分呢？重点就在于改造的成本。核心页面的业务复杂度决定了它不容易做全页面的动态化，它比较适合做局部的动态化方案。核心页面的复杂度在于业务本身复杂，最重要的是核心页面往往会有多个垂直业务团队共同的开发维护，大家各自有重点关注的模块，做全页面的动态化，无法做到有效的物理隔离。

而对于非核心页面，业务功能和交互相对简单，组织关系也较为确定，更适合做标准的MRN和Titans容器化。所以我们的策略是核心页面做到支撑页面模块级别的业务动态和复用，非核心页面可以做到页面级别的动态化和复用。页面容器化的核心含义就是把一个页面划分为若干个模块，每个模块成为一个业务容器，每个容器的填充既可以用Native的方式实现，也可以用Mach实现（Mach是外卖自研的页面局部动态化技术），可以支持iOS/Android/小程序三端跨平台运行。页面本身则化身为容器的管理者，负责子容器的编排和布局，并支持其动态化。

3.6.4.1 页面容器化设计思路

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



页面容器化设计中主要分为三个阶段，模块有序化、模块编排化、渐进式业务落地。

- **模块有序化**：将耦合的外卖业务代码按模块维度进行拆分，建立标准化的模块间组合和交互方案，降低模块内改动对其他模块的影响。这个阶段我们同时完成了Native原生模块和局部动态化模块的标准化改造。
- **模块编排化**：页面容器化的一个特点是页面具备编排模块的能力，在这个阶段我们在客户端增加了对业务模块结构编排能力的支持，同时我们跟后端的同学共建了配置平台，通过配置化的方式打通了AB实验平台、统一数据服务等多个平台。在标准化的数据协议的支撑下，页面支持了AB实验动态上线，大大降低了客户端在AB实验方面的开发成本，做到了客户端零成本，后端低成本，高效地支撑了外卖首页六周年的大改版。
- **渐进式业务落地**：页面容器化后最明显的收益是支持业务需求的动态上线，只有页面容器中的业务模块具备动态能力才能实现这个目标，这会涉及Native模块迁移为Mach模块的过程。在这个方面，我们的思路是跟随业务需求渐进式落地。在模块编排阶段，我们设计了Native模块向Mach模块迁移的能力，同时设计了覆盖模板维度和API接口维度的三重降级方案，来保障模块动态化迁移的稳定性。

3.6.4.2 业务构建模块标准化

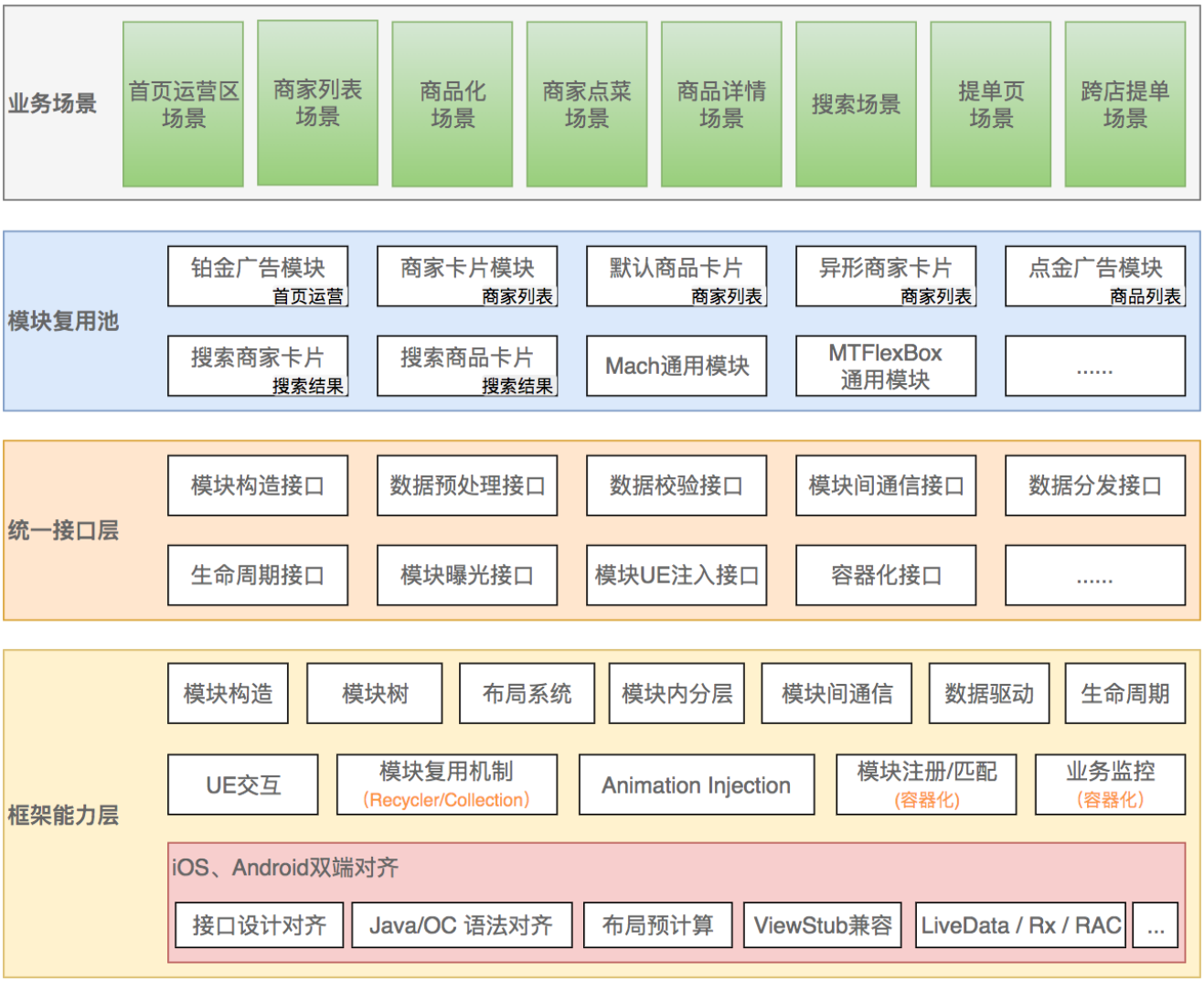
从App页面开发的角度看，一个完整的页面可以按照不同的功能及不同业务属性划分出多个不同的模块。

业务构建泛指由多个业务模块组合拼装为一个业务页面的过程，涉及页面本身（UIViewController/Activity）以及各个业务模块的构造过程，前后端业务数据以及页面和业务模块之间的数据交互过程，业务模块内部的数据处理以及视图刷新流程。

模块标准化指的将业务构建涉及到的多个过程通过规范化的方式确定下来，形成唯一的标准。模块标准化一方面能够在解决业务共性问题的基础上提供业务难点专项解决方案，另一方面能够在框架基础上形成能力约束，减少重复建设、低质量建设的问题。

业务构建模块标准化中我们抽象了四层，下面将分别进行解读。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



- 最底层是框架能力层，是外卖业务团队自研的符合外卖业务特点的双端模块化框架。框架解决了不同页面场景下的共性问题，对典型的业务痛点也进行了支持。它是一种页面框架设计在iOS、Android双端对齐的实现方案，这种双端对齐的能力为页面容器化设计的双端一致性提供了保障。
- 统一接口层是对框架能力层的标准化抽象，它可以保证任一模块调用的能力在各个业务场景下的实现都是一致的，有了这一层抽象任一模块都可以直接在各个场景下复用。
- 在往上就是App全局的模块复用层，标准化后的模块可以通过唯一标示向模块复用池注册模块，这种中心化的注册方式可以让业务模块在跨业务库的场景下可以灵活地复用。
- 最上层就是外卖的核心业务场景层，每一个场景都对应了一个标准化的页面容器，页面容器通过实现容器化接口来完成页面容器的构建。

通过业务构建模块标准化的建设，业务模块已经是标准化的了，可以在跨页面间自由组合，这为页面容器化打下了基础。

在页面容器化中最基础的能力有以下几点：页面中业务模块可编排能力，动态上线前端AB实验的能力，增量上线动态模块的能力。实现这些能力最重要的就是进行前后端数据协议标准化建设。客户端根据数据协议中的模块唯一标识匹配并构造业务模块，在完成模块数据的填充后会根据数据协议中的模块布局信息完成模块的布局。针对Mach动态模块，我们创建了基于模板ID的模块匹配和数据填充流程，可以支持Mach动态模板的增量上线。在数据协议中针对前端AB实验我们预留了AB实验和通参字段，在数据填充阶段通过容器化接口传入动态模块中，用于支持AB实验的动态上线。

在容器化上线的过程中属于接口的大版本升级，为了保证容器的高可用性，客户端从模块级别和API级别实现了两套降级容灾方案。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?tid=8674704361863085

模块级别的降级方案主要针对Mach动态模块，与Native模块不同，Mach动态模块需要预先下载动态模板才能正常地完成模块的载入和渲染。为了保证动态模块的加载成功率，我们一方面在接口上线前利用Eva（美团内部系统）对Mach模板的下载进行预热。另一方面，我们设计了动态模块的主动降级方案，针对动态模块的动态上线使用Native模块进行兜底降级，对于跟版动态模块使用App内置模板的方案进行兜底降级。

API级别的容灾方案主要为了保障客户端在新接口不稳定的情况下可以自行降级到旧接口。针对这个问题，我们对线上老接口设计了数据结构映射方案，在客户端通过配置化的方式可以把老接口的数据结构映射为新接口的数据结构。这样在上层业务无感知的情况下，可以做到容灾方案的上下线。

3.6.4.3 小结

通过页面容器化，使得页面只需要关心页面级的构造方式，而无需关心某一模块内部如何实现动态化的。把页面与页面的模块分离，也符合目前外卖客户端的组织结构，有利于业务组间的协作。同时，页面容器化使得外卖核心页面具备了符合外卖业务场景下的动态能力，渐进式把Native静态模块过渡到具备动态能力的模块，从模块的维度使整个页面具备了动态能力。这种渐进式的迁移方案把容器迁移跟业务模块的迁移分隔开，大大降低了页面容器化改造的风险。

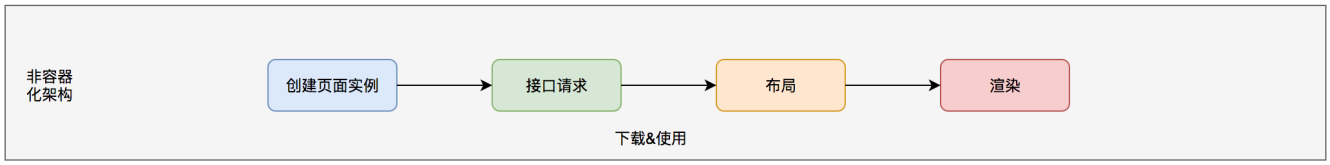
3.6.5 外卖容器化架构的衡量指标

3.6.5.1 容器化架构衡量指标的特点

质量和性能指标是衡量我们App开发质量和用户体验的重要依据，是我们一直都非常关注的重点数据。在非容器化时代，我们大多数的指标都和App的使用环节紧密相关，因为在非容器化时代，逻辑链路相对简单，例如我们打开一个新页面时，我们首先创建页面实例，然后发起网络请求，同时页面会经历一系列生命周期方法，最后渲染。这时我们可能会关注网络请求的成功率和请求时间，页面的渲染时间，和过成功是否发生Crash，这个过程相对更短暂，指标更少，所以监控起来也更容易。

外卖的容器化大大提升了外卖业务的复用能力、动态能力、模块化和开发效率，但同时也带来了更长的逻辑链路，链路从时间维度上划分是：下载链路、加载链路、使用链路。例如我们在使用MRN容器的时候，会涉及到bundle的启动下载或预热下载，bundle解压缩，MRN容器引擎初始化，bundle加载，JS的加载、执行，页面渲染等步骤，其中的每个步骤都可能存在性能问题和质量风险。因此，我们需要升级我们的衡量指标系统来应对容器化带来的新的挑战。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?tid=8674704361863085



3.6.5.2 链路指标

- **下载链路**：在下载链路中下载容器所需的各种资源，在MRN和Mach中主要是bundle的下载任务，只有bundle下载成功，才能进行后面的各项操作。所以bundle下载的成功率是下载链路中最重要的指标，同时bundle下载的时机也很重要。外卖业务中有各种bundel上百个，如果在启动时拉取所有bundle，对冷启动时间会造成极大的影响。我们采用了bundle预热的方法，发布bundle是给bundle打上相应的Tag，在适当的时机去下载，避免集中下载。
- **加载链路**：在加载链路中重要工作是对下载链路中下载的资源的解压和解析。例如在用PGA加载页面时，会进行模块的解析、模块匹配、模块降级、数据模型生成等步骤。在MRN中会进行bundle解压、引擎初始化、bundle加载等步骤。加载链路往往是比较消耗计算资源的步骤，对页面打开和加载时间影响较大，所以我们会比较关注加载链路的性能指标。
- **使用链路**：使用链路和非容器化的使用阶段基本相同，会主要关注页面的加载时间、Crash率、页面页面FPS、页面卡顿等指标。除此之外，还会关注和容器本身特性相关的一些指标，例如在MRN容器中，我们还会关注JS错误率、JS渲染时间、白屏率等指标。

3.6.5.3 关键指标

因为容器化的使用形成了一个串行的链路，所以如果某个关键节点失败，会导致容器功能不可使用，关键指标的任务就是从上述众多的指标当中筛选出这些关键节点。例如在下载链路中bundle下载的成功率和API的成功率，加载链路中bundle加载的成功率和模块匹配的成功率，下载或加载失败都无法再进行链路中的后续步骤，针对上面的成功率指标，我们会添加分钟级别的实时监控告警，做到及时发现，快速响应和紧急修复。

在使用链路中模块渲染的成功率、Native Crash率、JS错误率也属于关键指标，这些任务的失败也会导致容器的不可用，针对这些指标我们也会采用实时监控措施，并且添加降级手段，例如回滚bundle版本，或者把MRN和Mach容器降级为Native容器。

3.6.6 外卖容器化架构的监控运维

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?tid=8674704361863085

上面讲到了容器化架构的各项衡量指标，那么把这些指标具体落到实处的工作就是线上的运维监控工作。工欲善其事，必先利其器，对于监控运维工作，一定要有合适的监控工具辅助配合才能事半功倍，公司内有很多优秀的监控统计工具可供使用，这里的难点就是如何根据监控的需要判断选择合适的工具。还有就是合理的划分监控维度和数据指标的优先级，例如对于能够影响到链路稳定性的关键指标，我们需要做到分钟级的监控，一旦出现问题就能及时收到告警，对于非关键指标，则通过生成日报的方式，方便开发者的统计和分析。

工具的使用上主要分为大盘工具、具体异常工具、灰度降级工具、告警工具等（以下是美团内部使用的工具）。

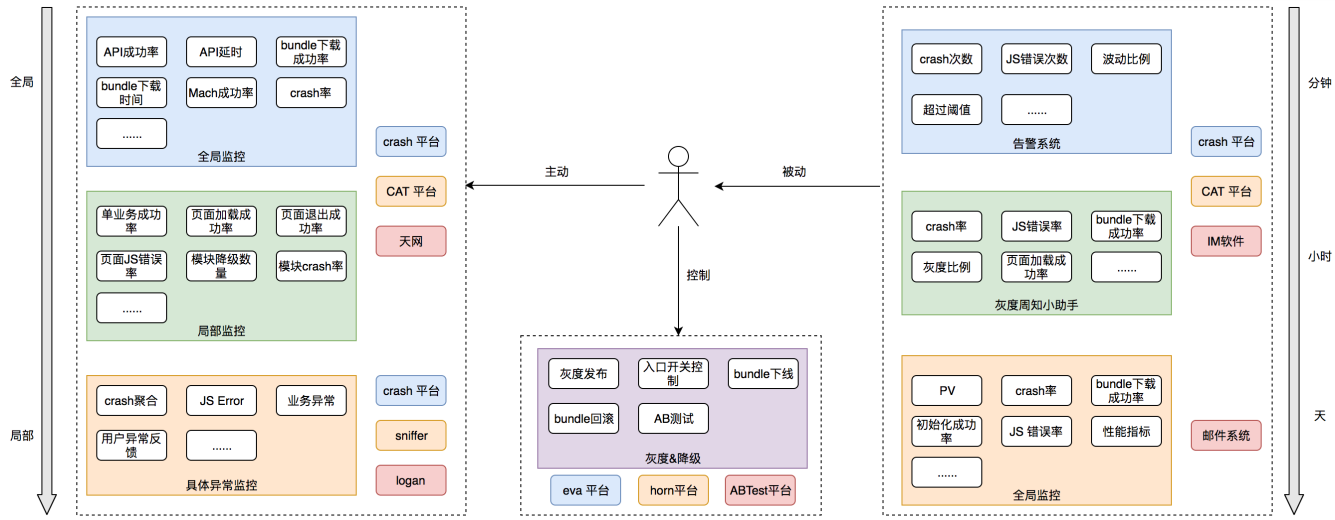
- **大盘工具**：主要使用CAT、天网、Crash平台等工具。这些工具收集、统计大盘数据，然后生成可视化的图标和曲线。方便开发者了解大盘的整体情况和变化趋势。
- **具体异常工具**：使用Sniffer、Logan等工具。这些工具可以用来获取发生异常时的上下文和设备信息，回捞用户行为日志，方便定位排查具体问题。
- **灰度降级工具**：使用ABTest平台、Horn等工具。用于下发配置，以进行灰度控制或开关控制。
- **告警工具**：告警小助手。将告警通过IM软件及时发送到个人或群组，做到及时发现及时处理。

业务覆盖维度监控可以分为全局监控和局部（单业务）监控。

- **全局监控**：监控各项容器化质量指标、性能指标，生成每日报表，方便跟踪监控容器化的整体质量。
- **局部（单业务）监控**：实时监控每个页面、每个容器的线上数据，做到有问题及时发现，及时定位，及时处理。

时间维度监控：可以按天、小时、分钟的时间维度。天级别的监控主要是一些非关键路径指标，例如一些性能指标，页面加载时间、页面FPS、JS渲染时间等，我们可以按天维度的生成数据报表，已邮件的数据发送日报。当App灰度上线时，我们会开始小时级别的监控，每过半小时通过IM软件向广播一些关键指标，方便开发者跟踪线上数据的稳定性。分钟级别的监控则是针对关键指标，观察分钟维度上的变化，如果关键指标超过阈值，或者波动过大，就会及时产生告警。

下面我们以一个开发者的视角去看一下外卖容器化架构的监控运维系统。从获取信息的方式上可以分为主动查询和被动推送，开发者可以通过监控工具监控全局和局部数据的变化趋势，也可以分析具体异常Case；也可以从IM工具，邮件等收到相关的推送数据，以便及时响应。在控制运维上，开发者可以通过Eva、Horn等美团内部的灰度系统进行灰度发布，当灰度期发现问题的时候，可以及时地通过停止灰度，版本回滚，关闭入口的方式进行降级容灾处理。



3.6.7 外卖容器化架构的发布能力

3.6.7.1 容器化架构发布体系

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?tid=8674704361863085

容器化使外卖业务具备了强大的动态化能力，但动态化能力又和需要相应的发布能力来支持，发布能力是我们业务开发质量和效率的重要保障，也是我们容器化建设工作过程中的重点环节，这一节主要介绍一下外卖容器化的发布能力。

从发布能力类型的角度看主要可以分为三种类型：（1）容器内容的发布，包括发布整个页面或者发布页面中的局部模块；（2）配置下发，通过API或其他配置平台，下发布局协议、AB测试、样式配置、功能配置、模板配置、容器配置等，大大提高了业务的灵活度和线上验证能力；（3）灰度、降级下发，通过UUID，用户画像等信息做到灰度发布，降级回滚等控制能力。

从发布资源的的角度看主要分为两种：一种是普通的资源，例如发布一个Web页面，或者通过发布新版API来控制页面局部容器的展示与否和展示的位置，同时我们也可以进行一些AB Test操作；另一种是bundle资源，主要是针对MRN容器和Mach容器，每个MRN容器和Mach容器的资源都会先被打包成一个bundle，然后通过发布系统下发到终端，然后终端解析bundle中的代码和资源，最终渲染页面。

从发布阶段的角度看，可以分为测试阶段、上线阶段、灰度阶段和全量阶段，其中上线阶段是最终的环节，我们增加了很多校验和保护手段来尽量保证上线操作的正确性。

3.6.7.2 跟版本发布流程

虽然我们具随时备动态发布能力，但正常的版本迭代还是会存在中，所以外卖这边的节奏是周动态迭代+双周版本迭代，这保证了我们的开发工作有个一清晰的周期。在动态发布阶段中最关键的阶段操作上线阶段。以MRN为例，目前外卖业务中应有70多个bundle，再算上测试环境的bundle就有接近150个bundle，只是管理这些bundle就是一个复杂的工作，况且在进行上线操作时还是涉及发布的目标App、App版本的上下界、MRN版本的上下界等，一不小心就会造成操作失误，所以进行上线操作时需要非常谨慎。

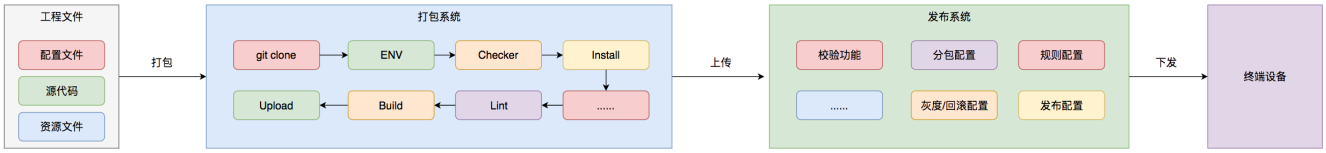
我们针对操作上线阶段进行了事务流水线，通过流水线建立保护措施，一个bundle的上线要经历一个流水线的若干操作。首先，操作人根据上线SOP手册进行若干检查操作，同时编写标准格式的发布说明，然后周知相关核心人员后在操作系统上发起上线申请，Leader和QA收到申请后会进行检查并审批，审批通过后还要避开App使用的高峰期或节假日上线，上线后通过灰度发布观察各项数据指标，指标正常后全量发布。

3.6.7.3 bundle资源发布

bundle是我们最常发布的资源类型，这里再结合发布工具讲解一下bundle的发布过程。MRN和Mach都是以bundle的形式下发到设备终端的，我们在发布bundle的时候主要会用到两个工具，打包工具Talos和发布工具Eva（美团内部工具）。一个bundle的工程文件主要由三个部分组成：配置文件、源代码和资源文件，其中配置文件用于指导Talos对工程文件进行打包，多个bundle可以共享一份配置文件。当我们准备发布一个bundle时，先找到该bundle在Talos的发布模板，选择发布环境（测试或线上），然后进行一键打包，然后Talos会进行一系列流水线操作，包括Clone代码、配置环境、进行Lint检查、构建和上传等。Talos打包完毕后将bundle上传到Eva系统，然后Eva负责bundle的分包、上线、下线、灰度等操作，最终下发到终端设备上。

未来，我们还将引入美团住宿的MRN-DevOps来进一步的屏蔽当前多系统的问题，降低整个周期管理的成本，特别是发布前的人工检查成本，逐渐实现RD在一个平台上操作从研发到发布运维的所有实现。尽可能地减少人工成本，提升自动化。

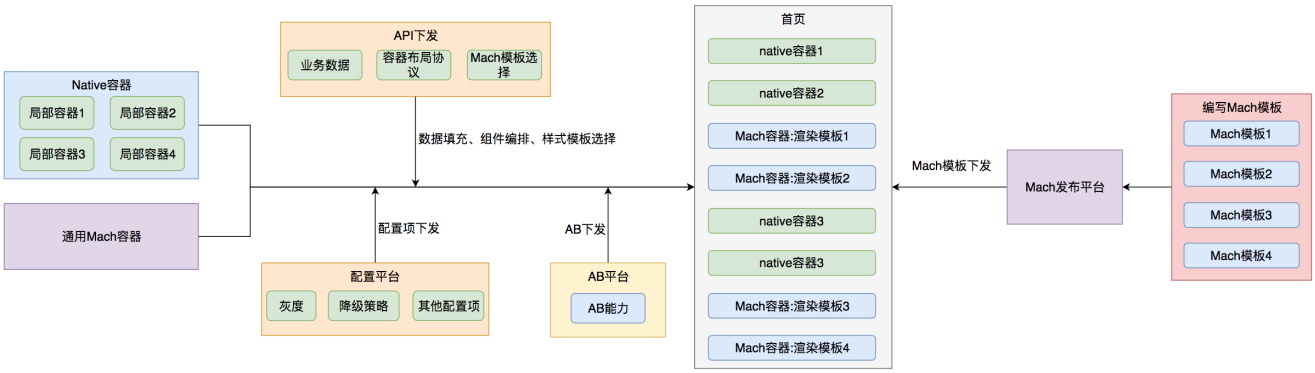
腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?tid=8674704361863085



3.6.7.4 多种发布能力综合使用

上面介绍的是以bundle资源形式的发布流程，过程较为清晰简单。下面再结合外卖首页，介绍一下局部容器化的发布方式。外卖首页是典型的流式列表，在局部容器化的架构下，首页就是由一个个矩形容器以ListView方式布局的，容器分两种，Native容器和Mach容器，Mach容器是一个通用容器，我们可以编写不同的样式模板，下发到终端后交由通用Mach容器来渲染，以此达到只使用通用容器展示不同UI样式的目的，这里涉及了Mach的发布系统。

首页各子容器相当于一块块积木，它们的位置排布、展示与否、模板的选择等最终交由API控制，API具备了控制首页布局，样式展示的能力，而不再是单纯的数据源。同时，首页也涉及了AB能力、灰度降级策略等其实配置项下发系统。可以看到外卖首页的容器化是由多种发布能力配合支撑的，是外卖发布能力体系的"集大成者"。



3.6.8 总结

好的架构是要随着业务的发展，不断演变去适应业务的发展。美团外卖从一个很小规模，每日单量只有几千的业务，逐渐地走到今天，每日单量峰值超过4000万，组织架构也从一个十几个人的团队，逐渐发展到现在多角色、多垂直业务方向，上千人共同协作的团队。移动端上的架构，为了适应业务的发展要求，也经历了组件化、平台化、RN混合化，再到现在向容器化的变迁。

容器化架构相对于传统的移动端架构而言，充分地利用了现在的跨端技术，将动态化的能力最大化的赋予业务。通过动态化，带来业务迭代周期缩短、编译的加速、开发效率的提升等好处。同时，也解决了我们面临着的多端复用、平台能力、平台支撑、单页面多业务团队、业务动态诉求强等业务问题。

当然，容器化架构带来好处的同时，对线上的可用性、容器的可用性、支撑业务的线上发布上提出了更加严格的要求。我们通过监控下载、加载、使用链路上的可用性，来保障线上动态业务的可用性。针对容器，我们利用成熟的测试基建，建设容器的自动化测试来保障容器的可用性。针对发布，我们建设迭代流程，配合发布流水线，将线上的发布变得更为可控。

截止到目前为止，外卖业务经过了几十个动态化业务上线窗口，累积共发版百次以上。未来半年，我们还将进一步从业务需求入手，将业务需求细分归类，让产品侧逐渐建立容器和动态化需求的概念，能够从源头上，逐渐的将业务进行划分，最终使得每个业务需求，都可以归类抽象成可以动态下发的业务和容器能力建设，从而进一步的完善容器化架构的能力和支持更多的的业务场景。

参考资料

- React Native在美团外卖客户端的实践
- 美团外卖iOS多端复用的推动、支撑与思考
- 美团外卖前端容器化演进实践
- UC浏览器客户端容器化架构演进

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

- 你知道支付宝容器化架构是怎么搭建的吗？
- 讲一讲移动端跨平台技术的演进之路
- 盘点主流移动端跨平台UI技术：实现原理、技术优劣、横向对比等

作者简介

- 郭赛，同同，徐宏，均为美团外卖iOS工程师。

3.7 美团Android组件化方案及组件消息总线modular-event实战

3.7.1 背景

组件化作为Android客户端技术的一个重要分支，近年来一直是业界积极探索和实践的方向。美团内部各个Android开发团队也在尝试和实践不同的组件化方案，并且在组件化通信框架上也有很多高质量的产出。最近，我们团队对美团零售收银和美团轻收银两款Android App进行了组件化改造。本文主要介绍我们的组件化方案，希望对从事Android组件化开发的同学能有所启发。

为什么要组件化

近年来，为什么这么多团队要进行组件化实践呢？组件化究竟能给我们的工程、代码带来什么好处？我们认为组件化能够带来两个最大的好处。

提高组件复用性

可能有些人会觉得，提高复用性很简单，直接把需要复用的代码做成Android Module，打包AAR并上传代码仓库，那么这部分功能就能被方便地引入和使用。但是我们觉得仅仅这样是不够的，上传仓库的AAR库是否方便被复用，需要组件化的规则来约束，这样才能提高复用的便捷性。

降低组件间的耦合

我们需要通过组件化的规则把代码拆分成不同的模块，模块要做到高内聚、低耦合。模块间也不能直接调用，这需要组件化通信框架的支持。降低了组件间的耦合性可以带来两点直接的好处：第一，代码更便于维护；第二，降低了模块的Bug率。

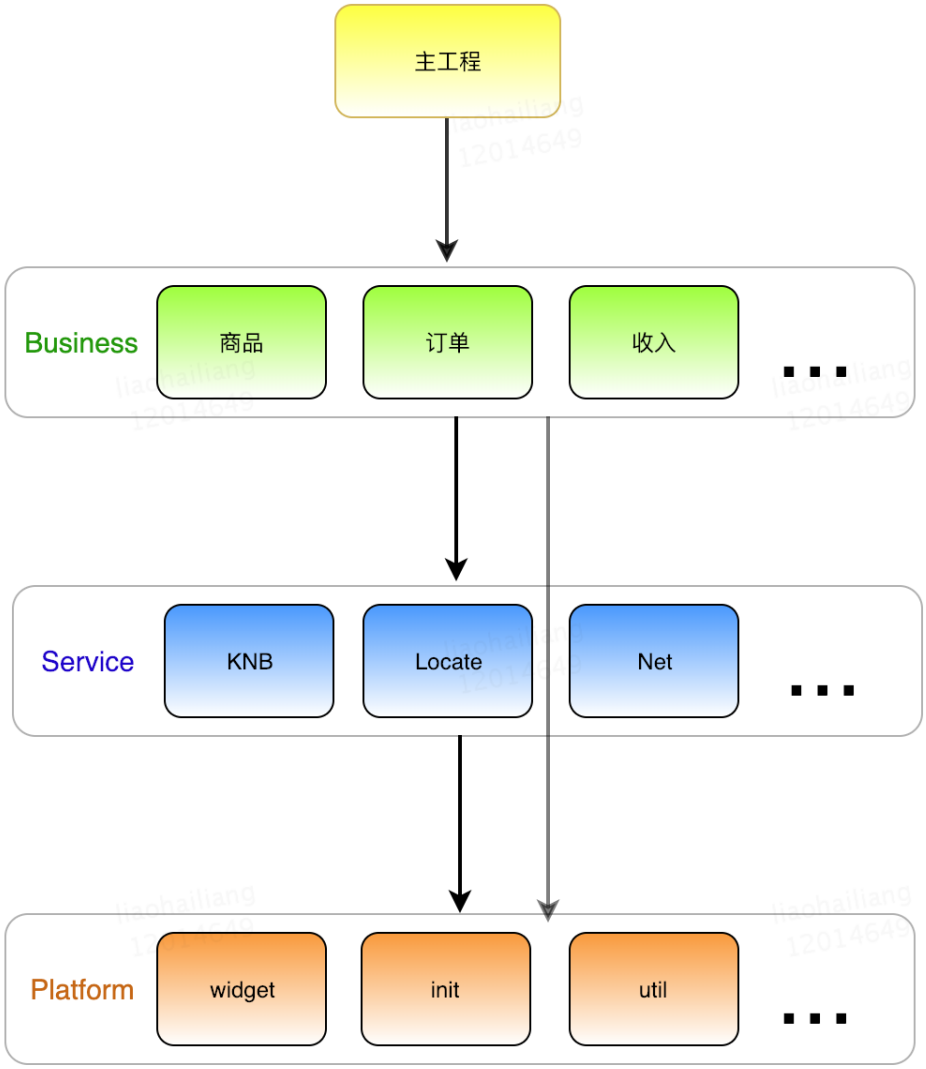
3.7.2 组件化之前的状态

我们的目标是要对团队的两款App（美团零售收银、美团轻收银）进行组件化重构，那么这里先简单地介绍一下这两款应用的架构。

总的来说，这两款应用的构架比较相似，主工程Module依赖Business Module，Business Module是各种业务功能的集合，Business Module依赖Service Module，Service Module依赖Platform Module，Service Module和Platform Module都对上层提供服务。

有所不同的是Platform Module提供的服务更为基础，主要包括一些工具Utils和界面Widget，而Service Module提供各种功能服务，如KNB、位置服务、网络接口调用等。这样的话，Business Module就变得非常臃肿和繁杂，各种业务模块相互调用，耦合性很强，改业务代码时容易“牵一发而动全身”，即使改一小块业务代码，可能要连带修改很多相关的地方，不仅在代码层面不利于进行维护，而且对一个业务的修改很容易造成其他业务产生Bug。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



组件化之前的状态

3.7.3 组件化方案调研

为了得到最适合我们业态和构架的组件化方案，我们调研了业界开源的一些组件化方案和公司内部其他团队的组件化方案，在此做个总结。

开源组件化方案调研

我们调研了业界一些主流的开源组件化方案。

- CC

号称业界首个支持渐进式组件化改造的Android组件化开源框架。无论页面跳转还是组件间调用，都采用CC统一的组件调用方式完成。

- DDComponentForAndroid

得到的方案采用路由 + 接口下沉的方式，所有接口下沉到base中，组件中实现接口并在IApplicationLike中添加代码注册到Router中。

- ModularizationArchitecture

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

组件间调用需指定同步实现还是异步实现，调用组件时统一拿到RouterResponse作为返回值，同步调用的时候用RouterResponse.getData()来获取结果，异步调用获取时需要自己维护线程。

- ARouter

阿里推出的路由引擎，是一个路由框架，并不是完整的组件化方案，可作为组件化架构的通信引擎。

- 聚美Router

聚美的路由引擎，在此基础上也有聚美的组件化实践方案，基本思想是采用路由 + 接口下沉的方式实现组件化。

3.7.4 美团其他团队组件化方案调研

美团收银ComponentCenter

美团收银的组件化方案支持接口调用和消息总线两种方式，接口调用的方式需要构建CCPData，然后调用ComponentCenter.call，最后在统一的Callback中进行处理。消息总线方式也需要构建CCPData，最后调用ComponentCenter.sendEvent发送。美团收银的业务组件都打包成AAR上传至仓库，组件间存在相互依赖，这样导致mainapp引用这些组件时需要小心地exclude一些重复依赖。在我们的组件化方案中，我们采用了一种巧妙的方法来解决这个问题。

美团App ServiceLoader

美团App的组件化方案采用ServiceLoader的形式，这是一种典型的接口调用组件通信方式。用注解定义服务，获取服务时取得一个接口的List，判断这个List是否为空，如果不为空，则获取其中一个接口调用。

WMRouter

美团外卖团队开发的一款Android路由框架，基于组件化的设计思路。主要提供路由、ServiceLoader两大功能。之前美团技术博客也发表过一篇WMRouter的介绍：《WMRouter：美团外卖Android开源路由框架》。WMRouter提供了实现组件化的两大基础设施框架：路由和组件间接口调用。支持和文档也很充分，可以考虑作为我们团队实现组件化的基础设施。

3.7.5 组件化方案

3.7.5.1 组件化基础框架

在前期的调研工作中，我们发现外卖团队的WMRouter是一个不错的选择。首先，WMRouter提供了路由+ServiceLoader两大组件间通信功能，其次，WMRouter架构清晰，扩展性比较好，并且文档和支持也比较完备。所以我们决定了使用WMRouter作为组件化基础设施框架之一。然而，直接使用WMRouter有两个问题：

1. 我们的项目已经在使用一个路由框架，如果使用WMRouter，需要把之前使用的路由框架改成WMRouter路由框架。
2. WMRouter没有消息总线框架，我们调研的其他项目也没有适合我们项目的消息总线框架，因此我们需要开发一个能够满足我们需求的消息总线框架，这部分会在后面详细描述。

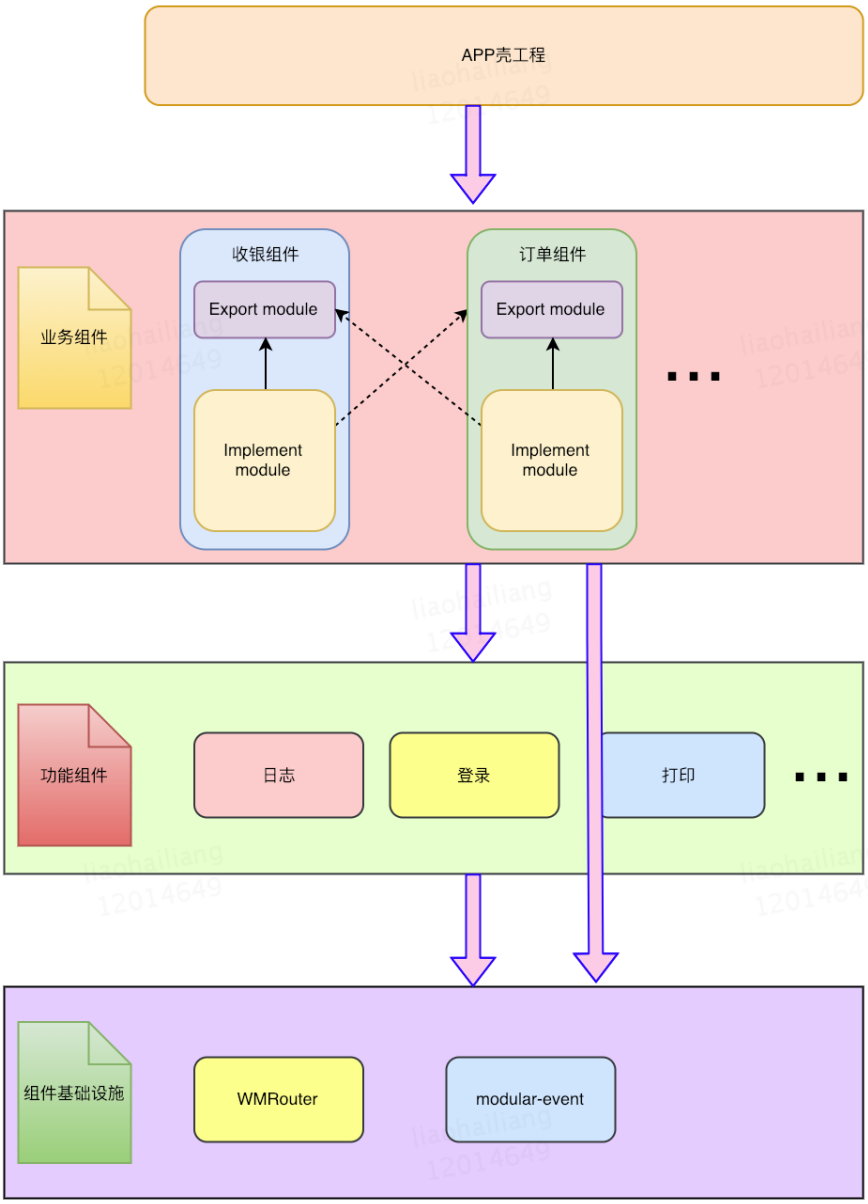
3.7.5.2 组件化分层结构

在参考了不同的组件化方案之后，我们采用了如下分层结构：

1. **App壳工程**：负责管理各个业务组件和打包APK，没有具体的业务功能。
2. **业务组件层**：根据不同的业务构成独立的业务组件，其中每个业务组件包含一个Export Module和Implement Module。
3. **功能组件层**：对上层提供基础功能服务，如登录服务、打印服务、日志服务等。
4. **组件基础设施**：包括WMRouter，提供页面路由服务和ServiceLoader接口调用服务，以及后面会介绍的组件消息总线框架：modular-event。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

整体架构如下图所示：



分层结构

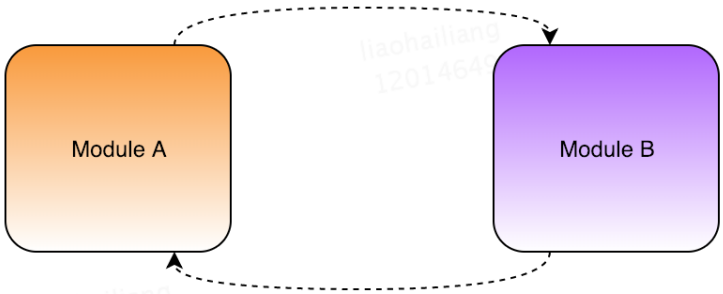
3.7.5.3 业务组件拆分

我们调研其他组件化方案的时候，发现很多组件方案都是把一个业务模块拆分成一个独立的业务组件，也就是拆分成一个独立的Module。而在我们的方案中，每个业务组件都拆分成了一个Export Module和Implement Module，为什么要这样做呢？

1. 避免循环依赖

如果采用一个业务组件一个Module的方式，如果Module A需要调用Module B提供的接口，那么Module A就需要依赖Module。同时，如果Module B需要调用Module A的接口，那么Module B就需要依赖Module A。此时就会形成一个循环依赖，这是不允许的。

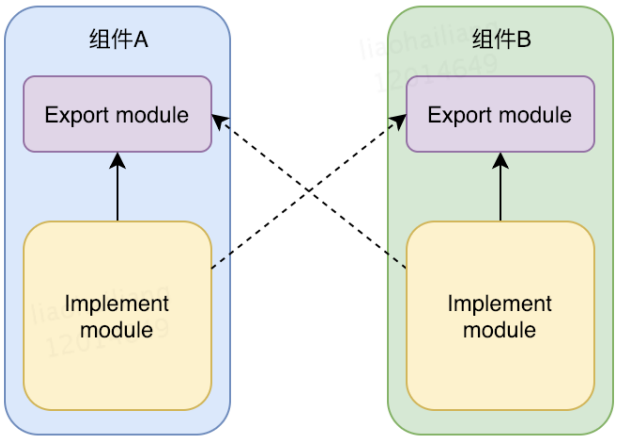
腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



循环依赖

也许有些读者会说，这个好解决：可以把Module A和Module B要依赖的接口放到另一个Module中去，然后让Module A和Module B都去依赖这个Module就可以了。这确实是一个解决办法，并且有些项目组在使用这种把接口下沉的方法。

但是我们希望一个组件的接口，是由这个组件自己提供，而不是放在一个更加下沉的接口里面，所以我们采用了把每个业务组件都拆分成了一个Export Module和Implement Module。这样的话，如果Module A需要调用Module B提供的接口，同时Module B需要调用Module A的接口，只需要Module A依赖Module B Export，Module B依赖Module A Export就可以了。



组件结构

2. 业务组件完全平等

在使用单Module方案的组件化方案中，这些业务组件其实不是完全平等，有些被依赖的组件在层级上要更下沉一些。但是采用Export Module+Implement Module的方案，所有业务组件在层级上完全平等。

3. 功能划分更加清晰

每个业务组件都划分成了Export Module+Implement Module的模式，这个时候每个Module的功能划分也更加清晰。Export Module主要定义组件需要对外暴露的部分，主要包含：

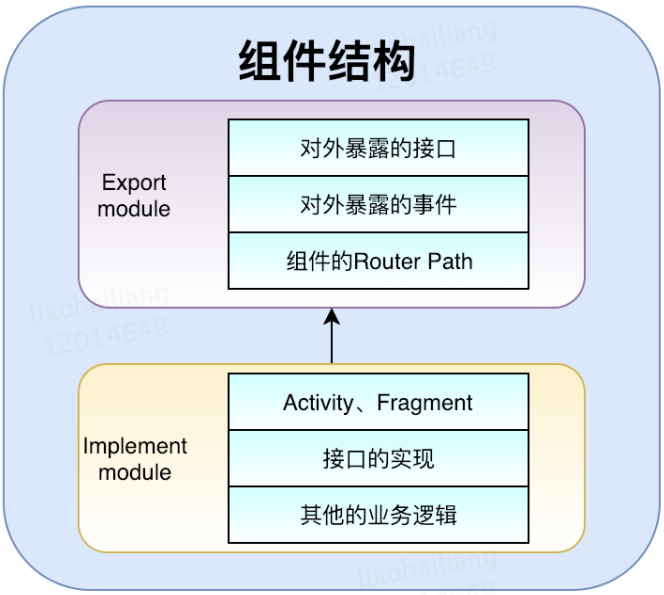
- **对外暴露的接口**，这些接口用WMRouter的ServiceLoader进行调用。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

- **对外暴露的事件**，这些事件利用消息总线框架modular-event进行订阅和分发。
- **组件的Router Path**，组件化之前的工程虽然也使用了Router框架，但是所有Router Path都是定义在了一个下沉Module的公有Class中。这样导致的问题是，无论哪个模块添加/删除页面，或是修改路由，都需要去修改这个公有的Class。设想如果组件化拆分之后，某个组件新增了页面，还要去一个外部的Java文件中新增路由，这显然难以接受，也不符合组件化内聚的目标。因此，我们把每个组件的Router Path放在组件的Export Module中，既可以暴露给其他组件，也可以做到每个组件管理自己的Router Path，不会出现所有组件去修改一个Java文件的窘境。

Implement Module是组件实现的部分，主要包含：

- 页面相关的Activity、Fragment，并且用WMRouter的注解定义路由。
- Export Module中对外暴露的接口的实现。
- 其他的业务逻辑。



组件功能划分

3.7.6 组件化消息总线框架modular-event

前文提到的实现组件化基础设施框架中，我们用外卖团队的WMRouter实现页面路由和组件间接口调用，但是却没有消息总线的基础框架，因此，我们自己开发了一个组件化消息总线框架modular-event。

3.7.6.1 为什么需要消息总线框架

之前，我们开发过一个基于LiveData的消息总线框架：LiveDataBus，也在美团技术博客上发表过一篇文章来介绍这个框架：《Android消息总线的演进之路：用LiveDataBus替代RxBus、EventBus》。关于消息总线的使用，总是伴随着很多争论。有些人觉得消息总线很好用，有些人觉得消息总线容易被滥用。

既然已经有了ServiceLoader这种组件间接口调用的框架，为什么还需要消息总线这种方式呢？主要有两个理由。

1. 更进一步的解耦

基于接口调用的ServiceLoader框架的确实现了解耦，但是消息总线能够实现更彻底的解耦。接口调用的方式调用方需要依赖这个接口并且知道哪个组件实现了这个接口。消息总线方式发送者只需要发送一个消息，根本不用关心是否有人订阅这个消息，这样发送者根本不需要了解其他组件的情况，和其他组件的耦合也就越少。

2. 多对多的通信

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

基于接口的方式只能进行一对一的调用，基于消息总线的方式能够提供多对多的通信。

3.7.6.2 消息总线的优点和缺点

总的来说，消息总线最大的优点就是解耦，因此很适合组件化这种需要对组件间进行彻底解耦的场景。然而，消息总线被很多人诟病的重要原因，也确实是因为消息总线容易被滥用。消息总线容易被滥用一般体现在几个场景：

1. 消息难以溯源

有时候我们在阅读代码的过程中，找到一个订阅消息的地方，想要看看是谁发送了这个消息，这个时候往往只能通过查找消息的方式去"溯源"。导致我们在阅读代码，梳理逻辑的过程不太连贯，有种被割裂的感觉。

2. 消息发送比较随意，没有强制的约束

消息总线在发送消息的时候一般没有强制的约束。无论是EventBus、RxBus或是LiveDataBus，在发送消息的时候既没有对消息进行检查，也没有对发送调用进行约束。这种不规范性在特定的时刻，甚至会带来灾难性的后果。比如订阅方订阅了一个名为login_success的消息，编写发送消息的是一个比较随意的程序员，没有把这个消息定义成全局变量，而是定义了一个临时变量String发送这个消息。不幸的是，他把消息名称login_success拼写成了login_seccess。这样的话，订阅方永远接收不到登录成功的消息，而且这个错误也很难被发现。

3.7.6.3 组件化消息总线的设计目标

1. 消息由组件自己定义

以前我们在使用消息总线时，喜欢把所有的消息都定义到一个公共的Java文件里面。但是组件化如果也采用这种方案的话，一旦某个组件的消息发生变动，都会去修改这个Java文件。所以我们希望由组件自己来定义和维护消息定义文件。

2. 区分不同组件定义的同名消息

如果消息由组件定义和维护，那么有可能不同组件定义了重名的消息，消息总线框架需要能够区分这种消息。

3. 解决前文提到的消息总线的缺点

解决消息总线消息难以溯源和消息发送没有约束的问题。

3.7.6.4 基于LiveData的消息总线

之前的博文《Android消息总线的演进之路：用LiveDataBus替代RxBus、EventBus》详细阐述了如何基于LiveData构建消息总线。组件化消息总线框架modular-event同样会基于LiveData构建。使用LiveData构建消息总线有很多优点：

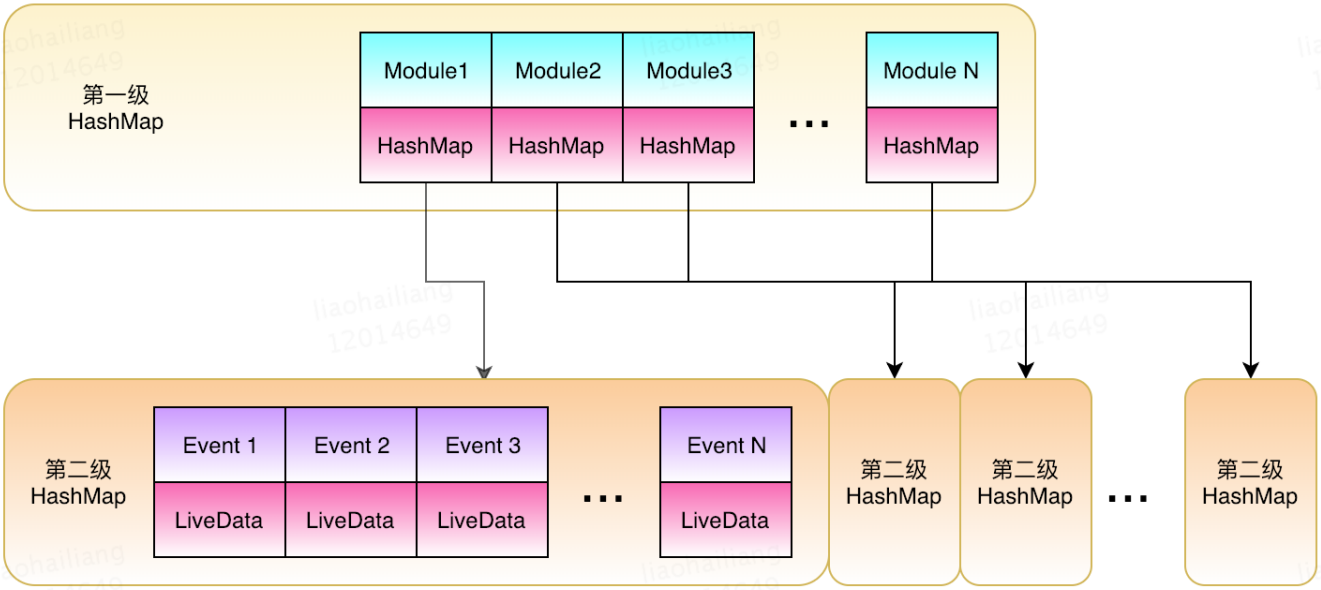
1. 使用LiveData构建消息总线具有生命周期感知能力，使用者不需要调用反注册，相比EventBus和RxBus使用更为方便，并且没有内存泄漏风险。
2. 使用普通消息总线，如果回调的时候Activity处于Stop状态，这个时候进行弹Dialog一类的操作就会引起崩溃。使用LiveData构建消息总线完全没有这个风险。

3.7.7 组件消息总线modular-event的实现

3.7.7.1 解决不同组件定义了重名消息的问题

其实这个问题还是比较好解决的，实现的方式就是采用两级HashMap的方式解决。第一级HashMap的构建以ModuleName作为Key，第二级HashMap作为Value；第二级HashMap以消息名称EventName作为Key，LiveData作为Value。查找的时候先用组件名称ModuleName在第一级HashMap中查找，如果找到则用消息名EventName在第二级HashName中查找。整个结构如下图所示：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



消息总线结构

3.7.7.2 对消息总线的约束

我们希望消息总线框架有以下约束：

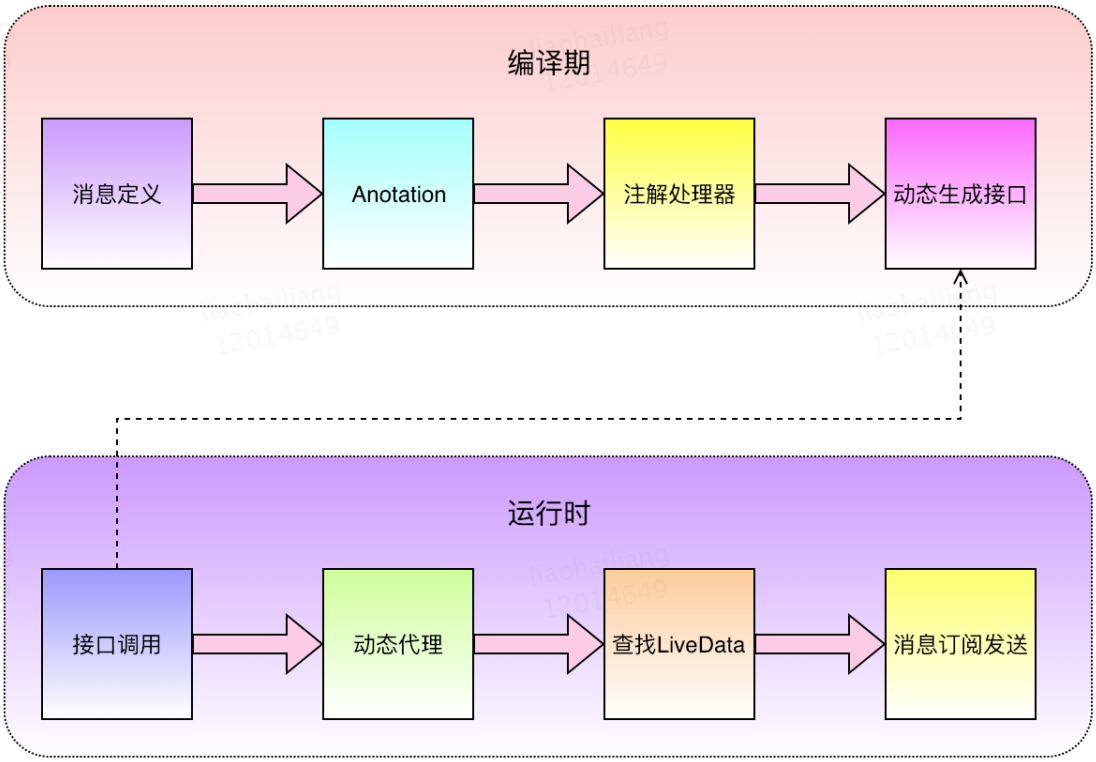
1. 只能订阅和发送在组件中预定义的消息。换句话说，使用者不能发送和订阅临时消息。
2. 消息的类型需要在定义的时候指定。
3. 定义消息的时候需要指定属于哪个组件。

3.7.7.3 如何实现这些约束

1. 在消息定义文件上使用注解，定义消息的类型和消息所属Module。
2. 定义注解处理器，在编译期间收集消息的相关信息。
3. 在编译器根据消息的信息生成调用时需要的interface，用接口约束消息发送和订阅。
4. 运行时构建基于两级HashMap的LiveData存储结构。
5. 运行时采用interface+动态代理的方式实现真正的消息订阅和发送。

整个流程如下图所示：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



实现流程

3.7.8 消息总线modular-event的结构

- **modular-event-base**：定义Anotation及其他基本类型
- **modular-event-core**：modular-event核心实现
- **modular-event-compiler**：注解处理器
- **modular-event-plugin**：Gradle Plugin

Anotation

- **@ModuleEvents**：消息定义

```
@Retention(RetentionPolicy.SOURCE)
@Target(ElementType.TYPE)
public @interface ModuleEvents {
    String module() default "";
}
```

- **@EventType**：消息类型

```
@Retention(RetentionPolicy.SOURCE)
@Target(ElementType.FIELD)
public @interface EventType {
    Class value();
}
```

消息定义

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

通过@ModuleEvents注解一个定义消息的Java类，如果@ModuleEvents指定了属性module，那么这个module的值就是这个消息所属的Module，如果没有指定属性module，则会把定义消息的Java类所在的包的包名作为消息所属的Module。

在这个消息定义java类中定义的消息都是public static final String类型。可以通过@EventType指定消息的类型，@EventType支持java原生类型或自定义类型，如果没有用@EventType指定消息类型，那么消息的类型默认为Object，下面是一个消息定义的示例：

```
//可以指定module，若不指定，则使用包名作为module名
@ModuleEvents()
public class DemoEvents {

    //不指定消息类型，那么消息的类型默认为Object
    public static final String EVENT1 = "event1";

    //指定消息类型为自定义Bean
    @EventType(TestEventBean.class)
    public static final String EVENT2 = "event2";

    //指定消息类型为java原生类型
    @EventType(String.class)
    public static final String EVENT3 = "event3";
}
```

interface自动生成

我们会在modular-event-compiler中处理这些注解，一个定义消息的Java类会生成一个接口，这个接口的命名是EventsDefineOf+消息定义类名，例如消息定义类的类名为DemoEvents，自动生成的接口就是EventsDefineOfDemoEvents。消息定义类中定义的每一个消息，都会转化成接口中的一个方法。使用者只能通过这些自动生成的接口使用消息总线。我们用这种巧妙的方式实现了对消息总线的约束。前文提到的那个消息定义示例DemoEvents.java会生成一个如下的接口类：

```
package com.sankuai.erp.modularevent.generated.com.meituan.jeremy.module_b_export;

public interface EventsDefineOfDemoEvents extends
com.sankuai.erp.modularevent.base.IEventsDefine {
  com.sankuai.erp.modularevent.Observable<java.lang.Object> EVENT1();

  com.sankuai.erp.modularevent.Observable<com.meituan.jeremy.module_b_export.TestEventBean>
EVENT2(
      );

  com.sankuai.erp.modularevent.Observable<java.lang.String> EVENT3();
}
```

关于接口类的自动生成，我们采用了square/javapoet来实现，网上介绍JavaPoet的文章很多，这里就不再累述。

3.7.9 使用动态代理实现运行时调用

有了自动生成的接口，就相当于有了一个壳，然而壳下面的所有逻辑，我们通过动态代理来实现，简单介绍一下代理模式和动态代理：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

- **代理模式**： 给某个对象提供一个代理对象，并由代理对象控制对于原对象的访问，即客户不直接操控原对象，而是通过代理对象间接地操控原对象。
- **动态代理**： 代理类是在运行时生成的。也就是说Java编译完之后并没有实际的class文件，而是在运行时动态生成的类字节码，并加载到VM中。

在动态代理的InvocationHandler中实现查找逻辑：

1. 根据interface的typename得到ModuleName。
2. 调用的方法的methodname即为消息名。
3. 根据ModuleName和消息名找到相应的LiveData。
4. 完成后续订阅消息或者发送消息的流程。

消息的订阅和发送可以用链式调用的方式编码：

- 订阅消息

```
ModularEventBus
        .get()
        .of(EventsDefineOfModuleBEvents.class)
        .EVENT1()
        .observe(this, new Observer<TestEventBean>() {
            @Override
            public void onChanged(@Nullable TestEventBean testEventBean) {
                Toast.makeText(MainActivity.this, "MainActivity收到自定义消息: " +
testEventBean.getMsg(),
                        Toast.LENGTH_SHORT).show();
            }
        });
```

- 发送消息

```
ModularEventBus
        .get()
        .of(EventsDefineOfModuleBEvents.class)
        .EVENT1()
        .setValue(new TestEventBean("aa"));
```

3.7.10 订阅和发送的模式

- 订阅消息的模式
    1. **observe**：生命周期感知，onDestroy的时候自动取消订阅。
    2. **observeSticky**：生命周期感知，onDestroy的时候自动取消订阅，Sticky模式。
    3. **observeForever**：需要手动取消订阅。
    4. **observeStickyForever**：需要手动取消订阅，Sticky模式。
- 发送消息的模式
    1. **setValue**：主线程调用。
    2. **postValue**：后台线程调用。

3.7.11 总结

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

本文介绍了美团行业收银研发组Android团队的组件化实践，以及强约束组件消息总线modular-event的原理和使用。我们团队很早之前就在探索组件化改造，前期有些方案在落地的时候遇到很多困难。我们也研究了很多开源的组件化方案，以及公司内部其他团队（美团App、美团外卖、美团收银等）的组件化方案，学习和借鉴了很多优秀的设计思想，当然也踩过不少的坑。我们逐渐意识到：任何一种组件化方案都有其适用场景，我们的组件化架构选择，应该更加面向业务，而不仅仅是面向技术本身。

3.7.12 后期工作展望

我们的组件化改造工作远远没有结束，未来可能会在以下几个方向继续进行深入的研究：

1. **组件管理**：组件化改造之后，每个组件是个独立的工程，组件也会迭代开发，如何对这些组件进行版本化管理。
2. **组件重用**：现在看起来对这些组件的重用是很方便的，只需要引入组件的库即可，但是如果一个新的项目到来，需求有些变化，我们应该怎样最大限度的重用这些组件。
3. **CI集成**：如何更好的与CI集成。
4. **集成到脚手架**：集成到脚手架，让新的项目从一开始就以组件化的模式进行开发。

参考资料

1. Android消息总线的演进之路：用LiveDataBus替代RxBus、EventBus
2. WMRouter：美团外卖Android开源路由框架
3. 美团外卖Android平台化架构演进实践

作者简介

- 海亮，美团高级工程师，2017年加入美团，目前主要负责美团轻收银、美团收银零售版等App的相关业务及模块开发工作。

3.8 美团Android消息总线的演进之路：用LiveDataBus替代RxBus、EventBus

3.8.1 背景

对于Android系统来说，消息传递是最基本的组件，每一个App内的不同页面，不同组件都在进行消息传递。消息传递既可以用于Android四大组件之间的通信，也可用于异步线程和主线程之间的通信。对于Android开发者来说，经常使用的消息传递方式有很多种，从最早使用的Handler、BroadcastReceiver、接口回调，到近几年流行的通信总线类框架EventBus、RxBus。Android消息传递框架，总在不断的演进之中。

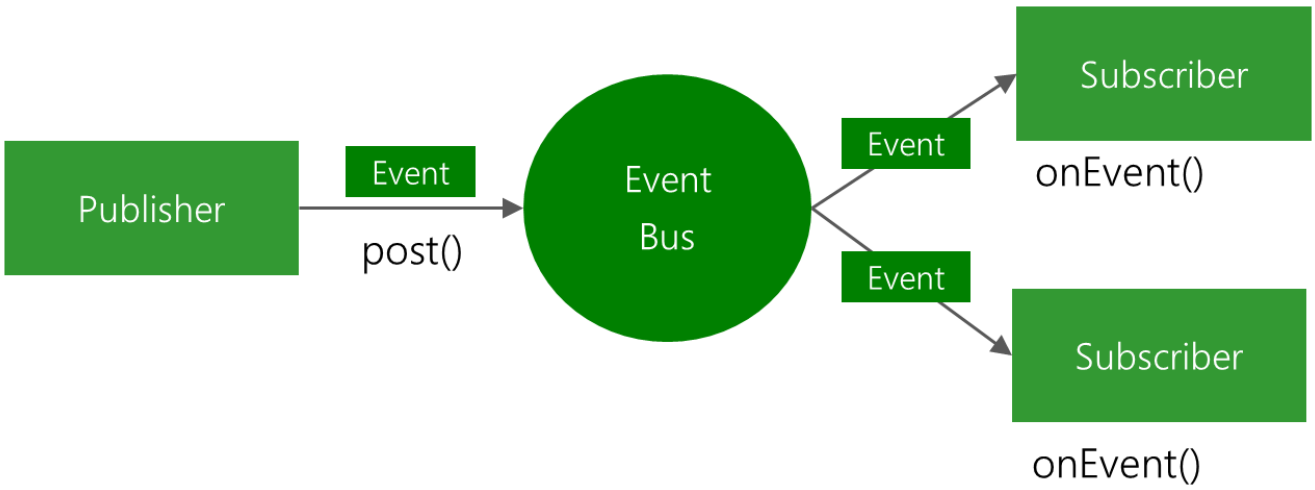
3.8.1.1 从EventBus说起

EventBus是一个Android事件发布/订阅框架，通过解耦发布者和订阅者简化Android事件传递。EventBus可以代替Android传统的Intent、Handler、Broadcast或接口回调，在Fragment、Activity、Service线程之间传递数据，执行方法。

EventBus最大的特点就是：简洁、解耦。在没有EventBus之前我们通常用广播来实现监听，或者自定义接口函数回调，有的场景我们也可以直接用Intent携带简单数据，或者在线程之间通过Handler处理消息传递。但无论是广播还是Handler机制远远不能满足我们高效的开发。EventBus简化了应用程序内各组件间、组件与后台线程间的通信。EventBus一经推出，便受到广大开发者的推崇。

现在看来，EventBus给Android开发者世界带来了一种新的框架和思想，就是消息的发布和订阅。这种思想在其后很多框架中都得到了应用。

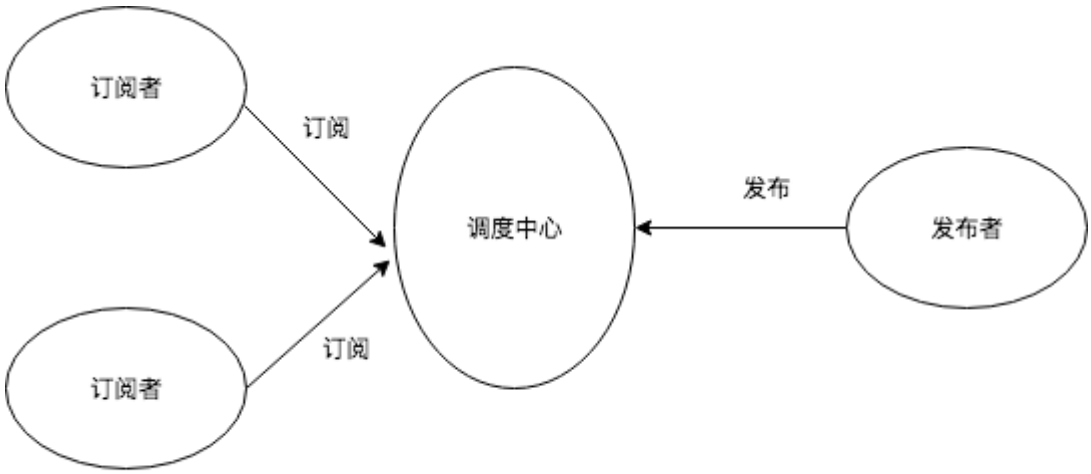
腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



图片摘自EventBus GitHub主页

3.8.1.2 发布/订阅模式

订阅发布模式定义了一种“一对多”的依赖关系，让多个订阅者对象同时监听某一个主题对象。这个主题对象在自身状态变化时，会通知所有订阅者对象，使它们能够自动更新自己的状态。



发布/订阅模式

3.8.2 RxBus的出现

RxBus不是一个库，而是一个文件，实现只有短短30行代码。RxBus本身不需要过多分析，它的强大完全来自于它基于的RxJava技术。响应式编程（Reactive Programming）技术这几年特别火，RxJava是它在Java上的实作。RxJava天生就是发布/订阅模式，而且很容易处理线程切换。所以，RxBus凭借区区30行代码，就敢挑战EventBus江湖老大的地位。

3.8.2.1 RxBus原理

在RxJava中有个Subject类，它继承Observable类，同时实现了Observer接口，因此Subject可以同时担当订阅者和被订阅者的角色，我们使用Subject的子类PublishSubject来创建一个Subject对象（PublishSubject只有被订阅后才会把接收到的事件立刻发送给订阅者），在需要接收事件的地方，订阅该Subject对象，之后如果Subject对象接收到事件，则会发射给该订阅者，此时Subject对象充当被订阅者的角色。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

完成了订阅，在需要发送事件的地方将事件发送给之前被订阅的Subject对象，则此时Subject对象作为订阅者接收事件，然后会立刻将事件转发给订阅该Subject对象的订阅者，以便订阅者处理相应事件，到这里就完成了事件的发送与处理。

最后就是取消订阅的操作了，RxJava中，订阅操作会返回一个Subscription对象，以便在合适的时机取消订阅，防止内存泄漏，如果一个类产生多个Subscription对象，我们可以用一个CompositeSubscription存储起来，以进行批量的取消订阅。

RxBus有很多实现，如：

AndroidKnife/RxBus（https://github.com/AndroidKnife/RxBus ）Blankj/RxBus（https://github.com/Blankj/RxBus ）

其实正如前面所说的，RxBus的原理是如此简单，我们自己都可以写出一个RxBus的实现：

3.8.2.2 基于RxJava1的RxBus实现

```
public final class RxBus {

    private final Subject<Object, Object> bus;

    private RxBus() {
        bus = new SerializedSubject<>(PublishSubject.create());
    }

    private static class SingletonHolder {
        private static final RxBus defaultRxBus = new RxBus();
    }

    public static RxBus getInstance() {
        return SingletonHolder.defaultRxBus;
    }

    /*
     * 发送
     */
    public void post(Object o) {
        bus.onNext(o);
    }

    /*
     * 是否有Observable订阅
     */
    public boolean hasObservable() {
        return bus.hasObservers();
    }

    /*
     * 转换为特定类型的Obserbale
     */
    public <T> Observable<T> toObservable(Class<T> type) {
        return bus.ofType(type);
    }
}
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

3.8.2.3 基于RxJava2的RxBus实现

```
public final class RxBus2 {

    private final Subject<Object> bus;

    private RxBus2() {
        // toSerialized method made bus thread safe
        bus = PublishSubject.create().toSerialized();
    }

    public static RxBus2 getInstance() {
        return Holder.BUS;
    }

    private static class Holder {
        private static final RxBus2 BUS = new RxBus2();
    }

    public void post(Object obj) {
        bus.onNext(obj);
    }

    public <T> Observable<T> toObservable(Class<T> tClass) {
        return bus.ofType(tClass);
    }

    public Observable<Object> toObservable() {
        return bus;
    }

    public boolean hasObservers() {
        return bus.hasObservers();
    }
}
```

3.8.3 引入LiveDataBus的想法

3.8.3.1 从LiveData谈起

LiveData是Android Architecture Components提出的框架。LiveData是一个可以被观察的数据持有类，它可以感知并遵循Activity、Fragment或Service等组件的生命周期。正是由于LiveData对组件生命周期可感知特点，因此可以做到仅在组件处于生命周期的激活状态时才更新UI数据。

LiveData需要一个观察者对象，一般是Observer类的具体实现。当观察者的生命周期处于STARTED或RESUMED状态时，LiveData会通知观察者数据变化；在观察者处于其他状态时，即使LiveData的数据变化了，也不会通知。

3.8.3.2 LiveData的优点

- **UI和实时数据保持一致** 因为LiveData采用的是观察者模式，这样一来就可以在数据发生改变时获得通知，更新UI。
- **避免内存泄漏** 观察者被绑定到组件的生命周期上，当被绑定的组件销毁（destroy）时，观察者会立刻自动清理自身的数据。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

- **不会再产生由于Activity处于stop状态而引起的崩溃**

例如：当Activity处于后台状态时，是不会收到LiveData的任何事件的。

- **不需要再解决生命周期带来的问题** LiveData可以感知被绑定的组件的生命周期，只有在活跃状态才会通知数据变化。
- **实时数据刷新** 当组件处于活跃状态或者从不活跃状态到活跃状态时总是能收到最新的数据。
- **解决Configuration Change问题** 在屏幕发生旋转或者被回收再次启动，立刻就能收到最新的数据。

3.8.3.3 谈一谈Android Architecture Components

Android Architecture Components的核心是Lifecycle、LiveData、ViewModel 以及 Room，通过它可以非常优雅的让数据与界面进行交互，并做一些持久化的操作，高度解耦，自动管理生命周期，而且不用担心内存泄漏的问题。

- **Room** 一个强大的SQLite对象映射库。
- **ViewModel** 一类对象，它用于为UI组件提供数据，在设备配置发生变更时依旧可以存活。
- **LiveData** 一个可感知生命周期、可被观察的数据容器，它可以存储数据，还会在数据发生改变时进行提醒。
- **Lifecycle** 包含LifeCycleOwer和LifecycleObserver，分别是生命周期所有者和生命周期感知者。

3.8.3.4 Android Architecture Components的特点

- **数据驱动型编程** 变化的永远是数据，界面无需更改。
- **感知生命周期，防止内存泄漏。**
- **高度解耦** 数据，界面高度分离。
- **数据持久化** 数据、ViewModel不与UI的生命周期挂钩，不会因为界面的重建而销毁。

3.8.4 为什么使用LiveData构建数据通信总线LiveDataBus

3.8.4.1 使用LiveData的理由

- **LiveData具有的这种可观察性和生命周期感知的能力，使其非常适合作为Android通信总线的基础构件。**
- **使用者不用显示调用反注册方法。**

由于LiveData具有生命周期感知能力，所以LiveDataBus只需要调用注册回调方法，而不需要显示的调用反注册方法。这样带来的好处不仅可以编写更少的代码，而且可以完全杜绝其他通信总线类框架（如EventBus、RxBus）忘记调用反注册所带来的内存泄漏的风险。

3.8.4.2 为什么要用LiveDataBus替代EventBus和RxBus

- **LiveDataBus的实现及其简单** 相对EventBus复杂的实现，LiveDataBus只需要一个类就可以实现。
- **LiveDataBus可以减小APK包的大小** 由于LiveDataBus只依赖Android官方Android Architecture Components组件的LiveData，没有其他依赖，本身实现只有一个类。作为比较，EventBus JAR包大小为57kb，RxBus依赖RxJava和RxAndroid，其中RxJava2包大小2.2MB，RxJava1包大小1.1MB，RxAndroid包大小9kb。使用LiveDataBus可以大大减小APK包的大小。
- **LiveDataBus依赖方支持更好** LiveDataBus只依赖Android官方Android Architecture Components组件的LiveData，相比RxBus依赖的RxJava和RxAndroid，依赖方支持更好。
- **LiveDataBus具有生命周期感知** LiveDataBus具有生命周期感知，在Android系统中使用调用者不需要调用反注册，相比EventBus和RxBus使用更为方便，并且没有内存泄漏风险。

3.8.5 LiveDataBus的设计和架构

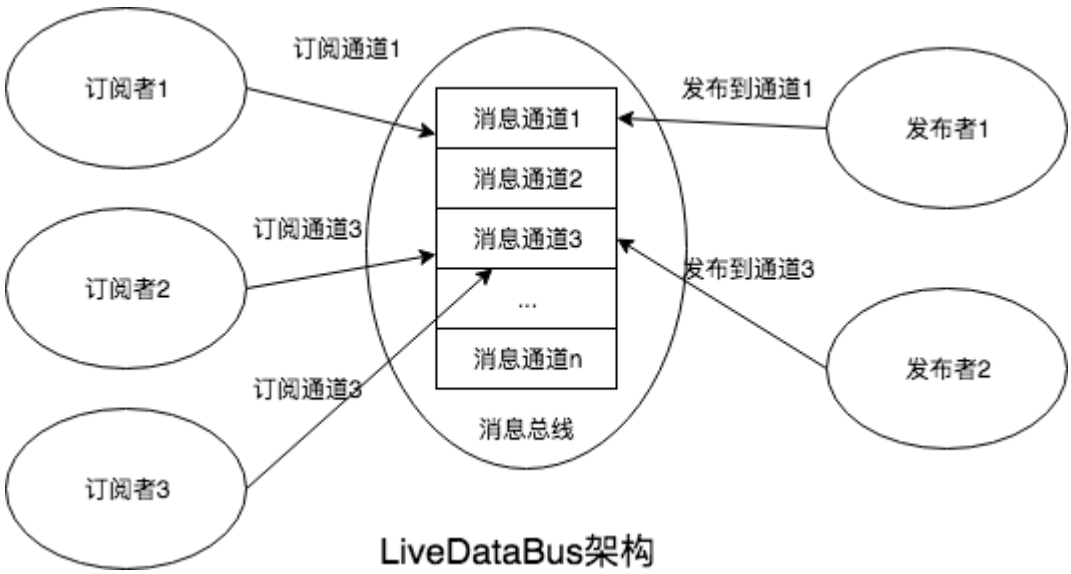
3.8.5.1 LiveDataBus的组成

- **消息** 消息可以是任何的Object，可以定义不同类型的消息，如Boolean、String。也可以定义自定义类型的消息。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

- **消息通道** LiveData扮演了消息通道的角色，不同的消息通道用不同的名字区分，名字是String类型的，可以通过名字获取到一个LiveData消息通道。
- **消息总线** 消息总线通过单例实现，不同的消息通道存放在一个HashMap中。
- **订阅** 订阅者通过getChannel获取消息通道，然后调用observe订阅这个通道的消息。
- **发布** 发布者通过getChannel获取消息通道，然后调用setValue或者postValue发布消息。

3.8.5.2 LiveDataBus原理图



LiveDataBus原理图

3.8.5.3 LiveDataBus的实现

第一个实现：

```
public final class LiveDataBus {

    private final Map<String, MutableLiveData<Object>> bus;

    private LiveDataBus() {
        bus = new HashMap<>();
    }

    private static class SingletonHolder {
        private static final LiveDataBus DATA_BUS = new LiveDataBus();
    }

    public static LiveDataBus get() {
        return SingletonHolder.DATA_BUS;
    }

    public <T> MutableLiveData<T> getChannel(String target, Class<T> type) {
        if (!bus.containsKey(target)) {
            bus.put(target, new MutableLiveData<>());
        }
        return (MutableLiveData<T>) bus.get(target);
    }
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
    public MutableLiveData<Object> getChannel(String target) {
        return getChannel(target, Object.class);
    }
}
```

短短二十行代码，就实现了一个通信总线的全部功能，并且还具有生命周期感知功能，并且使用起来也及其简单：

注册订阅：

```
LiveDataBus.get().getChannel("key_test", Boolean.class)
        .observe(this, new Observer<Boolean>() {
            @Override
            public void onChanged(@Nullable Boolean aBoolean) {
            }
        });
```

发送消息：

```
LiveDataBus.get().getChannel("key_test").setValue(true);
```

我们发送了一个名为"key_test"，值为true的事件。

这个时候订阅者就会收到消息，并作相应的处理，非常简单。

问题出现

对于LiveDataBus的第一版实现，我们发现，在使用这个LiveDataBus的过程中，订阅者会收到订阅之前发布的消息。对于一个消息总线来说，这是不可接受的。无论EventBus或者RxBus，订阅方都不会收到订阅之前发出的消息。对于一个消息总线，LiveDataBus必须要解决这个问题。

问题分析

怎么解决这个问题呢？先分析下原因：

当LifeCircleOwner的状态发生变化的时候，会调用LiveData.ObserverWrapper的activeStateChanged函数，如果这个时候ObserverWrapper的状态是active，就会调用LiveData的dispatchingValue。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
void activeStateChanged(boolean newActive) {
    if (newActive == mActive) {
        return;
    }
    // immediately set active state, so we'd never dispatch anything to inactive
    // owner
    mActive = newActive;
    boolean wasInactive = LiveData.this.mActiveCount == 0;
    LiveData.this.mActiveCount += mActive ? 1 : -1;
    if (wasInactive && mActive) {
        onActive();
    }
    if (LiveData.this.mActiveCount == 0 && !mActive) {
        onInactive();
    }
    if (mActive) {
        dispatchingValue( initiator: this);
    }
}
```

在LiveData的dispatchingValue中，又会调用LiveData的considerNotify方法。

```
private void dispatchingValue(@Nullable ObserverWrapper initiator) {
    if (mDispatchingValue) {
        mDispatchInvalidated = true;
        return;
    }
    mDispatchingValue = true;
    do {
        mDispatchInvalidated = false;
        if (initiator != null) {
            considerNotify(initiator);
            initiator = null;
        } else {
            for (Iterator<Map.Entry<Observer<T>, ObserverWrapper>> iterator =
                    mObservers.iteratorWithAdditions(); iterator.hasNext(); ) {
                considerNotify(iterator.next().getValue());
                if (mDispatchInvalidated) {
                    break;
                }
            }
        }
    } while (mDispatchInvalidated);
    mDispatchingValue = false;
}
```

在LiveData的considerNotify方法中，红框中的逻辑是关键，如果ObserverWrapper的mLastVersion小于LiveData的mVersion，就会去回调mObserver的onChanged方法。而每个新的订阅者，其version都是-1，LiveData一旦设置过其version是大于-1的（每次LiveData设置值都会使其version加1），这样就会导致LiveDataBus每注册一个新的订阅者，这个订阅者立刻会收到一个回调，即使这个设置的动作发生在订阅之前。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
private void considerNotify(ObserverWrapper observer) {
    if (!observer.mActive) {
        return;
    }
    // Check latest state b4 dispatch. Maybe it changed state but we didn't get the event
    //
    // we still first check observer.active to keep it as the entrance for events. So even
    // the observer moved to an active state, if we've not received that event, we better
    // notify for a more predictable notification order.
    if (!observer.shouldBeActive()) {
        observer.activeStateChanged( newActive: false);
        return;
    }
    if (observer.mLastVersion >= mVersion) {
        return;
    }
    observer.mLastVersion = mVersion;
    //noinspection unchecked
    observer.mObserver.onChanged((T) mData);
}
```

问题原因总结

对于这个问题，总结一下发生的核心原因。对于LiveData，其初始的version是-1，当我们调用了其setValue或者postValue，其vesion会+1；对于每一个观察者的封装ObserverWrapper，其初始version也为-1，也就是说，每一个新注册的观察者，其version为-1；当LiveData设置这个ObserverWrapper的时候，如果LiveData的version大于ObserverWrapper的version，LiveData就会强制把当前value推送给Observer。

如何解决这个问题

明白了问题产生的原因之后，我们来看看怎么才能解决这个问题。很显然，根据之前的分析，只需要在注册一个新的订阅者的时候把Wrapper的version设置成跟LiveData的version一致即可。

那么怎么实现呢，看看LiveData的observe方法，他会在步骤1创建一个LifecycleBoundObserver，LifecycleBoundObserver是ObserverWrapper的派生类。然后会在步骤2把这个LifecycleBoundObserver放入一个私有Map容器mObservers中。无论ObserverWrapper还是LifecycleBoundObserver都是私有的或者包可见的，所以无法通过继承的方式更改LifecycleBoundObserver的version。

那么能不能从Map容器mObservers中取到LifecycleBoundObserver，然后再更改version呢？答案是肯定的，通过查看SafeIterableMap的源码我们发现有一个protected的get方法。因此，在调用observe的时候，我们可以通过反射拿到LifecycleBoundObserver，再把LifecycleBoundObserver的version设置成和LiveData一致即可。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
@MainThread
public void observe(@NonNull LifecycleOwner owner, @NonNull Observer<T> observer) {
    if (owner.getLifecycle().getCurrentState() == DESTROYED) {
        // ignore
        return;
    }
    LifecycleBoundObserver wrapper = new LifecycleBoundObserver(owner, observer);
    ObserverWrapper existing = mObservers.putIfAbsent(observer, wrapper);
    if (existing != null && !existing.isAttachedTo(owner)) {
        throw new IllegalArgumentException("Cannot add the same observer"
                + " with different lifecycles");
    }
    if (existing != null) {
        return;
    }
    owner.getLifecycle().addObserver(wrapper);
}
```

对于非生命周期感知的observeForever方法来说，实现的思路是一致的，但是具体的实现略有不同。observeForever的时候，生成的wrapper不是LifecycleBoundObserver，而是AlwaysActiveObserver（步骤1），而且我们也没有机会在observeForever调用完成之后再去更改AlwaysActiveObserver的version，因为在observeForever方法体内，步骤3的语句，回调就发生了。

```
@MainThread
public void observeForever(@NonNull Observer<T> observer) {
    AlwaysActiveObserver wrapper = new AlwaysActiveObserver(observer);
    ObserverWrapper existing = mObservers.putIfAbsent(observer, wrapper);
    if (existing != null && existing instanceof LiveData.LifecycleBoundObserver) {
        throw new IllegalArgumentException("Cannot add the same observer"
                + " with different lifecycles");
    }
    if (existing != null) {
        return;
    }
    wrapper.activeStateChanged( newActive: true);
}
```

那么对于observeForever，如何解决这个问题呢？既然是在调用内回调的，那么我们可以写一个ObserverWrapper，把真正的回调给包装起来。把ObserverWrapper传给observeForever，那么在回调的时候我们去检查调用栈，如果回调是observeForever方法引起的，那么就不回调真正的订阅者。

3.8.6 LiveDataBus最终实现

```
public final class LiveDataBus {

    private final Map<String, BusMutableLiveData<Object>> bus;

    private LiveDataBus() {
        bus = new HashMap<>();
    }

    private static class SingletonHolder {
        private static final LiveDataBus DEFAULT_BUS = new LiveDataBus();
    }

    public static LiveDataBus get() {
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
        return SingletonHolder.DEFAULT_BUS;
    }

    public <T> MutableLiveData<T> with(String key, Class<T> type) {
        if (!bus.containsKey(key)) {
            bus.put(key, new BusMutableLiveData<>());
        }
        return (MutableLiveData<T>) bus.get(key);
    }

    public MutableLiveData<Object> with(String key) {
        return with(key, Object.class);
    }

    private static class ObserverWrapper<T> implements Observer<T> {

        private Observer<T> observer;

        public ObserverWrapper(Observer<T> observer) {
            this.observer = observer;
        }

        @Override
        public void onChanged(@Nullable T t) {
            if (observer != null) {
                if (isCallOnObserve()) {
                    return;
                }
                observer.onChanged(t);
            }
        }

        private boolean isCallOnObserve() {
            StackTraceElement[] stackTrace = Thread.currentThread().getStackTrace();
            if (stackTrace != null && stackTrace.length > 0) {
                for (StackTraceElement element : stackTrace) {
                    if ("android.arch.lifecycle.LiveData".equals(element.getClassName()) &&
                            "observeForever".equals(element.getMethodName())) {
                        return true;
                    }
                }
            }
            return false;
        }
    }

    private static class BusMutableLiveData<T> extends MutableLiveData<T> {

        private Map<Observer, Observer> observerMap = new HashMap<>();

        @Override
        public void observe(@NonNull LifecycleOwner owner, @NonNull Observer<T> observer) {
            super.observe(owner, observer);
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
        try {
            hook(observer);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    @Override
    public void observeForever(@NonNull Observer<T> observer) {
        if (!observerMap.containsKey(observer)) {
            observerMap.put(observer, new ObserverWrapper(observer));
        }
        super.observeForever(observerMap.get(observer));
    }

    @Override
    public void removeObserver(@NonNull Observer<T> observer) {
        Observer realObserver = null;
        if (observerMap.containsKey(observer)) {
            realObserver = observerMap.remove(observer);
        } else {
            realObserver = observer;
        }
        super.removeObserver(realObserver);
    }

    private void hook(@NonNull Observer<T> observer) throws Exception {
        //get wrapper's version
        Class<LiveData> classLiveData = LiveData.class;
        Field fieldObservers = classLiveData.getDeclaredField("mObservers");
        fieldObservers.setAccessible(true);
        Object objectObservers = fieldObservers.get(this);
        Class<?> classObservers = objectObservers.getClass();
        Method methodGet = classObservers.getDeclaredMethod("get", Object.class);
        methodGet.setAccessible(true);
        Object objectWrapperEntry = methodGet.invoke(objectObservers, observer);
        Object objectWrapper = null;
        if (objectWrapperEntry instanceof Map.Entry) {
            objectWrapper = ((Map.Entry) objectWrapperEntry).getValue();
        }
        if (objectWrapper == null) {
            throw new NullPointerException("Wrapper can not be bull!");
        }
        Class<?> classObserverWrapper = objectWrapper.getClass().getSuperclass();
        Field fieldLastVersion = classObserverWrapper.getDeclaredField("mLastVersion");
        fieldLastVersion.setAccessible(true);
        //get livedata's version
        Field fieldVersion = classLiveData.getDeclaredField("mVersion");
        fieldVersion.setAccessible(true);
        Object objectVersion = fieldVersion.get(this);
        //set wrapper's version
        fieldLastVersion.set(objectWrapper, objectVersion);
    }
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
    }
}
```

注册订阅

```
LiveDataBus.get()
        .with("key_test", String.class)
        .observe(this, new Observer<String>() {
            @Override
            public void onChanged(@Nullable String s) {
            }
        });
```

发送消息：

```
LiveDataBus.get().with("key_test").setValue(s);
```

源码说明

LiveDataBus的源码可以直接拷贝使用，也可以前往作者的GitHub仓库查看下载：https://github.com/JeremyLiao/LiveDataBus 。

3.8.7 总结

本文提供了一个新的消息总线框架——LiveDataBus。订阅者可以订阅某个消息通道的消息，发布者可以把消息发布到消息通道上。利用LiveDataBus，不仅可以实现消息总线功能，而且对于订阅者，他们不需要关心何时取消订阅，极大减少了因为忘记取消订阅造成的内存泄漏风险。

作者介绍

- 海亮，美团高级工程师，2017年加入美团，目前主要负责美团轻收银、美团收银零售版等App的相关业务及模块开发工作。

3.9 WMRouter：美团外卖Android开源路由框架

WMRouter是一款Android路由框架，基于组件化的设计思路，功能灵活，使用也比较简单。

WMRouter最初用于解决美团外卖C端App在业务演进过程中的实际问题，之后逐步推广到了美团其他App，因此我们决定将其开源，希望更多技术同行一起开发，应用到更广泛的场景里去。Github项目地址与使用文档详见 https://github.com/meituan/WMRouter。

本文先简单介绍WMRouter的功能和适用场景，然后详细介绍WMRouter的发展背景和过程。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

3.9.1 功能简介

WMRouter主要提供URI分发、ServiceLoader两大功能。

URI分发功能可用于多工程之间的页面跳转、动态下发URI链接的跳转等场景，特点如下：

1. 支持多scheme、host、path
2. 支持URI正则匹配
3. 页面配置支持Java代码动态注册，或注解配置自动注册
4. 支持配置全局和局部拦截器，可在跳转前执行同步/异步操作，例如定位、登录等
5. 支持单次跳转特殊操作：Intent设置Extra/Flags、设置跳转动画、自定义StartActivity操作等
6. 支持页面Exported控制，特定页面不允许外部跳转
7. 支持配置全局和局部降级策略
8. 支持配置单次和全局跳转监听
9. 完全组件化设计，核心组件均可扩展、按需组合，实现灵活强大的功能

基于SPI (Service Provider Interfaces) 的设计思想，WMRouter提供了ServiceLoader模块，类似Java中的`java.util.ServiceLoader`，但功能更加完善。通过ServiceLoader可以在一个App的多个模块之间通过接口调用代码，实现模块解耦，便于实现组件化、模块间通信，以及和依赖注入类似的功能等。其特点如下：

1. 使用注解自动配置
2. 支持获取接口的所有实现，或根据Key获取特定实现
3. 支持获取Class或获取实例
4. 支持无参构造、Context构造，或自定义Factory、Provider构造
5. 支持单例管理
6. 支持方法调用

其他特性：

1. 优化的Gradle插件，对编译耗时影响很小
2. 编译期和运行时配置检查，避免配置冲突和错误
3. 编译期自动添加Proguard混淆规则，免去手动配置的繁琐
4. 完善的调试功能，帮助及时发现问题

3.9.2 适用场景

WMRouter适用但不限于以下场景：

1. Native+H5混合开发模式，需要进行页面之间的互相跳转，或进行灵活的运营跳转链接下发。可以利用WMRouter统一页面跳转逻辑，根据不同的协议（HTTP、HTTPS、用于Native页面的自定义协议）跳转对应页面，且在跳转过程中可以使用UriInterceptor对跳转链接进行修改，例如跳转H5页面时在URL中加参数。
2. 统一管理来自App外部的URI跳转。来自App外部的URI跳转，如果使用Android原生的Manifest配置，会直接启动匹配的Activity，而很多时候希望先正常启动App打开首页，完成常规初始化流程（例如登录、定位等）后再跳转目标页面。此时可以使用统一的Activity接收所有外部URI跳转，到首页时再用WMRouter启动目标页面。
3. 页面跳转有复杂判断逻辑的场景。例如多个页面都需要先登录、先定位后才允许打开，如果使用常规方案，这些页面都需要处理相同的业务逻辑；而利用WMRouter，只需要开发好UriInterceptor并配置到各个页面即可。
4. 多工程、组件化、平台化开发。多工程开发要求各个工程之间能互相通信，也可能遇到和外卖App类似的代码复用、依赖注入、编译等问题，这些问题都可以利用WMRouter的URI分发和ServiceLoader模块解决。
5. 对业务埋点需求较强的场景。页面跳转作为最常见的业务逻辑之一，常常需要埋点。给每个页面配置好URI，使用WMRouter统一进行页面跳转，并在全局的OnCompleteListener中埋点即可。
6. 对App可用性要求较高的场景。一方面，可以对页面跳转失败进行埋点监控上报，及时发现线上问题；另一方面，页面跳转时可以执行判断逻辑，发现异常（例如服务端异常、客户端崩溃等）则自动打开降级后的页面，保证关键功能的正常工作，或给用户友好的提示。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

7. 页面A/B测试、动态配置等场景。在WMRouter提供的接口基础上进行少量开发配置，就可以实现：根据下发的A/B测试策略跳转不同的页面实现；根据不同的需要动态下发一组路由表，相同的URI跳转到不同的一组页面（实现方面可以自定义UriInterceptor，对匹配的URI返回301的UriResult使跳转重定向）。

3.9.3 基本概念解释

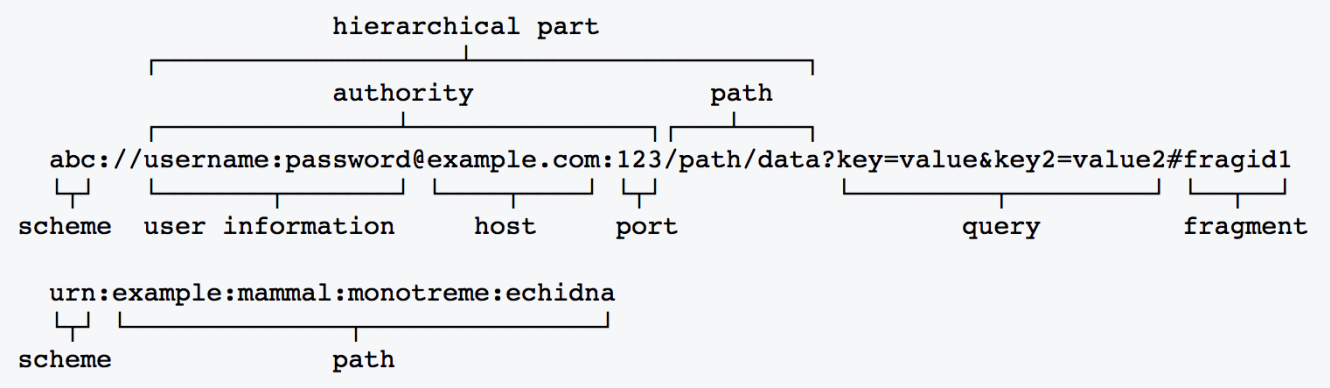
下面开始介绍WMRouter的发展背景和过程。为了方便后文的理解，我们先简单了解和回顾几个基本概念。

路由

根据维基百科的解释，路由（routing）可以理解成在互联的网络通过特定的协议把信息从源地址传输到目的地址的过程。一个典型的例子就是在互联网中，路由器可以根据IP协议将数据发送到特定的计算机。

URI

URI（Uniform Resource Identifier，统一资源标识符）是一个用于标识某一互联网资源名称的字符串。URI的组成如下图所示。



一些常见的URI举例如下，包括平时经常用到的网址、IP地址、FTP地址、文件、打电话、发邮件的协议等。

- http://www.meituan.com
- http://127.0.0.1:8080
- ftp://example.org/resource.txt
- file:///Users/
- tel:863-1234
- mailto:chris@example.com

在Android中也提供了`android.net.Uri`工具类用于处理URI，Android中URI常用的几个部分主要是scheme、host、path和query。

Android中的Intent跳转

在Android中的Intent跳转，分为显式跳转和隐式跳转两种。

显式跳转即指定ComponentName（类名）的Intent跳转，一般通过Bundle传参，示例代码如下：

```
Intent intent = new Intent(context, TestActivity.class);
intent.putExtra("param", "value")
startActivity(intent);
```

隐式跳转即不指定ComponentName的Intent跳转，通过IntentFilter找到匹配的组件，IntentFilter支持action、category和data的匹配，其中data就是URI。例如下面的代码，会启动系统默认的浏览器打开网页：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
Intent intent = new Intent(Intent.ACTION_VIEW);
intent.setData(Uri.parse("http://www.meituan.com"))
startActivity(intent);
```

Activity通过Manifest配置IntentFilter，例如下面的配置可以匹配所有形如`demo_scheme://demo_host/***`的URI。

```
<activity android:name=".app.UriProxyActivity" android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.VIEW"/>

        <category android:name="android.intent.category.DEFAULT"/>
        <category android:name="android.intent.category.BROWSABLE"/>

        <data android:scheme="demo_scheme" android:host="demo_host"/>
    </intent-filter>
</activity>
```

3.9.4 URI跳转

在美团外卖C端早期开发过程中，产品希望通过后台下发URI控制客户端跳转指定页面，从而实现灵活的运营配置。外卖App采用了Native+H5的混合开发模式，Native页面定义了专用的URI，而H5页面则使用HTTP/HTTPS链接在专门的WebView容器中加载，两种链接的跳转逻辑不同，实现起来比较繁琐。

Native页面的URI跳转最开始使用的是Android原生的IntentFilter，通过隐式跳转启动，但是这种方式存在灵活性差、功能缺失、Bug多等问题。例如：

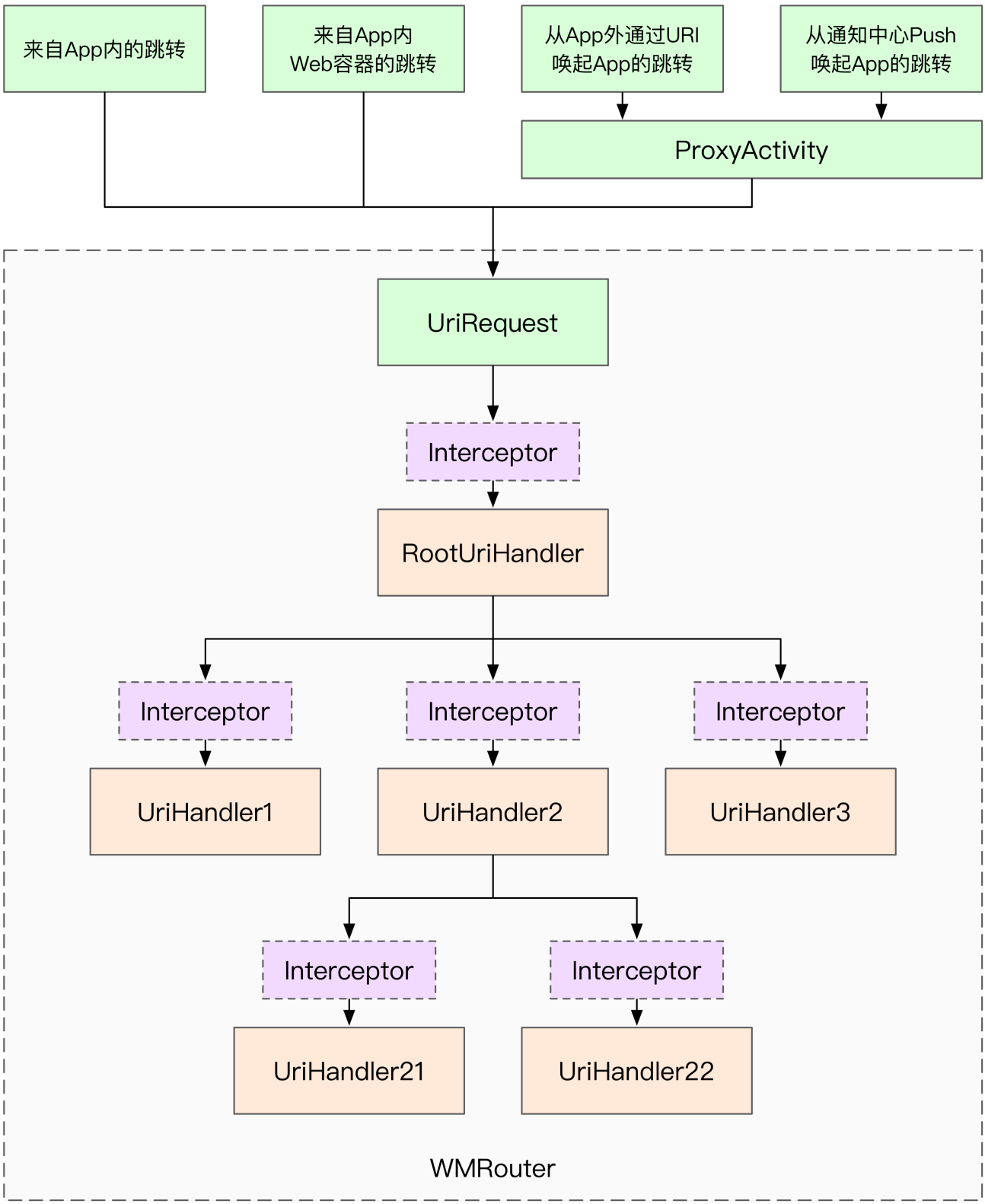
1. 从外部（浏览器、微信等）跳转外卖的URI时，系统会直接打开相应的Activity，而没有经过欢迎页的正常启动流程，一些代码逻辑可能没有执行，例如定位逻辑。
2. 有很多页面在打开前需要确保用户先登录或先定位，每个页面都写一遍判断登录、定位的逻辑非常麻烦，提高了开发维护成本。
3. 运营人员可能会配错URI，页面跳转失败，有些跳转的地方没有做try-catch处理，会产生Crash；有些地方虽然加了try-catch，但跳转失败后没有任何响应，用户体验差；跳转失败没有监控，不能及时发现和解决线上业务异常。

为了解决上述问题，我们希望有一个Android的URI分发组件，可以根据URI中不同的scheme、host、path，进行不同的处理，同时能够在页面跳转过程中进行更灵活的干预。调研发现，现有的一些Android路由组件主要都是在解决多工程之间解耦的问题，而URI往往只支持通过path分发，页面跳转的配置也不够灵活，难以满足实际需要。于是我们决定自行设计实现。

核心设计思路

下图展示了WMRouter中URI分发机制的核心设计思路。借鉴网络请求的机制，WMRouter中的每次URI跳转视为发起一个UriRequest；URI跳转请求被WMRouter逐层分发给一系列的UriHandler进行处理；每个UriHandler处理之前可以被UriInterceptor拦截，并插入一些特殊操作。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



3.9.5 页面跳转来源

常见的页面跳转来源如下：

1. 来自App内部Native页面的跳转
2. 来自App内Web容器的跳转，即H5页面发起的跳转
3. 从App外通过URI唤起App的跳转，例如来自浏览器、微信等
4. 从通知中心Push唤起App的跳转

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

对于来自App内部和Web容器的跳转，我们把所有跳转代码统一改成调用WMRouter处理，而来自外部和Push通知的跳转则全部使用一个独立的中转Activity接收，再调用WMRouter处理。

UriRequest

UriRequest中包含Context、URI和Fields，其中Fields为HashMap，可以通过Key存放任意数据。简单起见，UriRequest类同时承担了Response的功能，跳转请求的结果，也会被保存到Fields中。Fields可以根据需要自定义，其中一些常见字段举例如下：

- Intent的Extra参数，Bundle类型
- 用于startActivityForResult的RequestCode，int类型
- 用于overridePendingTransition方法的页面切换动画资源，int[]类型
- 本次跳转结果的监听器，OnCompleteListener类型

每次URI跳转请求会有一个ResultCode（类似HTTP请求的ResponseCode），表示跳转结果，也存放在Fields中。常见Code如下，用户也可以自定义Code：

- 200：跳转成功
- 301：重定向到其他URI，会再次跳转
- 400：请求错误，通常是Context或URI为空
- 403：禁止跳转，例如跳转白名单以外的HTTP链接、Activity的exported为false等
- 404：找不到目标(Activity或UriHandler)
- 500：发生错误

总结来说，UriRequest用于实现一次URI跳转中所有组件之间的通信功能。

UriHandler

UriHandler用于处理URI跳转请求，可以嵌套从而逐层分发和处理请求。UriHandler是异步结构，接收到UriRequest后处理（例如跳转Activity等），如果处理完成，则调用`callback.onComplete()`并传入ResultCode；如果没有处理，则调用`callback.onNext()`继续分发。下面的示例代码展示了一个只处理HTTP链接的UriHandler的实现：

```
public interface UriCallback {

    /**
     * 处理完成，继续后续流程。
     */
    void onNext();

    /**
     * 处理完成，终止分发流程。
     *
     * @param resultCode 结果
     */
    void onComplete(int resultCode);
}

public class DemoUriHandler extends UriHandler {
    public void handle(@NonNull final UriRequest request, @NonNull final UriCallback
callback) {
        Uri uri = request.getUri();
        // 处理HTTP链接
        if ("http".equalsIgnoreCase(uri.getScheme())) {
            try {
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
                // 调用系统浏览器
                Intent intent = new Intent();
                intent.setAction(Intent.ACTION_VIEW);
                intent.setData(uri);
                request.getContext().startActivity(intent);
                // 跳转成功
                callback.onComplete(UriResult.CODE_SUCCESS);
            } catch (Exception e) {
                // 跳转失败
                callback.onComplete(UriResult.CODE_ERROR);
            }
        } else {
            // 非HTTP链接不处理，继续分发
            callback.onNext();
        }
    }
    // ...
}
```

UriInterceptor

UriInterceptor为拦截器，不做最终的URI跳转操作，但可以在最终的跳转前进行各种同步/异步操作，常见操作举例如下：

- URI跳转拦截，禁止特定的URI跳转，直接返回403（例如禁止跳转非meituan域名的HTTP链接）
- URI参数修改（例如在HTTP链接末尾添加query参数）
- 各种中间处理（例如打开登录页登录、获取定位、发网络请求）
- ……

每个UriHandler都可以添加若干UriInterceptor。在UriHandler基类中，handle()方法先调用抽象方法`shouldHandle()`判断是否要处理UriRequest，如果需要处理，则逐个执行Interceptor，最后再调用`handleInternal()`方法进行跳转操作。

```
public abstract class UriHandler {

    // ChainedInterceptor将多个UriInterceptor合并成一个
    protected ChainedInterceptor mInterceptor;

    public UriHandler addInterceptor(@NonNull UriInterceptor interceptor) {
        if (interceptor != null) {
            if (mInterceptor == null) {
                mInterceptor = new ChainedInterceptor();
            }
            mInterceptor.addInterceptor(interceptor);
        }
        return this;
    }

    public void handle(@NonNull final UriRequest request, @NonNull final UriCallback
callback) {
        if (shouldHandle(request)) {
            if (mInterceptor != null) {
                mInterceptor.intercept(request, new UriCallback() {
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

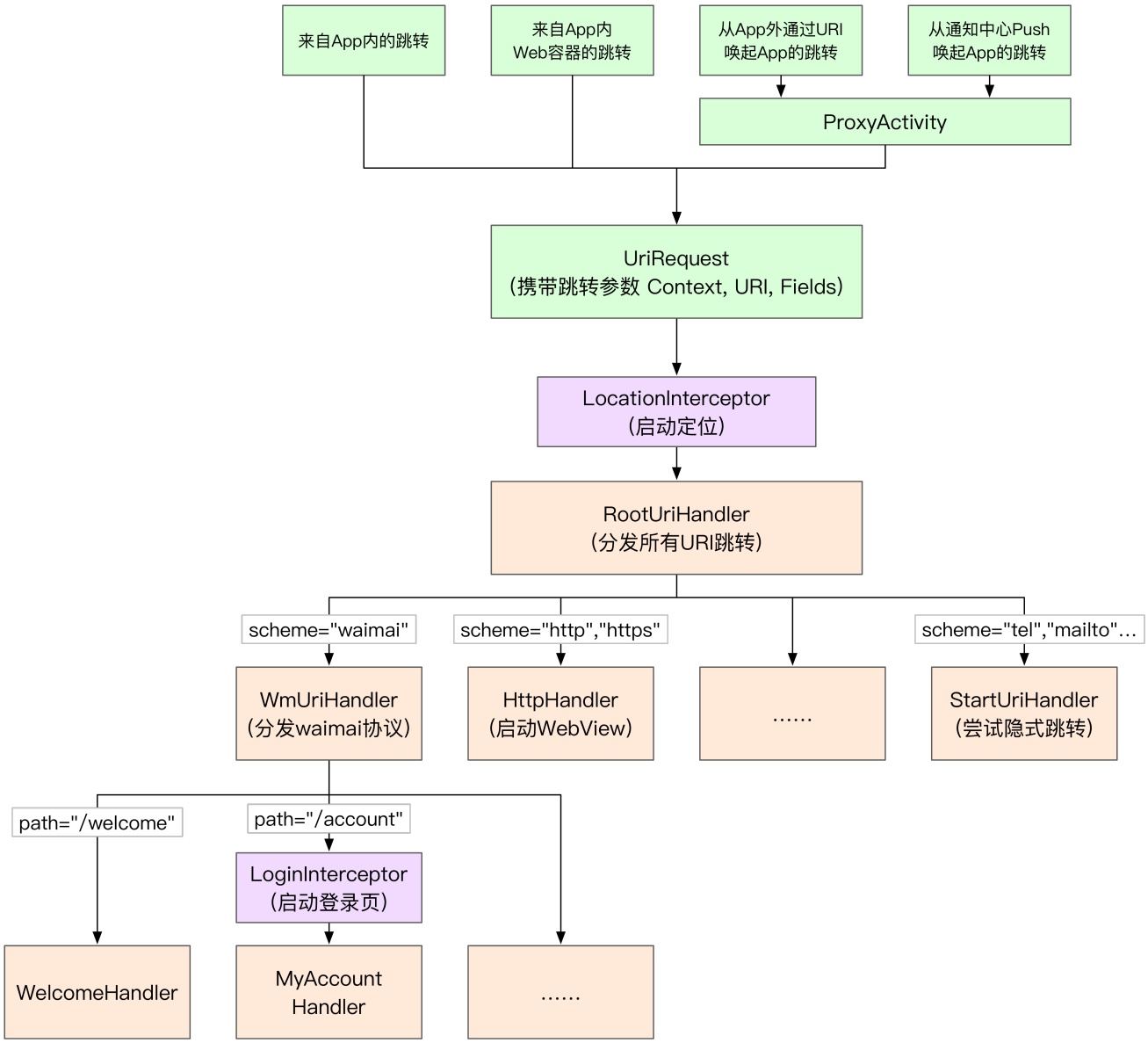
```
                    @Override
                    public void onNext() {
                        handleInternal(request, callback);
                    }

                    @Override
                    public void onComplete(int result) {
                        callback.onComplete(result);
                    }
                });
            } else {
                handleInternal(request, callback);
            }
        } else {
            callback.onNext();
        }
    }

    /**
     * 是否要处理给定的uri。在Interceptor之前调用。
     */
    protected abstract boolean shouldHandle(@NonNull UriRequest request);

    /**
     * 处理uri。在Interceptor之后调用。
     */
    protected abstract void handleInternal(@NonNull UriRequest request, @NonNull UriCallback
callback);
}
```

URI的分发与降级

在外卖C端App中的URI分发示意如下图。所有URI跳转都会分发到RootUriHandler，然后根据不同的scheme分发到不同的子Handler。例如waimai协议分发到WmUriHandler，然后进一步根据不同的path分发到子Handler，从而启动相应的Activity；HTTP/HTTPS协议分发到HttpHandler，启动WebView容器；对于其他类型URI（tel、mailto等），前面的几个Handler都无法处理，则会分发到StartUriHandler，尝试使用Android原生的隐式跳转启动系统应用。

每个UriHandler都可以根据实际需要实现降级策略，也可以不作处理继续分发给其他UriHandler。RootUriHandler中提供了一个全局的分发完成事件监听器，当UriHandler处理失败返回异常ResultCode或所有子UriHandler都没有处理时，执行全局降级策略。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

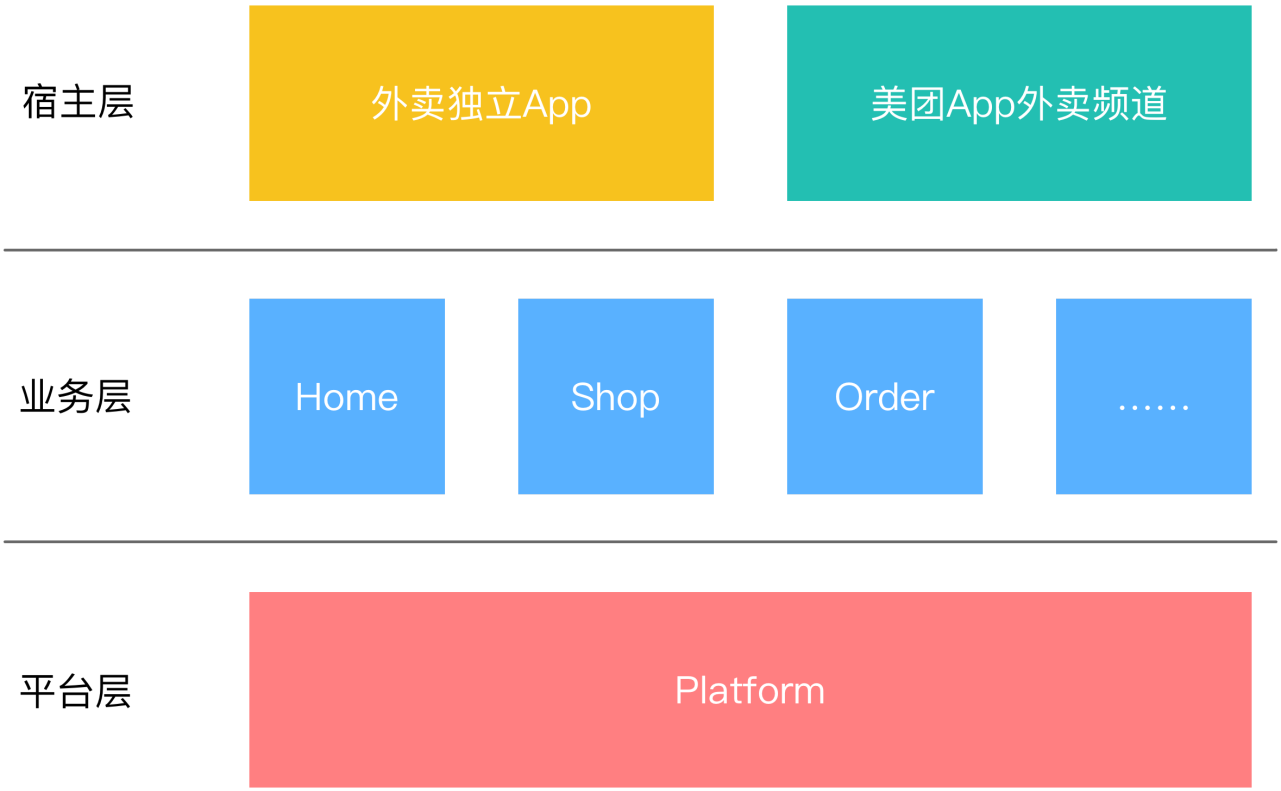


3.9.6 平台化与两端复用

随着外卖C端业务的演进，团队成员扩充了数倍，商超生鲜等垂直品类的拆分，以及异地研发团队的建立，客户端的平台化被提上日程。关于外卖平台化更详细的内容，可参考团队之前已经发布的文章 美团外卖Android平台化架构演进实践。

为了满足实际开发需要，在长时间的探索后，逐步形成了如图所示的三层工程结构。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



原有的单个工程拆分成多个工程，就不可避免的涉及到多工程之间的耦合问题，主要包括通信问题、复用问题、依赖注入、编译问题，下面详细介绍。

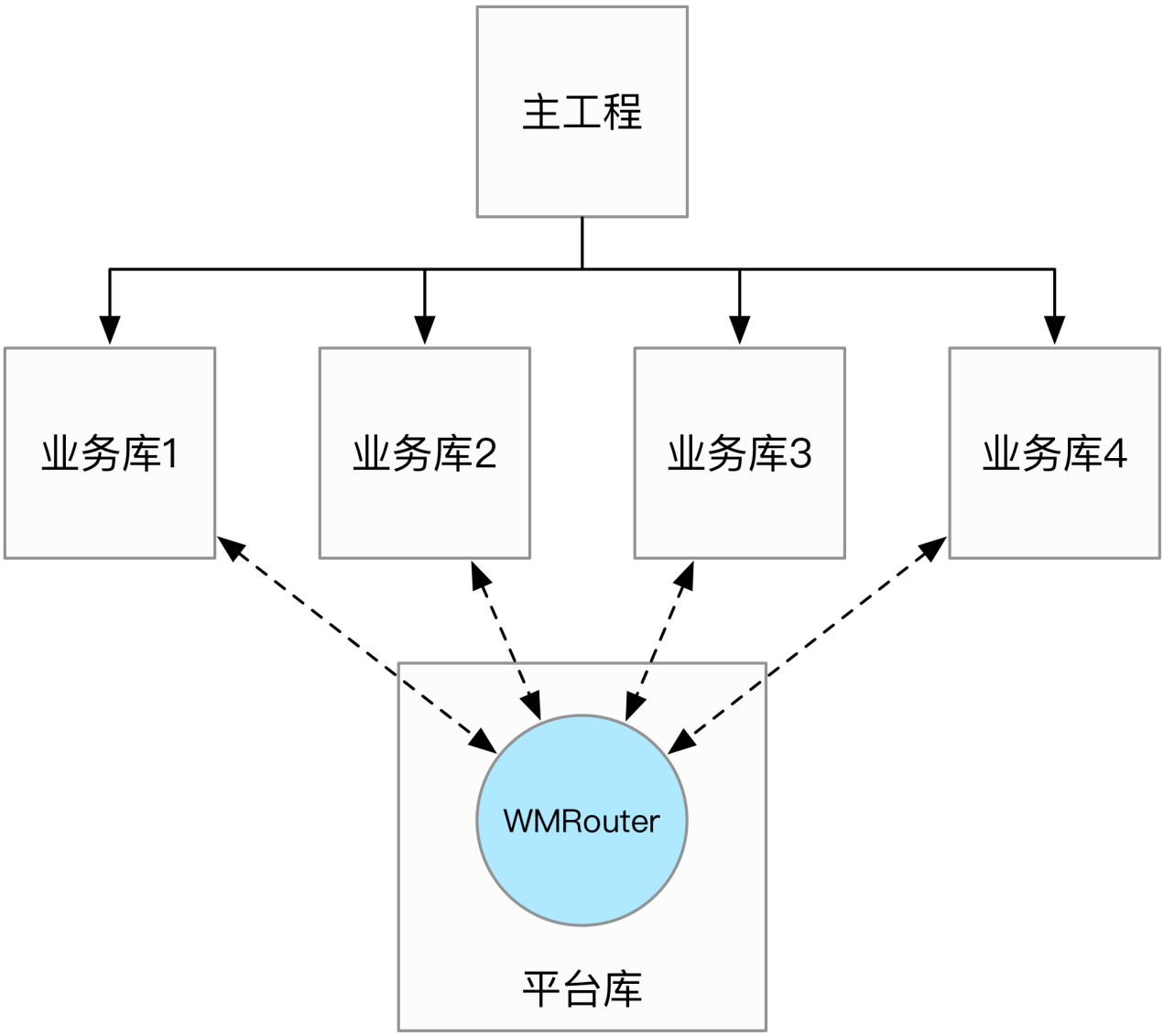
3.9.7 通信问题

当原先的一个工程拆分到各个业务库后，业务库之间的页面需要进行通信，最主要的场景就是页面跳转并通过Intent传递参数。

原先的页面跳转使用显式跳转，Activity之间存在强引用，当Activity被拆分到不同的业务库，业务库不能直接互相依赖，因此需要进行解耦。

利用WMRouter的URI分发机制，刚好可以很容易的解决这个问题。将将所有业务库的Activity注册到WMRouter，各个业务库之间就可以进行页面跳转了。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



此时WMRouter已经承载了两项功能：

1. 后台下发的运营URI跳转（`waimai://*`）
2. 内部页面跳转，用于代替原有的显式跳转，实现工程解耦（`wm_router://page/*`）

由于后台下发的URI是和产品、运营、H5、iOS等各端统一制定的协议，支持的页面、格式、参数等都不能随意改动，而内部页面跳转使用的URI，则需要根据实际开发需要进行配置，两套URI协议不能兼容，因此使用了不同的scheme。

3.9.8 复用问题与ServiceLoader模块

业务库之间经常需要复用代码。一些通用代码逻辑可以下沉到平台层从而复用，例如业务无关的通用View组件；而有些代码不适合下沉到平台层，例如业务库A要使用业务库B中的某个界面模块，而这个界面模块和业务库B的耦合很紧密。具体到外卖实际业务场景中，商家页在商家休息时会展示推荐商家列表，其样式和首页相同（如图），而两个页面不在一个工程中，商家页希望能直接从首页业务库中获取商家列表的实现。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

仅限紧急呼叫 下午3:45

披萨

综合排序 销量 距离 筛选

满减优惠 美团专送 品牌商家 点评高分

品牌 每日优鲜（阜通店）

4.6 月售2341 30分钟 | 1.9km

起送 ¥29 | 配送 ¥2.5 美团专送

首单减17 59减30 99减60 199减100 1.53折起

品牌 满记甜品（北京望京嘉茂店）

4.6 月售605 33分钟 | 3.1km

起送 ¥25 | 配送 ¥8 美团专送

甜品

首单减17 39减6 69减10 5折起 票 支持自取

品牌 CoCo都可（望京家乐福店）

4.8 月售3033 30分钟 | 2.3km

起送 ¥20 | 配送 ¥7 美团专送

奶茶果汁

首单减18 票

品牌 北京麦当劳广顺北餐厅

4.7 月售4110 30分钟 | 2.4km

起送 ¥0 | 配送 ¥9 | 人均 ¥33

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

本店打烊啦

相似好店

杭州小笼包

★★★★★ 月售1019　　30分钟 | 1.8km

起送 ¥20 | 配送 ¥5 | 人均 ¥16　　美团专送

首 新用户立减19元,首次使用美团支付最高再减3元

减 满25减7;满35减10;满50减15

粥饼屋

★★★★½ 月售2751　　36分钟 | 2.5km

起送 ¥15 | 配送 ¥3 | 人均 ¥18

首 新用户立减17元,首次使用美团支付最高再减3元

减 满25减10;满50减15

品牌 火齐 HUOJI

火齐潮汕砂锅粥（望京店）

★★★★½ 月售1710　　38分钟 | 3.1km

¥3

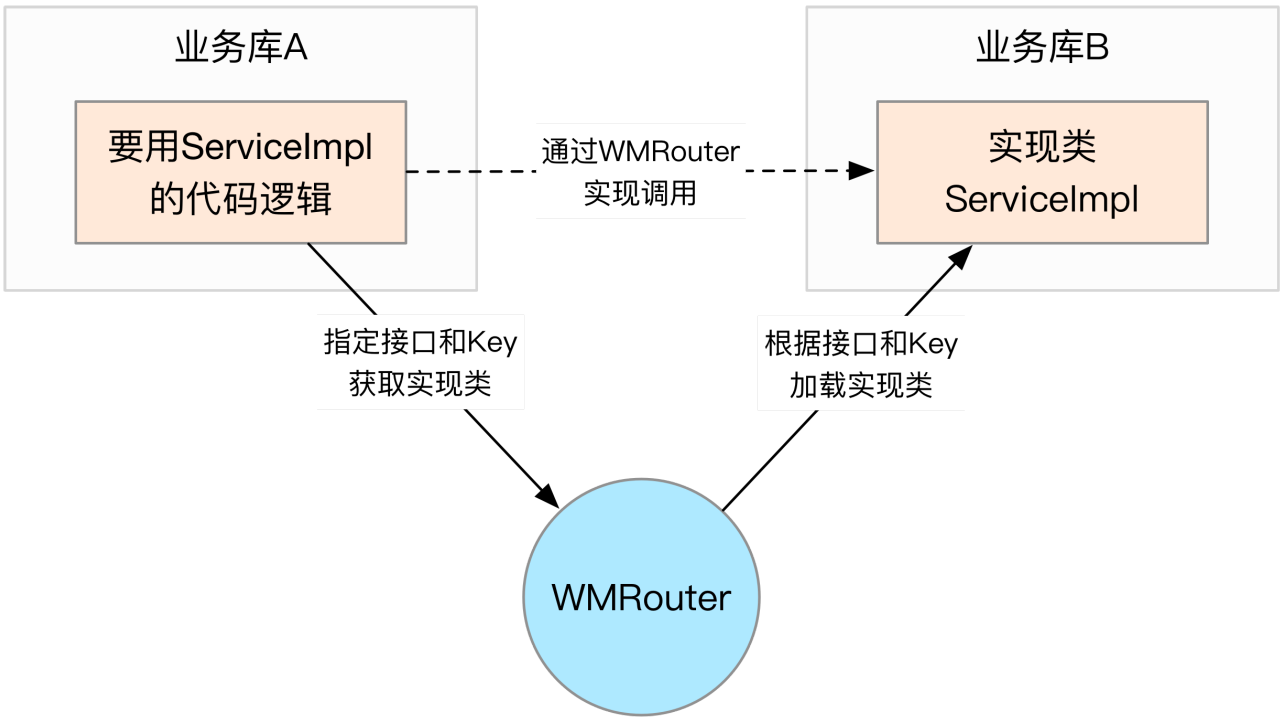
腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

为了解决上述问题，我们调研了解到Java中SPI (Service Provider Interfaces) 的设计思想与`java.util.ServiceLoader`工具类，还学习到美团平台为了解决类似问题而开发的ServiceLoader组件。

考虑到实际需求差异，我们重新开发了自己的ServiceLoader实现。相比Java中的实现，WMRouter的实现借鉴了美团平台的设计思路，不仅支持通过接口获取所有实现类，还支持通过接口和唯一的Key获取特定的实现类。另外WMRouter的实现还支持直接加载实现类的Class、用Factory和Provider创建对象、单例管理、方法调用等功能。在Gradle插件的实现思路上，借鉴了美团平台的ServiceLoader并做了性能优化，给平台提出了改进建议。

业务库之间代码复用的需求示意如图，业务库A需要复用业务库B中的ServiceImpl但又不能直接引用，因此通过WMRouter加载：

1. 抽取接口（或父类，后面不再重复说明）下沉到平台层，实现类ServiceImpl实现该接口，并声明一个Key（字符串类型）。
2. 调用方通过接口和Key，由ServiceLoader加载实现类，通过接口访问实现类。



URI跳转和ServiceLoader看起来似乎没有关联，但通信和复用需求的本质都可以理解成路由，页面通过URI分发跳转时的协议是Activity+URI，在这里ServiceLoader的协议是Interface+Key。

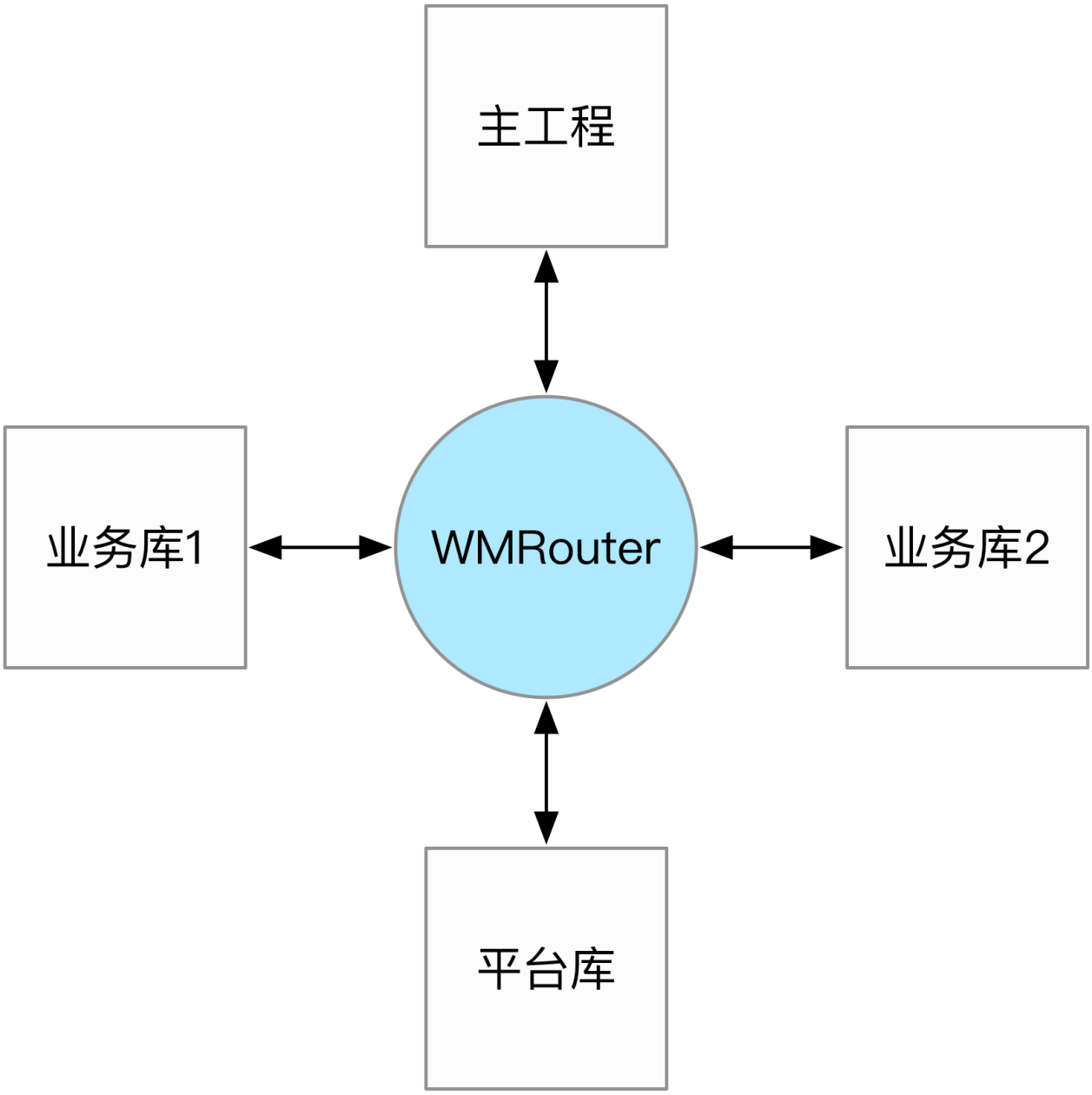
3.9.9 依赖注入

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

为了兼容外卖独立App和美团App外卖频道的两端差异，平台层的一些接口要在两个主工程分别实现，并注入到底层。常规Java代码注入的方式写起来很繁琐，而使用WMRouter的ServiceLoader功能可以更简单的实现和依赖注入类似的效果。

对于WMRouter来说，所有依赖它的工程（包括主工程、业务库、平台库）都是一样的，任何一个库想要调用其他库中的代码，都可以通过WMRouter路由转发。前面的通信和复用问题，是同级的业务库之间通过WMRouter调用，而依赖注入则是底层库通过WMRouter调用上层库，其本质和实现都是相同的。

ServiceLoader模块在加载实现类时提供了单例管理功能，可用于管理一些全局的Service/Manager，例如用户账户管理类`UserManager`。



3.9.10 编译问题

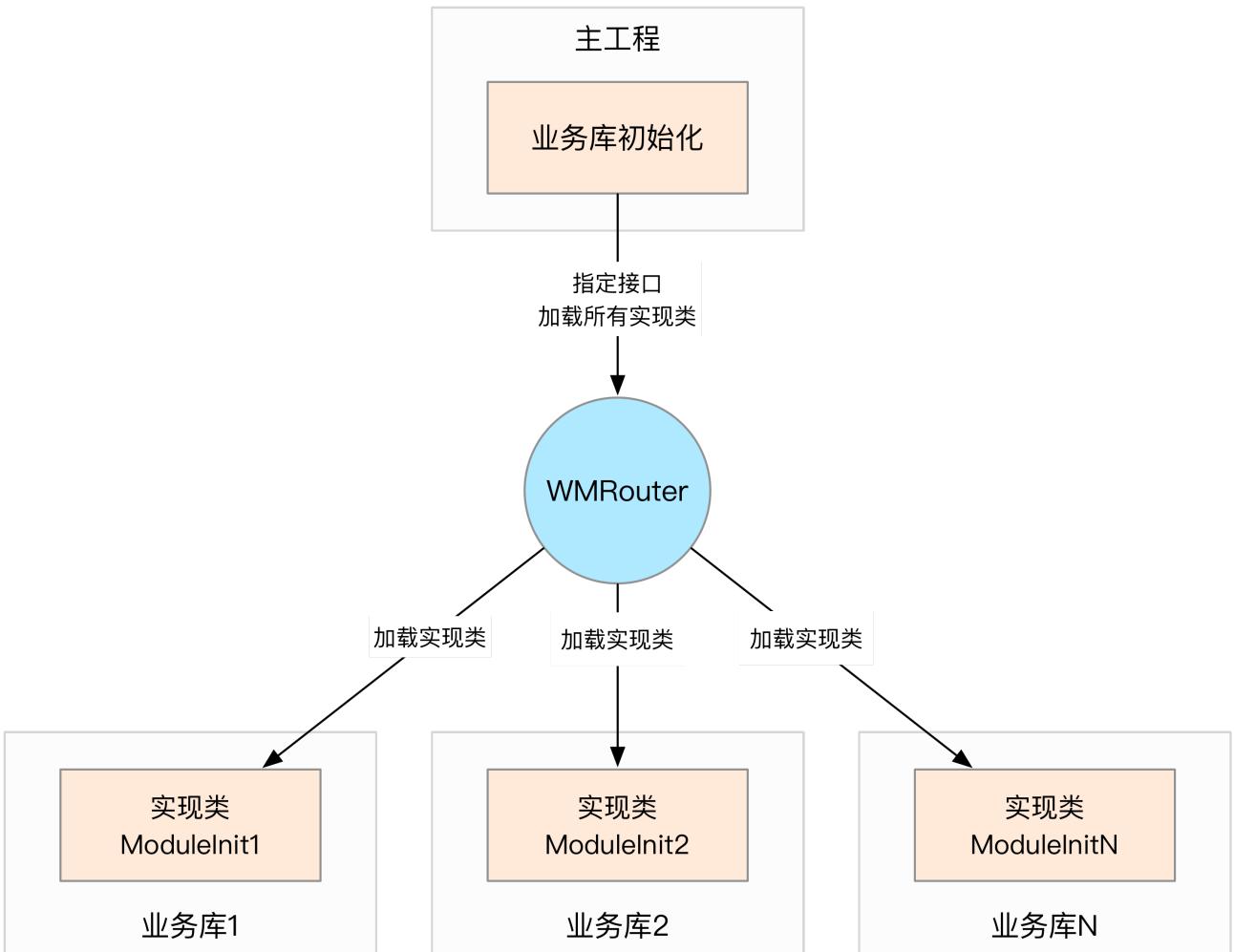
腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

由于历史原因，主工程作为一个没有业务逻辑的壳工程，对业务库却有较多依赖，特别是对业务库的初始化配置繁琐，和各业务库耦合紧密。另一方面，WMRouter跳转的页面、加载的实现类，需要在Application初始化时注册到WMRouter中，也会增加主工程和业务库的耦合。开发过程中各业务库频繁更新，经常出现无法编译的问题，严重影响开发。

为了解决这个问题，一方面WMRouter增加了注解支持，在Activity类、ServiceLoader实现类上配置注解，就可以在编译期间自动生成代码注册到WMRouter中。

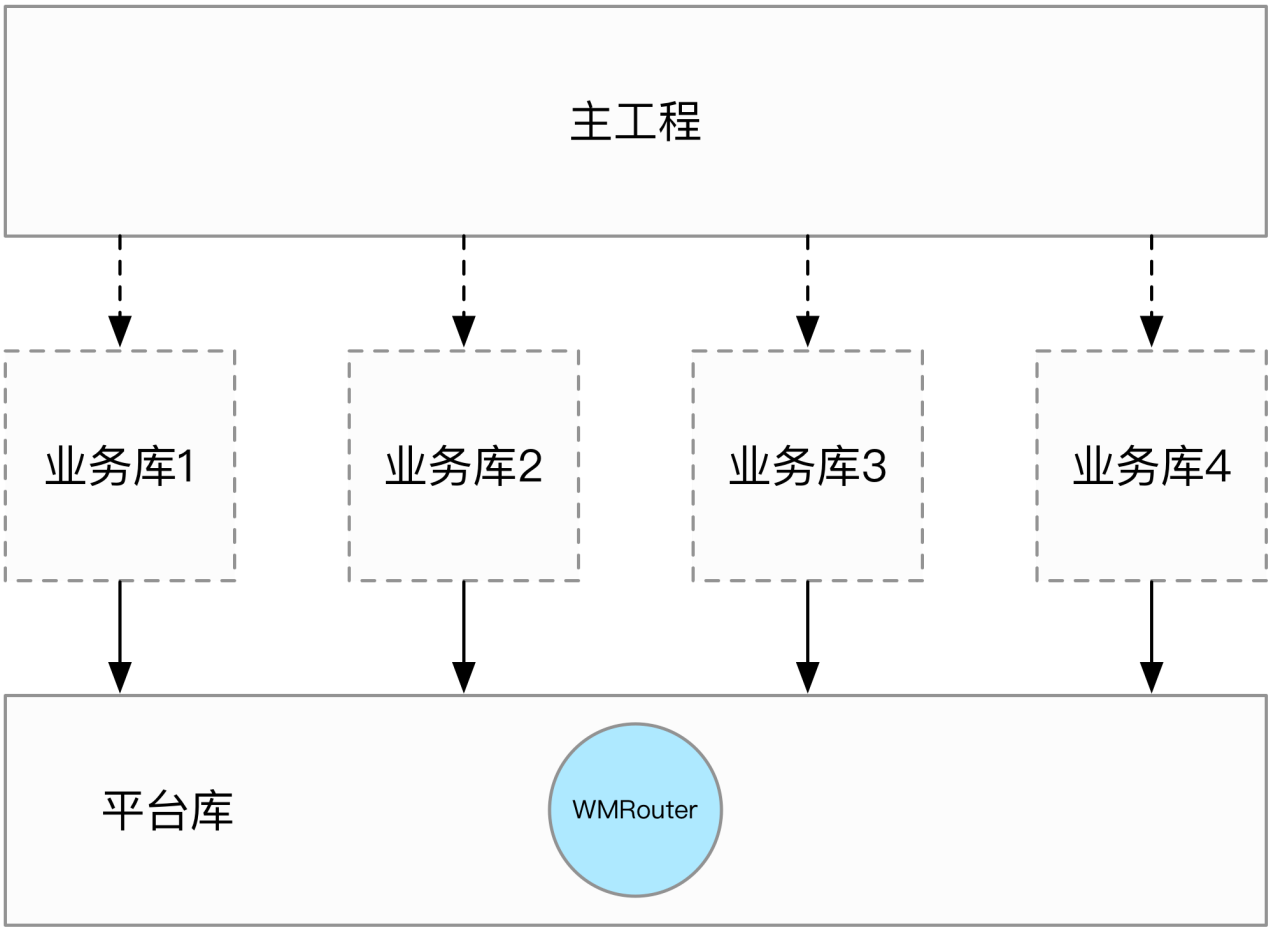
```
// 没有注解时，需要在Application初始化时代码注册各个页面，耦合严重
register("/home", HomeActivity.class);
register("/order", OrderListActivity.class);
register("/shop", ShopActivity.class)
register("/account", MyAccountActivity.class);
register("/address", MyAddressActivity.class);
// ...
// 增加注解后，只需要在各个Activity上通过注解配置即可
@RouterUri(path = "/shop")
public class ShopActivity extends BaseActivity {

}
```

另一方面，ServiceLoader还支持指定接口加载所有实现类，主工程可以通过统一接口，加载各个业务库中所有实现类并进行初始化，最终实现和业务库的彻底隔离。



腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

开发过程中，各个业务库可以像插件一样**按需集成到主工程**，能大幅减少不能编译的问题，同时由于编译时可以跳过不需要的业务库，编译速度也能得到提高。



3.9.11 WMRouter的推广

在WMRouter解决了外卖C端App的各种问题后，发现公司内甚至公司外的其他App也遇到了相似的问题和需求，于是决定对WMRouter进行推广和开源。

由于WMRouter是一个开放式组件化框架，UriRequest可以存放任意数据，UriHandler、UriInterceptor可以完全自定义，不同的UriHandler可以任意组合，具有很大的灵活性。但过于灵活容易导致易用性的下降，即使对于最常规最简单的应用，也需要复杂的配置才能完成功能。

为了在灵活性与易用性之间平衡，一方面，WMRouter对包结构进行了合理的划分，核心接口和实现类提供基础通用能力，尽可能保留最大的灵活性；另一方面，封装了一系列通用实现类，并组合成一套默认实现，从而满足绝大多数常规使用场景，尽可能降低其他App的接入成本，方便推广。

3.9.12 总结

目前业界已有的一些Android路由框架，不能满足外卖C端App在开发过程中的实际需要，因此我们开发了WMRouter路由框架。借鉴网络请求的思想，设计了基于UriRequest、UriHandler、UriInterceptor的URI分发机制，在保证功能灵活强大的同时，又尽可能的降低了使用难度；另一方面，借鉴SPI的设计思想、Java和美团平台的ServiceLoader实现，开发了自己的ServiceLoader模块，解决外卖平台化过程中的四个问题（通信问题、复用问题、依赖注入、编译问题）。在经过了近一年的不断迭代完善后，WMRouter已经成为美团多个App中的核心基础组件之一。

参考资料

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

1. Routing - Wikipedia
2. 统一资源标志符 - 维基百科
3. RFC 3966 - The tel URI for Telephone Numbers
4. RFC 6068 - The 'mailto' URI Scheme
5. Intent 和 Intent 过滤器
6. Introduction to the Service Provider Interfaces
7. 美团外卖Android平台化架构演进实践

作者简介

- 子健，美团高级工程师，2015年加入美团，先后负责外卖客户端首页、商家容器、评价等业务模块的开发维护，以及平台化、性能优化等技术工作。
- 渊博，美团高级工程师，2016年加入美团，目前作为外卖商家端Android App主力开发，主要负责商家端和蜜蜂端业务技术需求开发。
- 云驰，美团高级工程师，2016年加入美团，目前负责外卖客户端搜索、IM等业务库，及外卖多端统一工作。

3.10 美团猫眼android模块化实战-可能是最详细的模块化实战

3.10.1 前言

首先一句话概括：我想把这几个月做的事情记录下来，并且希望尽量详细，希望读者读了这篇文章能够知道项目进行模块化，项目改业务框架可能会遇到哪些问题，具体每个步骤都做什么，而不是大致的了解。

现在很多人都在谈模块化，网上有一大堆的博客实践都在讲这个。很多谈的只是模块与模块之间的解耦，并且大部分讲的是通过router路由进行解耦，其他谈的不多，而且不乏泛泛而谈。但将一个app真正做到解耦，运行。需要解决的事情远远不止解耦。业务架构、进程间通信、资源等处理、解耦方式等都需要解决。恰好对于猫眼模块化整个过程的实施，从头到尾，分析解决各种问题，我陆陆续续的做了几个月。猫眼app的历史版本是一个耦合度很高的一个工程。从这样的一个历史版本到最终的各个业务模块能够独立运行并且能够做进程间通信，会涉及到各个方面的解耦和一些其他东西。我今天我就以该app为例（其他的app进行解耦可能会遇到不同的问题，这点注意一下），**完整的讲下猫眼模块化的整个过程**。每一个方面**没有照搬网络的一些做法，而是分析对比，采用更好的设计方式。比如解耦使用serviceloader，而不是路由进行；比如架构使用更适合我们业务的一种带生命周期的mvp变种**。我还会说下具体的**花费时间和一些经验**，这样大家以后做模块时也心中有数。（提示一下，其实模块化过程所涉及的东西除了文章提及的还有很多。有些未提及，是因为之前已经完成，比如网络库的缓存由数据库->文本，这点读者注意一下。如果还有遗漏的地方，可以交流~）。

主要内容：serviceloader解耦，mvp变种框架，模块通信，lib独立运行，多端复用。

3.10.2 为什么做模块化

首先要说一点：做模块化不是为了炫技。如果没有业务场景需求，不建议做。 为什么要做模块化，网上已经阐述了很多原因了。这里我简单说下猫眼为什么要做：

- 猫眼需要快速移植到其他app（美团，点评..）。
- 解耦首页，减少冷启动时间。
- 开发时减少build时间，代码责任制。
- 服务快捷替换

3.10.3 解耦到什么程度？

首先说下，模块化究竟是什么呢？这个大家肯定都耳熟能详了：能够将不同的业务分离成不同的lib module。那么做完模块化，我们的某个业务lib 具有哪些功能呢？我认为是：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

总结一句话：**无沟通成本，快速，傻瓜式的在任何app上运行。**具体就是：这个lib不耦合具体app的服务，不耦合具体app的activity。只要给我一个app（或假的app壳子），通过它的baseActivity，和他们的服务，我就可以非常快速的将这个lib在那个app上运行。停！你可能会说这个服务是什么东西，让我详细的说下吧~

3.10.3.1 可以无侵入式的配置各种服务

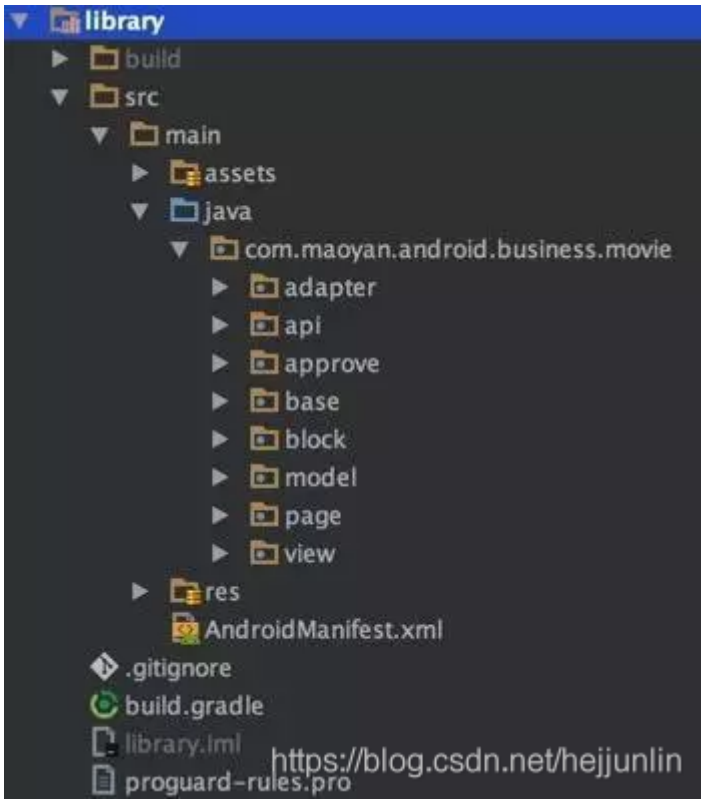
我们知道每个app都会提供账户信息，设备信息，网络服务，图片加载服务，打点服务，下拉刷新样式，错误状态等。每一个app的这些服务可能都不一样，比如美团使用的网络服务是okhttp，而点评使用的是长连接。所以我们的业务逻辑lib不能耦合这些具体的服务。只能耦合服务抽象而来的接口。在具体app使用的时候，我们再把app的服务提供给这个lib。那么这些服务怎么给呢？如果当需要服务时，我都留了一个传参的口子，这样我就需要把app的服务一个个塞到lib中需要的地方。这样成本太大了。我不希望这么麻烦，我希望的方式是直接把服务实现作为txt文本放在app的某个文件夹，你这个lib就能给我运行。这样我几乎不用管lib里面是什么东西。你只要给我一个业务lib，我添加一个txt文本，就能运行了。

3.10.3.2 lib快速便捷多端使用

说下不耦合activity。我们知道每个app有自己的baseAtivity，在里面做统计，处理异常、某些库的初始化等功能。除此之外，每个app的actionBar也不一样，每个页面在不用的app中manifest的schema也不一样。所以在lib中的业务，如果是一个业务，我们不能直接写成Activity，而应该是一个view/fragment，这样对于任何一个app，我们直接新建一个activty，然后把lib中的页面放到那个activity中即可。同样，考虑的是协同合作的成本问题，我不希望在放这个页面的时候，我需要处理很多其他的东西，比如数据加载。我希望你给我这个业务页面pager（其实是一个view），我放到activity onCreate（）的setContentView（）中即可，它就能运行。别让我做其他处理生命周期，数据绑定，销毁等的事，那都是你pager内部需要做的。

3.10.3.3 demo示例

前面这两点说的可能云里雾里的。最近我写一个猫眼问答需求，涉及5个页面，所以做成了一个lib。那么就结合我最近写的lib来图文阐述一下。



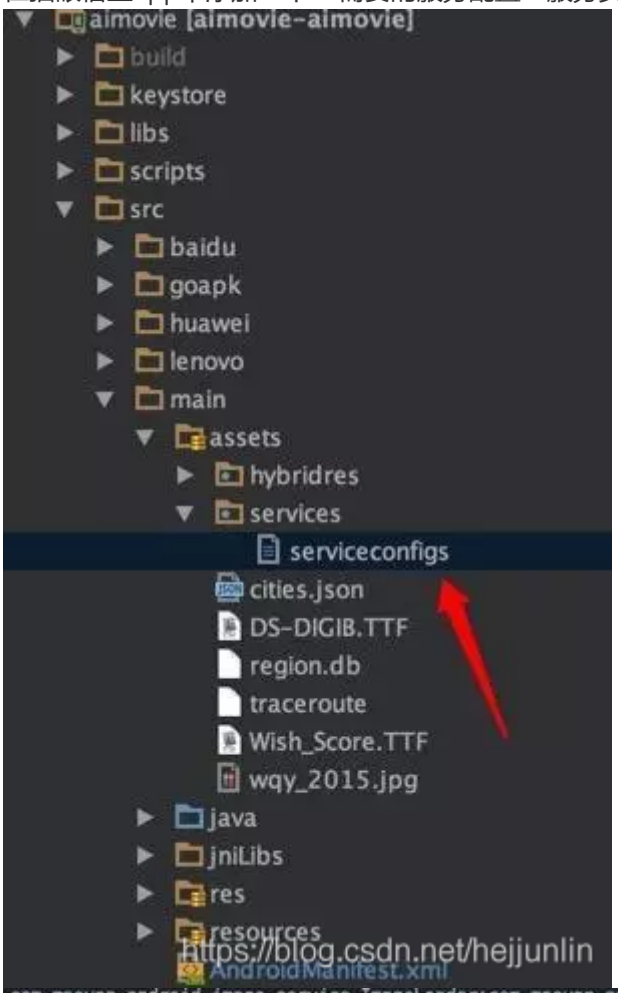
这个lib就是问答的业务lib。这不耦合具体的服务，只耦合服务接口。里面的页面（page包下）不是activity，而是view。 那么，这时候另外一个同事想把这个lib用到猫眼app上。怎么做呢？

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

- 在猫眼app的build.gradle下添加这个lib的依赖：

```
  compile 'com.maoyan.android.business:movie:1.0.2.3'
1
```

- 在猫眼宿主app中添加一个lib需要的服务配置：服务实现txt文本（因为是宿主app，之前其实已经存在）。



https://blog.csdn.net/hejjunlin

txt里面是：

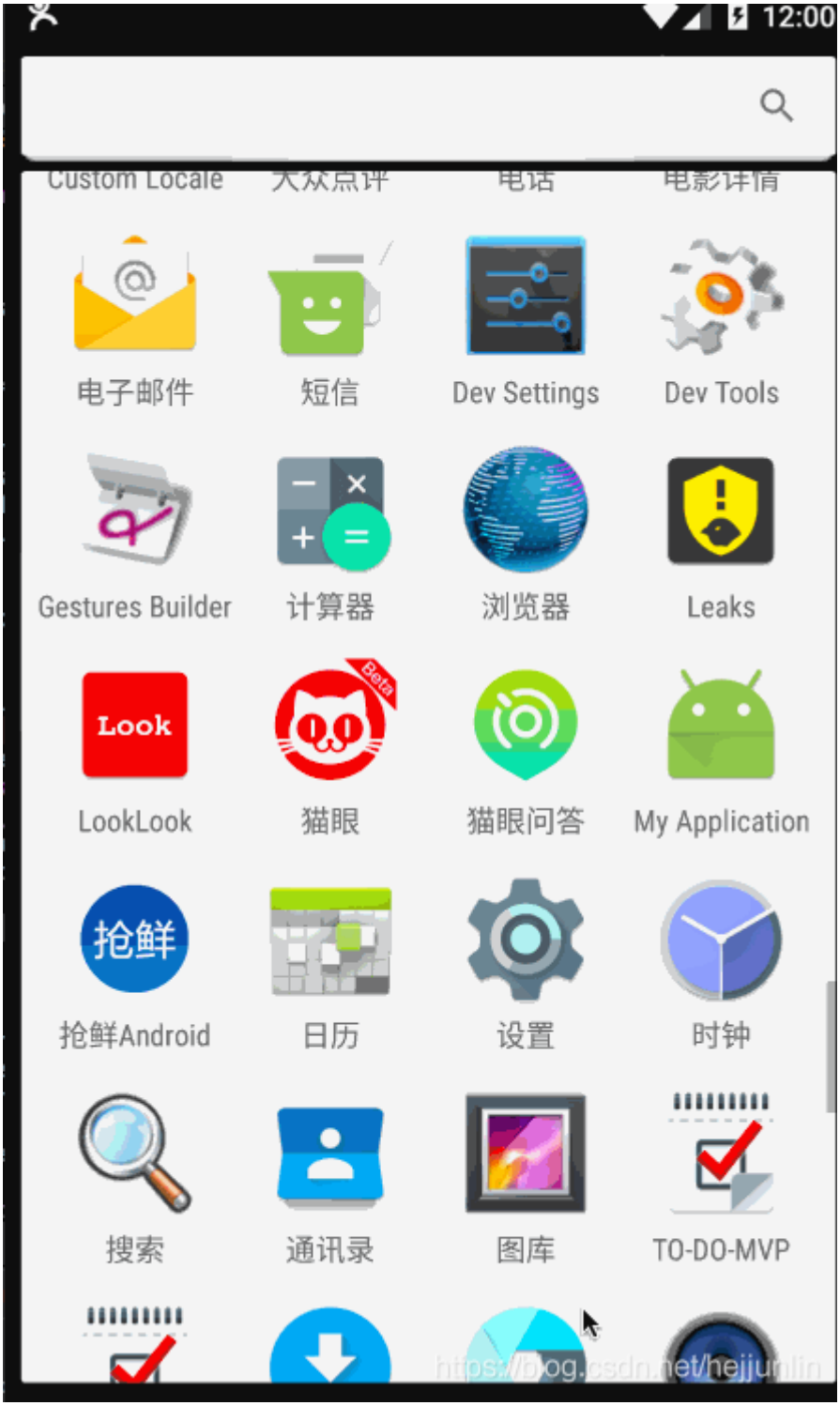
```
com.maoyan.android.image.service.ImageLoader:com.maoyan.android.imageloader.glide.GlideImageLoader
com.maoyan.android.local.service.LocalWishProvider:com.maoyan.android.local.serviceimpl.LocalWishProviderImpl
com.maoyan.android.service.login.ILoginSession:com.sankuai.common.serviceimpl.ILoginSessionImpl
com.maoyan.android.service.environment.IEnvironment:com.sankuai.common.serviceimpl.IEnvironmentImpl
com.maoyan.android.service.gson.IGson:com.sankuai.common.serviceimpl.IGsonImpl
com.maoyan.android.service.net.INetService:com.maoyan.android.serviceimpl.cachednet.CachedNetServiceProviderImpl
com.maoyan.android.serviceimpl.cachednet.ICallFactoryProvider:com.sankuai.movie.net.shark.MovieNVCallFactoryImpl
com.sankuai.movie.mine.mine.IMyCommentProvider:com.sankuai.movie.mine.mine.MyCommentManager
com.maoyan.android.service.approve.IApproveDataProvider:com.maoyan.android.impl.approve.ApproveDataProviderImpl
com.maoyan.android.service.broadcast.IBroadCastManager:com.maoyan.android.service.localbroadcast.LocalBroadCastManager
com.maoyan.android.service.viewstatus.IStatusViewProvider:com.sankuai.common.serviceimpl.IStatusViewImpl
com.maoyan.android.service.viewstatus.IPullToRefreshView:com.sankuai.common.serviceimpl.IPullToRefreshViewImpl
com.maoyan.android.business.movie.base.IMovieAskAndAnswerRouter:com.sankuai.movie.movie.qa.MovieAskAndAnswerRouterImpl
com.maoyan.android.business.movie.base.IMovieInfo:com.sankuai.common.serviceimpl.ImovieInfoImpl
```

- 在宿主app中创建activity，并放置lib中的页面，填写manifest，比如（可能有时候需要在里面写入actionBar的交互逻辑）

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

这就完成了，就运行了。所使用的各种服务，下拉刷新等都是这个app提供的。是不是快速、无需沟通写作、傻瓜。如果我们想测试这个app，那么也很简单。随便建一个app壳子，新建activity，把lib中的页面page放进去。然后添加所需要的服务实现txt文本（因为是测试，所以服务实现可以自由一些，可随意配置），就大功告成了。这种方式来修bug调ui，比启动宿主app修改代码节省很多时间。 我们看下我随意写了一个app来测试lib：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



我们可以看到下拉刷新，状态服务等和猫眼app中的都不一样，都可以定制。如果都这么写，其实所有的模块我们都可以快速，傻瓜，可定制的做成app~这种解耦程度是不是更好呢~

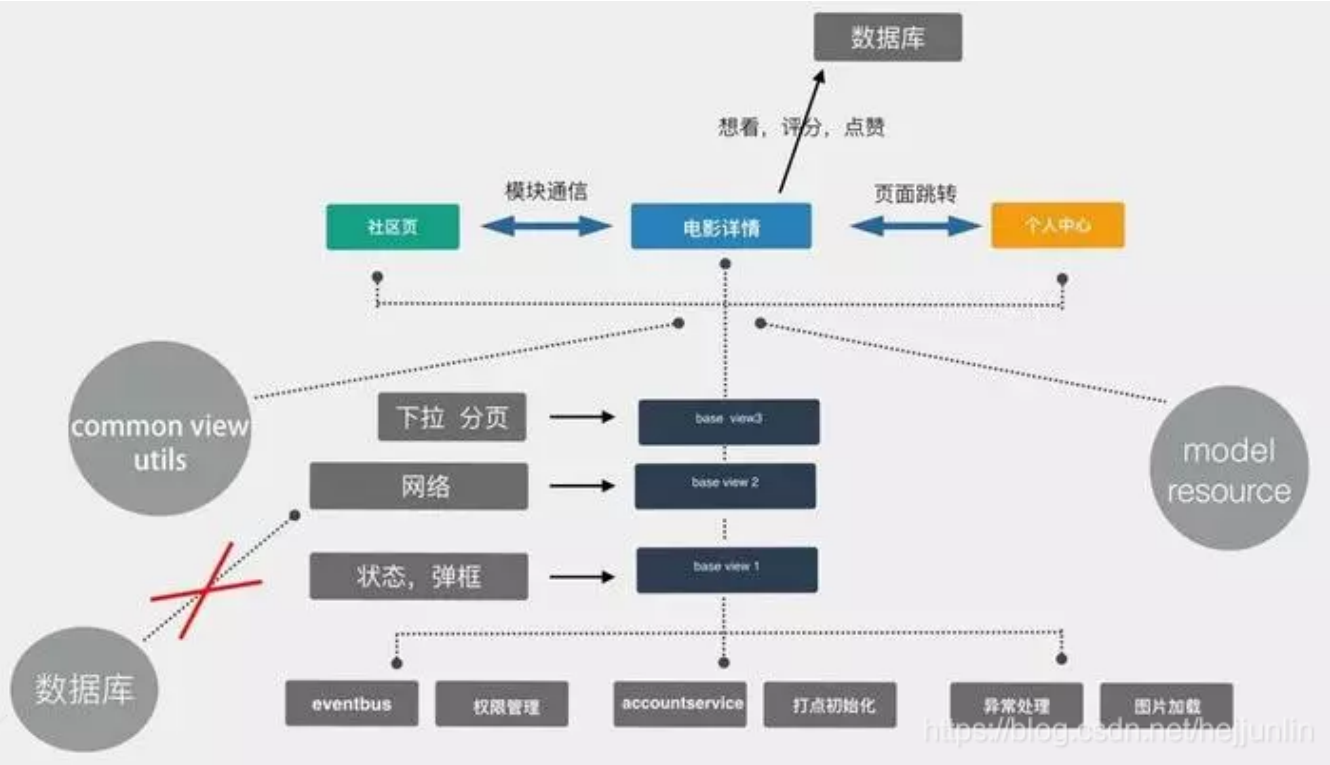
如果感觉还不错的话，那么我们开始工作吧~

3.10.4 开始模块化之旅

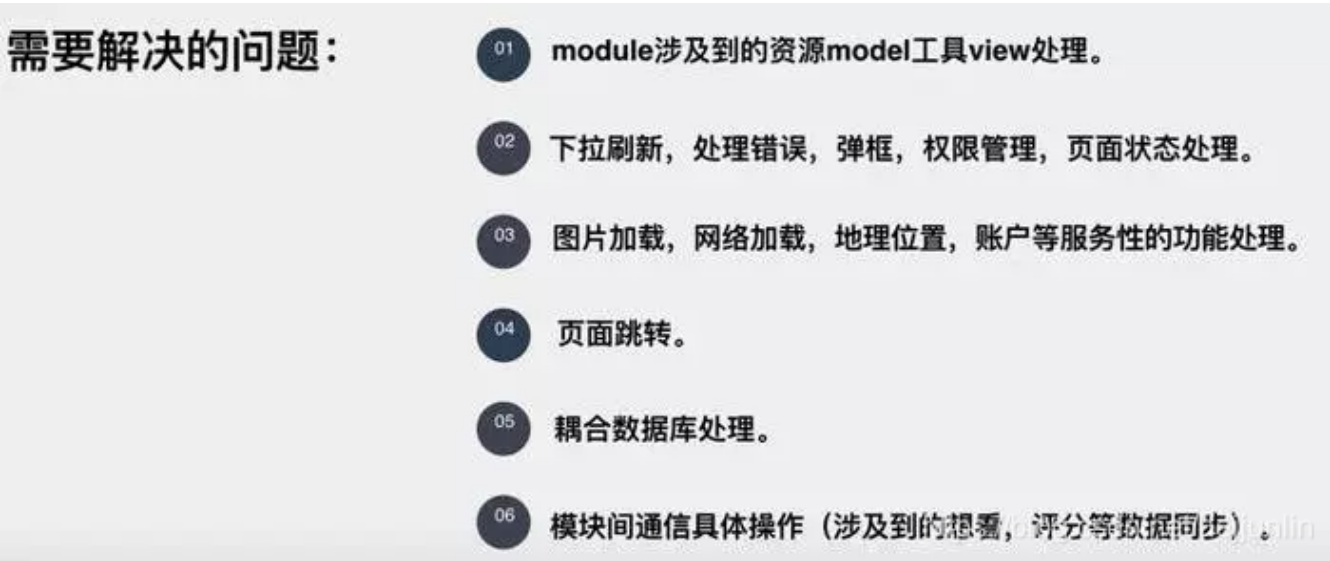
3.10.4.1 原项目耦合结构

开始模块化工作，我首先得给大家呈现下之前未模块时高度耦合的猫眼app。我们这里以电影详情页为例，看看他的耦合情况：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



电影详情页是建立在一层层的基类之上，这些基类耦合了具体的网络加载等各种服务。因为详情页有想看、评分、点赞等可编辑状态，所以还耦合了greendao数据库（以前网络加载也耦合了这个数据库，后来换成了retrofit+rxjava，所以替换到了这层耦合，谢天谢地）。该页面因为需要和其他页面互动（比如跳转、评分同步等），所以也同时耦合了其他页面的类。除此之外，还有utils，view，model等。如果想把电影详情页抽离出来，这些所有的耦合都要剥离。具体需要解决的问题，如下：

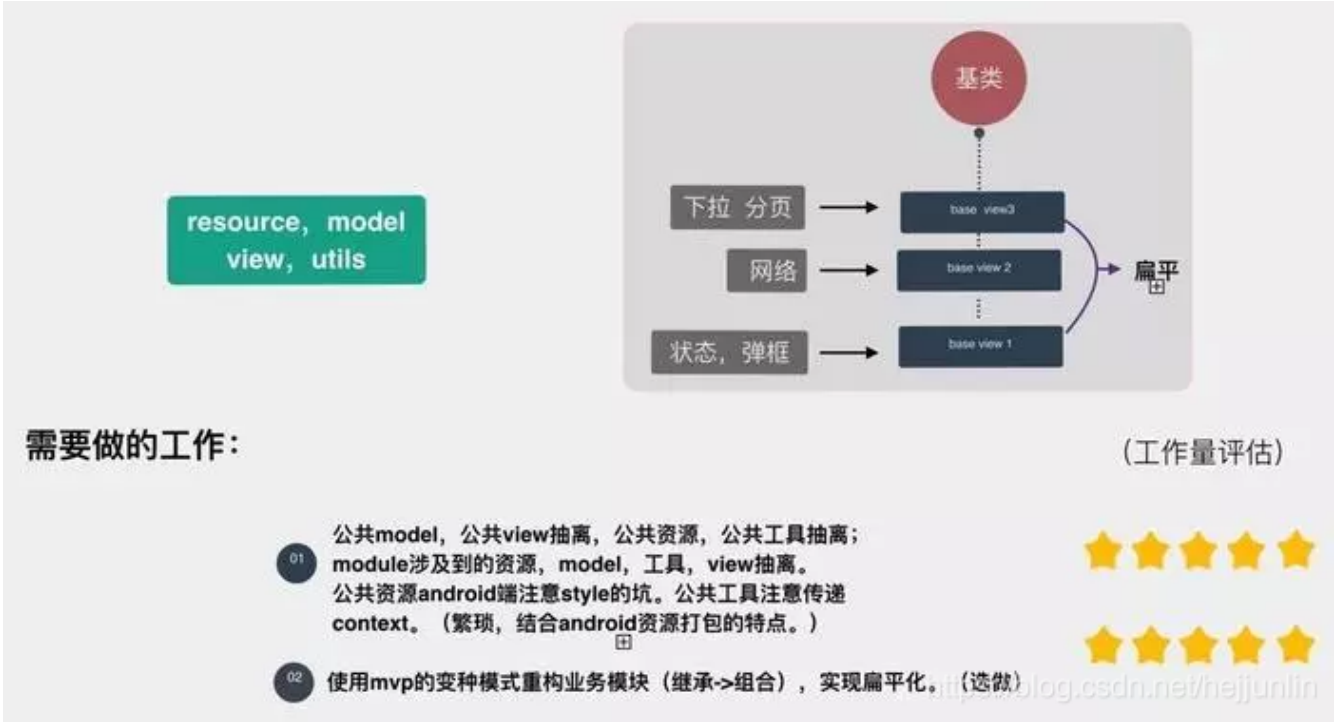


3.10.4.2 准备工作

4.2.1 工作量评估

首先我们说下解耦时需要做的准备工作。因为这些工作是解耦拆分的基础。有两点需要做，如下所示：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



首先说明一下，并不是我喜欢打五颗星。确实是这部分工作量比较大~~~

4.2.2 公共资源，model，utils等的拆分

4.2.2.1 耦合情况示例

第一点是公共资源，model，utils等的拆分。这些事情虽然不用考虑太多事情，但是很繁琐。在做模块化的时候，这个地方耗费了不少时间。很大一部分原因是，之前的猫眼历史版本代码不够规范，对代码耦合这些事情不够敏感。举几个例子吧：

- 我们之前的utils基本都写在一个类MovieUtils里面了。这个类就像大染缸。什么都向里面放。在传入的参数方面也不够规范，甚至MaoyanBaseFragment这种业务代码都作为参数传入。导致这个东西及其难拆。
- utils的方法不传context。前人写的时候图省事，在项目中统一加了一个静态的context，导致几乎所有的utils都没有传入context，这样的后果是这些工具方法直接以来宿主app。
- 之前写的common view 不够独立。既然想写common view，那么就尽量让这个view能够独立，不要耦合其他第三方库，尽量使用android 官方库。

4.2.2.2 资源拆分经验

对于资源的拆分，其实是非常繁琐的。尤其是如果string， color，dimens这些资源分布在了代码的各个角落，一个个去拆，非常繁琐。其实大可不必这么做。因为android在build时，会进行资源的merge和shrink。res/values下的各个文件（styles.xml需注意）最后都只会把用到的放到intermediate/res/merged/../valus.xml，无用的都会自动删除。并且最后我们可以使用lint来自动删除。所以这个地方不要耗费太多的时间。刚才说了，styles.xml需注意。那么需要注意什么呢？这个东西是这么写的：

```
<declare-styleable name="BezelImageView">
    <attr name="maoyan_common_view_maskDrawable" format="reference" />
    <attr name="maoyan_common_view_borderColor" format="color|reference" />
    <attr name="maoyan_common_view_border_thickness" format="dimension|reference" />
    <attr name="maoyan_common_view_placeholder" format="reference" />
    <attr name="maoyan_common_view_errorholder" format="reference" />
    <attr name="maoyan_common_view_radius" format="dimension" />
</declare-styleable>
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

我们在写属性名字的时候，一定要加上前缀限定词。如果不加的话，你这个lib打包成aar后给其他app使用的时候，会出现属性名名字重复的冲突，为什么呢？因为BezelImageView这个名字根本不会出现在intermediate/res/merged/../valus.xml里， 所以不要以为这是属性的限定词！

4.2.3 集成式vs组合式（选做）

前面说了资源utils等的拆分，那么接下来说下第二点，基类的处理。我们看到电影详情页是建立在一堆的基类之上。每一层的基类都做了一些事情。（当时这么写是为了页面的快速开发）如果我们想将电影详情页独立出来，就需要把这些基类打包成一个aar，下沉到基础库，供所有页面使用。但是我们以前的这种基类耦合了很多猫眼的东西，像下拉刷新，页面状态什么的都是写死的，并且如果我需要写个页面，我就需要继承那么一大堆的fragment。当然这种改一改也可以移植。但对以后的代码迭代肯定是不好的（修改，添加业务）。因为它灵活性差。比如如果点评app上需要猫眼某个页面的一部分而不是整个页面，原来那种改起来就不是很方便。我希望的方式是这些页面都是view，而不是fragment。并且也不是这种继承方式，而是组合方式。即如果我想要一个带下拉刷新的列表view，那么我直接build出来这么一个view，需要什么配置就set进来，它就能够使用。这个view你可以放到任何一个view，fragment中和其他view进行组合。即：

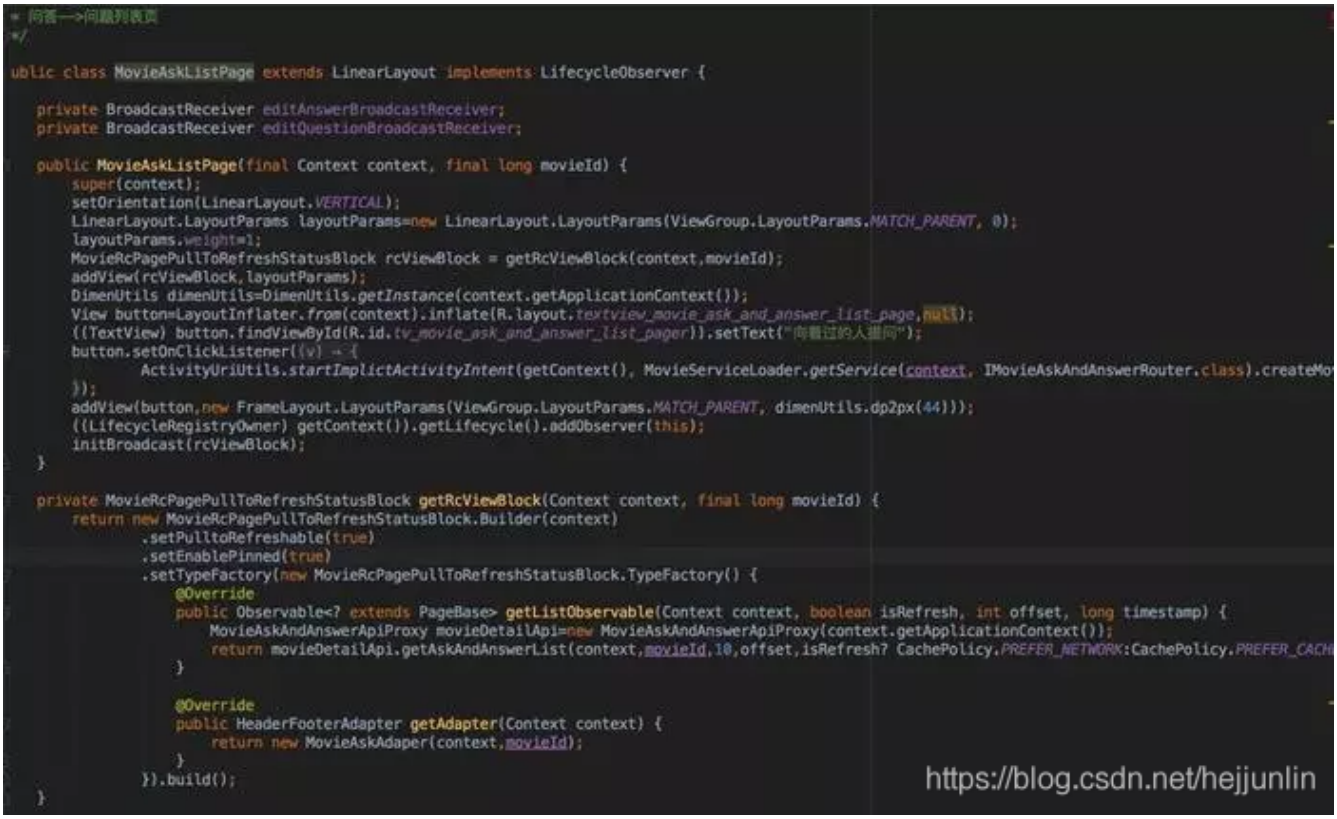
```
return new MovieRcPagePullToRefreshStatusBlock.Builder(context)
        .setPulltoRefreshable(true)
        .setEnablePinned(true)
        .setTypeFactory(new MovieRcPagePullToRefreshStatusBlock.TypeFactory() {
            @Override
            public Observable<? extends PageBase> getListObservable(Context context, boolean isRefresh, int offset, long timestamp) {
                MovieAskAndAnswerApiProxy movieDetailApi=new MovieAskAndAnswerApiProxy(context.getApplicationContext());
                return movieDetailApi.getAskAndAnswerList(context,movieId,10,offset,isRefresh? CachePolicy.PREFER_NETWORK:CachePolicy.PRE
            }

            @Override
            public HeaderFooterAdapter getAdapter(Context context) {
                return new MovieAskAdaper(context,movieId);
            }
        }).build();
```

这个MovieRcPagePullToRefreshStatusBlock是一个view，可以用在任何页面进行view的组合。

4.2.3.1 组件的插拔式，组合式设计

其实我的做法更大胆，或者更“懒”一些。我希望我这个MovieRcPagePullToRefreshStatusBlock build成功以后，放到页面中就能显示运行，自动加载数据了。就像小时候玩积木那样，组件与组件都是插拔即用式的。至于这个block是怎么加载数据的，使用者无需关系。使用者只需要拿到这个block，然后build时set进去需要的东西。放到页面中就可以运行了。可以参考这个作为示例：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?tid=8674704361863085



我们可以看到这个页面，我只是build出来了两个view，然后放到这个page中，并没有关心数据加载什么的，数据加载是在这个block内部完成的。然后这个page就像前面说的那样，放到某个app的activty中就可以运行了。插拔式、傻瓜式的思想，可能我这个人比较“懒”~~

那这种架构怎么实现呢？，接下来粗略的看看这种框架大体的实现思路吧（具体的可以看下我写的这一篇android 官方mvp框架优化：lifecycle-mvp，像前端那样组合式写页面）。其实这个框架大体也是mvp框架的思想，不过同时解决了业务场景的一些问题，比如，生命周期，移植性，沟通成本，使用方便与否等。既然要说下实现思路，那么从开始说起，对自己是个总结，对读者们可能有有些许帮助。先说下mvp框架的含义：

4.2.3.2 mvp框架含义

mvp框架总体来说适用于android的场景需求。m代表model，提供数据；v即view，提供的是供presenter调用的view相关的方法；p 即presenter，提供的是页面里触发动作的逻辑方法。

4.2.3.3 官方mvp框架的缺点

mvp框架网上有很多，官方也推荐了mvp框架。和一般的区别是：用contract来承载view和presenter的接口定义。用fragment来实现view接口。不过官方使用fragment来实现view，也有它的无奈。为什么说它无奈呢？对于view层的接口，使用fragment来进行实现，主要是因为fragment有生命周期。但fragment太笨重了。试想一下，我有一个页面，里面有四五块内容。为了以后的各块内容的移动、去除、移植更方面，我希望每一块内容都做成mvp形式，块与块之间不耦合。那么官方的这个mvp框架就不适用了。因为你不可能在一个页面写5个fragment把。android的activity中不建议写那么多的fragment，fragment典型的使用场景是ViewPager。

4.2.3.4 常规变通

那么变通一下，5块内容的view层，不再用fragment实现，而只是一个个普通的view，每个view监听事件的响应还是在view中进行（调用各自的presenter方法）。而对于整个页面的初始化加载或者下拉刷新加载，这5块内容共用一个fragment，在这个fragment的onStart（ ）和下拉刷新的监听回调中加载5块内容对应的presenter的方法。然后在fragment的onCreateView（ ）中把5块内容的view填充进来。5块内容之间可能还需要通信，数据交流，这些借助presenter在fragment中进行。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

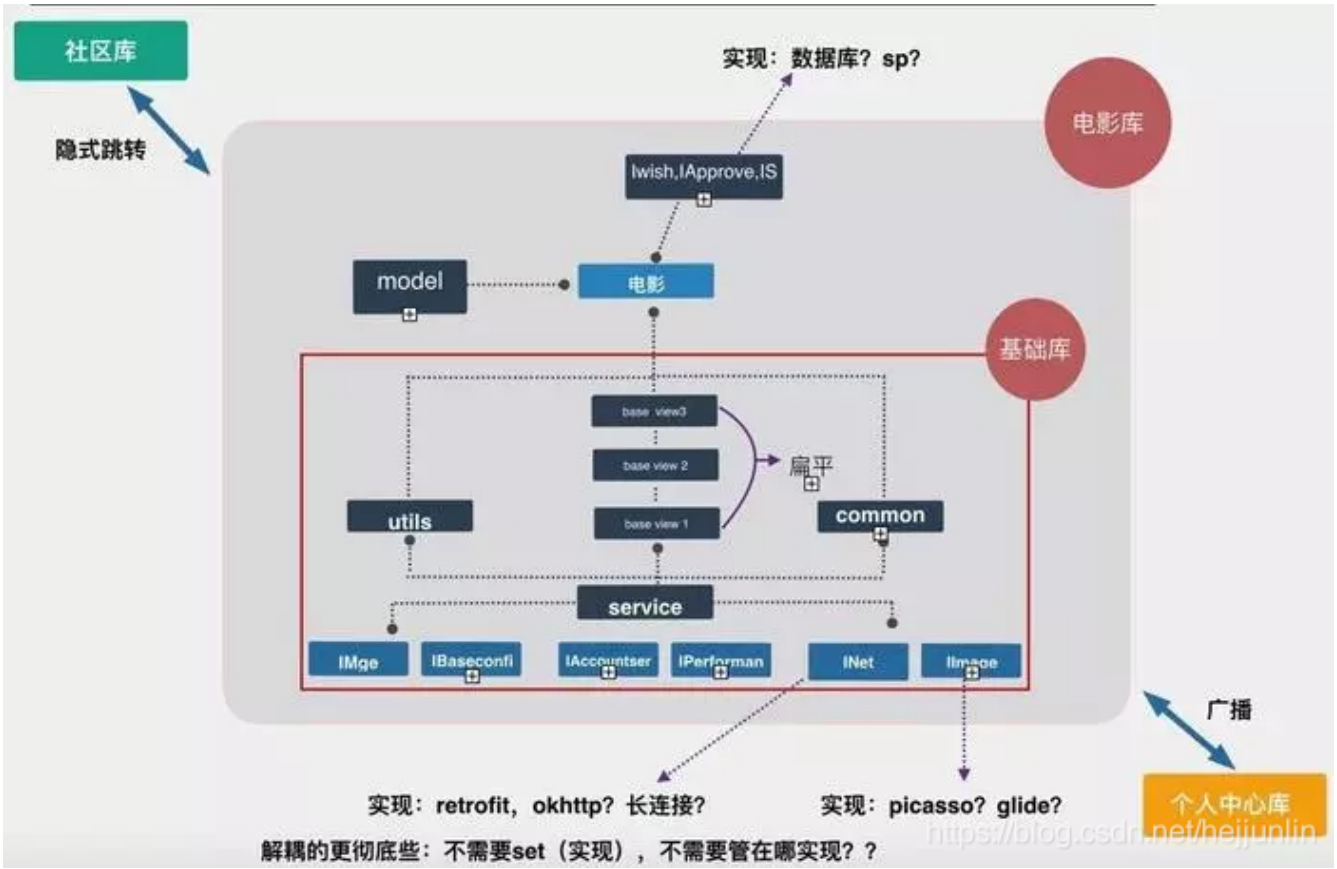
4.2.3.5 带生命周期的mvp：lifecycle-mvp

上面那么做完全没有问题，并且上面那种做法也存在于我们的项目中。但通过几个版本的迭代，我发现了一些问题：presenter太乱，太散。fragment需要持有所有的presenter，在onStart（）时load（）数据。各自的view也需要持有各自的presenter。并且view和presenter之间需要互相set（）。你还需要在activty或者fragment的onDetroy（）方法中管理presenter。总体让人觉得很乱。尤其是如果你的组件需要被别人使用，或组件用需要用到其他app时，其他人拿到你的组件，你要关心两个东西view和presenter，他得知道这两个东西里面的方法，并且他需要在activty/fragment的生命周期中关联他们并调用一些方法。嗯。这个过程肯定存在的大量沟通成本~ 所以才想到了前面讲的那种build方式来实例化组件，然后用pager组合组件。特点是（具体可以看下[android 官方mvp框架优化：lifecycle-mvp，像前端那样组合式写页面]）：

- 使用lifecycle-component这个组件提供生命周期。
- presenter被view层内部持有，不向外暴露。
- build创建view实例时，提供TypeFactory，用于业务的扩展。
- 业务代码分层。 用这种mvp的变种框架改写项目的原代/写新业务，就可以使页面更容易移植、拓展，页面内的模块也可以移动改变。当然，这种框架是建立在我们的业务基础之上，框架还是需要因项目而已，没有最好，只有更适合~

3.10.4.3 接口的抽离

前面已经阐述了模块化的准备工作，接下来我们需要做什么呢？根据前面介绍的原项目耦合结构，我们知道我们以前的项目直接依赖了各种service的具体实现。我们接下来要做的是把这些具体service实现用接口来剥离：

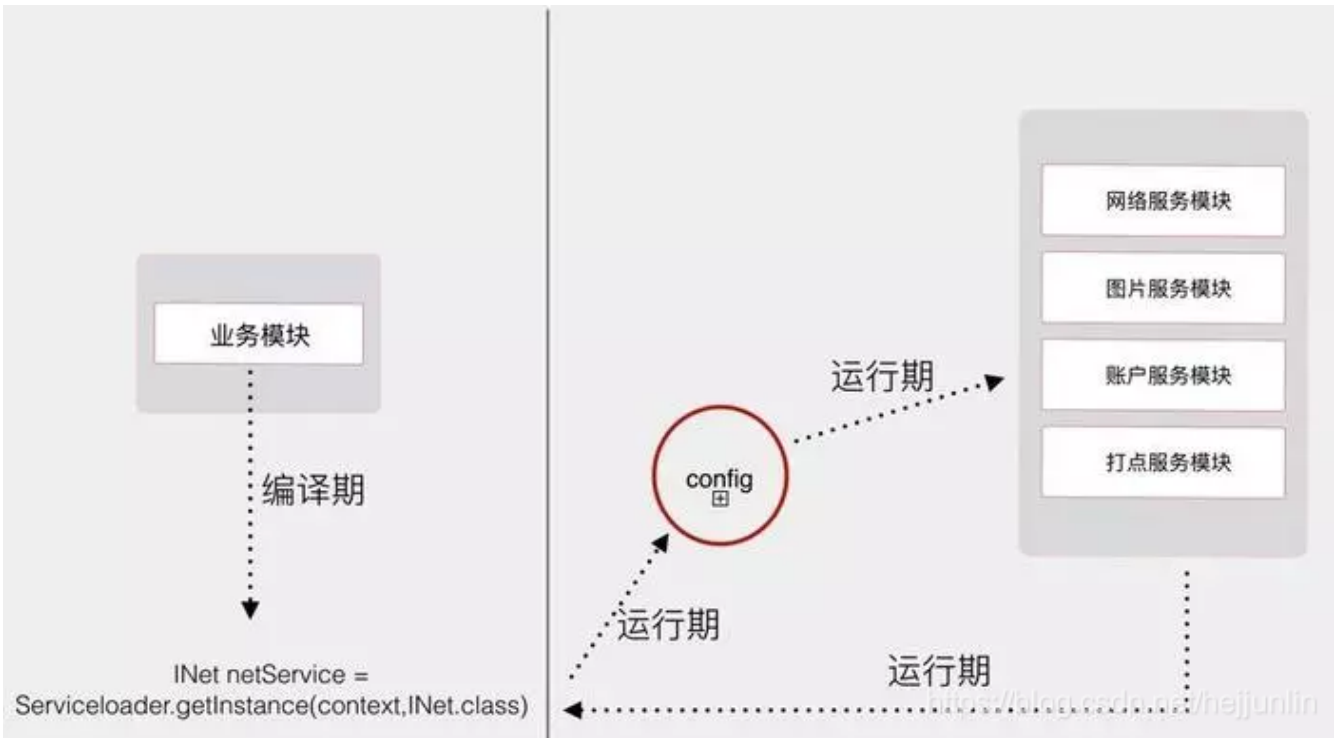


4.3.1 使用servieloader进行解耦—非显式的调用服务实现类

4.3.1.1 官方serviceloader

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?tid=8674704361863085

从图上可以看到，我们的实现类都被对应的接口所代替。但就这一步本身来说，并没有太大的难度：找到以前服务调用的地方，然后换成接口调用。无非就是有些服务用的比较多，换起来繁琐一些。但我们现在需要考虑一个问题：服务的实现，我们怎么给？首先想到的是，我们留一个参数来传入。但这种方式会导致将来使用lib的时候，沟通成本太大：你需要告诉别人哪里哪里我需要传入什么类型的参数。不然你这个lib就没法运行。我不希望别人在使用你lib的时候，还需要去内部查下你的代码是什么，应该怎么传参数。我希望 别人在使用的时候，对他们来说，lib是尽量透明的。不需要知道lib内部写的是什么，只需要在外部配置一个txt的文本就可以运行lib！那应该怎么做呢？ 其实java很早就提供了这种类似的功能：在资源目录META-INF/services中放置提供者配置文件，然后在app运行时，遇到Serviceloader.load(XxxInterface.class)时，会到META-INF/services的配置文件中寻找这个接口对应的实现类全路径名，然后使用反射去生成一个无参的实例。 我们大体的使用方式也是基于java提供的这种功能：



4.3.1.2 对官方serviceloader改造

4.3.1.2.1 官方serviceloader缺陷

从前面的阐述来看，java官方提供的serviceloader至少有三个地方需要改进。

- serviceloader没有缓存功能。因为对于服务来说，大部分我们都需要使用单例模式，而不会频繁的生成新的实例。
- serviceloader使用无参的构造方法进行构建实例。这点不用多说，肯定需要改进。谁的服务构建的时候不需要传入参数呢？
- serviceloader没有预检查等问题。因为在运行时，需要在配置文件中去寻找接口对应的实现类名。那么肯定会遇到接口名写错了，类名写错了，配置方式写错了，找不到接口实现类等，这些错误在编译器是发现不了的。同时，使用serviceloader是一种非显式的调用服务实现类方式，如果不在proguard中保护这些实现类，那么肯定会被shrink掉。除了proguard问题外，配置文件写在资源目录META-INF/services下对于一些手机（三星）也有兼容问题。最后，考虑servic配置文件手动注册的缺点，serviceloader需要提供自动注册功能。

对于上面三种情况的处理，第一点很容易解决。提供一个缓存就可以了，不多说。

4.3.1.2.2 serviceloader构造实例

第二点我们是这么解决的：我们让所有使用serviceloader加载服务的接口都实现Iprovider接口。Iprovider接口提供了一个init(Context context)方法。这样所有的服务实现类都需要实现init（Context）方法，在里面做原构造方法里需要做的初始化逻辑。因此，我们在调用serviceloader加载服务的时候就类似这样：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
          ImageLoader  imageLoader = MovieServiceLoader.getService(context,
ImageLoader.class);
1
```

在MovieServiceLoader内部，生成的实例会调用一下init（Context）方法。这样我们就解决了第二个问题。这里可能也会有一些朋友有些疑问（比如和美团平台的童鞋就此事讨论过）：为什么只传入context参数。如果一个服务实现类还需要其他参数怎么办？就我们的服务和而言，我认为只需要传入context，基本上通过context能够获得android绝大部分的参数。并且对于服务来说，既然它是一种服务，按理说不会依赖你项目一个具体的一个组件。所以我认为传入context就够了，而不是传入不定格式的object参数：

```
    MovieServiceLoader.getService(Object... params, ImageLoader.class);
1
```

这种方式固然能够解决所有问题。但是这种设计的思想已经违背了接口和实现的隔离概念。比如说，我想使用图片加载服务，按理说我只需要调用一下

```
    imageLoader = MovieServiceLoader.getService(context, ImageLoader.class);
1
```

就ok了，你这个具体的服务是Picasso还是glide别让我知道，我也不想知道。如果使用第二种方案，我难道还要知道你这个具体服务需要哪些参数，然后传入吗？这感觉太不友好了。使用Iprovider还有一个好处，那就是我们只需要在MovieServiceLoader仓库的proguard中添加：

就可以了。其他的地方在使用或新建服务接口时，都不用再考虑proguard问题了。

4.3.1.2.3 serviceloader预处理—gradle插件

第三个需要解决的问题是serviceloader的预检查等。这个解决方法就是写一个gradle插件。插件的大体流程是

- 我们在build的某个阶段拿到所有编译后的class文件（夹）和jar包。
- 使用javassit确定哪些类被@autoService修饰，配置文件中如果不存在，在其添加。
- 查看serviceConfig配置文件里面的格式是不是正确。
- 通过javassit来确定serviceConfig配置文件里面的类是不是在项目中存在，接口类是不是实现了Iprovider接口。

4.3.1.2.4 需要用到的知识：build流程，javassit，groovy

本来这里不想说太多东西，但是考虑到这三样很多读者可能不熟悉。直接去网上google这三方面的东西，单就这些东西，可能还需要学上一学。那么我还是把我的一些经验写上（为了切题，就不详细展开了），读者可以参考参考，些许能够事半功倍。 因为你需要拿到编译后的class文件和jar包，你需要知道build的大体流程，各个task的输入输出是什么，是以文件夹的形式还是jar包的形式。 比如说拿所有class的时机，可以在assembleXxx这个task时（dex task已经完成了），从dex task的输入文件夹/jar包中拿到所有的class。同理javac task的输入也可以。但javac task的输出就不可以，因为javac task输出的intermediate/class文件夹只包含项目中的class文件，不含有aar对应的intermediate/exploded-aar文件里面的class文件。当然transform也是一种实现方式。transform的输入，输出文件路径已经给好了，输入的class为所有的class。 除build流程之外，你还可能使用groovy来写插件逻辑。不过如果你实在不想用groovy，那么也可以用java，两者兼容，只是groovy的很多特性像循环等就没法用了。这里有个小经验：写goovy，ide不能很好提示错误，比如你使用了一个变量或一个方法，如果方法用错了，变量没定义。也不会给你提示找不到。所以最好还是使用先写到.java里面，然后再移动到.groovy里面吧。 最后你还需要知道javassit的一些知识，这个是处理class文件的工具。很强大，和java很像，大部分的使用都会落脚到ctClass的使用。所以这个类最

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

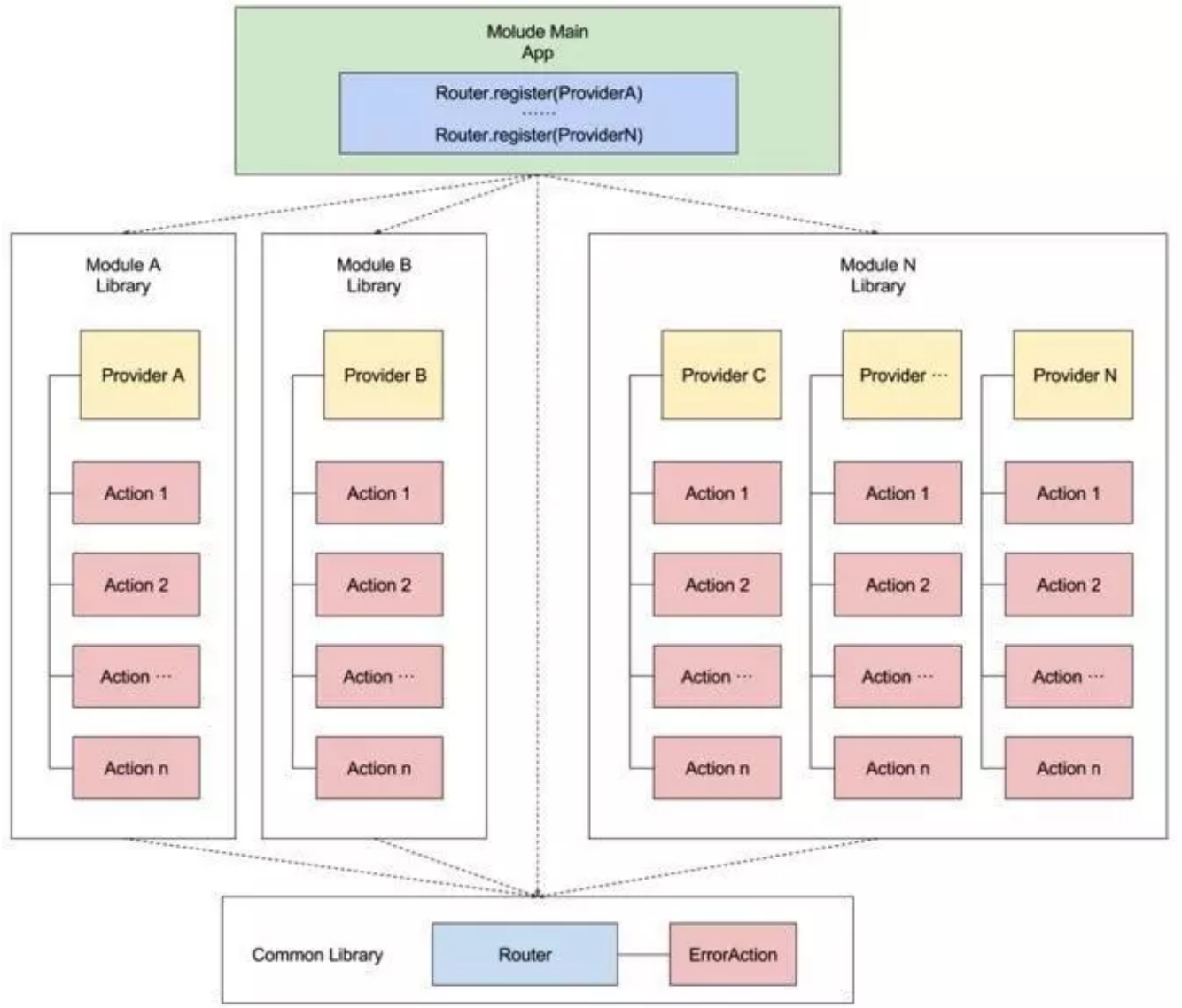
好熟悉。这里有个小经验：有时候需要ctClass->class的转化，记得使用静态变量储存这个class对象，不然会报classloader多次加载同一个路径的异常。 ok，使用serviceloader来进行解耦的原理，改进，好处已经说完了。

4.3.2 serviceloader解耦 vs 路由方式解耦

网上有关模块化的博客，大部分使用的是路由的方式解耦。路由的方式解耦是怎么一回事呢？

4.3.2.1 路由方式解耦阐述

我们看下大体的路由框架图 ,截取于 Android组件化之通信 ：



路由架构

https://blog.csdn.net/hejjunlin

那么这个路由框架是怎么工作的呢？这里的action是一个服务，provider是一个map集合，盛放一个lib里的所有(action名字：action实例）键值对。在宿主app中注册各个lib的provider。这样module A请求moduleB的服务时，通过（代码来源）：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
LocalRouter.getInstance(MaApplication.getMaApplication())
        .rxRoute(MainActivity.this, RouterRequestUtil.obtain(MainActivity.this).provider("main")
                .action("sync")
                .data("1", "Hello")
                .data("2", "World"))
        .subscribeOn(Schedulers.from(ThreadPool.getThreadPoolSingleton()))
        .observeOn(AndroidSchedulers.mainThread())
        .subscribe((Consumer) (maActionResult) → {
                Toast.makeText(MainActivity.this, maActionResult.getMsg(), Toast.LENGTH_SHORT).show();
        }, (Consumer) (throwable) → {
                Toast.makeText(MainActivity.this, "error", Toast.LENGTH_SHORT).show();
        });
```

即通过提供provider的名字，action的名字，参数名，值，到注册的map中寻找对应的action实例，然后调用其对应的方法。核心就是使用字符串来匹配对应的实例进行解耦。

4.3.2.1.1 路由方式解耦优点

这种方式的最大好处是，新建一个服务时，不需要写接口，所有的都用字符串来进行标志，进行匹配，两个model之间不需要耦合任何东西，甚至接口声明都不需要耦合。如果一个lib中有很多需要被外界调用的服务，并且调用的次数不多，或者我不仅仅对服务解耦，那么用这种路由的方式很好，因为不用写接口了。

4.3.2.1.2 路由方式在服务解耦方面不适用性的讨论

但为什么没选择这种解耦方式呢？因为这种方式，对于android整体的服务解耦来说，我还是提出了如下的顾虑（仅代表自己的观点，可能比较粗鄙，并不是说人家的项目不够优秀）：

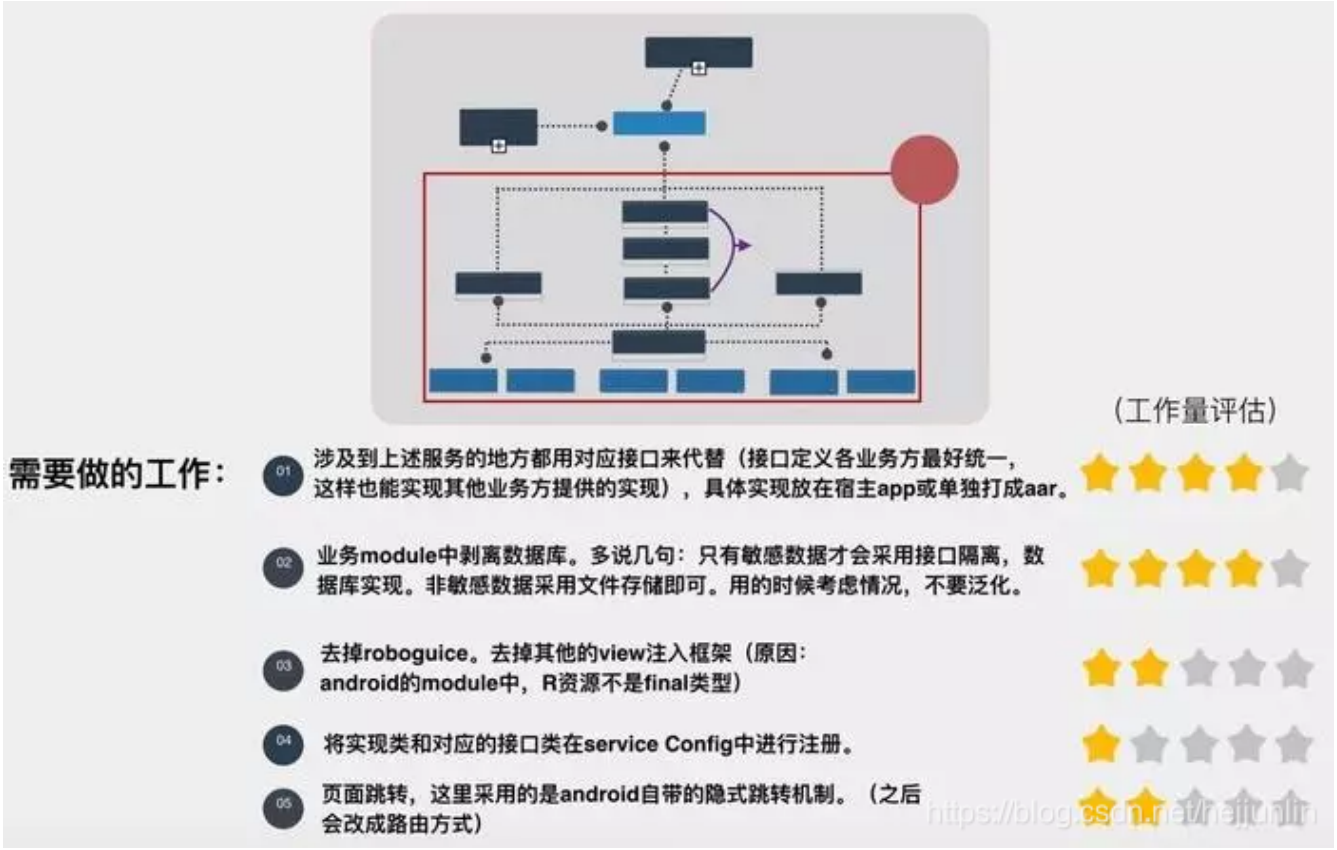
- 对于大面积的解耦，肯定大部分是app界别的服务进行解耦。特点是大量使用，这时候我写几个接口，下沉到base库，无伤大雅。这样我在使用的时候，serviceloader好处就突出来了：使用服务的时候，我不需要关心实现类的类名，包名是什么，需要传入什么参数，调用的方法的名字是什么。如果使用路由方式接口，我需要关心的事情就多了，如果我需要关心这么多东西，它就不应该叫服务了。如果另一个lib在你不知情的情况下改了名字怎么办？并且在代码移植到其他app或独立运行时，配置方式也不够友好。serviceloader只需要写个配置txt文件放在apk中即可，并且每一个lib的服务写到自己的serviceCinfig即可，不需要宿主app关心。使用路由方式，即使action可以自动注册，也需要在application处理一些注册的事情。
- 路由这种服务框架和serviceloader，本质来说，并不能进行真正意义上的模块间的通信。说的通俗点，路由框架能做的是：b lib可以在不依赖a lib项目的情况下，b可以new 出来a中一个类的实例（或提前new好），然后调用那个实例的方法。这并非通信，只是能够调用其他仓库的方法。而通信指的是监听状态，回调。serviceloader同样也做不到真正意义上的通信。模块间通信只能通过非显式的监听机制才能进行，比如eventbus，广播，contentprovider等来进行。为什么要说这一点呢？因为我看到很多模块化的博客都在说使用路由框架进行模块间通信。但就前面提到的这种路由框架，确实做不到真正意义上的模块间通信。 ok，serviceloader解耦 vs 路由方式解耦就到这里。

3.10.4.4 解耦方面的其他工作

4.4.1 工作评估

前面很大一个篇幅都在讲使用serviceloader进行服务的解耦。那么除了这个，还需要做什么？这里我先大体总结一下，再逐个阐述：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



4.4.2 服务实现的抽离

第一点的后半句需要注意一下：如果你希望所有模块都能够独立打包运行，那么需要把所有的服务实现也抽离出来。如果不想独立运行，只是想进行解耦，那么还是留在宿主app中即可。虽然说这么一句话很轻松。但是抽离一个服务实现，真正实施起来却需要花费很多的时间，因为一个服务可能耦合了很多的东西，不留神不好拆。这一点读者们心里要先有个数。 不过能抽离就尽量抽离吧，不只是lib的独立运行。对之后服务的替换也有很大的好处。比如网络加载库，以前使用的是retrofi+okhttp，后来升级成了retrofit+长连接。替换的时候只是在服务配置文件中改一句话的事情。如果打算抽离，要注意接口的定义，不要耦合具体某个库的类，考虑要全面，设计要合理。比如INet库，接口定义为：

```
        public interface INetService extends IProvider {
     <T> T create(final Class<T> service, String getdataPolicy, String        cacheTime);
      }
123
```

虽然retrofit是一个很棒的库，但接口也没有耦合这个库。说不定哪一天就替换了。

4.4.3 数据库的抽离

第二点说起来很痛苦，数据库的抽离真的是很麻烦。不知道在哪个版本开始，猫眼耦合了greendao。这个数据库本身来说挺优秀的，但是架不住它太大！如果我想把一个lib给别人用，难道我这个lib还得耦合一个大的第三方数据库？！！因为之前没有考虑过模块化，所以基本所有的网络数据，敏感数据等都进行了grrendao的保存。所以解耦的时候每每看到daossion，我都是虎躯一震。网络数据使用文件存储且对业务代码透明。敏感数据使用数据库存储，但用接口隔离，并且数据库建议使用官方的数据库sqlite或者room。

4.4.4 和butterKnife说再见

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

第三点的意思是如果你想将业务代码独立模块化，那么就得跟像butterknife框架的view注入功能说拜拜了。因为android adt14开始，library的R资源不再是final类型的了，所以在library中你不能使用R.id.xx，需要使用findViewById()来代替;也不能使用switch（R.id.xx）,需要使用if..else来代替。 第四点是第一点的后续工作。不存在多少工作量。

4.4.5 页面跳转

4.4.5.1 页面跳转需要做的事情

页面跳转也是app中需要重视的一个事情，因为它是模块化的门户，涉及到页面与页面，其他app、i版到页面之间的通信问题。虽然看起来简单，但如果设计不合理，那么模块化入口的代码优雅度，crash数量，页面降级，运营协作等方面都会受到影响。 对于页面间的跳转，我们的一般做法：

- 如果这个类页面没有隐式跳转功能：
  - 那么直接在其他页面首先 获取intent（getContext（），TargetActivity.class），然后intent添加参数。最后starActivity（getContext（），intent）。
  - 在目标activty 的onCreate（）里面getIntent（）.getString(xx_key,defaultValue)等获取参数;
  - 如果xx_key对应的value不合法或者解析错误，比如movieId=0，或者等于“”。那么应该跳转到一个其他页面或者跳转失败。
- 如果这个页面配置了隐式跳转功能：
  - 那么在其他页面你首先得创建一个createXxxActivityIntent（）的utils方法，在里面传入落地页的path，参数key，参数value。
  - 在manifest中声明。
  - 在目标activty 的onCreate（）里面getIntent（）.getData（）.parseBoolean(xx_key,defaultValue)...等获取参数
  - 如果xx_key对应的value不合法或者解析错误，比如movieId=0，或者等于“”。那么应该跳转到一个其他页面或者跳转失败。

4.4.5.2 android原生页面跳转存在的问题

下面说下这种使用原生页面跳转存在的问题~

- 在获取参数的时候，需要写一大推的intent.get(xx)，如果这个页面既含有隐式跳转，又含有显示跳转， 那么肯定上面那个过程都需要，这样在onCreate（）里面就会非常的乱。要进行if else
- 如果想进行隐式跳转，那么都需要在manifest进行注册intent-filter。一是麻烦，二是我需要在另外一个地方去配置某一个activity的东西，管理不方便。
- 需要另外写一个utils获取隐式intent。
- 没有降级策略，如果运营配错了，那么只能到错误页面，而无法进行一个补救措施，比如进入i版页面。
- 开发人员或者后台配置错误参数的时候，我们需要写兜底逻辑。每一个页面解析都需要写一段相同的逻辑。
- 如果一个页面需要登录用户才可以打开的权限，那么我们经常会写if（isLogin()）{//跳转页面}else{//跳转到登录页面} ，每次操作都要写这些个相同的逻辑。

如果觉得在这方面没那么多要求，针对页面间的跳转，为了不耦合其他的模块的类，所有页面都可以采用隐式跳转机制来进行。这基本已经可以满足情况了。但我这里还是想说下阿里推出的开源框架Arouter。其具有拦截功能，这样跳转失败可以有降级处理（比如呈现i版页面），让页面具有登录用户可打开权限；获取参数方式统一等。还是挺不错的。基本解决了上面所面临的问题。具体就不展开了，具体可以看开源最佳实践：Android平台页面路由框架ARouter

3.10.4.5 模块间/页面间通信

4.5.0 使用ViewModel来进行页面间数据共享

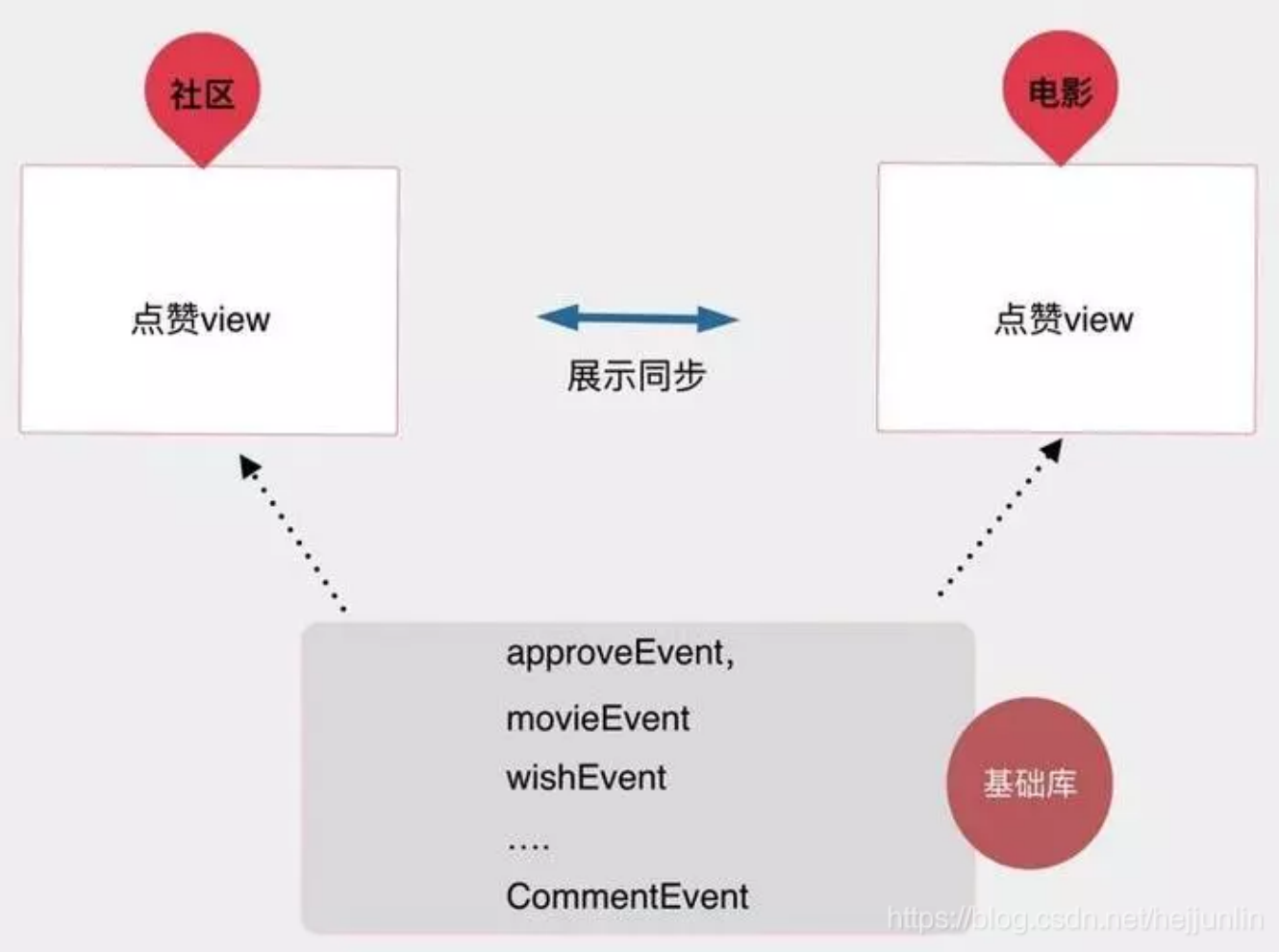
腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

这一段是新加的内容。我觉得放到这里比较合适。ViewModel是google新推出的lifecycle-component中的类，官方文档中阐述使用ViewModel可以解决页面旋转等配置改变时数据保存的问题。我思考了下，觉得它在解耦页面内数据共享的问题也能发挥作用。

举一个我以前遇到过的例子：一个页面做完了，pm找我做页面的埋点。埋点需要页面的movieId信息，但是需要埋点的那个block中并没有movieId。并且我这个block层级很深。如果想拿到movieId，我需要从activity页面层级一层层传到我这个block中，免不了中间层级的耦合和方法的创建。当时觉得这件事真是让人头大。那时候多么需要有个像事件监听形式的eventbus那样的东西，我只需要把数据放到bus里面，然后这个页面的任何一个地方都能很方便的获取。总结一下：**直白点说就是页面block/fragment之间需要使用对方的数据/view时，无需之间硬性的引用，只需要activity的context参数就可以获取对方的数据/view，从而进行数据交流、view访问。而页面的context是系统类型且是很容易获取的，并不存在耦合。** 具体使用可以参考我之前写的一篇文章使用ViewModel共享页面内的数据：ActivityDataBus

4.5.1 为什么要去掉eventbus，使用广播

如果已经到了这一步，那么大体上一个页面已经抽离出来了，剩下的是与其他模块、其他页面间的互动了。 前面说了serviceloader和路由方式都没办法做这些事情。我们首先想到的是使用eventbus来做这些事情。使用eventbus的前提是，需要定义一些Event事件。比如：



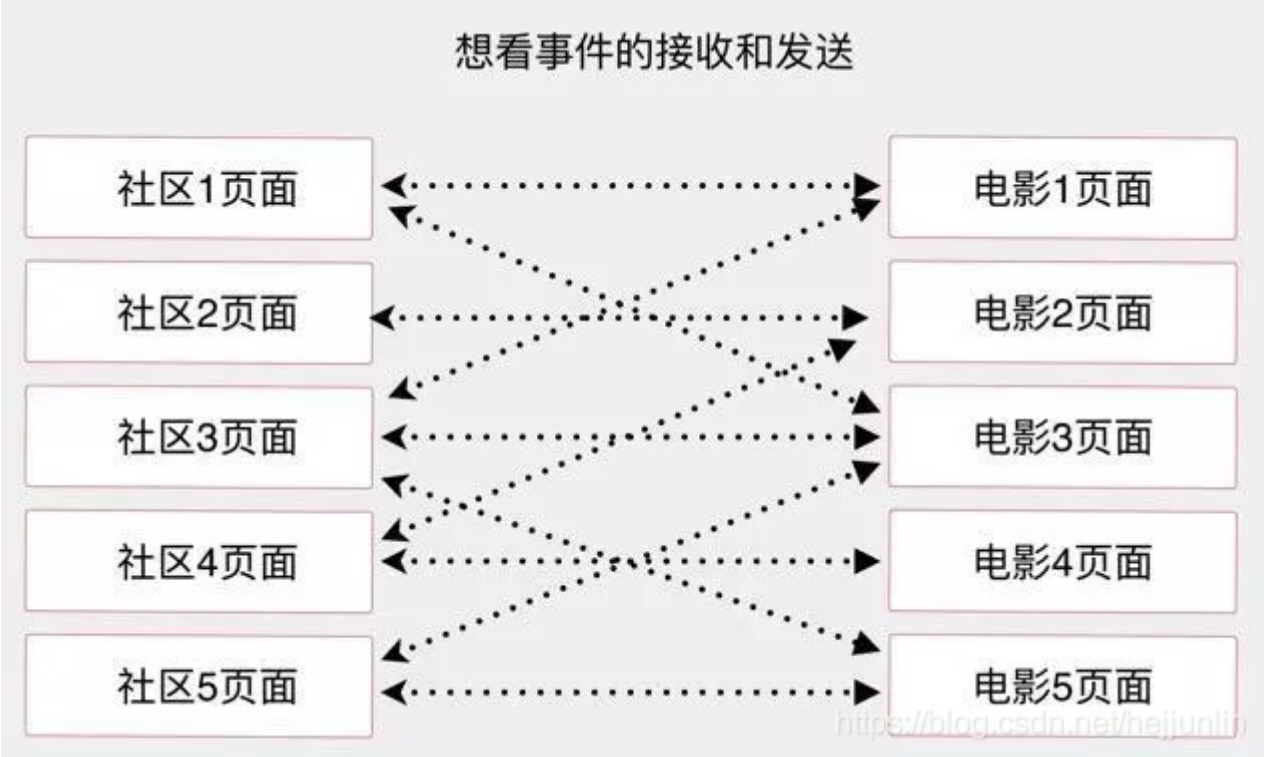
但如果你将业务代码各自模块化之后，就有一个尴尬的问题摆在面前：Event事件放在哪里？因为很多库都需要收听这个Event事件，所以只能将Event下沉到基础库。这样导致的结果是基础库越来越大，还无法拆分。关于这点 微信Android模块化架构重构实践也提到了这件事情，并且自创性的使用了一种叫".api"化的方式来解决这件事情。原理是在编译期将公用接口下沉到基础库，供其他module使用，而这段代码的维护仍然放到非基础库中。这种base库不会膨胀，代码维护的责任制更明确，确定挺不错。可惜最近没有那么多时间来写这个gradle插件了。不知道哪个读者有时间和兴趣可以实现这个插件。意义还是很大的，基础库的代码不会越来越膨胀。eventbus除了使基础库膨胀之

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

外，还有一个问题是，不能进行app间的进程通信。 我们使用广播来取代eventbus。android推出的LocalBroadcast实现机制简单来说是looper-handler并维护一个全局的map。性能上和eventbus类似，使用字符串而不是Event model来匹配事件。我们如果使用 一个接口来包装BroadcastManager，那么我们在app内部可以使用域内广播进行，对于模块化后的lib，我们可以使用域外广播来进行app间的通信。

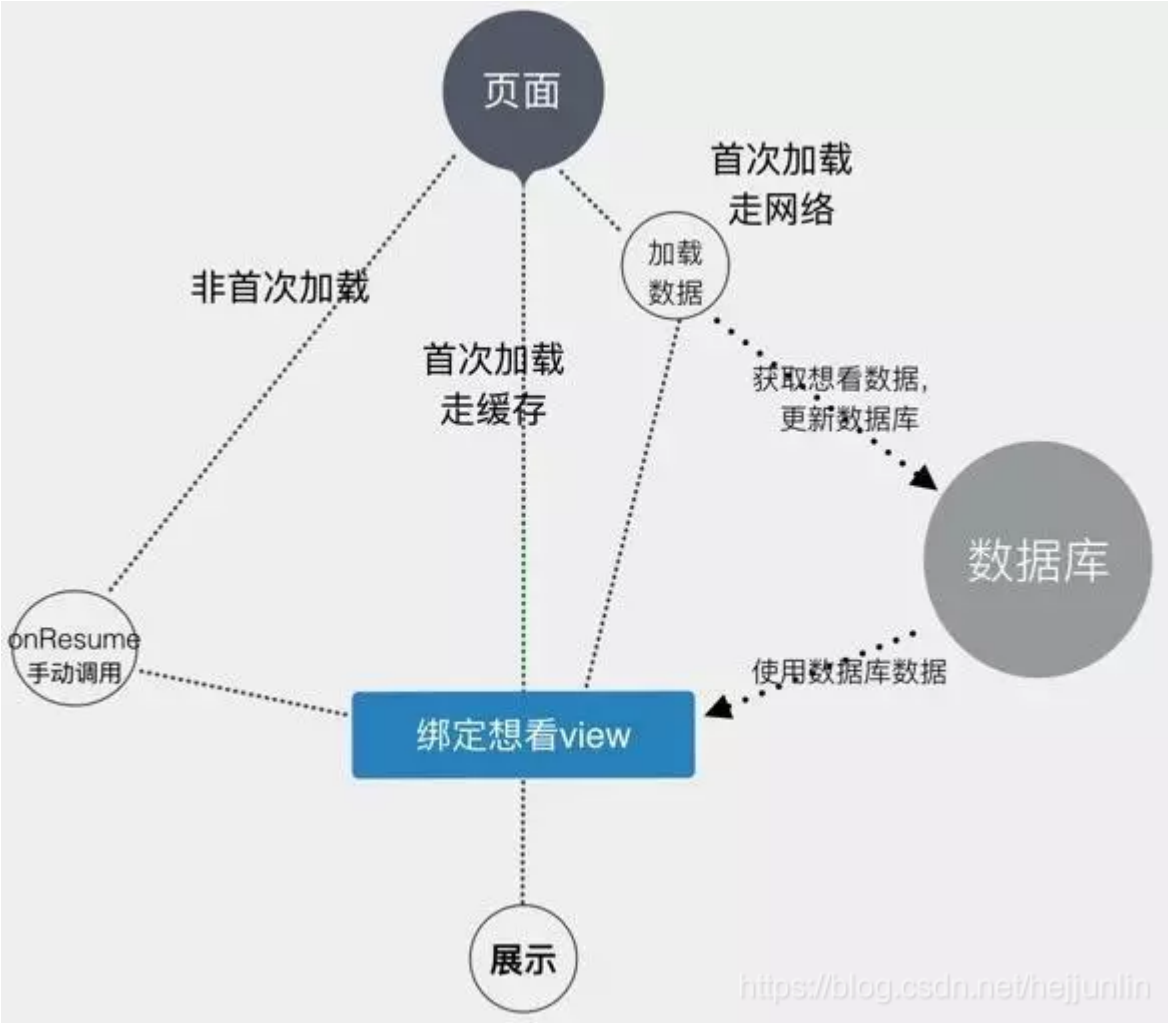
4.5.2 不要乱发广播

如果你项目中大量的使用eventbus，那么你会看到一个类中有大量的onEventMainThread（ ）方法，写起来很爽，阅读起来很痛苦。如果项目中发送这个Event的地方非常多，接收这个Event的地方也很多。在进行代码拆分时，你都不敢轻举妄动，生怕哪些事件没有被接收。广播和eventbus类似，如果项目中存在同一事件的大量发送和接收，那么项目的可读性和可维护性就会变得相当差。这种情况在敏感数据的同步问题上尤为突出：

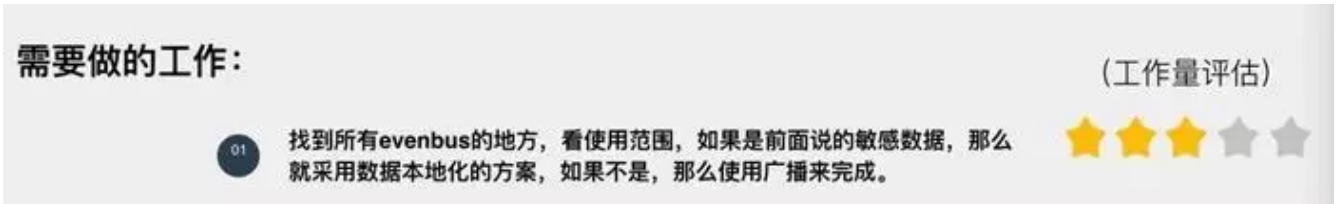


其实对于敏感数据的同步，不需要发送广播或eventbus来进行同步。可以借助数据库将想看数据本地化来完成同步。大体的思想就是我们从网络中获取的数据都同步到数据库。在进行敏感数据填充view时，采用的数据都来自数据库。在页面返回时，如果页面不触发填充敏感数据view的逻辑，那么在onResume（ ）手动调用，即：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



那么模块间/页面间通信大体的就讲完了，这里需要做的工作不多：



3.10.4.6 lib独立运行

4.6.1 为什么需要与宿主app进程通信

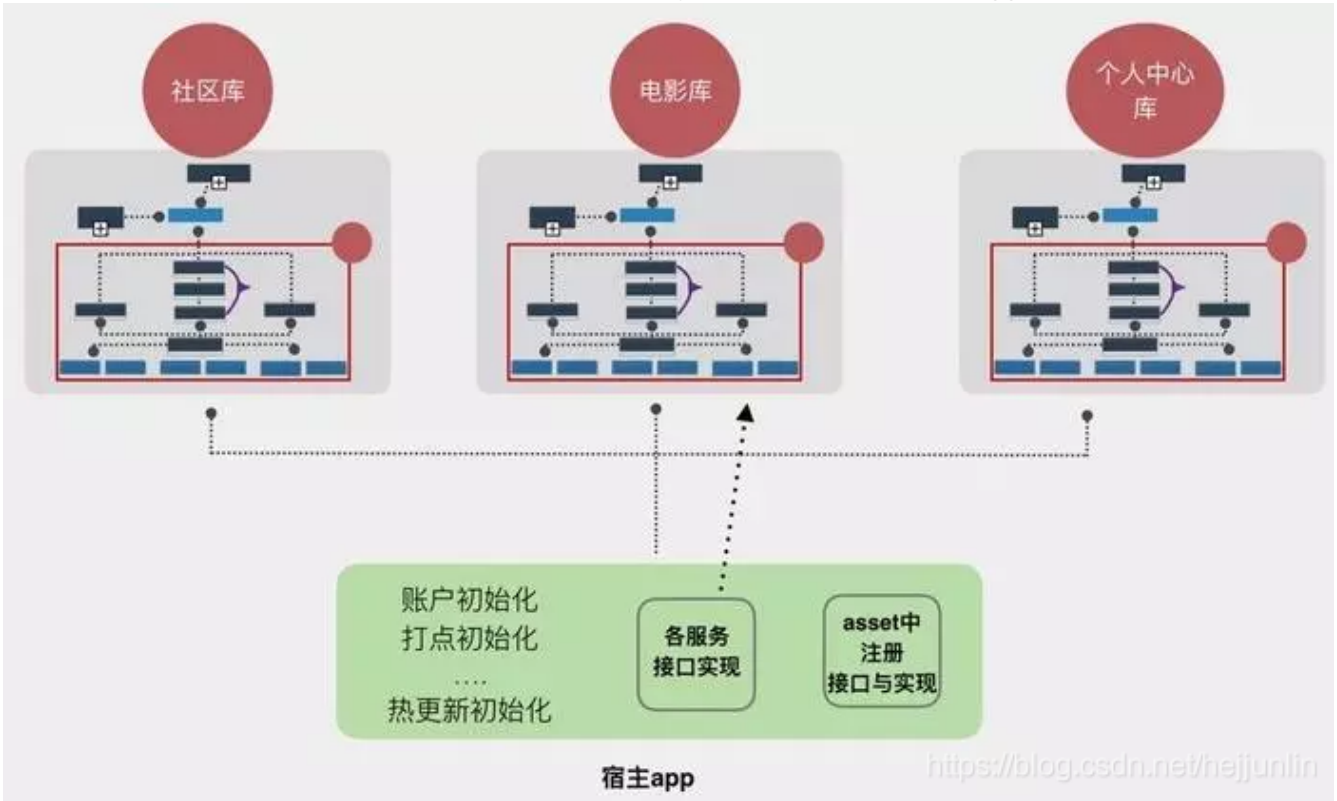
腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?tid=8674704361863085

到这一步，一个业务模块既可以作为library放在宿主app中，也可以作为application独立运行了。作为library很容易理解，和文章前面的问答模块阐述的一样，做宿主app中添加几个activty的壳子，然后添加上lib中的page，然后在manifest中注册即可，即：

```
/**
 * Created by chenwenchao on 17/8/7.
 *猫眼问答的问题列表页
 */

public class MovieAskListActivity extends MaoYanBaseActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        if (getIntent()!=null&&getIntent().getData()!=null){
            long movieId= ActivityUriUtils.parseLong(false,getIntent().getData(),"movieId",()-> showUriParseExceptonActivity());
            getSupportActionBar().setTitle("猫眼问答-"+ MovieMediaUtils.getCutedMovieName(getApplicationContext(),movieId));
            setContentView(new MovieAskListPage(this,movieId));
        }
    }
}
```

https://blog.csdn.net/hejjunlin

当然如果还需要做一些actionBar的交互，需要在宿主activty中写入相应的逻辑。整个app的框架图如：



https://blog.csdn.net/hejjunlin

当业务lib需要调试时，我们需要让这个lib独立运行，就如同文章前面的问答业务模块demo所示。那这时候就有一个问题，我们lib独立运行时，账户的数据从何而来，和app相关的地理位置，城市等等这些数据怎么得到？读者可能会说这些不是服务吗？服务的话，不应该和网络加载，图片加载的服务一样使用serviceloader加载吗？按道理讲是这样的，但账户等一些信息的服务实现类并不是那么容易从宿主app中抽离出来，因为那么服务实现类需要application中进行初始化，还要考虑很多其他东西。所以真实的账户信息并不那么容易通过以前的那种方式获取，那怎么办呢？最简单的办法是制作假数据，比如造一个我自己账户的信息，作为服务实现类使用。但这样的话，账户信息只能是一个人的，对账户信息的修改不可行，账户也不能退出登录。所以还得想新的办法。

4.6.2 与宿主app进程通信过程

最后发现如果我们独立运行的lib能够监听宿主app的账户，位置，城市，登录类型，设备等信息并能够进行同步，那么独立运行的lib中的这些信息就都是真实信息了，并且是动态的。当宿主app退出登录，lib中也是无登录状态。具体的操作是：

- 在宿主app中，我们提供一些contentProvider，各方法提供的内容就是宿主app真实的的账户等数据。当对宿主app账户等信息改变时，通知contentProvider的监听者，比如：

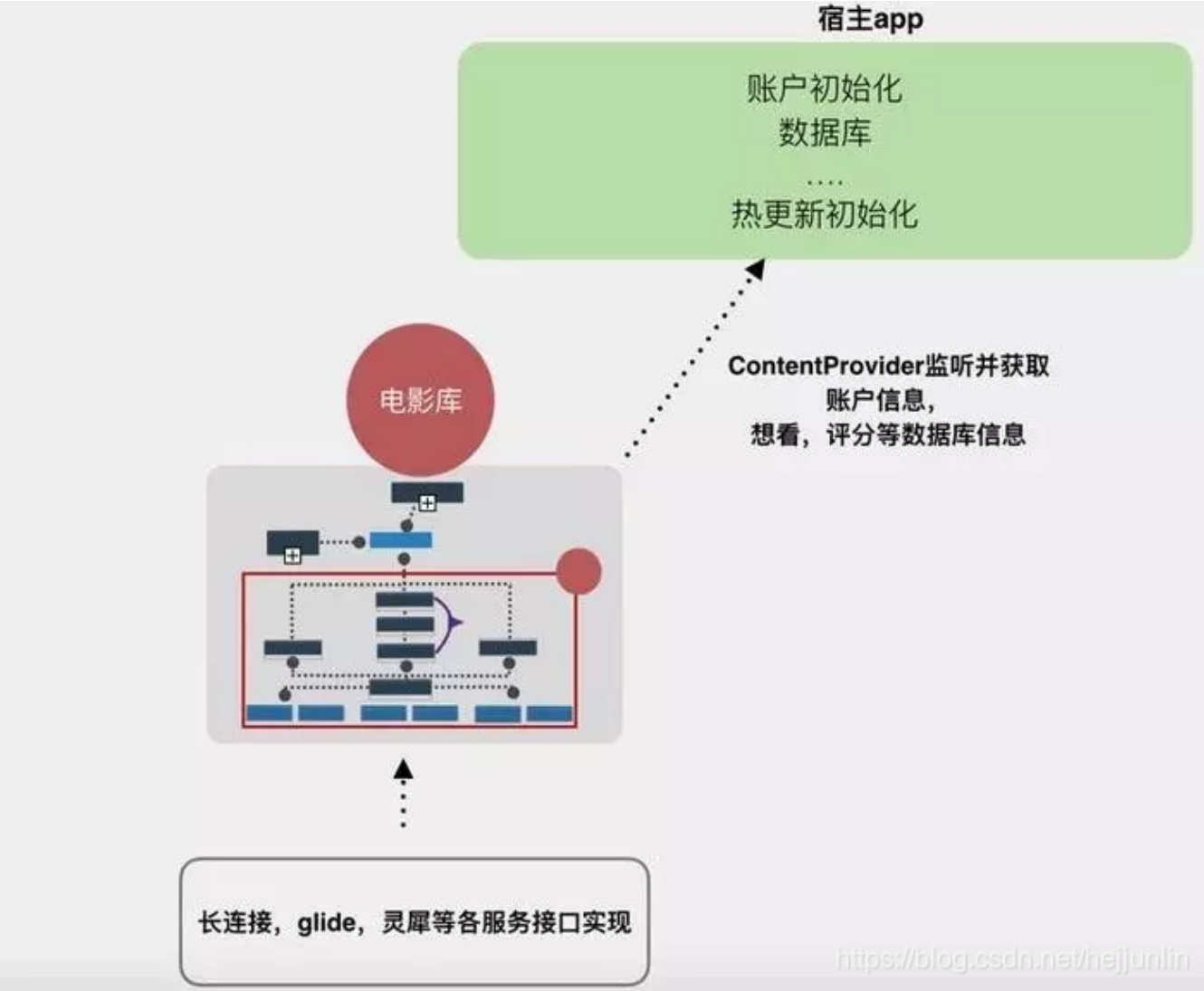
腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
      public void onEventMainThread(LoginEvent loginEvent) {
getContext().getContentResolver().notifyChange(Uri.parse("content://com.maoyan.android.r
unenv/loginsession"),null);
 }
123
```

- 在独立app中，其扮演contentProvider的监听者：

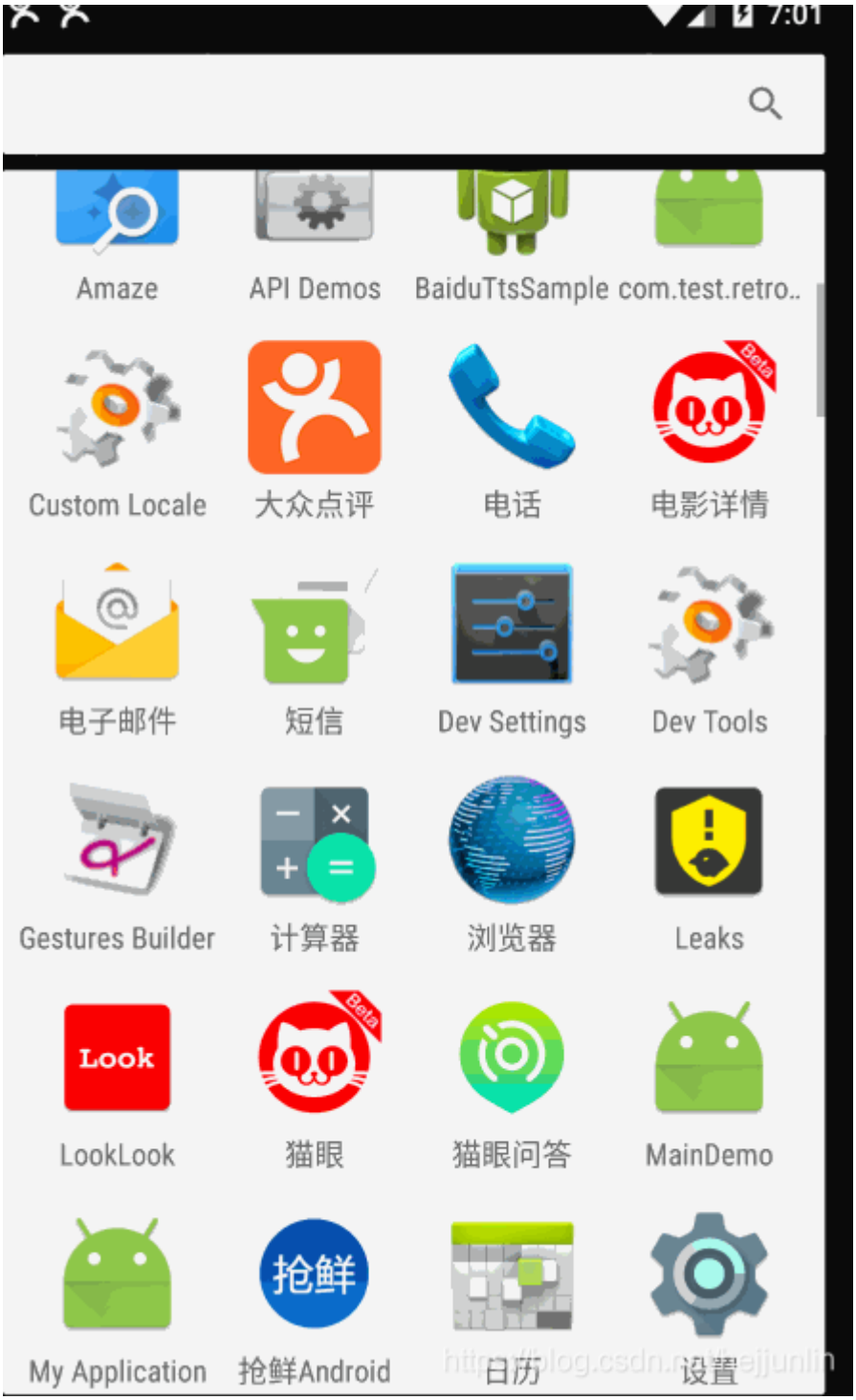
```
mContentResolver.registerContentObserver(Uri.parse("content://com.maoyan.android.runenv/
devicesession"), false, new ContentObserver(null) {
    @Override
    public void onChange(boolean selfChange) {
        super.onChange(selfChange);
        reloadEnviroment();
    }
});
1234567
```

这样的话，lib中账户等数据就和宿主app的数据保持一致了。我们使用服务接口包一层，这样使用方式和之前的服务使用方式就一致了。 大体的示意图如：

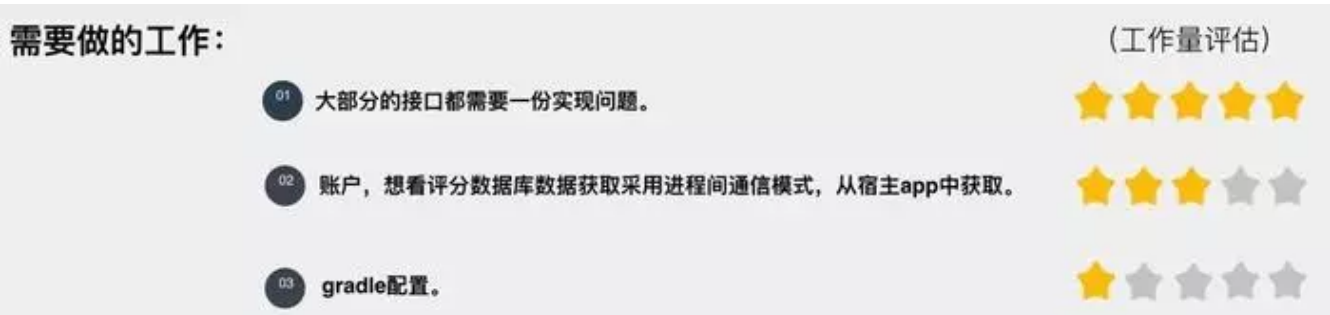


腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

当宿主app退出登录，lib中也是无登录状态，我们看下demo：



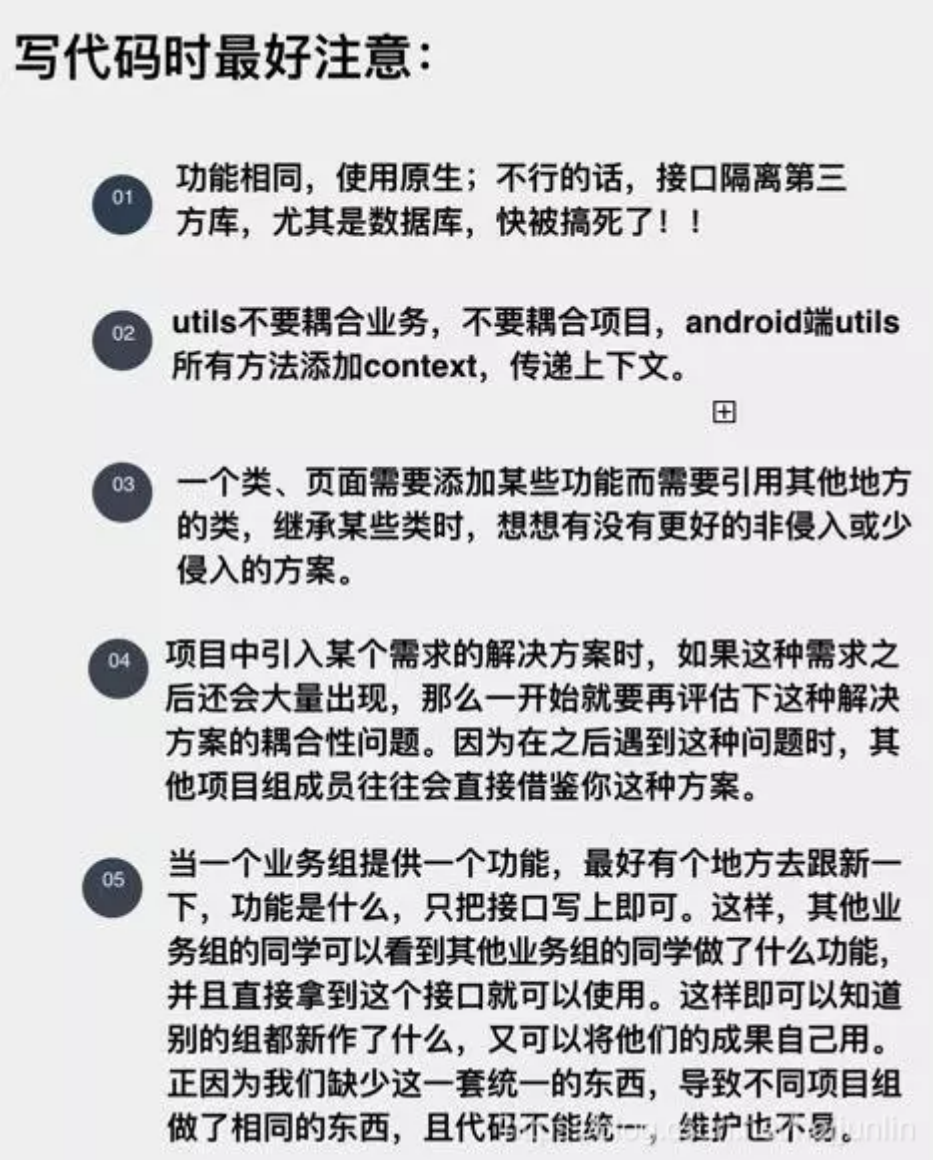
最后，按照惯例，当一个模块要独立运行时，需要做的事情评估：



腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

3.10.5 总结

整个流程终于结束了，希望读者看完后，对模块化有个整体的认识，对每一步需要做什么，耗时多少都有个大致的了解。进行模块化并不是 为了炫技，表明自己多厉害。如果只是这样，那大可不必这么做。因为模块化是一个繁琐，枯燥，耗费时间长，你做了大量的工作，但是在 表面功能上，老板们可能看不到。还不如花一点时间，引入一个第三方库看着花哨。很大一部分工作量是为以前欠设计的代码逻辑买单。我做这件事件也是为了业务服务，因为猫眼电影需要服务的客户端不少。所示做业务解耦，业务进行模块化是必然的事情。通过模块化后，可以很方便的将代码移植到其他端，app内页面的调整也变得简单。 最后的最后，在整个模块化的过程中，有一些经验感悟可以分享给大家，道理都很简单，更重要的是落实：



参考资料

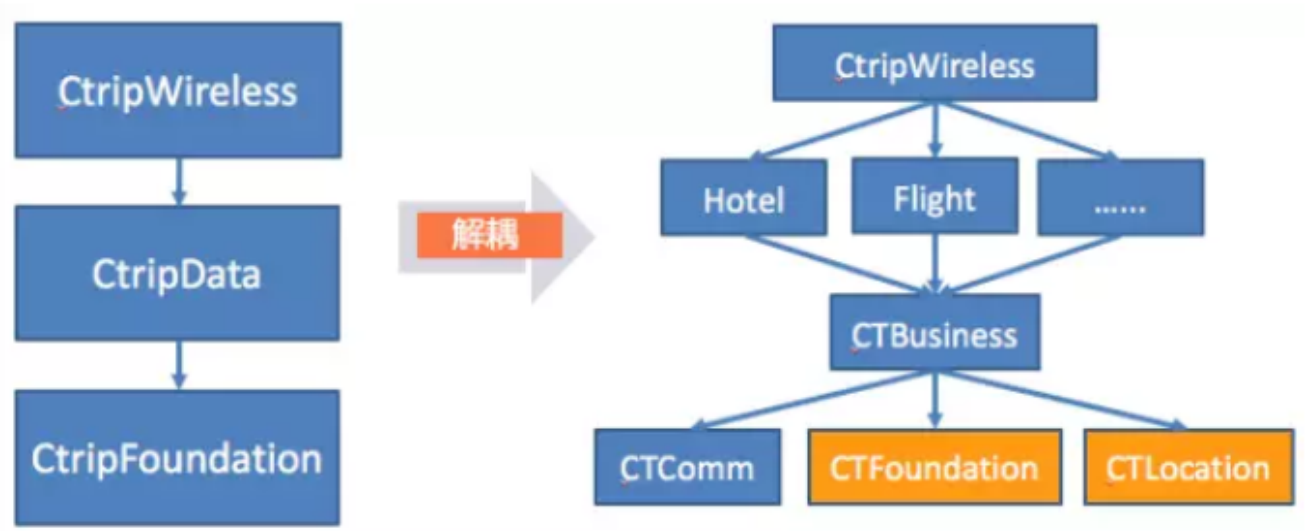
- 微信Android模块化架构重构实践
- 开源最佳实践：Android平台页面路由框架ARouter
- Android组件化之通信
- ServiceLoader
- AutoService
- Android Architecture Components
- 关于Android业务组件化的一些思考
- 从零开始的Android新项目11 - 组件化实践

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

原址：https://blog.csdn.net/qq_22744433/article/details/77948442

3.11 携程App

- 提到了使用数据总线或者 URL 总线实现页面和功能跳转
- 提到了定制 Gradle 插件来定制化打包



3.11.1 携程移动App架构优化之旅

> 本文为携程移动开发总监陈浩然在2015年10月份的ArchSummit全球架构师峰会上的演讲总结。由于面向受众为架构师，因此不会涉及到很多技术细节。通过本文，你可以了解携程通过哪些手段来优化它的App架构的。

『携程旅行App』作为携程超级App产品，是公司全品类旅行产品的核心售卖入口，过去两年为了更好支撑无线业务的快速发展，携程移动App在产品和技术架构方面也做了大量的优化。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

产品

产品方面，携程App从原先的iPhone、iPad、Android Phone、Android Pad和Windows Phone共五个版本精简为Universial iOS和Universial Android两个版本，以便于集中研发和市场资源发布新产品。

无线后端服务架构

无线后端服务架构方面，原有的无线架构（图一）是所有无线服务耦合成一个统一的服务模块，囊括了所有提供给客户端访问的API。

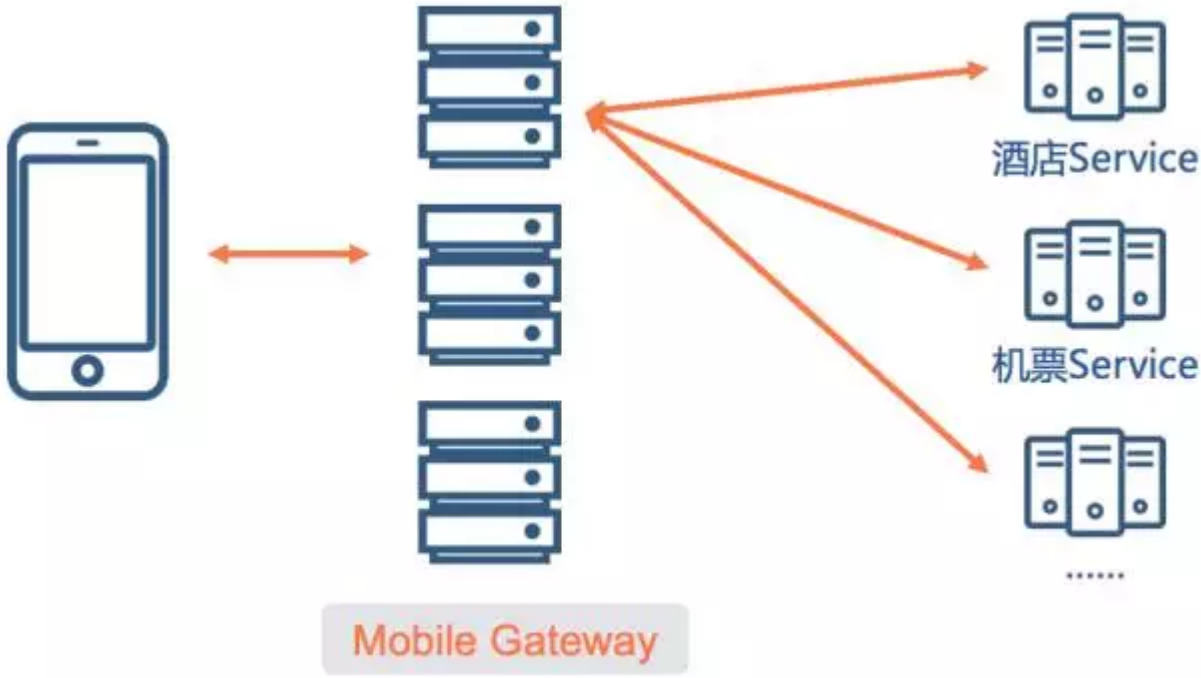
（图一）

在统一服务模块功能较少的时候，这套架构完全符合业务功能需求，但当功能迭代加快，问题不断出现：

- 每个API都是DLL动态库形式（.net）实现，在某些公共逻辑接口发生变化时，不同时期上线的API可能使用了不同版本的逻辑接口定义，会导致运行时出现诡异结果或者进程Crash，影响稳定性；
- 每次大版本发布上线，从测试到全面发布完毕，都需要全量回归测试过程，开发和测试进度严重耦合，影响发布效率；
- 新增的低优先级API可能稳定性不好，出现问题时会影响到整个服务进程的稳定性。

除了这种完全耦合的弊端之外，还存在其他诸如缺少负载均衡、越少监控、缺少熔断等影响后端稳定性的问题。于是我们开始尝试使用一种新的无线架构：Gateway（图二）。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

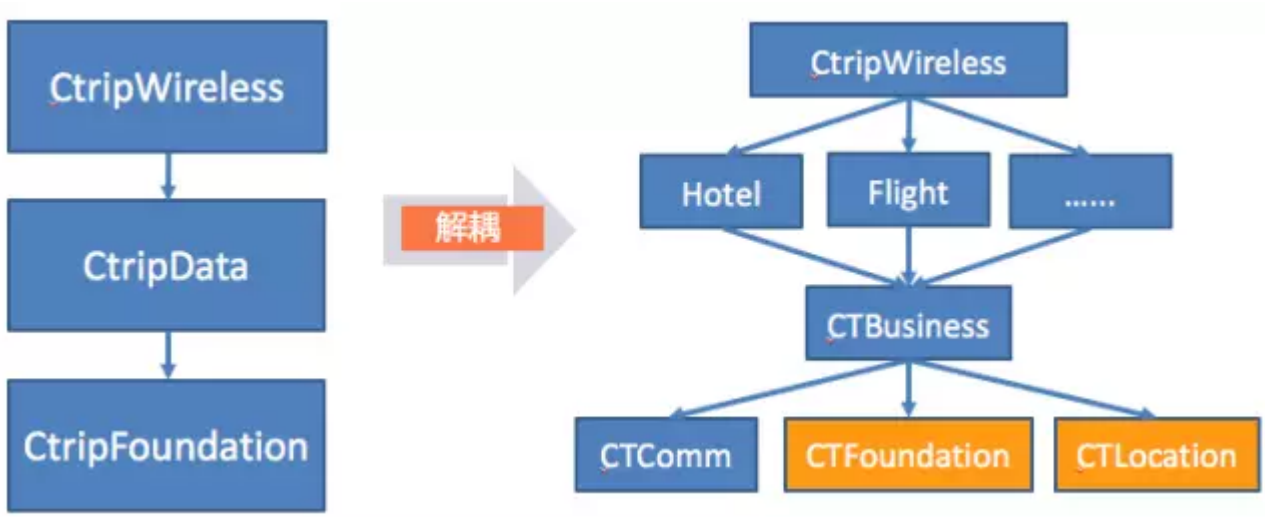


（图二）

携程基于Netflix的开源项目Zuul开发了无线Gateway（曾在2015年QCon上海会议中分享）。Gateway的职能是负责接收来自无线端的所有API请求，并将他们路由到正确的目标应用服务器，并且提供限流、隔离、熔断等功能，保证了无线服务的长期稳定运行，拥有的弹性容错机制也减少了日常运维工作。同时该Gateway提供了多维度的监控数据，并与报警系统对接，实时监控线上情况，达到运维自动化。

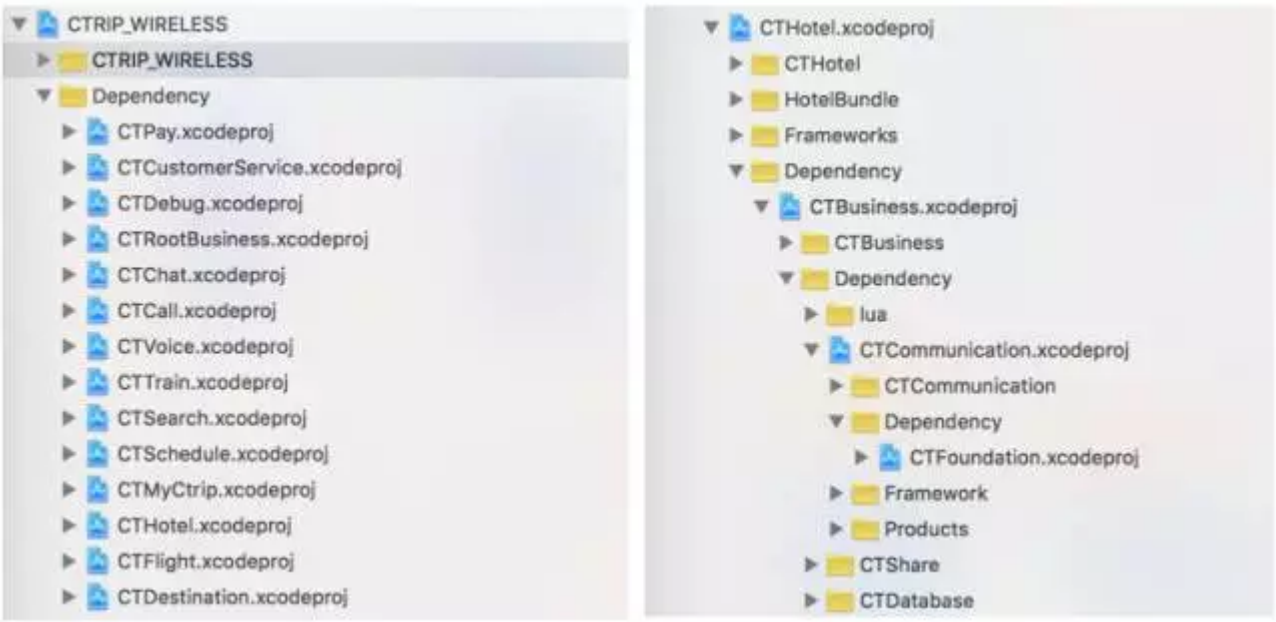
App工程架构

App工程架构方面，原先的App开发还是单工程单构建的方式，各产品间的代码耦合严重，于是进行了App工程的解耦（图三和图四）。



（图三）

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



（图四）

解耦后各产品间代码完全独立，相互页面和功能的跳转走数据总线或者URL总线方式来实现，打包时的资源问题可以通过Run Script（iOS）和Gradle（Android）来定制化解决。

在App工程架构解耦后，紧接着是两个问题：

- 开发框架是否满足产品开发需求？
- 性能和质量是否达到用户体验要求？

为了解决第一个问题，我们梳理了现有的App功能，将通讯、定位、Hybrid框架、数据库、登录、分享、基础库等核心功能实现了SDK化，也提供给公司其他App直接使用；同时将App内部的公共业务组件例如地图、日历、城市、图片、通讯录等实现了统一。

要解决第二个问题『App性能和质量』，首先需要了解App性能的现状，即App端性能的采集。常规做法有两种：1. 使用第三方性能采集SDK，例如OneAPM、听云等第三方工具；2. 自建。携程为了完整掌握用户数据采用了自建的方式：App通过日志SDK采集日志，上传至日志服务端，日志消息经Kafka消息队列存入HDFS（RCFile格式）分布式文件存储系统，后期的数据查询均基于Hive系统来实现。

App端的性能数据会在多种纬度（操作系统、App版本、网络状况、位置）下采集网络（网络服务成功率、平均耗时、耗时分布等）、定位（经纬度成功率、城市定位成功率等）、启动时间、内存流量等各类指标。其中像网络服务性能是对于用户体验至关重要的端到端性能，也是优化的核心依据。

性能数据采集后需要采用简单直观的Portal进行展示，携程为无线业务开发了Web端和App端性能展示Portal，图五是网络性能的截屏，数据会每小时进行更新聚合并展示。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?tid=8674704361863085

（图五）

基于完备的性能采集数据，很多App性能便可以依据数据进行迭代优化，例如App网络服务方面，携程App采用了以下多种优化手段：

- 使用TCP长连接实现网络服务，减少网络连接时间
- 根据网络状况2G/3G/4G/WIFI进行调优参数
- 根据连接/读/写不同阶段使用重试机制
- 使用IP列表避免DNS解析失败或者劫持，无需进行DNS解析
- 根据网络延迟选择服务端IP（使用Ping）
- 使用ProtocolBuffer+Gzip减少Payload

经过这些优化手段，携程App的端到端网络服务成功率从最初很差的95.32%提升至99.87%，用户体验得到明显提升。

携程App的Hybrid框架也是经过多个版本的迭代，已经支持强大的插件功能，已经做到凡是可以用Native的组件通通使用Native组件来优化Hybrid业务的体验。携程Hybrid框架在设计之初即采用了离线包功能：Hybrid业务整体打包在App中，节省了用户打开页面时的资源加载时间；同时离线包支持查分增量更新，并通过7z压缩方式进一步降低了增量更新包的大小，相对zip压缩减少30%大小。

Native的插件化和HotFix

Native的插件化和HotFix方面，我们在iOS端使用开源的解决方案JSPatch，在Android端采用了自研的解决方案DynamicAPK，可以支持各组件的资源及代码的更新。DynamicAPK方案无需做任何activity/fragment/resource的proxy实现，使得原有的业务代码无需修改即可支持，迁移代码很小，同时可以提升App启动时间，详情请参考GitHub。

其他优化都是针对特定的业务场景展开，例如Android的海外地图没有好的解决方案，iOS的地图控件又存在人在国外看国外和人在国外看国内地图数据精度低的问题，携程基于Google地图开发了Hybrid版地图，用于在特定场景的业务需求；Hybrid网络性能优化，通过Hybrid接口发送Native网络服务，可以避免DNS劫持并且利用Native端的TCP长连接；海外网络性能优化，通过TCP海外加速产品实现链路优化；图片性能优化方面使用了WebP图片格式，可以降低30-40%图片大小，图片分片上传等。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

移动端技术发展很快，携程也在积极尝试新技术，例如React Native（已在账户信息部分页面使用），新的网络协议SPDYHTTP/2.0，Apple/Huawei/Samsung Watch App等都做了大量尝试，以期能够提升产品品质。

总之，App架构的技术优化没有尽头，我们会继续以开发效率、性能、质量、新技术几个纬度不断推进，希望未来可以有更多内容分享给业内同行。

3.12 支付宝 App

- 介绍了基于 Bundle 和 mPaaS 打包插件的 Quinox 客户端框架，用于解决模块化和动态化两大问题；
- 提到了解决资源 id 重复的解法：改造 aapt，为每个 Bundle 指定不同 packageId；

你知道支付宝容器化架构是怎么搭建的吗？

3.12.1 前言

由本章节开始，我们将从支付宝客户端的架构设计方案入手，细分拆解客户端在“**容器化框架设计**”、“**网络优化**”、“**性能启动优化**”、“**自动化日志收集**”、“**RPC 组件设计**”、“**移动应用监控、诊断、定位**”等具体实现，带领大家进一步了解支付宝在客户端架构上的迭代与优化历程。

本节将介绍支付宝 Android 容器化框架设计的基本思路。

3.12.1.1 开发背景

随着 Android 应用程序所能实现的功能越来越强大和复杂，随之而来的是：

- Android 程序的的代码和资源越来越多，APK 文件的 size 越来越大，Android 程序也越来越复杂；
- 随着应用的迭代、项目的扩张，团队数量以及团队人数的同时增多，基于传统架构模式的并行开发也变得愈加困难。

此外，移动客户端通常需要面对动态化开发的挑战；Bug 紧急修复等运维需求；同时也有一些在线运营的需求，如动态下发广告，推送接入活动等。如果每次有运维、运营需求，都需要一次客户端发版，那将是传统的开发人员的梦魇。

Android 开发者们深切体会到一个稳健可靠、可扩展的、支持大规模并行开发的客户端开发框架对于平台级别的客户端 App 的重要性。事实上，客户端框架设计的健壮性和扩展性，在面对上述需求和解决困难上，往往能达到事倍功半的效果，尤其是 Android 客户端开发人员将深受其利。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

那么，作为平台级别的 Android 客户端 App 究竟该如何的进行框架设计，才能满足千变万化的移动互联网时代的困难和需求？

3.12.1.2 平台级客户端框架面临的问题

No.	问题	描述
1	项目工程复杂度高，开发、编译、测试、集成都非常困难	支付宝 App 代码 200W 行+
2	平台级 App 的内部微应用（团队）非常多，并行开发要求高	内部多达几十个应用
3	APK size 庞大	支付宝 APP 60+M，导致在某些厂商 Rom 中安装不上
4	线上版本出现各种问题	如：发版后，UCSDK 被乌云平台暴露出安全漏洞
5	线上活动运营需求	春节红包扫福活动，预案要动态推送新 so 文件到客户端

我们可以归纳为：平台级客户端框架必须要解决的是**模块化**和**动态化**这两大核心问题。（本篇文章我们着重关注**模块化**相关内容，后续我们通过其他文章分析**动态化**的能力。）

3.12.1.3 框架设计原则

为了解决上述模块化的问题，我们要遵循以下原则去设计客户端框架：

- 根据基础技术层级、客户端的业务线等原则，对客户端应用程序进行模块化拆分。
- 每一个模块由独立的小团队或者个人来进行开发、维护、测试、集成。
- 模块与模块之间要做到彻底解耦，模块之间可以通过接口进行依赖。
- 每一个模块可以进行热插拔，单个模块的插拔不影响整体的工程的编译运行。

3.12.2 Quinox 简介

Quinox 客户端框架是类 OSGi（ like-as）框架的实现。Quinox 一词来源于著名的 OSGi 框架的实现 Equinox。

基于此框架的客户端 App，都是由一个个的积木搭建而成，这些积木被称之为：Bundle。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

3.12.3 Bundle 介绍

3.12.3.1 什么是 Bundle

Bundle 是 OSGi 规范的模块化基本单位，与 Android 里的 android.os.Bundle 是两个完全不同的概念。 OSGi 里的 Bundle 指的是 Java 应用程序的基本单位，它是一个模块单元（Jar 格式），也是上文 Quinox 简介里提到的积木。基于 Quinox 容器框架开发的应用程序也是由众多的 Bundle（APK 格式）构成。

本章节将从项目开发的三个不同的时期对 Bundle 的形态进行阐述：

时期	形态
开发期	Bundle 工程
构建期	Bundle 包
集成期	集成客户端的 Bundle 基线

3.12.3.2 Bundle 工程

常规的 Android 项目开发，代码工程通常有两种(两级)类型

工程类型	Library	Application
工程输出	Aar	Apk

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

基于 Quinox 容器框架开发的 Android 项目，代码工程则有三种(三级)类型

工程类型	Library	Bundle工程包	Application（测试包/安装包/Final APK）
工程输出	Aar	Apk(.jar)	Apk

关于 Bundle 工程，我们需要了解以下三点：

- Bundle 工程跟常规的 Android Application 工程非常的类似：它内部也会有多个 Library（Android Module）；它的输出形式也是 APK 格式。
- 虽然 Bundle 包文件本质上是 APK 格式，但是该 APK 是无法运行的。同时，Bundle 工程被 deploy 到 mvn 仓库里时，它的后缀名是会改为.jar。
- 基于 Quinox 容器的 Application 工程（可称之为 Portal 工程）则是将众多 Bundle（APK）合并成一个 APK（Final）的过程。这里是合并，而不是编译，所以生成最终 APK 的速度将会非常快，因为编译已经被分布式的进行在各个 Bundle 中了。基于 Quinox 容器开发的客户端程序，需使用 mPaaS 定制的构建工具（即打包插件）。

关于 Bundle 工程的结构图请参考：

3.12.3.3 Bundle 包

如上所述，Bundle 工程的输出也是 APK 文件。

在 OSGi 规范中，Bundle 是有很多属性的。Bundle 工程输出的 APK 与常规的 APK 有一个不同点，mPaaS 插件会将 Bundle 的所有属性生成一个特殊的文件放在这个 APK 中，供容器去解读。

除了 APK 文件之外，Bundle 工程的构建结果还包含：

- AndroidMannifest.xml
- Bundle 接口包（可以理解为一个 jar 包，它包含且暴露该 Bundle 提供的接口类）
- mapping.txt

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

Bundle 包文件，在构建完成之后，通常要 deploy 到本地/远程的 mvn 仓库中，以供其他 Bundle 工程引用，或是被 Portal 工程集成。

3.12.3.4 Bundle 基线

前面已经讲述过，构建 Final APK 其实主要就是将很多的 Budnle APK 合并成最终的 APK 的过程，而这些众多的 Bundle APK 们都存放于 mvn 仓库中。

因此我们将这些 Bundle 的 GAV（GroupId，ArtifactId，Vesion）的集合，称之为基线。

当某一个团队/个人开完一个 Bundle 工程的新功能，并经过测试达到可发布状态，就可以更新基线里的版本号。我们将这个过程称之为进基线。我们认为：基线里打出来的 APK 是稳定可运行的；没有稳定 Bundle 工程包不应该进基线。

Bundle 基线机制可以很好的隔离了模块之间的相互影响，保障了不同团队间开发环境的和谐与稳定，达到了我们之前的设计的初衷，因此可以很好的支持多团队并行开发。

3.12.3.5 Bundle 包属性及配置办法

关于 Bundle 属性，我们可以参考 OSGi 的 Bundle 属性。Quinox 容器框下定义的 Bundle 属性要简单的多。

下表将列举 Bundle 的所有属性以及配置方法：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

名称	说明	配置办法
Bundle-Name	Bundle 的名称，作为 key 值存在。同一个客户端 apk 中，不允许同名的 Bundle 存在	由 mPaaS 插件根据 Bundle 工程的 GAV 的 GroupId、ArtifactId ，以一定的规则生成而来。
Bundle-Version	Bundle 的版本号，各个 Bundle 的接口包必须做到 API 版本向下兼容。	由 mPaaS 插件根据 Bundle 工程的 GAV 中的 Version 生成而来
Init-Level	已废弃	配置为 1 即可
Package-Name	已废弃	配置为 '' 即可
Component-Name	Bundle 中声明的 Android Component。它跟 Export-Pacakges 属性一样，是 Bundle 的入口类。	由 mPaaS 插件根据 AndroidManifest.xml 文件中定义的 Activity，Service，BroadcastReceiver，ContentProvider 等生成
Package-Id	Bundle 工程的资源的 packageid，具体技术细节请参考4.2章节	必须由开发同学在 Bundle 工程中设置属性 packageId，其值的设置区间为【27, 127】，如果没有资源，则设置为 127，如果 Bundle 为 Bundle 依赖关系树上根节点的 Bundle，则设置为27。
Contains-Dex	此 Bundle 中是否包含代码（classes.dex）	由 mPaaS 插件根据 Bundle 文件中是否包含 classes.dex 节点判断得来。备注：静态链接的 Bundle 由于 classes.dex merge 到了主 apk 中，所以该属性会被修正为 false
Contains-Res	此 Bundle 中是否包含资源（resources.arsc）	由 mPaaS 插件根据 Bundle 文件中是否包含 resources.arsc 文件判断得来。
Native-Library	此 Bundle 中是否包含 native so（lib/xxx/libxxx.so）	由 mPaaS 插件根据 Bundle 文件中 native so 文件判断得来。备注：在构建最终 apk 时，Bundle 中所有的 so 文件在构建 Final apk 时，会 merge 到 Final apk 中，所以该属性会被修正为 null
Required-Bundle	此 Bundle 依赖的 Bundle 列表：为 Budnle-Name@Bundle-Version 的格式。	由 mPaaS 插件根据Bundle项目工程依赖其他Bundle接口包，来生成此属性。
Export-Pacakges	Bundle 导出包（请参考 OSGi 的导出包概念）。Quinox 容器将根据导出包，从对应的 Bundle 中加载类。	必须由开发同学在 Bundle 工程中设置属性 exportPackages。例如：某个非静态连接的 Bundle 提供了类:`com.alipay.android.phone.framework.api.A`作为接口给其他 Bundle 使用，则须将`com.alipay.android.phone.framework.api`配置为导出包（反射被使用的类也应纳入导出包）。有多个导出包的用 ',' 隔开。为了性能考虑，导出包不应设置太多，或者范围太广。

3.12.4 资源管理

3.12.4.1 资源管理器

作为 Android 开发人员，我们知道通过 android.content.res.Resources 对象可以获取字符串、布局、图片、动画等资源。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

在 Quinox 容器化的框架内，原生的资源管理肯定无法实现多 Bundle 的资源管理，这时候，我们就构建了 Bundle 资源管理器，来专门处理各个 Bundle 的资源的加载、调用等工作，替代了 Android 原生的资源管理逻辑。

但是，由于所有 Bundle 包都是独立编译的，它们中的资源极可能存在着相同的资源 id。因此，当存在相同资源时，就可能存在冲突，那么如何解决资源 id 的冲突呢？

3.12.4.2 资源 id

作为 Android 程序员，我们都知道资源 id 是一个 int 值，它包含4个 byte。它是在构建 APK 工程时，由 aapt 工具生成，定义在 R 文件中。

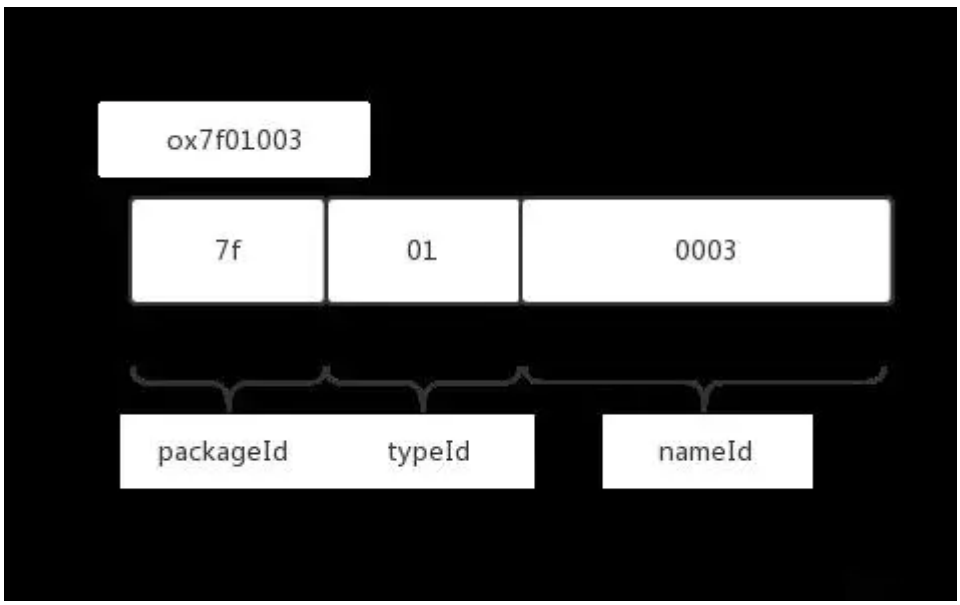
示例代码：

```
public final class R {    public static final class drawable {        public static int
xxx_bg=0x1e020000;    }    public static final class id {        public static int
xxx_id=0x1e050001;    }    public static final class layout {        public static int
xxx_layout=0x1e030000;    }    public static final class string {        public static int
xxx_str=0x1e040001;    }}复制代码
```

这四个 byte 含义如下：

- 第一个字节为：pacakgeId。
- 第二个字节为：typeId。它表示的是不同的资源类型，如字符串，布局，图片，动画等。
- 第三第四两个字节合起来为：资源名称的 id

如下图所示：



到这里，很多读者应该已经理解到了，Quinox 容器框架关于资源 id 冲突的解决方案是，让 mPaaS 打包插件使用改造过 aapt 工具，对每一个 Bundle 工程都指定不同 packageId，进行分区隔离，从而确保不同的 Bundle 之间资源 id 是不会重复的。这也是为什么 Bundle 工程里需要指定 packageId 的缘故。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

3.12.5 容器化

关于 Quinox 容器化这里，由于目前为止，Quinox 暂未开源，所以本章节内，我们暂时不涉及到源码分析。

上面我们聊了很多关于 Bundle 的话题，那么整个容器化的核心，也是如何管理各个 Bundle。这时就要引出我们的容器管理器了，容器管理器的主要工作就是协调各个 Bundle，对各种信息进行增删改查。

在应用启动后，我们的容器管理器会读取配置信息，生成各 Bundle 的信息实例。

3.12.5.1 容器管理器：增、删

Quinox 容器框架的目标是解决 Android 客户端 App 模块化和动态化这两大核心问题。增、删这两项能力，更多的是用来实现动态化能力的，方便容器对各个 Bundle 进行动态添加、删除。由于本文着重描写模块化的能力，所以这部分，我们后续单开专题来分析容器的动态化能力。

3.12.5.2 容器管理器：改

关于容器管理器的改的能力，Quinox 主要是利用改的能力，做一些启动性能的优化，以及机型适配上的工作，这里涉及源码较多，我们不做过多分析。

3.12.5.3 容器管理器：查

容器管理器使用最频繁的功能接口应该是查询接口：

- 根据 BundleName 进行查询（还记得 Bundle 的 Require-Bundle 属性么？）
- 根据 packageId 进行查询（非27的 Bundle）
- 根据 Android Component 类名进行查询（还记得 Bundle 的 Component-Name 属性么？）

通过管理器的查询接口，我们进行各个 Bundle 之间的协调、通信，完成容器化的功能。

3.12.5.4 组件的启动

除了容器管理器，还有一个重要的点就是组件的启动器。Quinox 容器定制类原生 Android Activity 的启动流程，从而自主管理 Activity 的创建以及生命周期。

同时，由于 Activity 是我们自主的启动器进行的创建，我们还可以对 Activity 进行一些定制化的改造，方便其更好的适配容器这套体系。比如说给 Activity 赋予我们自定义的资源管理器，管理 Activity 堆栈并对外提供接口，对 Activity 各生命周期做一些切面工作等等。

定制化的组件启动器，还有一个好处就是可以做到 Activity 的动态运行。所谓动态运行，是指运行出厂未注册在 Manifest 中的 Activity。这块功能，更多是为了支持容器动态化的能力。

由于 Activity 动态运行的实现逻辑涉及较多的核心技术点，所以我们暂时不进行具体实现的剖析。

3.12.6 总结

通过本节内容，我们已经初步了解了 安卓端容器化框架的设计思路和相应模块。由于篇幅限制，很多技术要点我们无法一一展开。

关于安卓端容器化框架的设计思路和具体实践，同样期待你们的反馈，欢迎一起探讨交流。

3.13 Atlas-手淘组件化框架的前世今生和未来的路

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

摘要：Atlas是古希腊神话中的天神，是波士顿动力公司的机器人，借助搜索引擎，得以发现这个名词背后许许多多的含义。在手机淘宝，Atlas是一个扎根于Android客户端的一个组件化容器框架，相比神话中用手和头支撑起苍天的泰坦神族，Atlas在手淘默默无闻地承载着手淘上丰富业务的运行，伴随着数不清的功能在...

今天手淘技术团队宣布正式开源它们的容器框架Atlas，项目地址：

https://github.com/alibaba/atlas

同时他们还推出了项目官网，上线了技术文档：

http://atlas.taobao.org/

下面让Atlas团队来介绍下该项目的历史、原理和未来。

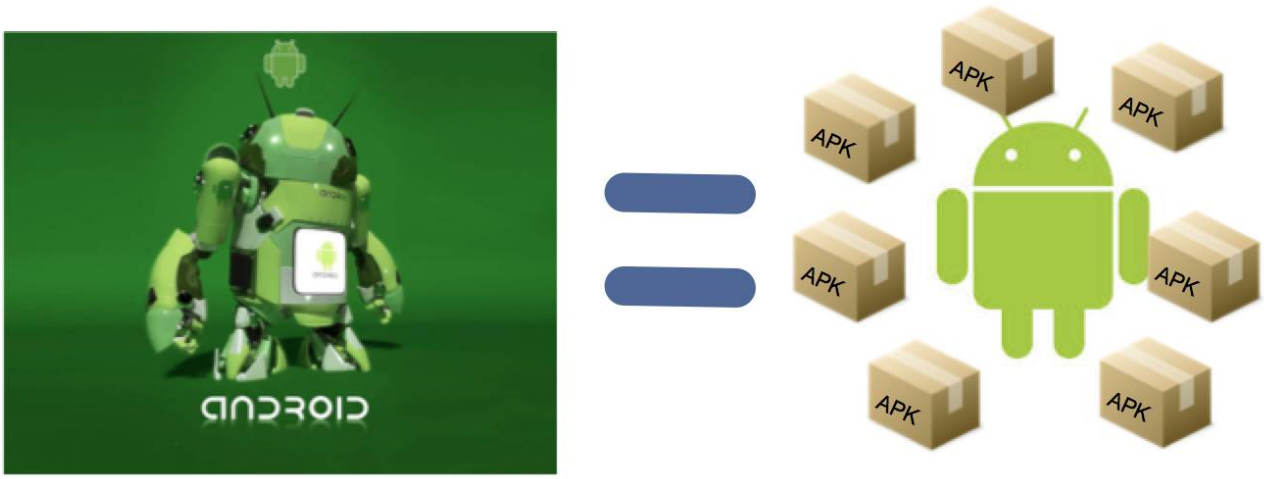
3.13.1 Atlas是什么

Atlas是古希腊神话中的天神，是波士顿动力公司的机器人，借助搜索引擎，得以发现这个名词背后许许多多的含义。在手机淘宝，Atlas是一个扎根于Android客户端的一个组件化容器框架，相比神话中用手和头支撑起苍天的泰坦神族，Atlas在手淘默默无闻地承载着手淘上丰富业务的运行，伴随着数不清的功能在用户手中经历新老交替。

插件化的历史

在阿里，在手机淘宝，移动客户端的迭代更新可以说是整个移动互联网从新起到繁荣的见证。伴随着越来越多业务的诞生，越来越多的代码开始往小小的客户端涌入，而无论是iOS还是Android，各种客户端的体积也经历着快速的增长。而由于客户端的推广成本较高，各个业务线都有很强的需求接入手机淘宝客户端。所以在当时航母战略的背景下，迫切需要一种技术能把客户端化整为零，这些模块可以自由组合，并且当部分功能变更时只需要更新对应模块。

我们在2012年底的时候就开始研究Android上的插件化技术，并与2013年初把Atlas作为插件化框架接入手机淘宝。基于插件化的能力，一个普通的Android应用可以低成本地转为符合Atlas规范的插件，使Apk既可以以插件方式运行也可以独立安装运行。一个大的Android客户端项目可以分割成数个插件，做到代码隔离，降低了开发、维护、部署的成本。



（基于native的插件化构想）

当时插件化的方案可分为三类：Web、WebApp、Native。各种方案在手淘内都在有同学进行深入的研究。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

Native形式的插件由于技术门槛较高，当时业界采用的并不多见。但是由于其比WebApp有良好的性能和体验，同时对于许多已经成型的小客户端推广的迫切需求，尤其值得我们去探索。

上图是当时插件化的技术原理的简图，其实跟现在业界的插件化方案主要的设计思路基本一致。Atlas作为一个模拟的Android运行环境，每个插件以一个独立的进程运行在各自的沙箱环境中。同时由于接入成本较低，其他的业务应用基本可以算是零成本的方式接入到手淘环境，同时也可以随时发布新的版本进行更新。

插件化的方案有其特有的优势，独立的进程保证了业务的绝对隔离，同时也便于隔离风险。聚划算、天猫、彩票一个个独立应用开始起降于手淘这架航母。甜蜜的成长总是短暂，插件化发展很快，随着手淘All In方案的深入，一些隐患开始展现。

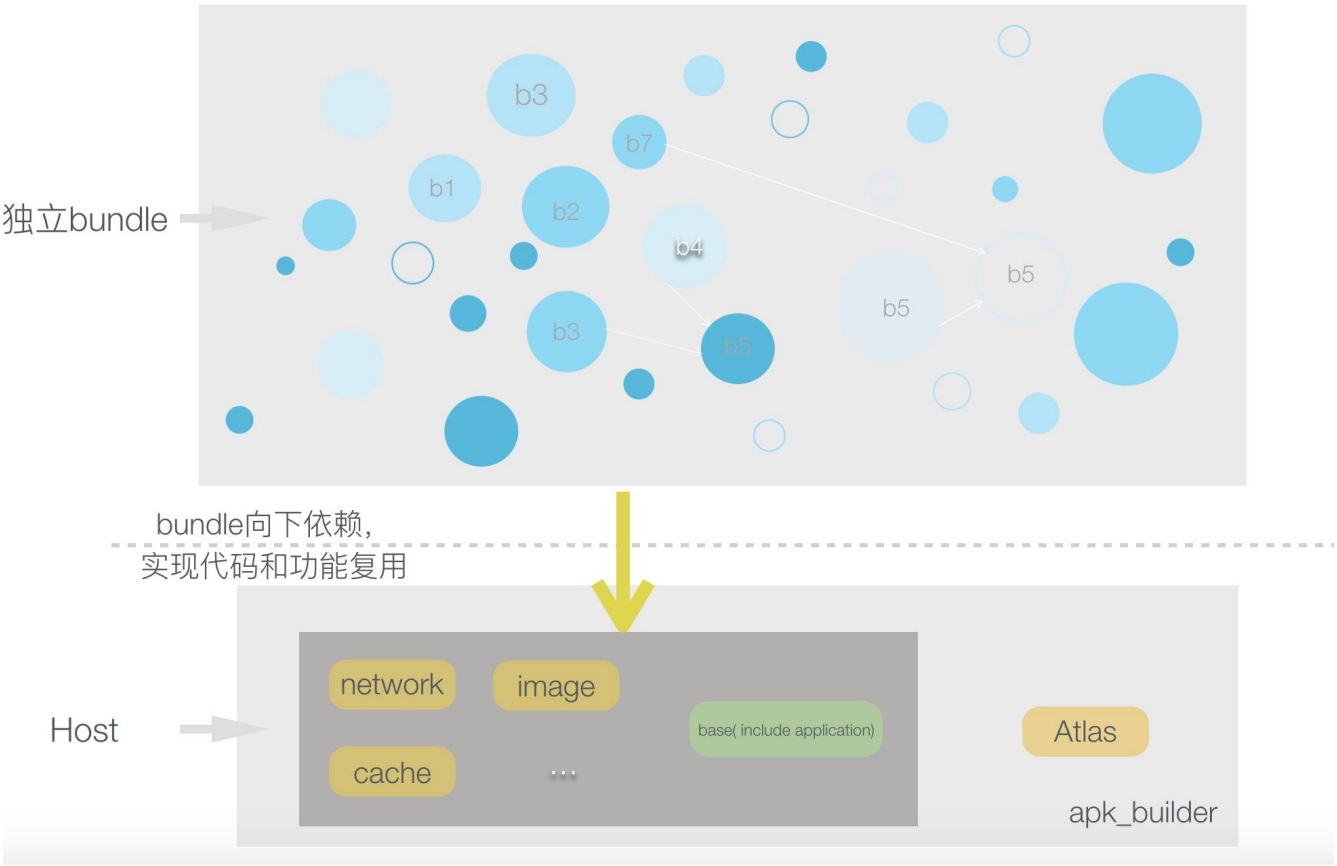
1. 性能：新进程的开辟极大影响每个业务的进入速度，同时由于手淘的插件并不是用完即走的场景，比如说从首页->聚划算->详情->店铺->下单各个环节是存在着一定的关联，各个插件的进程无法退出。而多进程的机制又极大的占用了内存，同时也引起许多用户的质疑
2. 复用：插件的独立限制了许多中间件的复用，AIDL的方式并不适合中间件能力的输出，独立的apk直接进入的同时也携带了大量重复的二方库，也一方面极大增大的应用的体积
3. 稳定：插件化需要一个模拟的Android运行环境，在Android多版本以及国产ROM的兼容中需要做大量的工作，同时多插件运行过程中在低端设备上很容易遇到进程被回收或者三方应用强杀的情况，没有良好的恢复机制会极大降低用户的体验

3.13.2 组件化的诞生

伴随着All In的进行，在手淘内部发起了对大型app进行重构的计划。我们需要这样一个框架：

1. 支持大量丰富业务的接入，同时业务之间能够保持清晰的边界，各自可以继续灵活迭代；
2. 用一批统一的中间件去支撑起各种业务的底层功能，保持中间件代码的全面复用；
3. 能够尽量保持对系统的低侵入，尊重原生运行机制以降低后期的维护成本；
4. 在用户设备上尽量体现一个简单客户端的特性，同时特定的业务功能按需获取，保持体积的可控;

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

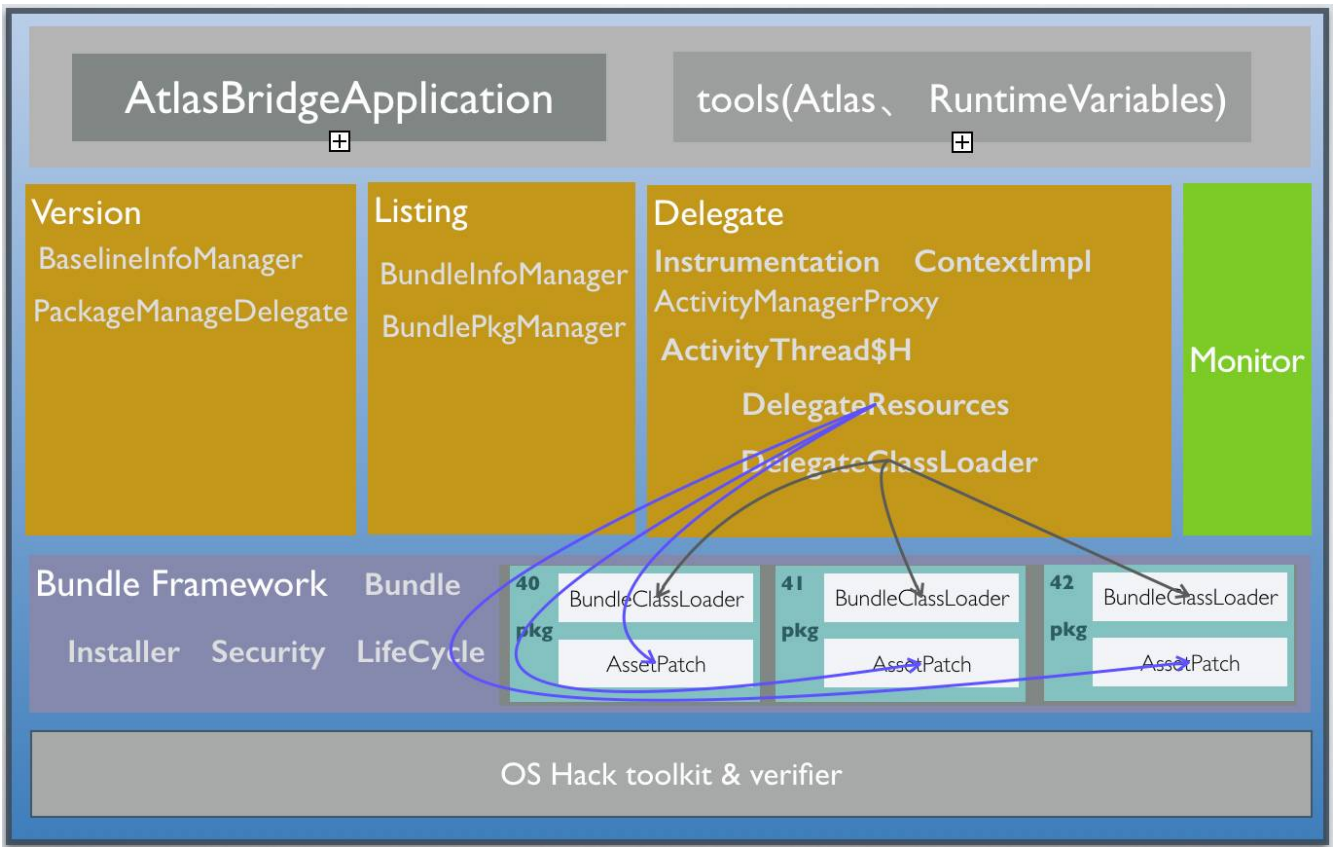


↑新容器化结构设想↑

基于对插件化框架对Android运行机制的理解，参考了OSGI在服务端框架，在开发IDE等领域"高复用、低耦合、可插拔"的优势，我们借鉴了OSGI规范开发了基于组件化的Atlas容器化框架。

下图是基于组件化框架的系统结构

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

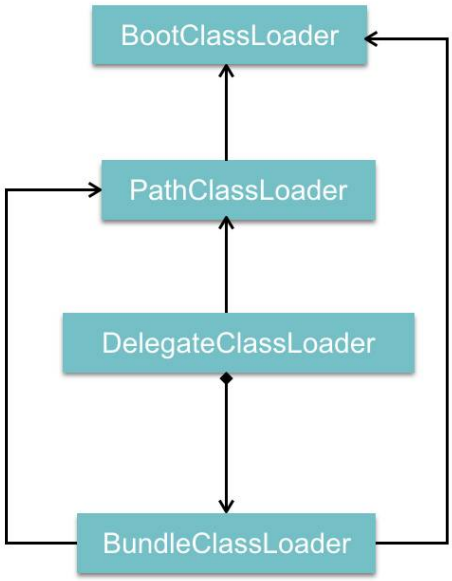


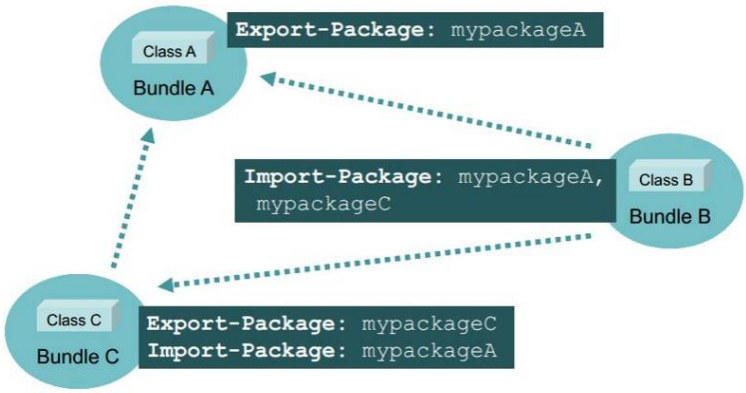
- 最底层的tookit verifier全面罗列了上层需要反射使用的注入和代理的Api，会在应用启动时先进性全局性的校验，以避免程序运行中遇到不兼容的情况；
- 往上Bundle Framework负责组件的安装 更新 操作以及管理所有组件的生命周期,这里组件的边界隔离就遵循了OSGI的规范，每个组件分配独立的classloader，同时组件有各自的资源，每个资源在构建期间由AAPT分配独立的package ID；
- Runtime层 主要包括清单管理、版本管理、以及系统代理三大块：
    1. 版本：每个组件在构建期间就由构建插件分配自己的版本号，同时安装期间也有各自的版本目录，每个bundle的启动加载都需要经过版本的校验，组件发生更新同时也下发最新的版本信息。依托版本管理机制组件的热更新能力水到渠成。
    2. 清单：OSGI规范中每个组件通常通过OSGI.MF来暴露自身的component,这是与Atlas容器所不同的地方。在Android设备上，多文件的形式很容易受IO异常的影响干扰bundle正常运行，所以我们采用了构建期间集中生成清单的方式，清单里面记录bundle所有的component(Android四大组件)，依赖、packagename等内容。
    3. 代理：各个系统关键点的注入使得bundle可以做到按需加载，避免了像原生MultiDex方案由于首次启动时多dex同步安装而造成UI卡顿的情况，代理层的核心DelegateClassLoader负责类的查找和路由，DelegateResource管理所有bundle的资源，它们在容器启动时进行注入，并在运行过程中随着bundle的不断载入进行更新。
- 接入层 简单是美-复杂的东西留给自己。为了方便，Atlas容器有自身独立的Application负责启动，同时在构建期间会由插件替换应用原有的Application。运行期间应用首先由AtlasBridgeApplication负责启动，并在容器启动完毕后全权代理给应用真正的Application；同时对需要自定义和由外部决策的功能，容器开放接口由接入方简单设置。

3.13.3 解耦和依赖

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

依托classloader，组件的独立性有了充分的保证，同时由于BundleClassloader保持了对PatchClassLoader的引用，使得宿主的中间件就如同普通APK开发一样可以被组件简单调用；同时组件之间除了基于AIDL的服务调用，组件之间提供了静态依赖和运行时依赖，相互依赖的组件可以通过配置打成，这在一定程度上使某些偏功能的代码或者UI的重用成为可能，同时配置的方式使组件之间的关系可以清晰追溯。





3.13.4 性能的演进

1. **异步按需**：由于每个组件是一个小型Apk的结构，每个组件安装使其涉及到文件的拷贝、native lib的解压以及校验载入等过程。特别是Service等后台component触发组件安装和前台Activity引发组件安装并行时UI的流畅很受影响。为了降低对UI的影响，每个组件的安装都在一个统一的异步安装线程中进行。Activity、Service、Receiver等的发起都被进行了异步的处理。
2. **解释执行**组件的代码在安装后还需要dexopt才能使用，这一过程在ART上的时间消耗尤为明显，为了降低用户等待时间，Atlas框架对dalivk系统上首次使用bundle时关闭了verify，在ART系统上首次使用时关闭了dex2oat走解释执行，保障bundle首次进入尽可能地高效，同时后台通过异步任务走原生的dexopt过程，为下次使用做好准备。

动态性的增强

1. **远程组件**：插件化时代的按需下载并没有被废弃，独立的远程组件可以满足用户的个性化需求。基于用户的操作和清单机制远程组件在用户需要时只需要简单地等待或者授权就可以从远端拉倒本地。同时当用户设备因为空间紧张时容器也可以清理掉一些长期不用的组件以释放拥挤的空间
2. **动态部署**：动态部署是容器一个最重要的功能。基于此，业务可以灵活发布自己的需求；有故障的业务可以及时修复或者回滚；同时动态部署的快速覆盖能力在灰度等场景下可以更快地收到所需的效果。动态部署的范围也通过改造进行扩大，从最初的组件部署到现在支持除Atlas容器小部分核心代码外的所有代码，部署patch包的体积和性能的改进也在不断的进行改进。

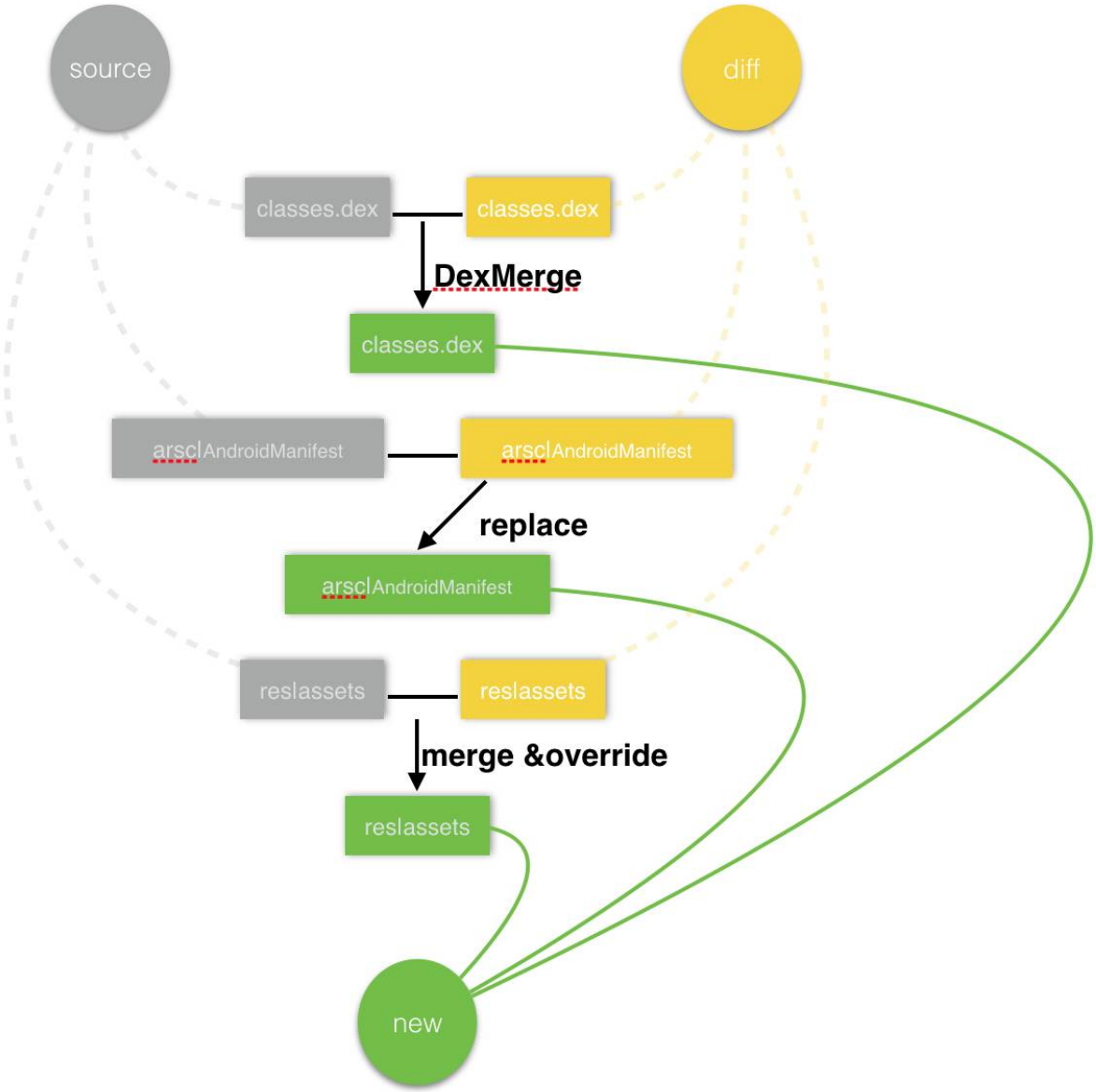
diff

每次大版本的发版，集成平台会保留该次发版的AP（Ap中记录所有参与构建的二方库的版本，组件的依赖，混淆的mapping等各种信息），在手淘接下来每周一次的动态部署迭代中，动态部署的构建会与之前的dex进行字节码级别的diff，生成tpatch包，最终下发到用户手机的patch仅包含变化class组成的dex和更改或者新增的资源文件

Bundle的Merge

patch收到后，客户端的merge过程会根据patch信息，取到source 组件，source dex和patch dex进行dex合并，合成新的dex，同时变化的资源覆盖老的资源最终形成新的组件。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



中间件部署

动态部署能力的增强少不了对底层中间件的支持，部署的机制也与bundle截然不同

1. 中间件代码部署

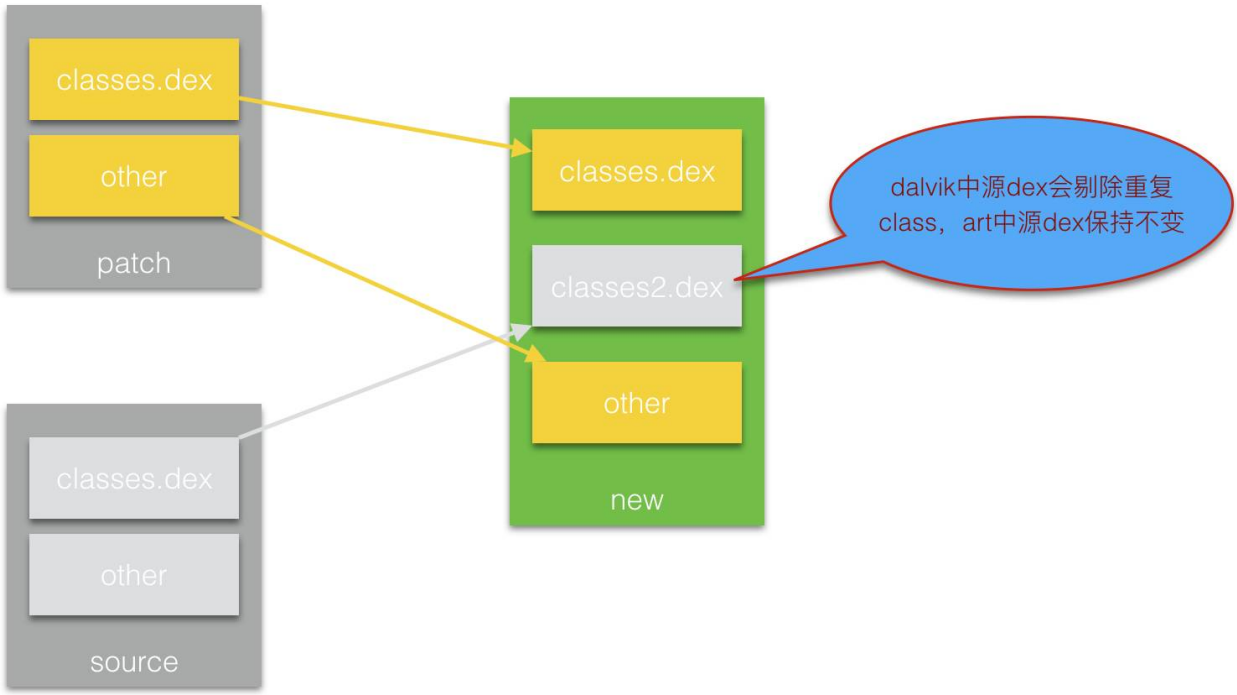
Dalvik和Art在merge过程中都会形成新patch zip，zip中dex以多dex的方式命名，patch dex为classes.dex,原始apk中的classes.dex以此根据序号+1放入zip中，不同的是dalvik由于preVerify的限制写入zip中的源dex会先经过处理，剔除在patchdex中已有的class，确保被patch的dex无法被打上preVerify的标签。

安装过程中则多个dex依次dexopt，同时插入到dexpatch的第一位。而ART由于无preVerify校验，但是Android到ART后原有的dexopt会改为dex2oat.

运行时代码由原来的解释执行改为直接运行native代码，之前的使用过程中发现单纯得往前面追加patch的dex并不能完全解决动态部署的问题，dex2oat的过程优化了class的执行代码，比如说内敛，虚函数的校验等。就有可能在运行过程中直接在native层执行老的优化过的class代码而不是从新patch的dex中load新的class去使用。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?tid=8674704361863085

所以在ART上类似Dalivk，放入源dex，只是源dex不会做任何处理，然后合并在一起做一遍dex2oat，这样能使新的class覆盖老的class参与dex2oat并完成优化的过程，

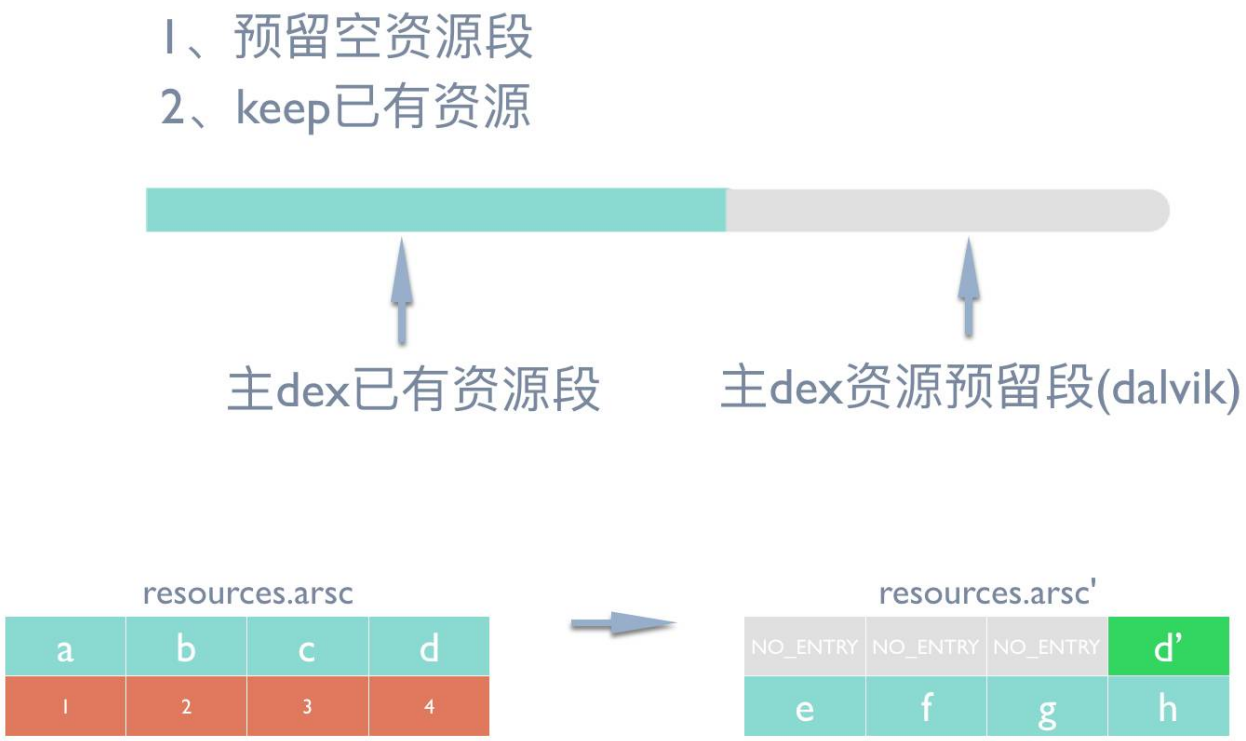


2. 中间件资源部署

资源部署基于overlay机制达成，不过dalivk上这个机制并不支持新增资源，在overlay的包里面如果读到了一个资源，dalvik系统会去校验该资源ID在base中值，如果不存在则抛错，所以动态部署为了利用这一特性同时支持新增资源的需求，在打基线包的时候就在每个不同的type里面预留了128个资源ID供后续动态部署使用。

同时打动态部署的patch时会以之前的ID分配的内容作为输入，保证已有的资源分配到的ID保持不变，同时如果资源没有发生变更，则剔除该资源，所以aapt dump resource patch的时候我们看到的INVALID TYPE 的资源段就表示没有发生变更的资源，如果有发生了变更，且之前有这个资源，就会在原有的资源ID分配过去，同时新的资源文件打入patch包或者新的资源文本写入arsc，如果是新增的资源，则使用预留的资源段进行分配。整个替换过程如下图所示

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



Atlas动态部署的能力随着手淘的需求不断进行着演进，随着整个框架的开源，自然需求会变得更加广泛。比如在手淘内部一周一发版的节奏下，动态部署对新增component的某些需求并不强烈，而开源后此类功能可能会尤为重要，所以接下来容器的更新迭代中此类的功能也会及时上线。拥抱开源社区会是Atlas重新审视自己，继续发展的关键一步。

3.13.5 未来的路

相比于iOS，Android的推陈出新的速度是极快的。Google在Android系统的改进深入到方方面面，组件化的发展也需要紧跟Android步伐。

Atlas起步的时候MultiDex还未出来，解决方法数的问题就能带来足够的愉悦。现在Atlas基于组件化减少主dex方法数的同时，也支持主dex原生MultiDex的能力。目前Atlas内部集成了MultiDex的能力，外部只需要打开MultiDex开关即可。这么做的目的一方面避免外部初始化实际的误解，另一方面我们也意识到原生的mulitdex在性能上还有很多优化的点，我们会在后面及时进行优化。

在开发调试方面，Atlas容器框架基于动态部署的能力支持单个组件的独立编译调试，但是相比instantrun的速度，目前还有很大提高。而基于Instant Run改造的单组件秒级编译也已经在紧锣密鼓的开发中，后续基于框架的独立调试插件也将会在Android Studio插件库中上线。

Android系统机制的演进也鞭策着动态部署前行。比如16年Android 7.0的混合编译当时也给Atlas造成了不小的挑战，服务于亿级的UV，动态部署的能力的覆盖面、到达率都还有进一步提升的空间，相比andfix等热部署修复方案，动态部署需要重启来打到更新的能力可能也需要有新的转变。

市面上许许多多的容器框架，基本思路就大致两三种。Atlas本身也没有特别新的技术，有的是基于已有的知识进行不断的重组和优化。在手淘内部，追求更高的稳定性和性能，亿级的UV需要及其稳定的容器来承载，同时Atlas也不断尝试新的方案，在创新与实用之间不断权衡。

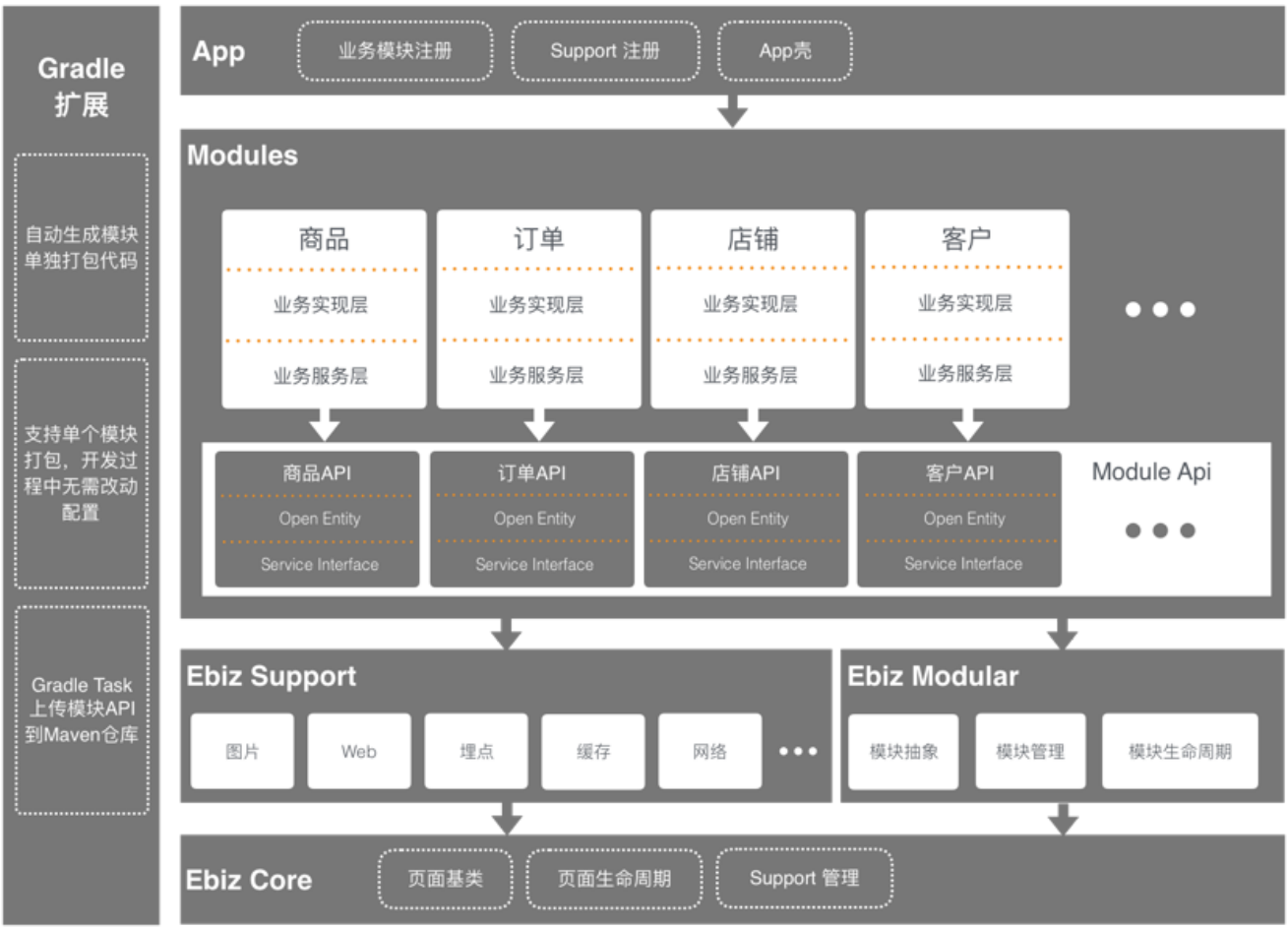
Atlas是一个结构简明的容器框架，通过尽量简单的思路去解决移动应用从开发到运维中的的桎梏，让开发中的复杂简单化，让维护中的复杂简单化，让开发从流水线上忙碌的工作回到专注于对最终目标的精耕细作，抛开嘈杂的干扰，拨开虚无缥缈，到达云和山的彼端！

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

3.14 有赞微商城

组件化改造概述

- 提出了 5 个调整方向：抽象基础模块、公共服务去中心化、业务模块服务化、抽象基础组件、单/多模块打包
- 介绍了基于 3 个基础组件依赖和 1 个 Gradle 插件的落地方案



3.14.1 有赞移动应用如何给页面安上“任意门”

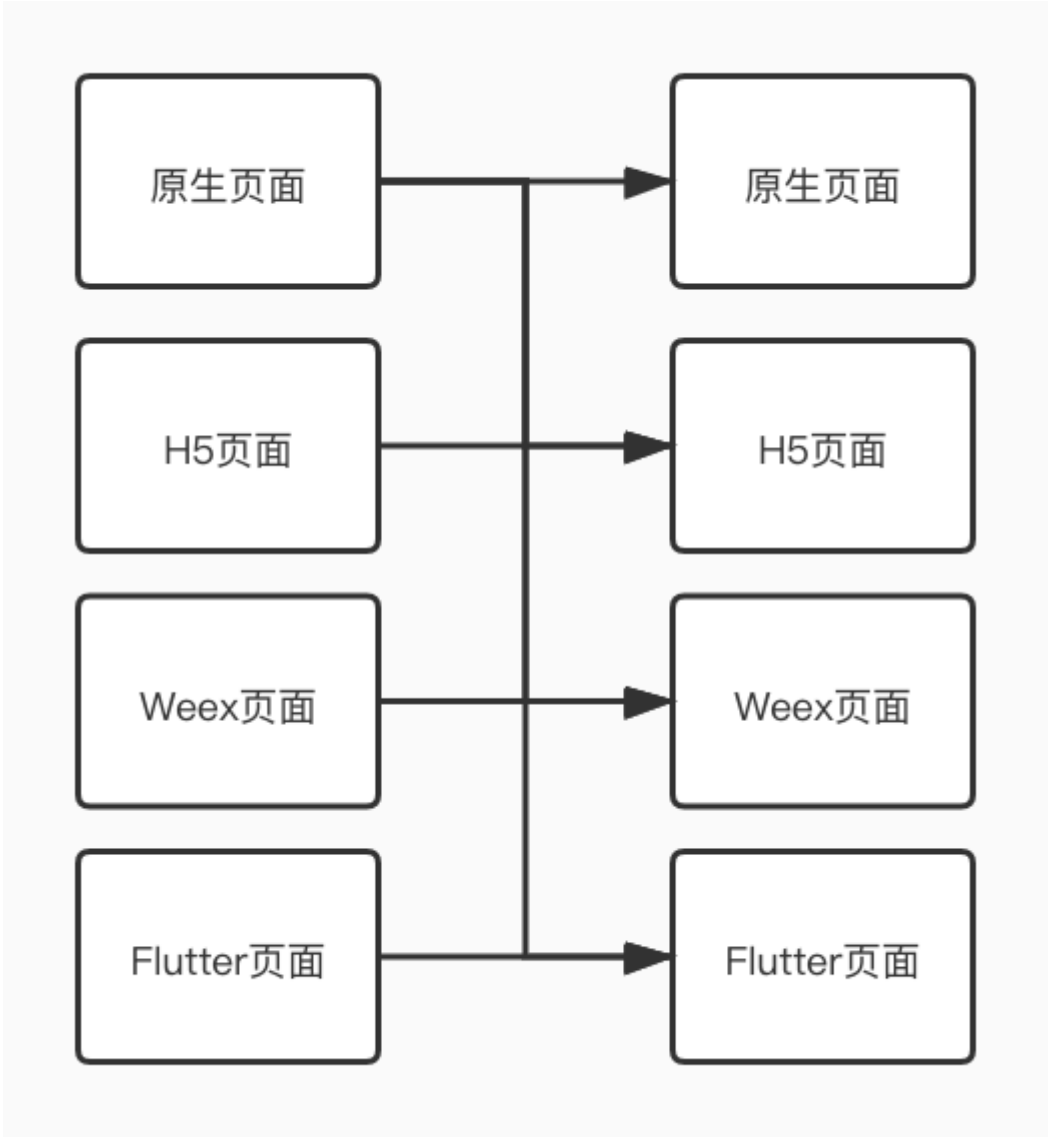
“任意门”：一行配置实现页面跳转重定向。

3.14.1.1背景 & 痛点 & 价值

动态路由组件，处理的是 App 中最最常见的一种行为的问题，那就是：跳转。

随着 App 技术栈的扩展，从原本最最简单的原生到原生的跳转，扩展到目前同一个 App 中包含原生页面、H5 页面、Weex 页面、Flutter 页面之间的跳转。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



随之而来的问题就是：随着 App 的版本迭代，很多原本原生实现的页面，需要通过新的 H5 或者 Weex 页面进行升级/降级。而这些原本都是硬编码的跳转逻辑，可能需要随着版本不停改动。总结下来，现有的，各个技术栈隔离的页面跳转逻辑面临的直接问题有：

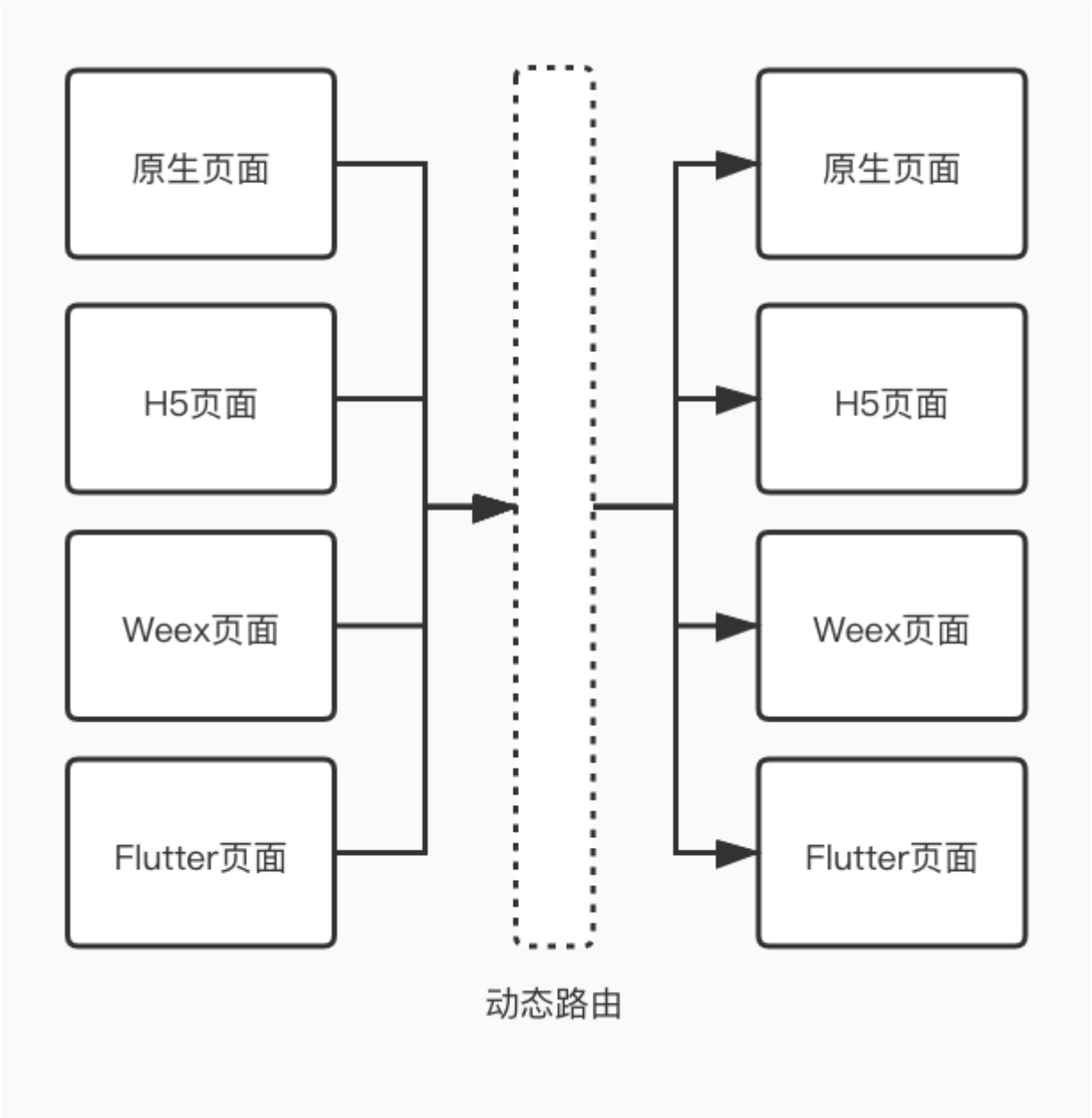
- 跳转逻辑跟着版本走，无法统一进行改动
- 跨技术栈跳转的实现成本比较高，必须在桥接模块中进行特殊适配
- 在一些 H5 需要使用专门 WebView 页面打开的场景下，很难去适配，也必须通过各个 Web 跳转的拦截做特殊处理

为了解决以上硬编码以及灵活性差的问题，我们决定梳理现有的各技术栈跳转逻辑，将这些跳转整合，能够满足动态性、可配置的需求。

得益于项目中原有的路由跳转组件，各种页面之间的页面都可以通过 URL 的方式进行路由，于是我们基于 URL 跳转，开发了一套动态路由组件，它完成的工作有：

- 承担 App 内所有跳转逻辑
- 通过配置中心组件，支持获取/配置路由替换规则
- 匹配所有的路由跳转规则，命中规则的，替换成新的目标路由地址
- 将实际跳转目标地址传递给路由组件执行实际的跳转行为

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



3.14.1.2实现方案

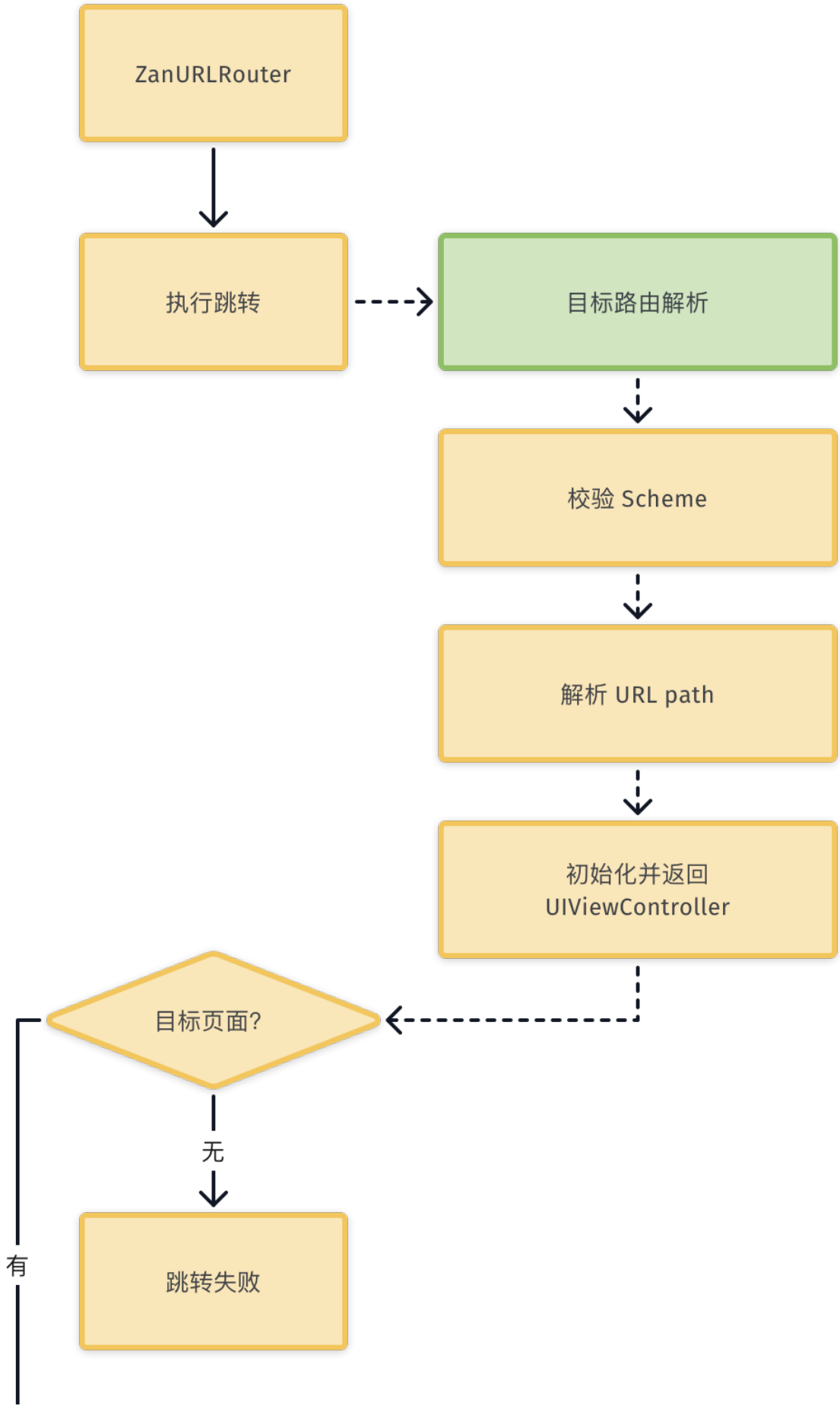
路由拦截+替换

微商城客户端目前已经有一套稳固的组件化实现方案，组件之间的页面跳转通过路由的方式进行解耦，这是一种比较常见的方式。

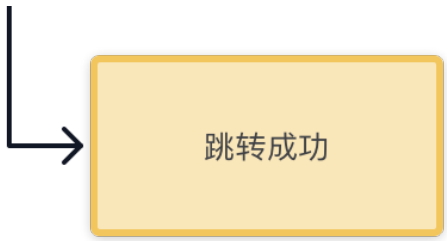
在微商城项目中，负责实现的路由组件为 `ZanURLRouter` ，它的职责很简单：

- 启动时注册路由和页面
- 找寻正确的页面进行跳转

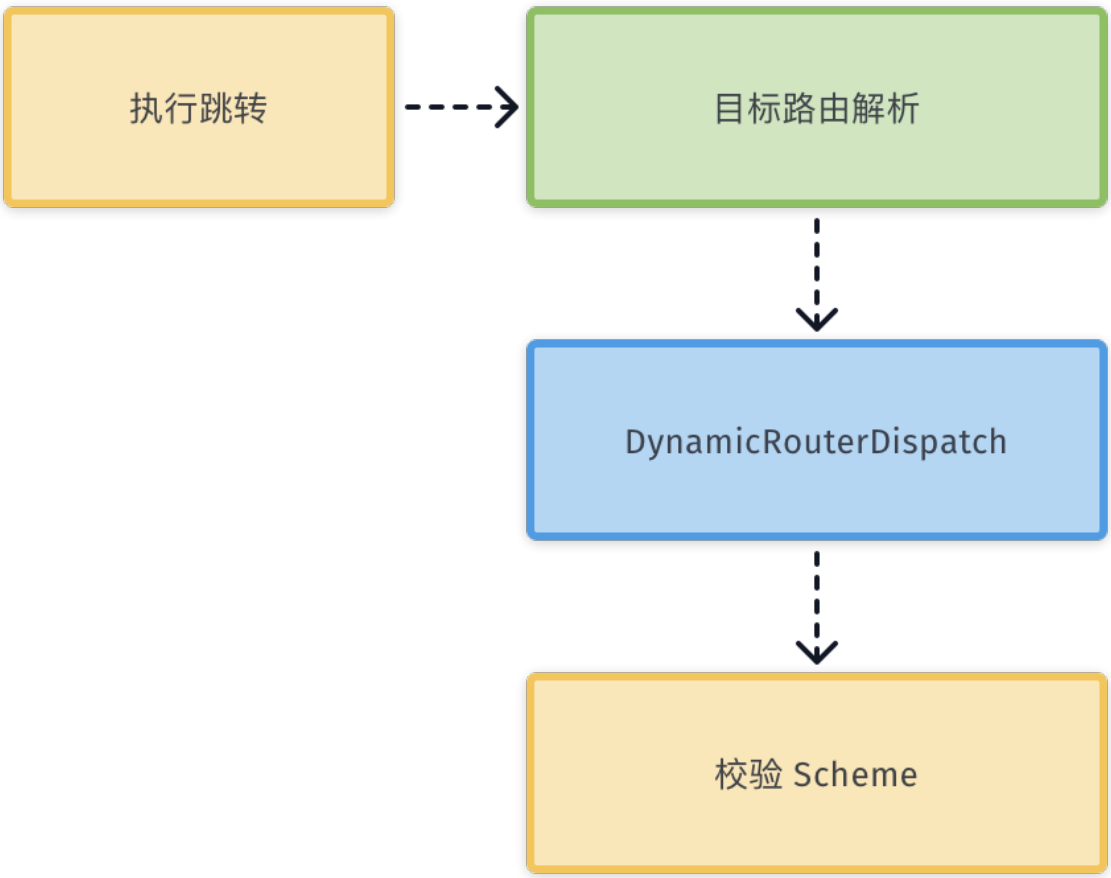
腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



在不影响外部接口的前提下，我们在目标路由解析这一步，引入了动态路由



对于移动端的路由重定向，实际上就是将一个路由转换为另一个路由，如：

```
youzan://orderlist?type=1&status=2
```

转换为：

```
wsc://orderlist/v2?type=1&status=2
```

跳转规则配置

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

路由的拦截和替换中的一个关键节点就是“配置”，我们需要一个路由规则列表来记录和下发匹配规则。为了方便下发路由规则表，我们将这份配置表存放在有赞移动配置中心，根据客户端的版本进行区分，动态地下发给不同版本的客户端。

一条路由规则，分为一个 Key 和对应的 Value，Key 为匹配方式，使用正则表达式进行匹配，Value 为替换方式，使用 JSON 格式定义。

实际代码实现中，我们将“路由规则”和“路由替换行为”分别抽象成实体类和接口方法。

抽象实体类

关于替换路由跳转的规则，我们可以这样配置：

```
Key: ^youzan://orderlist\?type=(\d+)&status=(\d+)$
Value: {"template": "wsc://orderlist/v2?type=$1&status=$2"}
```

即：一条匹配规则 + 一条替换模板。我们将之抽象为一个实体类，`Rule`：

```
class Rule {
    // url 匹配规则（正则表达式）
    String pattern;
    // url 匹配规则（正则表达式）
    String template;
}
```

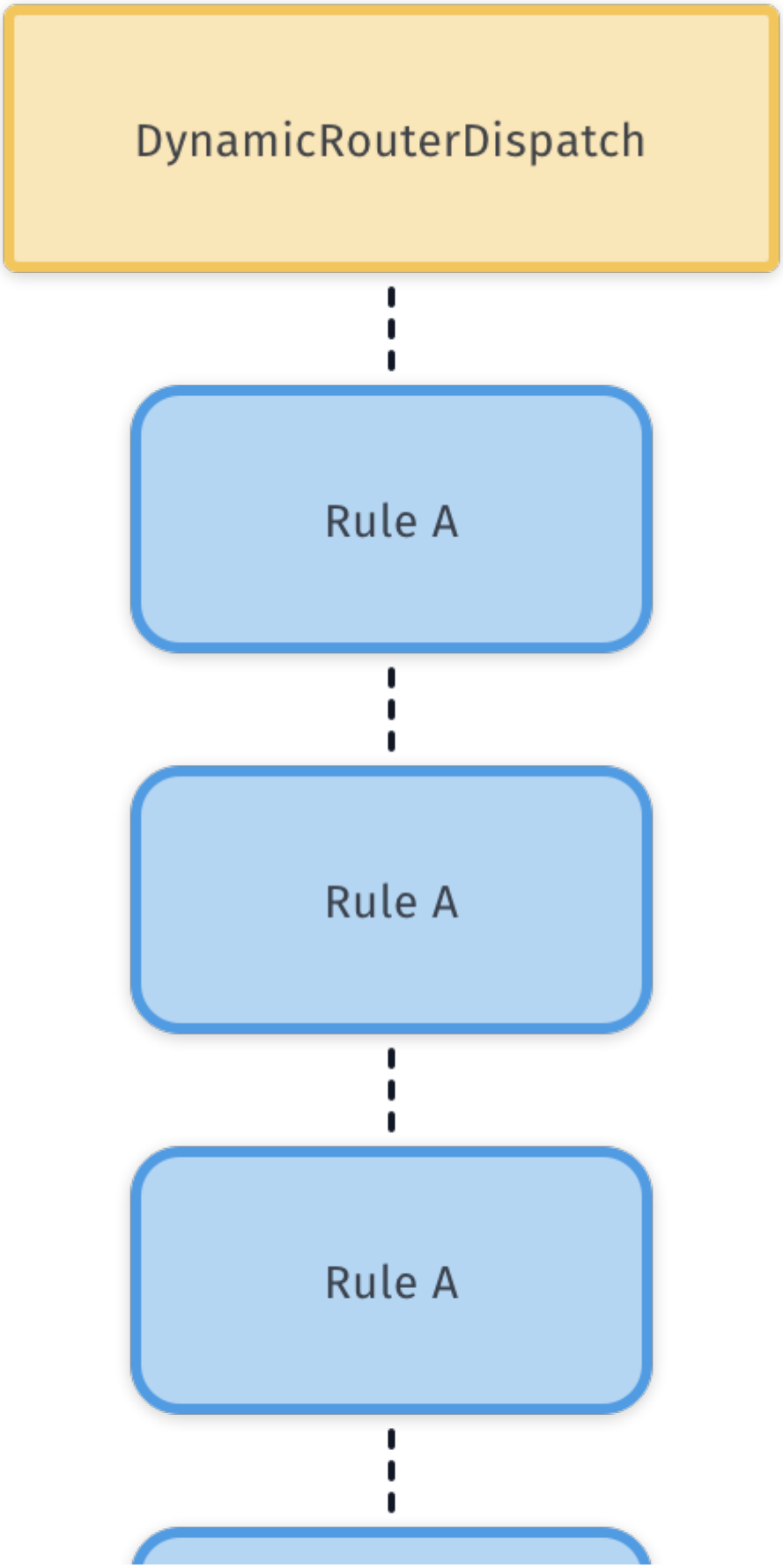
抽象接口

有了规则配置之后，就需要对动态路由的行为进行抽象，核心就是初始化规则、匹配规则和替换路由三个方法：

```
// 注册替换规则
fun initWithPattern(Rule rule)
// 校验是否命中已经注册的路由配置的 pattern 正则
fun testWithRoute(String routeUrl): Boolean
// 获取替换后的跳转地址
fun appliedWithRoute(String routeUrl): String
```

动态路由器会在应用启动阶段拉取正确的规则表，解析并记录下来：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

在 `ZanURLRouter` 解析目标路由的时候，对每一个规则进行匹配测试，命中则应用匹配的规则，返回替换后的路由，再继续接下来的工作。

路由替换

实体类、接口类都抽象完成之后，就是动态路由的核心实现了，这里依赖到一个的核心工具就是：正则表达式。这里用到正则的场景有两个： - 正则验证是否命中规则 - 正则替换url文本

在 Android 和 iOS 开发中，字符串正则相关的 API 都是自带的，开箱即用：

```
/* ------------ Android ------------ */

// 正则匹配校验方法
Pattern.matcher(String text)
// 正则匹配校验方法
Regex.replace(String input, String replacement)

/* ------------   iOS   ------------ */
(NSString *)stringByReplacingMatchesInString:(NSString *)string options:
(NSMatchingOptions)options range:(NSRange)range withTemplate:(NSString *)templ;
```

疑难问题：参数处理

大部分情况下，跳转本身都是带参数的，那么动态替换跳转的 URL 之后，参数的获取就成了一个问题，尤其是原生和其他页面页面的跳转。

我们主要以 Android 为例，Android 原生跳转都是通过一个关键类：Intent 来实现参数的存取。这里需要注意的是，由于 Intent 传值存在多种复杂的数据接口，包括 Parcelable 这种复杂参数的场景，由于降级之后都是以 URL 的形式传值，所以我们目前约定动态路由的参数只支持基本数据类型，复杂参数类型的需要接入方来做兼容。

参数处理我们分两个典型的场景来讨论： - 原生跳转 H5 参数传递 - H5 跳转原生的参数传递

原生跳转H5

这里的方式主要是将 Intent 中的基本数值类型参数取出来，拼接成带参数的 URL 来实现将 Intent 里面的参数传递给 H5，主要实现代码如下：

```
fun appendBundleParams(strBuilder: StringBuilder, bundle: Bundle) {
    val ketSet = bundle.keySet()
    for (key in ketSet) {
        bundle[key]?.let { value ->
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
            when (value) {
                is Bundle -> appendBundleParams(strBuilder, value)
                is String, is Int, is Long, is Double, is Float, is Char, is Boolean, is
Short
                -> {
                    if (strBuilder.isNotEmpty()) {
                        strBuilder.append("&")
                    }
                    strBuilder.append("$key=$value")
                }
                else -> {
                    // do nothing
                }
            }
        }
    }
}
```

H5跳转原生

同理的，H5 跳转原生做的就是将 URL 中携带的参数塞到 Intent 中来进行。

这里比较关键的一个问题是：Intent 的取值都是带类型的，而 URL 的参数都是字符串。我们目前解决方案也很简单，就是封装 Intent 的取值方法，由于目前有赞 Android 主要使用 Kotlin 来开发，可以使用 Kotlin 的扩展函数特性来实现（Java 可以使用工具类的方式）：

```
fun Intent.getIntFromRouter(key: String, defaultValue: Int): Int {
    val extras = this.extras;
    if (extras == null || !this.hasExtra(key)) {
        return defaultValue
    }
    return extras.get(key) as? Int ?: (this.getStringExtra(key)?.toInt() ?: defaultValue)
}
```

碰到的坑：UrlEncode

在匹配和替换 URL 规则的场景中，我们经常会碰到这么一种情况，URL 是被 UrlEncode 过的。由于字符串的正则匹配和正则替换是不会判断字符串是否被 UrlEncode 过，所以这里的逻辑需要由路由组件来实现。

UrlEncode 字符串的正则匹配逻辑实现比较简单，即直接将字符串 Decode 之后进行匹配。

比较复杂的是 UrlEncode 字符串的正则替换，有些情况下，路由中的url是必须进行 UrlEncode 的，如果直接 Decode 进行替换，那么可能会导致实际跳转的目标 URL 被错误地截断，导致无法跳转，所以这里的替换必须保留 UrlEncode 的字符。

我们的解决思路是：记录 URLEncode 前后被 encode 字符的下标，然后再手动实现 replace 方法去挨个替换字符串中的字符，核心代码如下：

```
private fun getEncodeCharMap(url: String, encodeUrl: String): Map<Int, IntRange> {
    if (Uri.decode(encodeUrl) != url) {
        return mapOf()
    }
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
    val urlChars = url.toCharArray()
    val urlEncodeChars = encodeUrl.toCharArray()
    var i = 0
    var j = 0
    val encodeMap = mutableMapOf<Int, IntRange>()
    while (i < urlChars.size && j < urlEncodeChars.size) {
        // text:   [www:] => [www%23]
        // length: [0123] => [012345]
        if (urlChars[i] != urlEncodeChars[j]) {
            val s = Uri.encode(urlChars[i].toString())
            val range = IntRange(j, j + s.length - 1)
            encodeMap[i] = range
            j += s.length
        } else {
            j++
        }
        i++
    }
    return encodeMap
}
```

3.14.1.3 实际应用案例

应用中心

微商城App应用中心，应该是应用动态路由的最佳场景，应用中心存在大量跳转的场景。

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

先来说下使用动态路由的背景，应用中心中应用列表都是由服务端统一下发的，后端为每个应用配置的跳转地址是统一的，而 Android 和 iOS 本地路由配置的 URL 是不一致的，如果直接下发配置的话，会存在有一端无法跳转的问题。以店铺管理应用跳转为例：

- iOS中店铺管理的路由 URL：wsc://shop/management
- Android 中的路由URL：wsc://team/management
- 服务端下发的URL：wsc://team/management

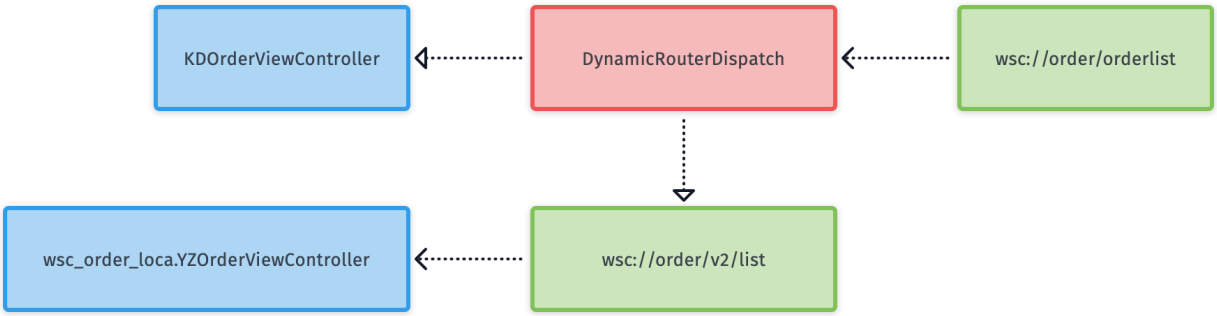
那么解决同一套配置跳转不同 URL 的这个问题，就交给动态路由来完成了，我只需要在iOS的动态路由添加一个规则，将 wsc://shop/management 动态替换成 wsc://team/management 就可以搞定！

订单项目

在微商城客户端的订单模块重构项目中，考虑到订单是使用频次很高的核心场景之一，且代码历史较久，所以新的模块上线后与旧订单列表模块共存，直到灰度完全结束。

由于微商城已经是组件化拆分，业务组件之间的跳转使用路由完成，我们在设计灰度方案时，利用动态路由来实时进行目标路由的映射：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



具体可见《微商城订单模块重构实践》一文。

3.14.1.4 总结

“上线只是开始”，随着业务迭代，历史业务也越来越多，为了保证不同平台版本的用户能够平滑过渡到新的功能上去，动态路由组件扮演了一个客户端的 URL 重定向服务的角色，避免因服务下线、功能更新、平台差异、项目重构等原因导致的功能不可用。

动态路由组件，核心就是非常简单的正则匹配和正则替换，而这个非常简单和核心代码逻辑，实现了业务场景下非常重要的路由重定向。这整套解决方案，也是有赞移动端在应用组件化、动态化的一个重要组成部分，我们也希望这个技术方案能够抛砖引玉，启发更多优秀的移动端动态化解决思路。

3.14.2 有赞微商城-Android组件化方案

3.14.2.1 概述

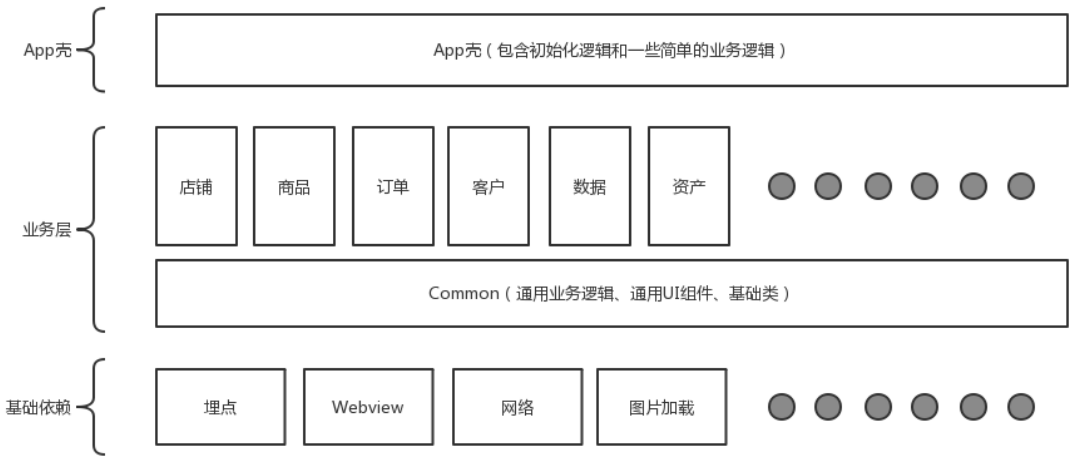
目前有赞移动端的主要工作内容是在“有赞微商城”和“有赞零售”两条公司主要的业务线，随着有赞 Saas 业务的增长，客户端也不断迭代，支持越来越多的功能。 在这个业务快速增长的情况下，移动端技术的整体架构也是一直在不断调整，来保证开发效率和业务的快速迭代。

这篇文章，主要是介绍有赞微商城 Android组件化的一些思路和实现。

3.14.2.1.1 现状

客户端的架构，从一开始的“All IN ONE” 模式（即所有代码都在 App 中），逐渐演变到目前的一个单 Project 多 Module 结构：

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



3.14.2.1.2 痛点

新的项目架构，也带了了新的问题： - 日益复杂的 Common 模块，逻辑复杂，依赖不清晰，不敢随便改动 Common 代码，造成大量冗余代码和无法维护的业务逻辑 - 随着业务模块的增多，打包速度一发不可收拾；从倒杯水的时间到下楼吃个饭的时间，大大减慢了开发节奏 - 由于业务模块跟项目中的上层（App 壳）和下层（Common 模块）耦合 - 业务模块增多，由于业务模块没有自己的生命周期，无法实现模块之间的隔离，整体模块控制比较混乱

3.14.2.1.3 需要解决的问题

- Common模块轻量化，需要将 Common 层的业务向上抽离，通用的底层和基础组件、公用 UI 组件抽成单独的依赖
- 移动端业务服务化，解耦现有业务，抽象出业务接口，业务模块只向外暴露自己的接口，并实现跨模块之间的调用
- 能够配置单模块或者多模块打包，不用每次调试都全量打包，费时费力，又影响开发的节奏
- 业务模块的依赖和发布管理

3.14.2.2 架构调整

我们之前虽然有在做整个工程模块化的开发，但是目前的模块化框架可以说是不够彻底的：

- 模块只是项目结构的概念（一个模块一个 Module），在逻辑层并没有模块这个概念
- 模块本身并没有生命周期控制
- 公用服务中心化，公用逻辑部分全部都在 Common 模块中
- 模块对外暴露的服务不可知，都是直接依赖模块内部的代码逻辑
- 模块无法单独打包，针对模块的代码改动，只能全量打包之后才能看到效果

为了解决以上的问题，我们需要对现有的架构进行调整。

3.1.2.2.1模块的抽象

将模块的功能抽象出一些基础类，组成了模块化支持组件，它提供的功能有：

- 抽象出模块本身作为某一类业务的容器，即所有业务模块需要实现自己的模块类，继承自我们的 BaseModule，并在 App 壳工程中进行注册
- 模块对象跟 Activity 一样，拥有生命周期的概念，需要在生命周期的不同阶段处理自己相应的逻辑（注册服务、初始化数据等）
- 模块可以注册的对外暴露的服务的实现，在注册模块的时候，模块携带的服务也会被注册到 App 的服务中心

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

3.14.2.2.2 公共业务去中心化

跟很多客户端的同学聊过，很多 APP 发展到一定阶段之后，必然会诞生一个所谓的 Common 模块。它就像一个大储物柜，每个人都把一些其他人可能用到的东西一股脑儿塞进去。 这么个塞法，会有两个问题：

1. 冗余：比如一些工具类，很多时候，当你找不到需要的工具类的时候，你可能会塞一个新的进去
2. 维护成本高：所有公用的业务逻辑的实现都在 Common 中，对一些公用业务逻辑的影响面无法掌控

2.2.1 Common 里面都有什么？

- 工具类
- 公用的 UI 组件
- 多个业务模块都公用的业务类
- 基础组件的封装类（图片库、网络库、Webview）
- 封装的一些基类（BaseActivity，BaseFragment 什么的）

2.2.2 解决的思路

- 将公用的业务模块向上抽离到业务模块中（所谓业务模块的服务化）
- 将基础组件抽象到一个独立的组件中
- 将一些基础类下沉到不包含业务逻辑的底层核心库中

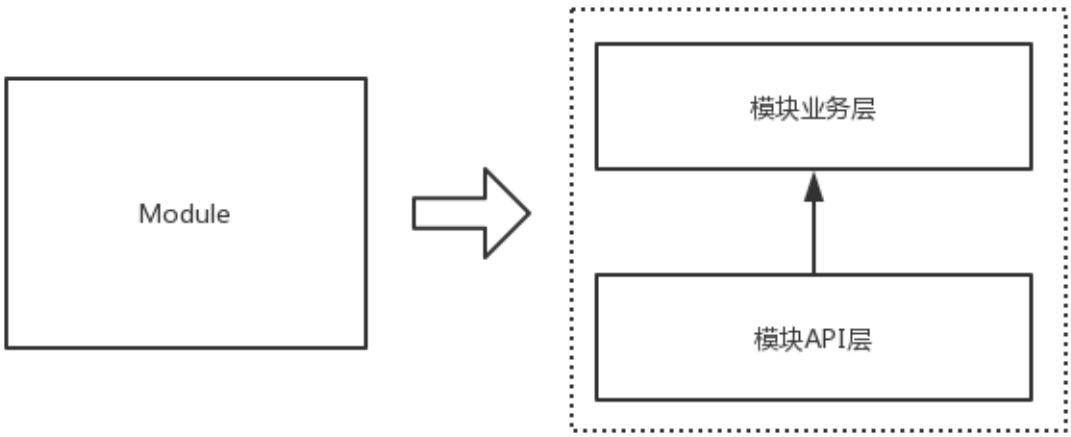
3.14.2.2.3 业务模块服务化

“服务化”这个词，在服务端的开发中经常被提到，简单来说，就是根据业务划分为多个模块，模块之间的交互以互相提供服务的方式来完成。 而客户端随着业务模块的增多，也必然存在业务模块之间存在业务依赖的情况，而 Android 端依赖的方式有：

1. A 模块直接依赖 B 模块，直接调用 B 模块的代码逻辑
2. 将 A 和 B 模块中的公用部分放到 Common 模块中，通过调用 Common 模块的代码实现依赖

2.3.1 业务模块服务依赖的实现

- 后端的服务化是借助于 Dubbo 来构建的 RPC 服务，依赖某个服务，只需要依赖其对外暴露的 API 模块（只包含接口和数据结构的 Maven 依赖），不需要依赖其具体实现，具体服务调用的实现由框架来实现
- 客服端的依赖也可以参考这样的方式来实现模块之间的依赖，例如商品模块，可以提供一个 API 层，用来对外暴露数据结构和服务



腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

2.3.2 API 层实现方式

对外暴露服务的方式有很多种：

- 协议的方式：例如"app://order/detail/get?id=100"，数据可以用 JSON 来进行传递，请求本地服务就像调用一个 Http 服务一样，根据请求协议来获取数据，然后解析数据进行操作
- 接口的方式：像后端使用 Dubbo 服务那样，订单模块对外提供一个独立的 Maven 依赖，里面包含了数据接口和对外提供的服务接口，适用方依赖之后直接调用

2.3.3 接口的方式实现 API

协议的方式的问题：如果服务提供的地方更改了之后，需要手动去查询所有调用到的地方，进行更改，而且没有版本管理，而且数据解析都需要手动进行转换，改动的成本比较高，也有一定稳定性风险。 接口的方式的问题：需要额外提供一个依赖（单独把 API 层打包成一个 aar 包），使用方需要添加 Mave 依赖，所以引入依赖和发布的成本比较高。

我们最终选择了接口的方式，这种方式的稳定性和版本控制做的更好，对于改动来说，编译过程自动会帮你校验改动的影响面，而引入依赖和发布成本高的问题，完全可以交给构建工具（Gradle Plugin）来解决。

2.3.4 业务实现层

业务实现层需要做的，就是实现自己模块本身的业务逻辑，并实现自己提供的 API 接口，暴露对外的服务。

3.14.2.2.4 基础组件抽象

2.4.1 现有的基础组件实现

项目中现在有很多的基础组件都是统一在 Common 里面进行封装的，例如：账号库、网络库、图片加载、Web 容器库等等，这也带来了一些问题：

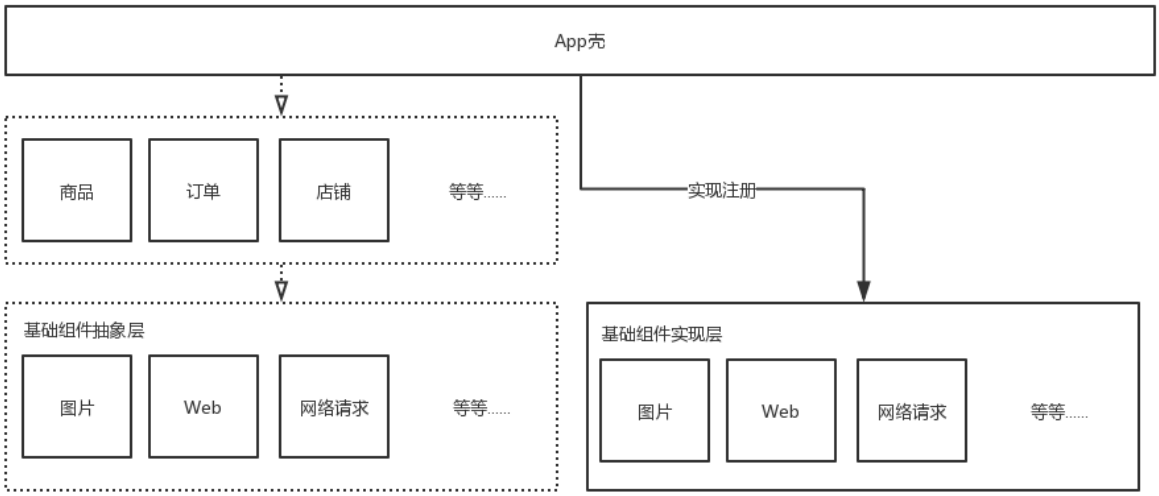
1. Common 太重
2. 业务模块跟基础组件强耦合，在开发一些跨团队的组件过程中，如果碰到使用的基础库不同的时候，需要比较多的时间来做封装
3. 升级基础组件或替换依赖的成本比较高，一些 API 的更改需要改动每个调用的地方

2.4.2 实现思路

- 将常用的基础组件整理，抽象成单独的一个抽象层，里面定义了一系列基础组件接口（图片加载、Web 容器、JsBridge 调用、账号等等）
- 把统一实现的组件放到另一个依赖里面，可以在 App 中进行具体实现的注册，而业务模块本身，可以只依赖抽象

2.4.3 依赖结构

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



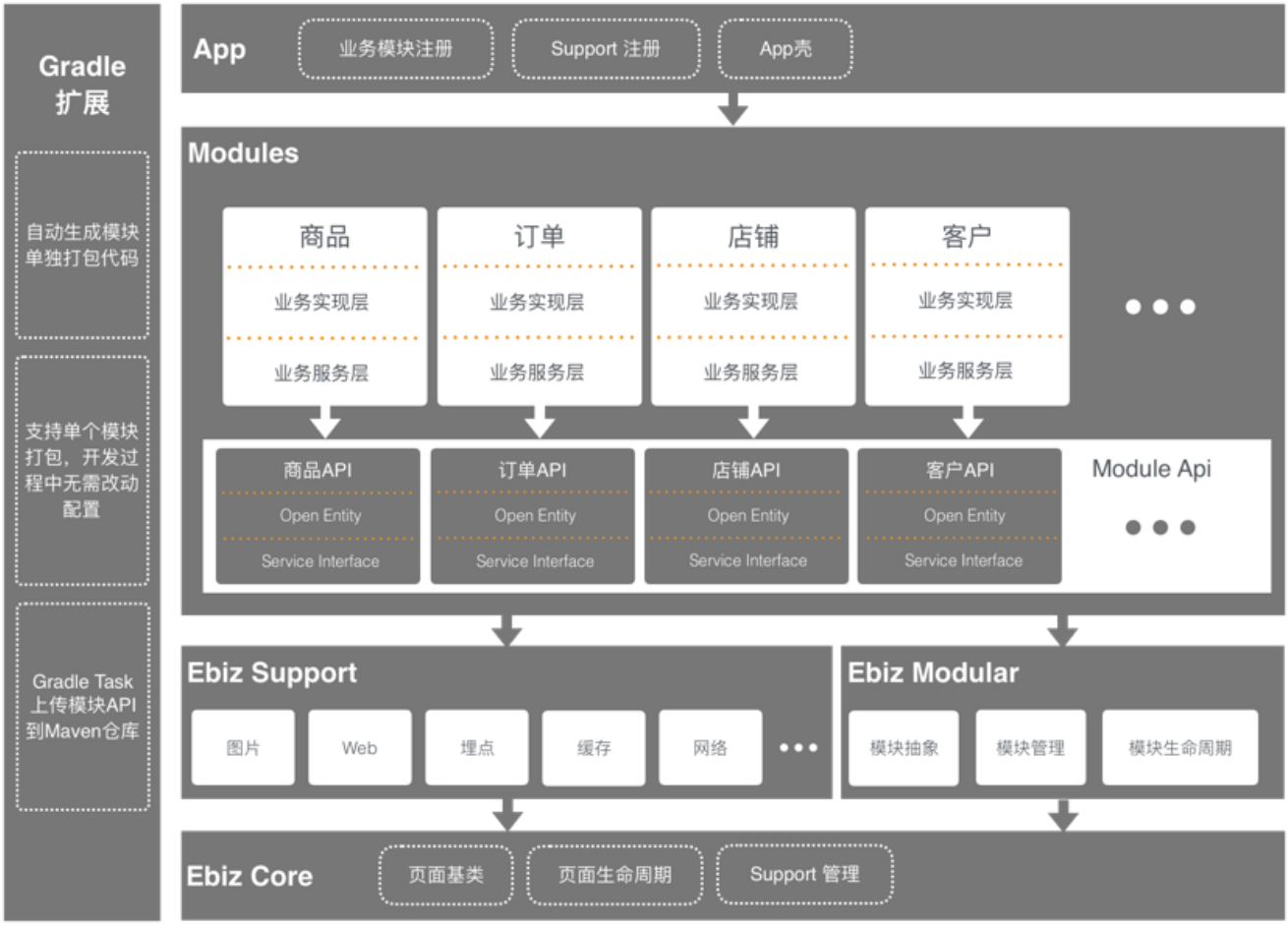
3.14.2.5 单/多模块打包

随着业务量和业务复杂度的增长，还有多个三方组件的引入，客户端工程代码量也变得越来越庞大，直接造成的一个问题是：打包慢！一个简单的场景：当你开发了一个商品模块内部的功能之后，你需要打整个 App 的包才能进行测试，而打一个包的时间可能是 5 ~ 10 分钟，如果一天打包 10 次，也是比较酸爽。我们的组件也需要支持单模块或者选定的某些进行打包，其中的思路也是通过自定义 Gradle Plugin 在编译阶段，动态去更改 Module 实际依赖的 Android Gradle 插件来实现的。 经测试，同一台电脑，完整打包（clean之后再安装）耗时 4 分钟，而单模块打包（同样也是 clean 之后安装）耗时 1 分钟，整体打包时间降低了 70% 以上。

3.14.2.6 架构图

上面的一些改进点，总结成一张图，就是这样的:

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085



3.14.2.3 实现方案

目前我们的方案提供 3 个基础组件依赖和 1 个 Gradle 插件：

- modular-core: 提供组件模块化生命周期和模块服务注册相关的模块化基础组件
- modular-support: 对项目中二方、三方包接口的抽象
- modular-support-impl:对项目中二方、三方包接口的默认抽象
- modular-plugin: 支持模块生成 API 层目录，生成 APP 运行环境，以及管理模块发布的 Gradle 插件

3.14.2.3.1 Modular-core

3.1.1 实现模块类

业务模块类需要继承 BaseModule：

```
public class ModuleA extends BaseModule {

        @Override
        public void onInstalled() {
            registerBusinessService(ModuleAService.class, new CachedServiceFetcher() {
                @Override
                public ModuleAService createService(@NotNull ModularManage manager) {
                    if (service == null) {
                        service = new ModuleAServiceImpl();
                    }
                    return service;
                }
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
            });
        }
    }
```

3.1.2 模块生命周期

模块有以下几个生命周期：

- onInstalled() -> 模块被注册的时候调用：Module 在 App 中被注册的时候
- onCreate() -> 模块第一次启动的时候调用：Module 所属的某个 Activity 第一次启动的时候
- onStart() -> 模块启动的时候调用：模块第一次启动之的时候
- onStop() -> 模块停止的时候调用：Activity 栈里面没有模块所属 Activity 的时候

3.1.3 模块生命周期的实现

其实组件内关于生命周期捕获和监听，都是借助于 Google 的 Android Architecture Components 中的 Lifecycle 库来实现的。

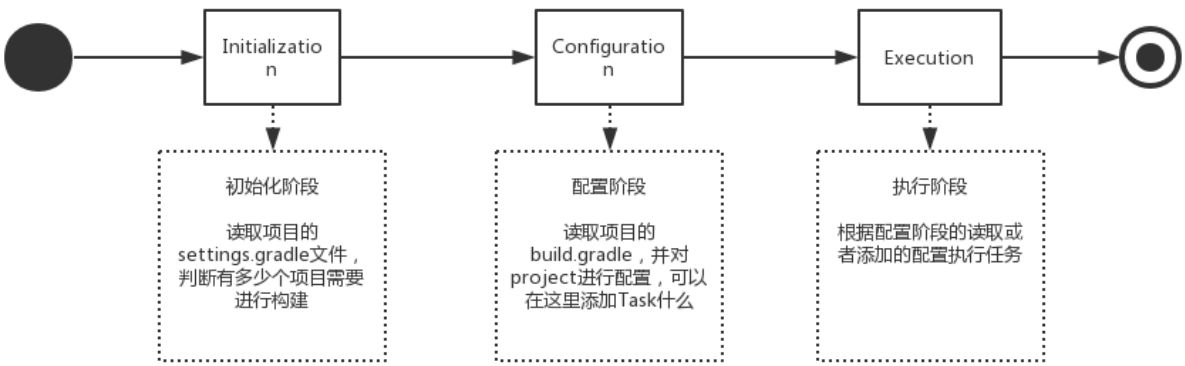
- 模块生命周期的捕获：首先需要将 Activity 的类注册到 Module 中，然后全局监听 Activity 的 Lifecycle.Event 事件，就可以获取到模块内 Activity 的运行情况
- 模块生命周期的监听：BaseModule 本身继承了 LifecycleOwner 接口，可以对其添加观察者，来实现对模块生命周期的监听

3.14.2.3.2 Modular-plugin

这里需要依赖对于 Android 的构建工具 Gralde 的扩展，它支持的高度可扩展特性，帮助我们在组件化开发中更加高效，不需要关系一些额外的工作，只需要关注开发的内容即可，对现有的代码逻辑基本没有侵入。

3.2.1 Gralde 的生命周期

这里必须要提一些的就是 Gradle 的生命周期，因为我们的很多扩展功能，都是在对 Gradle 执行的生命周期的各个阶段做一些改动来实现的，大概的生命周期如图：



3.2.2 单模块打包

Android 打包成 Apk 并运行的条件有：

- AndroidManifest.xml 的配置支持（application 标签的配置）
- 主 Activity 的配置

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

3.2.2.1 实现原理

- 自动生成模块自己的 Application 类
- 自动读取 Module 的 AndroidManifest 文件并修改成可以打包成 App 的配置
- 在打包的时候动态更改 SourceSet，使打包的时候使用生成的文件进行打包
- 在打包的时候动态更改支持的 Plugin 类型（'com.android.application'或是'com.android.library'）

3.2.2.2 修改模块 build.gradle 的配置

将以下配置添加到模块目录下的 build.gradle 文件中

```
modular {
        // 模块包名
        packageName = "com.youzan.ebizcore.plugin.demoa"

        app {
            // 单模块打包开关
                asApp = true
                // 运行的 App 的名称
                appName = "Module A"
                // 入口 Activity
                launchActivity = "com.youzan.ebizcore.plugin.demoa.ModuleAActivity"

                // 配置只在单模块打包时需要引入的依赖
                requires {
                    require "com.squareup.picasso:picasso:2.3.2"
                }
            }
        }
```

3.2.2.3 生成单模块运行需要的环境

运行 modular 的 createApp Task，就会自动生成需要的类（以 module_a 为例）

```
    自动生成的文件目录结构：
    ./module_a
        --src
        ----main
        ------app # 自动生成 app 目录
        --------java # 自动生成 Application 类
        --------res # 自动生成资源
        --------AndroidManifest.xml # 自动生成 Manifest 文件
```

3.2.2.4 执行单模块打包并安装的 Task

运行 modular 的 runAsApp Task，模块就会被单独达成一个 apk 包，并安装到你的手机上，如果模块有上下文依赖（比如登录）的话可以额外提供依赖，加到模块的 app 的 requires 中。 这里的打包执行是在 build 目录下生成了一个打包脚本，并调用 Gradle 的 API 执行脚本来实现打包安装的。

3.14.2.3.3模块 API 管理

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

模块 API 层提供的接口和数据结构代码是可以直接在模块内部被引用到的，方便开发，但是在暴露给外部的模块时候的时候是需要打包成 aar 上传到 Maven 来提供的，Modular-Plugin 分别针对这两个步骤提供了两个 Task，方便开发者快速进行开发和发布。

3.2.3.1 在 build.gralde 中添加相关配置

```
modular {
        packageName = "com.youzan.ebizcore.plugin.demoa"
        // 模块 API 支持相关参数
        api {
            // 是否需要提供 API 支持的开关（会影响到是否可以运行自动生成代码的 Task）
            hasApi = true
            // 对外提供的 API Service 类名
            apiService = "ModuleAService"
            // API 层的依赖
            requires {
                require "com.google.code.gson:gson:2.8.2"
            }
        }
    }
```

3.2.3.2 生成 API 打包需要的文件

运行 modular 的 createApi Task，就会自动生成需要的类（以 module_b 为例）

```
./module_b
      --src
      ----main
      ------service # 自动生成 service 目录，用来存放对外接口和数据对象
      --------java # 自动生成 Application 类
      --------AndroidManifest.xml # 自动生成 Manifest 文件，为了单独打成 aar 包
```

3.14.2.3.4 模块发布

发布功能内部使用了 'maven-publish' 插件来进行依赖的上传，开发者只关心上报的配置就好

3.2.4.1 在 build.gralde 中添加发布配置

```
modular{
        // 模块发布需要的参数
        publish {
            // 是否打开模块发布
            active = true
            // 上报地址，支持本地路径和远程 Mave 仓库地址
            repo = "../release"
            groupId = "com.youzan.ebizmobile.demo"
            artifactId = "modular-a"
            // 上报的业务模块 aar 包的版本号
            moduleVersion = "0.1.4"
            // 上报的 API 层 aar 包的版本号
            apiVersion = "0.1.5"
```

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

```
                // Maven 登录名和密码，可以从 local.properties 中取
                userName = ""
                password = ""
            }
        }
```

3.2.4.2 执行发布的 Task

运行 modular 的 uploadModule Task，Module-Plugin 会执行打包上传的任务，执行顺序是这样的： 1. 首先打包并上传 Module 的 API 模块（SourceSet 只包含 API 的类） 2. 将 Module API 的代码从模块的 SourceSet 中去除，并添加刚才上报的 API 模块的 Maven 依赖到 Module 的 dependencies 中

3.14.2.4Modular-support

3.3.1 基础组件抽象

以图片组件为例，一般业务模块中使用到的图片相关的功能有：图片加载、图片选择等，可以把这些功能抽象成接口

```
    interface IImageLoadSupport {
        fun <IMAGE : ImageView> loadImage(imageView: IMAGE?, imgUrl: String)
        fun <IMAGE : ImageView> loadImage(imageView: IMAGE?, @DrawableRes drawableId: Int)
        fun <IMAGE : ImageView> loadImage(imageView: IMAGE?, imgUrl: String, callback:
ImageLoadCallback<IMAGE>)
        fun imagePicker(activity: Activity?, selectedImgUris: List<Uri>)
        fun onImagePickerResult(requestCode: Int, resultCode: Int, intent: Intent?):
List<String>?
    }
```

3.3.2 基础组件的实现

基础组件的实现可以在 App 中进行注册，如果需要单模块组件中使用 Support 相关功能，可以提供一套默认实现，在但模块运行时引入，在全局有一个 Support 注册中心，以 Map 的形式维护运行中的 Support 对象：

```
    fun <SUPPORT : Any, SUPPORTIMPL : SUPPORT> registerProvider(supportCls: Class<SUPPORT>,
provider: SupportProvider<SUPPORTIMPL>) {
        synchronized(Lock) {
            supportsProviderMap[supportCls] = provider
            if (supportsMap.containsKey(supportCls)) {
                supportsMap.remove(supportCls)
            }
        }
    }
```

3.14.2.5规划

开发到现在，这边的三个组件已经能够基本完成我们对于组件化核心需求，但是，也是有一些方向可以进一步优化整套方案的使用：

- Modular-Support 组件引入依赖注入的方式实现 API 的调用，使用方可以不再需要关心实例对象的获取
- Modular-Support 组件可以提供给 Weex、RN、H5、Flutter 业务一些原生的功能
- Modular-Plugin 能够进一步压缩打包时间，并且让开发中的依赖配置更加灵活
- Modular-Plugin 继续优化管理和依赖打包的功能，提高效率

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

3.14.2.6 总结

组件化的道路千万条，项目的架构也是在不断得调整优化，来达到提升团队开发效率，保证项目稳定性的目的。以上的这些想法和在实际项目中的一些方案，希望能给正在进行模块化探索的同学提供一些灵感。

致谢

书稿终告断落，掩卷思量，饮水思源，在此谨表达自身的殷切期许与拳拳谢意。非常感谢各位读者耐心的看到本身的最后部分。

首先，第三章实战部分几乎已经包含高级Android 组件化的全部内容，本书的第三章通过多个大厂实例，从不同的角度引领读者亲身实战，真正地掌握高级编程的核心开发技巧。但是，实例的数量终究有限，希望读者更多地关注于实战中的开发思想，而不是具体的代码逻辑，代码总会不断地更迭，解决问题的思维却历久弥新。本书中的实例更多的是以点带面，读者可以一边阅读和思考，一边编写代码，相信读完本书，一定受益匪浅；同时，通过本书的实例可以解决一些常见的开发需求。

衷心希望每位读者在阅读完本书之后，都“不虚此行”！在书籍撰写过程中，有赞技术团队、腾讯技术团队、阿里巴巴技术团队、美团技术团队、爱奇艺技术团队、得到技术团队、携程技术团队、蘑菇街技术团队倾情无私分享的项目实战案例。同时也感谢其他博主的在组件化分享的心得笔记。在本书引用的文章和作者这里都一一表示感谢。

最后，得感谢本书得以付梓的幕后英雄彭丑丑和腾讯享学技术团队，包括关键技术资料和项目实战案例搜集整理，谢谢您们！对所引用的书籍表示感谢。

参考资料

《Android组件化架构》 —— 苍王 著

有赞技术团队：

《有赞移动应用如何给页面安上“任意门”》 —— qiezi 著

《有赞微商城-Android组件化方案》 —— qiezi 著

《有赞移动 iOS 组件化（模块化）架构设计实践》 —— jackie 著

腾讯技术团队：

《微信Android模块化架构重构实践》 —— carlguo 著

阿里巴巴技术团队：

《你知道支付宝容器化架构是怎么搭建的吗？》 —— 奶盖

《Atlas-手淘组件化框架的前世今生和未来的路》 —— 手淘团队 著

美团技术团队：

《外卖客户端容器化架构的演进》 —— 郭赛 同同 徐宏 著

《Android组件化方案及组件消息总线modular-event实战》 —— 海亮 著

《Android消息总线的演进之路：用LiveDataBus替代RxBus、EventBus》 —— 海亮 著

《WMRouter：美团外卖Android开源路由框架》 —— 子健 渊博 云驰 著

《美团猫眼android模块化实战-可能是最详细的模块化实战》 —— happylion_heart 著

腾讯享学课堂 Android 高级架构系列免费直播公开课
https://ke.qq.com/course/341933?taid=8674704361863085

爱奇艺技术团队：

《Android组件化跨进程通信框架Andromeda解析》—— wanderingguy 著

得到技术团队：

《Android彻底组件化demo发布》—— 格竹子 著

《Android彻底组件化—代码和资源隔离》—— 格竹子 著

《组件化：代码隔离也难不倒组件的按序初始化》—— leobert-lan 著

《浅谈Android组件化》—— 张明庆 著

携程技术团队：

《携程移动App架构优化之旅》—— 陈浩然 著

蘑菇街技术团队：

《蘑菇街 App 的组件化之路》—— limboy 著

《蘑菇街 App 的组件化之路·续》—— limboy 著

其他：

《Android 组件化/模块化 的理解！》—— 前行的乌龟 著

《Android组件化开发思想与实践》—— popular_linda 著

《关于Android模块化我有一些话不知当讲不当讲》—— 流水 著

《Android组件化方案》—— 张华洋 著

《蘑菇街、滴滴、淘宝、微信的组件化架构解析》—— 刘小杜 著

《Android业务组件化开发实践》—— kymjs张涛 著

《Android实现模块 api 化》—— Tyhj 著

《Android | 用力过猛！为了组件化改造学习十几家大厂的技术博客》](https://juejin.cn/post/6896099703474749453) —— 彭丑丑 著

腾讯享学课堂Android移动互联网架构开发 https://ke.qq.com/course/341933

作者：ana生 链接：https://www.jianshu.com/p/96830179c418